??????????????

??????????????, ??????????????
??? ?????????????????? ??????????
?????? ?????????????? ???? ESH27 Lab

2026-11-11

- פרק 0: הקדמה
  - החזון במשפט אחד
  - למה עכשיו: נקודת המפנה של 2024–2026
  - עיקרון היושר: רמות טענה
  - מה המעבדה איננה
  - למי הספר מיועד
  - איך לקרוא את הספר
  - מוסכמות בספר
  - מילה על זמן
  - בקצרה
  - רעיונות לתוכן
- פרק 1: מהי הוכחה?
  - הוכחה כשרשרת: אקסיומות ואוקלידס
  - המאה ה-19: מהפכת הקפדנות
  - משבר היסודות: הפרדוקס של ראסל
  - התוכנית של הילברט
  - גדל: משפטי אי-השלמות
  - טיורינג: מה מכונה יכולה לחשב
  - ZFC: היסוד הסטנדרטי
  - תורת הטיפוסים והתאמת קרי–הווארד
  - מהי "הוכחה פורמלית"?
  - הוכחה חברתית מול הוכחה פורמלית
  - איך כל זה מתחבר למעבדה

- בקצרה
- רעיונות לתוכן
- פרק 2: הבעיות הגדולות
  - 1900: עשרים ושלוש הבעיות של הילברט
  - 2000: בעיות המילניום
  - בעיות מפורסמות נוספות
  - למה המעבדה לא תוקפת את הבעיות האלה (עדיין)
  - בקצרה
  - רעיונות לתוכן
- פרק 3: מכונות והוכחות
  - הענף הראשון: מכונות שמוצאות הוכחות
  - הענף השני: מכונות שבודקות הוכחות
  - משפט ארבעת הצבעים: ההוכחה שפתחה את הוויכוח
  - השערת קפלר ו-Flyspeck
  - עידן ה-SAT: הוכחות בגודל של טרה-בייטים
  - BB(5): הבונה העסוק ואי-הכריעות
  - Lean ו-mathlib
  - הוויכוח הפילוסופי: האם זו הוכחה?
  - מה המעבדה לוקחת מההיסטוריה הזו
  - בקצרה
  - רעיונות לתוכן
- פרק 4: AI ומתמטיקה 2019–2026
  - המצב בנקודת ההתחלה
  - מערכה ראשונה: Lean מתבגר (2020–2023)
  - מערכה שנייה: המוכיחים הנוירוניים (2020–2025)
  - מערכה שלישית: חיפוש וגילוי (2021–2025)
  - מערכה רביעית: פורמליזציה בקנה מידה (2024–2026)
  - שני קולות: Tao ו-Buzzard
  - פרשת בעיות ארדש
  - בנצ׳מרקים: מדידה, זיהום והייפ
  - ובפיזיקה התיאורטית
  - הלקחים: מה המעבדה לומדת מכל זה
  - מהיסטוריה לפרקטיקה
  - בקצרה
  - רעיונות לתוכן

- פרק 5: Lean 4 בידיים
  - מה זה Lean, בשתי פסקאות
  - התקנה
  - המשפט הראשון
  - לקרוא מטרות
  - עשר טקטיקות שכדאי להכיר
  - סיור בקובץ Basic.lean של המעבדה
  - איך mathlib מאורגנת
  - איך מוצאים את הלמה הנכונה
  - sorry: ההבטחה שלא קוימה
  - כשהניסוח עצמו שגוי
  - מכאן לאן
  - בקצרה
  - רעיונות לתוכן
- פרק 6: ארכיטקטורת המעבדה
  - העיקרון: גרעין אמון
  - הלולאה
  - רמות טענה
  - סגל הסוכנים
  - היומן
  - מודל הנתונים
  - ה-orchestrator בשימוש
  - איך הפיזיקה משתלבת
  - ה-analogy-agent: מחולל ניסוחים בלבד
  - מבנה המאגר
  - עקרונות היושר, בסיכום
  - בקצרה
  - רעיונות לתוכן
- פרק 6½: חמשת היסודות
  - א. לא בשמים היא
  - ב. בשם אומרו
  - ג. למד לשונך לומר "איני יודע"
  - ד. או חברותא או מיתותא
  - ה. עבד ולא אדון
  - חמישה רבדים, חמישה יסודות

    - בקצרה
    - רעיונות לתוכן
- פרק 7: בנק המטרות
    - קודם כל, בכנות
    - חמש השכבות
    - השדות
    - איך מחושב הדירוג
    - איך הנתונים זורמים
    - סיור בבנק
    - שלוש המטרות הראשונות
    - בקצרה
    - רעיונות לתוכן
- פרק 8: ארגז הכלים
    - איך הקטלוג בנוי
    - סיור לפי קטגוריות
    - ה-Starter Stack: חמישה-עשר כלים ליום הראשון
    - צינורות: איך כלים מתחברים
    - עקרונות לבחירת כלים
    - בקצרה
    - רעיונות לתוכן
- פרק 9: דפוסי הוכחה יצירתיים
    - לפני הכול: הדפוס שאינו דפוס
    - דפוס 1: לשנות את שפת הבעיה
    - דפוס 2: כוח גס שהופך לאמין
    - דפוס 3: לייבא כלי מתחום אחר
    - דפוס 4: למצוא את האינווריאנט הנכון
    - דפוס 5: המכונה מחפשת, האדם מסביר
    - דפוס 6: לחפש תוכניות, לא אובייקטים
    - דפוס 7: לקודד כ-SAT, ולאמת את הקידוד
    - דפוס 8: קיום בלי בנייה
    - דפוס 9: אינטואיציה פיזיקלית כמחולל השערות
    - דפוס 10: נחיל, פיגום ומבט מבחוץ
    - סיפור צד: האלכסון
    - ה-playbook של הסוכנים
    - בקצרה

    - רעיונות לתוכן
- פרק 10: פיזיקה תיאורטית
    - שתי תרבויות של "נכון"
    - רמות ודאות בפיזיקה
    - מפת הבעיות הפתוחות
    - איך AI ושיטות פורמליות משתלבים
    - חמישה פרויקטים להתחלה
    - גבולות ויושר
    - בקצרה
    - רעיונות לתוכן
- פרק 11: גשרי חוכמה
    - ההבחנה היסודית: שני מישורים
    - חלק א: דרך הגמרא כשיטת חשיבה
    - חלק ב: ספר יצירה, קומבינטוריקה עתיקה
    - חלק ג: הרמב"ם, הלכות קידוש החודש כאלגוריתם
    - חלק ד: הלוח העברי כתורת מספרים
    - חלק ה: צירוף, מילוי והרש"ש
    - חלק ו: מסורות עתיקות אחרות
    - חלק ז: מנוע המשלים
    - חלק ח: גבולות
    - בקצרה
    - רעיונות לתוכן
- פרק 12: יושרה, אתיקה ופרסום
    - למה צריך מדיניות כתובה
    - רמות הטענה כמדיניות עריכה
    - איך מכריזים על תוצאה
    - קרדיט: בני אדם, מודלים ועבודה קודמת
    - טעויות, תיקונים והסרות
    - נתונים וקוד פתוחים
    - כבוד לטקסטים קדושים
    - הימנעות מהייפ
    - צינור התוכן: מהספר לסרטון
    - רשימת בדיקה עריכתית
    - בקצרה
    - רעיונות לתוכן

- פרק 13: יומן מבצעים
  - איך כותבים רשומה
  - הרשומות
  - בקצרה
  - רעיונות לתוכן
- נספח א: מילון מונחים
  - א
  - ב
  - ג
  - ד
  - ה
  - ז
  - ח
  - ט
  - י
  - כ
  - ל
  - מ
  - נ
  - ס
  - ע
  - פ
  - צ
  - ק
  - ר
  - ש
  - ת
  - בקצרה
  - רעיונות לתוכן
- נספח ב: ביבליוגרפיה
  - 1. פורמליזציה ועוזרי הוכחות
  - 2. מוכיחי משפטים נוירוניים ומערכות AI למתמטיקה
  - 3. חיפוש, גילוי ובנייה בעזרת מחשב
  - 4. בעיות ארדש
  - 5. בנצ׳מרקים

- 6. הוכחות יצירתיות קלאסיות ומודרניות
- 7. פיזיקה תיאורטית
- 8. בנק המטרות: מקורות למטרות הראשונות
- 9. גשרי חוכמה: מקורות יהודיים, היסטוריה של המתמטיקה ומתודולוגיה
- 10. כלים
- 11. מקורות פנימיים של המעבדה
- בקצרה
- רעיונות לתוכן

פרק 0: הקדמה

הספר הזה מתעד ניסוי. קבוצה קטנה, בלי מחלקה אוניברסיטאית, בלי תקציב של מעבדת מחקר גדולה ובלי דוקטורט במתמטיקה טהורה, מנסה לעשות מחקר אמיתי במתמטיקה ובפיזיקה תיאורטית, בעזרת סוכני בינה מלאכותית ושפת הוכחות פורמלית בשם Lean 4. הקדמה טובה צריכה להסביר שלושה דברים: למה מישהו חושב שזה אפשרי, למה דווקא עכשיו, ואיך נדע אם הצלחנו. בלי התשובה לשאלה השלישית, השתיים הראשונות הן סתם הייפ. לכן רוב הפרק עוסק בה.

החזון במשפט אחד

המשפט שפותח את ה-README של המעבדה הוא:

> **להפוך לחוקרים במתמטיקה ובפיזיקה תיאורטית בעזרת AI ובזכותו.**

לכל מילה במשפט הזה יש תפקיד.

"להפוך לחוקרים": המטרה איננה לצרוך מתמטיקה, לסכם מאמרים או להסביר משפטים מפורסמים ברשתות החברתיות (אף שגם את זה נעשה). המטרה היא לייצר ידע חדש: חסם חדש, הוכחה חדשה, פורמליזציה שלא הייתה קיימת, תיקון סטטוס של בעיה פתוחה. חוקר הוא מי שמוסיף משהו לספרות שאחרים יכולים לבדוק ולהשתמש בו.

"במתמטיקה ובפיזיקה תיאורטית": שני התחומים שבהם הקשר בין רעיון לבין אמת הוא הדוק במיוחד. במתמטיקה טענה מוכחת או שאינה מוכחת. בפיזיקה תיאורטית המצב מורכב יותר, ופרק 10 מוקדש להבחנות בין "נגזר", "מוכח" ו"מתאים לניסוי". שני התחומים נבחרו כי בשניהם אפשר, לפחות בחלק מהבעיות, להחליף את השאלה "האם אנחנו מאמינים לזה?" בשאלה "האם המחשב בדק את זה?".

"בעזרת AI": הסוכנים הם כלי עבודה. הם קוראים ספרות, מציעים בניות, כותבים קוד Lean, מריצים פותרי SAT ומנסחים מחדש בעיות. הם עושים בשעות את מה שלצוות קטן היה לוקח חודשים.

"ובזכותו": וזה החלק הנועז. בלי הכלים האלה, צוות כמו שלנו לא היה יכול בכלל להיכנס לזירה. ההון שנדרש כדי להשתתף במחקר מתמטי ירד, לא כי המתמטיקה נעשתה קלה, אלא כי שני דברים קרו

במקביל: מודלים שמסוגלים לבצע חלק מהעבודה, ומוודא (verifier) מכני שמסוגל לבדוק את מה שהם עשו. ההימור של המעבדה הוא שהשילוב בין השניים מייצר דלת כניסה חדשה.

למה עכשיו: נקודת המפנה של 2024–2026

אפשר היה לכתוב ספר כזה ב-2015, אבל הוא היה ספר על חלום. ב-2026 הוא ספר על כלים שעובדים, עם גבולות ידועים. הנה כמה ציוני דרך שמסבירים את השינוי. הפירוט המלא נמצא בפרק 4 וב-`docs/ai-math-lessons.md`.

יולי 2024: AlphaProof

באולימפיאדה הבין-לאומית במתמטיקה (IMO) של 2024 הציגה Google DeepMind את AlphaProof ו-AlphaGeometry 2. המערכות פתרו 4 מתוך 6 בעיות וצברו 28 מתוך 42 נקודות, ציון ברמת מדליית כסף[1]. שני פרטים חשובים לא פחות מהתוצאה: ההוכחות של AlphaProof נכתבו ב-Lean ונבדקו מכנית, אבל הניסוח הפורמלי של הבעיות נעשה בידי בני אדם, וחלק מהבעיות דרשו ימים של חישוב, הרבה מעבר ל-9 השעות שעומדות לרשות מתחרה אנושי. זו הייתה הפעם הראשונה שמערכת AI הגיעה לרמת מדליה באולימפיאדה, והיא עשתה זאת דרך שפת הוכחות פורמלית.

יולי 2025: רמת זהב בשפה טבעית

שנה אחר כך, ב-IMO 2025, שתי מערכות שונות, Gemini Deep Think של Google DeepMind ומודל ניסיוני של OpenAI, השיגו 35 מתוך 42 נקודות, ציון ברמת מדליית זהב, וכתבו את ההוכחות בשפה טבעית[2]. במקביל, Aristotle של Harmonic ו-Seed-Prover של ByteDance הציגו פתרונות פורמליים ב-Lean לחלק גדול מהבעיות. ההבדל בין שני המסלולים הוא לב הספר הזה: הוכחה בשפה טבעית צריך אדם מומחה כדי לבדוק, והוכחה ב-Lean בודקת את עצמה.

מאי 2025: AlphaEvolve

ב-14 במאי 2025 הציגה DeepMind את AlphaEvolve, מערכת שבה מודל שפה כותב ומשכתב תוכניות בלולאה אבולוציונית, ומעריך אוטומטי נותן לכל תוכנית ציון[3]. בכ-50 בעיות פתוחות בקומבינטוריקה, גאומטריה ואנליזה, המערכת שחזרה את התוצאה הטובה הידועה בכ-75% מהמקרים ושיפרה אותה בכ-20%. בין השאר, החסם התחתון למספר הנשיקה במימד 11 עלה מ-592 ל-593. השיפורים היו לרוב קטנים, אבל הם היו אמיתיים, ניתנים לבדיקה, והגיעו בבעיות שבני אדם עבדו עליהן שנים. זה בדיוק סוג הבעיות שהמעבדה מסמנת כ-Tier C בבנק המטרות.

ספטמבר 2026: FLT ב-Lean

ב-4 בספטמבר 2026 דיווחה Anthropic על הוכחה מלאה ובדוקה במחשב של המשפט האחרון של פרמה (FLT) ב-Lean, שהושלמה בעזרת סוכני Claude[4]. לפי הדיווח, מדובר בכ-13 מיליון שורות קוד וכ-30 אלף משפטי ביניים, שנכתבו בכ-11 ימי עבודה של סוכנים, וההוכחה נשענת רק על שלוש האקסיומות הסטנדרטיות של Lean. Kevin Buzzard, שמוביל מאז 2024 את פרויקט הפורמליזציה של FLT באימפריאל קולג', בדק את התוצאה. לפי הדיווחים, הוא תיאר אותה כתוצאה גדולה לאוטו-פורמליזציה, שמתמטית "לא מלמדת אותנו כמעט דבר חדש" (הציטוט נלקח ממקורות משניים: [לא אומת מילולית]).

ההערה הזו של Buzzard חשובה לא פחות מההישג. פורמליזציה של משפט ידוע איננה גילוי של משפט חדש. ובכל זאת, היא מוכיחה שסוכנים יכולים כעת לבצע עבודה הנדסית-מתמטית בהיקף שהיה נחשב בלתי אפשרי, ושהתוצאה ניתנת לבדיקה בלי לסמוך על הסוכנים.

מה השתנה באמת

אם מצמצמים את ארבעת האירועים לתובנה אחת: **ה-AI נעשה מחולל טוב של רעיונות והוכחות, אבל מחולל לא-אמין. מה שהפך אותו לשימושי הוא שהוצמד לו מוודא אמין.** כל הצלחה אמיתית בשנים האלה נשענה על מוודא כזה: Lean, פותר SAT עם תעודה, או פונקציית ניקוד מדויקת. וכמעט כל פרשת הייפ, למשל ההכרזה מאוקטובר 2025 ש-GPT-5 "פתר 10 בעיות ארדש" (כשבפועל הוא מצא מאמרים קיימים שכבר פתרו אותן), נבעה מהיעדר מוודא כזה. פרק 4 מספר את הסיפורים האלה בפירוט.

עיקרון היושר: רמות טענה

המעבדה מפרסמת התקדמות בפומבי, באתר וב-S27 News. פרסום פומבי בתחום שמלא הייפ דורש משמעת. לכן כל טענה שהמעבדה מפרסמת נושאת **רמת טענה (claim level)**, מ-L0 ועד L4. ההגדרות המלאות נמצאות ב- `docs/architecture.md` , והנה הן בקצרה:

רמה	משמעות	מה מותר להגיד	מי רשאי לקבוע
L0	רעיון, השערה, השראה	"יש לנו כיוון"	כל סוכן
L1	עדות נומרית או חישובית	"בדקנו עד N"	סוכני חיפוש, בנייה ופיזיקה
L2	הוכחה לא-פורמלית	"יש טיוטת הוכחה"	lean-agent, אדם
L3	מאומת מכונה: Lean בלי `sorry` , או תעודה חישובית שנבדקה	"הוכח (מאומת מכונה)"	רק ה-**verifier**

רמה	משמעות	מה מותר להגיד	מי רשאי לקבוע
L4	עבר ביקורת עמיתים, פורסם, או אומץ בידי הקהילה	"פורסם"	אדם

כמה כללים נגזרים מהטבלה, ולאורך הספר נחזור אליהם שוב ושוב:

1. **רק תוכנה מעלה טענה ל-L3.** לא סוכן, לא חבר צוות נלהב ולא המייסד. ה-verifier מריץ את Lean, בודק שאין `sorry` , בודק שההוכחה משתמשת רק באקסיומות המותרות, ובודק תעודות SAT בכלים כמו `drat-trim` . אם הבדיקה נכשלת, הטענה נשארת ב-L2.
2. **L3 על חלק מבעיה הוא תוצאה לגיטימית.** חסם משופר או מקרה פרטי מאומת הם הישג אמיתי, והאתר מציג אותם בדיוק כך, ולא כ"פתרנו את הבעיה".
3. **בודקים את הטענה, לא רק את ההוכחה.** הוכחה מושלמת של ניסוח שגוי היא הכשל השקט המסוכן ביותר. כל ניסוח פורמלי עובר בדיקה מול המקור.
4. **רעיון מ"גשרי חוכמה" הוא תמיד L0.** פרק 11 עוסק בגמרא, בקבלה ובחוכמות עתיקות כמקור השראה ושיטת חשיבה. שם, יותר מבכל מקום אחר, חשוב לומר במפורש: השראה איננה ראיה.
5. **כל ניסיון נרשם.** הצלחה וכישלון נרשמים ביומן (`data/ledger.jsonl`), שמאפשר רק הוספה. דו"ח שלילי הוא תוצר, לא בושה.

רמות הטענה אינן רק כלל פנימי. הן גם הבטחה לקוראים: כשתראו בספר, באתר או בכתבה את המילה "הוכח", תדעו בדיוק מה עומד מאחוריה.

מה המעבדה איננה

כדי שלא יהיו אי-הבנות, כדאי לומר גם מה איננו מתיימרים לעשות.

- **איננו מתכוונים לפתור את השערת רימן.** בעיות המילניום מסומנות בבנק המטרות כ-Tier A, "מונשוטים". הן משמשות ללמידה, לתרגול פורמליזציה ולהשראה, ולא מקבלות משאבי חישוב משמעותיים בלי החלטה מפורשת של הצוות. פרק 2 מסביר למה.
- **איננו מחליפים מתמטיקאים.** כל ההצלחות של 2024–2026 שאנחנו מכירים כללו בני אדם: בניסוח הטענה, בבדיקת הסטטוס, בכתיבת ה-blueprint או בפרשנות.
- **איננו מפרסמים "תיאוריות של הכול".** לקח מפורש ממחקר AI בפיזיקה: ההצלחות נמצאות במקום שבו יש בדיקה עצמאית. טקסט פיזיקלי מבריק של מודל שפה, בלי בדיקה, הוא רעש.

למי הספר מיועד

הספר נכתב לכמה קהלים, והוא מנסה לשרת את כולם בלי לוותר על דיוק.

הקורא הסקרן המשכיל. מי שקרא על AlphaProof בעיתון ורוצה להבין מה באמת קרה. בשבילו פרקים 1–4 מספקים את הרקע: מהי הוכחה, אילו בעיות פתוחות חשובות, איך מחשבים נכנסו לעולם ההוכחות, ומה ה-AI עשה בשנים האחרונות. לא נדרש ידע מוקדם מעבר למתמטיקה של תיכון.

המפתח או מהנדס ה-AI. מי שיודע לתכנת ורוצה להיכנס לעבודה עם Lean או לבנות סוכנים למתמטיקה. בשבילו פרקים 5, 6 ו-8 הם מדריך מעשי, עם קוד ועם הפניות לקבצים במאגר.

הסטודנט או החוקר. מי שכבר יודע מתמטיקה ומחפש מקום לתרום. בשבילו בנק המטרות (פרק 7) ודפוסי ההוכחה היצירתיים (פרק 9) הם נקודת הכניסה.

הצוות עצמו והסוכנים. הספר הוא גם זיכרון ארגוני. סוכן חדש, או חבר צוות חדש, צריך להבין למה הכללים הם מה שהם. פרק 13, יומן התפעול, מתעד מה קרה בפועל.

יוצרי התוכן של S27. כל פרק מסתיים בסעיף "רעיונות לתוכן", עם הצעות קונקרטיות לכתבות, פוסטים וסרטונים. הספר הוא מקור, וממנו נגזרים התכנים.

איך לקרוא את הספר

הספר בנוי בשלושה חלקים: רקע (פרקים 1–4), המעבדה (פרקים 5–9), והרחבות (פרקים 10–13), ובסופו שני נספחים.

חלק ראשון: רקע

- **פרק 1: מהי הוכחה?** מאקסיומות של אוקלידס, דרך המשבר ביסודות המתמטיקה, ועד גדל, טיורינג ותורת הטיפוסים. הפרק מסתיים באנלוגיה בין הוכחות לתוכניות מחשב, הגשר שמוביל ל-Lean.
- **פרק 2: הבעיות הגדולות.** 23 הבעיות של הילברט, שבע בעיות המילניום, ובעיות מפורסמות נוספות כמו גולדבך, קולץ והשערת abc. בסוף: למה המעבדה מסמנת אותן כ"מונשוטים".
- **פרק 3: מכונות והוכחות.** ההיסטוריה של הוכחות בעזרת מחשב: מה-Logic Theorist ב-1956, דרך משפט ארבעת הצבעים והשערת קפלר, ועד הוכחות SAT בגודל של טרה-בייטים ופורמליזציות מחקריות ב-Lean. וגם הוויכוח הפילוסופי: האם הוכחה שאף אדם לא יכול לקרוא היא הוכחה?
- **פרק 4: AI ומתמטיקה, 2019–2026.** הסיפור המפורט של שבע השנים האחרונות, כולל ההצלחות, הכישלונות ופרשות ההייפ.

חלק שני: המעבדה

- **פרק 5: Lean 4 בידיים.** מדריך מעשי: התקנה, משפט ראשון, טקטיקות, mathlib, ואיך קוראים הודעות שגיאה.
- **פרק 6: ארכיטקטורת המעבדה.** הגרעין האמין, הלולאה, הנחיל, היומן והמתזמר. מבוסס על `docs/architecture.md`.
- **פרק 7: בנק המטרות.** איך נבחרות מטרות, מה פירוש השכבות A, B, C, D ו-P, ושלוש המטרות הראשונות. מבוסס על `data/goals.csv` ו-`docs/goal-bank.md`.
- **פרק 8: ארגז הכלים.** פותרי SAT, מוכיחים אוטומטיים, ספריות, מערכות אלגברה ממוחשבת והכלים שבנינו. מבוסס על `data/tools.csv` ו-`docs/tools.md`.
- **פרק 9: דפוסי הוכחה יצירתיים.** עשרה דפוסים חוזרים בהוכחות פורצות דרך, ואיך מלמדים אותם לסוכנים. מבוסס על `docs/creative-proofs.md`.

חלק שלישי: הרחבות

- **פרק 10: פיזיקה תיאורטית.** איפה AI ושיטות פורמליות יכולים לתרום באמת: conformal bootstrap, אמפליטודות, PhysLean, ומה כדאי להשאיר בצד.
- **פרק 11: גשרי חוכמה.** דרך הגמרא כשיטת חשיבה, הקבלה וסוד הצירוף כמבנה קומבינטורי, וכללי ההפרדה בין השראה לבין ראיה.
- **פרק 12: אתיקה, יושרה ופרסום.** ייחוס כן בין אדם למכונה, ניהול טענות ציבוריות, טעויות ותיקונן.
- **פרק 13: יומן תפעול.** מה קרה בפועל, מטרה אחר מטרה, כולל כישלונות.

נספחים

- **נספח א: מילון מונחים** (`98-glossary.md`): כמאה מונחים בעברית ובאנגלית, מלוגיקה ועד פיזיקה.
- **נספח ב: ביבליוגרפיה** (`99-bibliography.md`): המקורות המרכזיים.

מסלולי קריאה מומלצים

אם אתם...	קראו
רוצים להבין "על מה כל הרעש"	0, 1, 3, 4
רוצים לתרום קוד	0, 5, 6, 8, ואז 7
רוצים לתרום מתמטיקה	0, 2, 7, 9
מגיעים מפיזיקה	0, 1, 10, 6
כותבים תוכן ל-S27	0, ואז סעיפי "רעיונות לתוכן" בכל פרק

מוסכמות בספר

- **מונחים** מופיעים בפעם הראשונה בעברית ובאנגלית בסוגריים, למשל "עוזר הוכחות (proof assistant)". כולם מרוכזים במילון שבנספח א.
- **נוסחאות** כתובות ב-LaTeX, וקוד Lean מופיע בבלוקים מסומנים.
- **מקורות** מופיעים כהערות שוליים בסוף כל פרק, והביבליוגרפיה המרכזית בנספח ב.
- **[לא אומת]**: כל טענה שלא הצלחנו לאמת מול מקור ראשוני מסומנת כך. אנחנו מעדיפים סימון מביך על פני ביטחון מזויף.
- **הפניות למאגר** כתובות כנתיבים, למשל `data/goals.csv`. מקור האמת של המעבדה הוא קובצי ה-CSV; האתר, האקסל וגיליון Google הם תצוגות שנבנות מהם.

מילה על זמן

הספר נכתב בסוף 2026, בתקופה שבה התחום משתנה בקצב של חודשים. חלק מהעובדות בפרקים 4 ו-7 יתיישנו מהר. למשל, `docs/goal-bank.md` מזכיר שבמאי 2026 הופרכה השערת מרחק היחידה, ושב-2026 הושלמה ב-Lean הפורמליזציה של אריזת הכדורים במימד 8. ייתכן שכשתקראו את הספר, יהיו תוצאות חדשות שמשנות את התמונה. לכן הספר מבחין בין **עקרונות**, שאמורים להחזיק מעמד (אימות לפני הכרזה, רמות טענה, חיפוש ספרות קודם לכל), לבין **תמונות מצב**, שמתוארכות במפורש.

בקצרה

- החזון: להפוך לחוקרים במתמטיקה ובפיזיקה תיאורטית בעזרת AI ובזכותו, כלומר לייצר ידע חדש שאחרים יכולים לבדוק.
- למה עכשיו: בין 2024 ל-2026 הגיעו מערכות AI לרמת מדליה ב-IMO (AlphaProof, ואחריו רמת זהב), שיפרו חסמים בבעיות פתוחות (AlphaEvolve), ופרמלו משפט ענק כמו FLT ב-Lean (Anthropic, ספטמבר 2026).
- התובנה המרכזית: AI הוא מחולל לא-אמין; מה שהופך אותו לשימושי הוא מוודא אמין כמו Lean או תעודת SAT.
- עיקרון היושר: כל טענה נושאת רמה L0–L4, ורק תוכנת ה-verifier רשאית להעלות טענה ל-L3, "מאומת מכונה".
- הספר בנוי מרקע (1–4), מהמעבדה עצמה (5–9) ומהרחבות (10–13), עם מילון וביבליוגרפיה בסוף.

רעיונות לתוכן

1. **"מה זה אומר שמחשב 'הוכיח' משהו?"** זווית: הסבר רמות הטענה L0–L4 דרך שלוש דוגמאות מהחדשות (AlphaProof, פרשת בעיות ארדש של אוקטובר 2025, FLT ב-Lean). קהל: הציבור הרחב, סרטון קצר או קרוסלה ברשתות.
2. **"ארבעה רגעים ששינו את המתמטיקה, 2024–2026"** זווית: ציר זמן ויזואלי עם הסבר של מה היה חדש באמת בכל רגע, ומה לא. קהל: קוראי S27 News, כתבת רקע.
3. **"המניפסט של ESH27 Lab"** זווית: פוסט מייסד שמסביר את המשפט "בעזרת AI ובזכותו" ואת ההימור על שילוב מחולל ומוודא. קהל: קהילת ה-AI והטכנולוגיה, פוסט לינקדאין או בלוג.
4. **"מה אנחנו לא נעשה"** זווית: כתבה נגד הייפ, שמסבירה למה המעבדה לא תוקפת את השערת רימן ובמה היא כן מתחילה. קהל: מתמטיקאים וספקנים, פוסט ארוך.
5. **"איך לקרוא את הספר: חמישה מסלולים"** זווית: מדריך קצר שמפנה כל סוג קורא לפרקים שמתאימים לו. קהל: קוראים חדשים, עמוד נחיתה באתר.

פרק 1: מהי הוכחה?

כל מה שהמעבדה עושה נשען על מילה אחת: "הוכחה". כשמערכת AI מכריזה שהיא "הוכיחה" משהו, כשמתמטיקאי מקבל פרס על הוכחה, וכשהמעבדה מסמנת טענה כ-L3, "מאומת מכונה", כולם משתמשים באותה מילה. אבל המילה הזו שינתה משמעות לאורך 2,300 שנה, והשינויים האלה לא היו קוסמטיים. כל אחד מהם נולד ממשבר: גילוי שמה שנחשב הוכחה איננו מספיק. הפרק הזה עובר על המשברים האלה, מאוקלידס ועד תורת הטיפוסים, ומסביר למה בסוף הדרך עומדת תוכנה כמו Lean, שבה הוכחה היא, פשוטו כמשמעו, תוכנית מחשב שעוברת קומפילציה.

הוכחה כשרשרת: אקסיומות ואוקלידס

לפני היוונים, מתמטיקה הייתה בעיקר אוסף של מתכונים. הבבלים ידעו לחשב שלשות פיתגוריות ולפתור משוואות ריבועיות, והמצרים ידעו לחשב נפחים. אבל איש לא שאל *למה* המתכון עובד תמיד.

החידוש היווני, שהגיע לשיאו ב"יסודות" (Elements) של אוקלידס, בסביבות 300 לפני הספירה, היה שיטה: מתחילים ממספר קטן של **אקסיומות (axioms)**, הנחות יסוד שמקבלים בלי הוכחה, ומהן גוזרים טענות חדשות בצעדים שכל אחד מהם הוא מובן מאליו. כל טענה שנגזרת כך נקראת **משפט (theorem)**, והשרשרת עצמה היא **הוכחה (proof)**.

היופי של השיטה הוא שהיא מעבירה את האמון. אם אתם מקבלים את חמש האקסיומות ואת כללי ההיסק, אתם חייבים לקבל את כל 465 המשפטים של היסודות. אינכם צריכים לסמוך על אוקלידס, רק על השרשרת.

המשבר הראשון: האקסיומה החמישית

האקסיומה החמישית של אוקלידס, אקסיומת המקבילים, הייתה מסורבלת יותר מהאחרות. בניסוח מודרני (של פלייפייר) היא אומרת: דרך נקודה שמחוץ לישר עובר בדיוק ישר אחד שמקביל לו. במשך כאלפיים שנה ניסו מתמטיקאים להוכיח אותה מתוך ארבע האחרות, ונכשלו.

במאה ה-19 התברר למה. לובצ'בסקי (1829), בויאי (1832), ובאופן לא-פורסם גם גאוס, בנו גאומטריות שבהן ארבע האקסיומות הראשונות מתקיימות והחמישית לא. **הגאומטריה הלא-אוקלידית** הייתה עקבית בדיוק כמו האוקלידית. המסקנה הייתה עמוקה: אקסיומות אינן "אמיתות מובנות מאליהן" על העולם, אלא הנחות שבוחרים. מערכת אקסיומות אחת מתארת עולם אחד, ומערכת אחרת מתארת עולם אחר.

המשבר השני: הצעדים ה"מובנים מאליהם"

גם בתוך היסודות עצמם היו חורים. כבר בהוכחה הראשונה בספר, בניית משולש שווה צלעות, אוקלידס מניח ששני מעגלים נחתכים, בלי שום אקסיומה שמבטיחה זאת. הוא הסתמך על הציור. ב-1899 פרסם דויד הילברט את "יסודות הגאומטריה" (Grundlagen der Geometrie), ובו מערכת של 20 אקסיומות שסוגרת את החורים האלה. לפי האגדה, הילברט אמר שאפשר להחליף את "נקודות, ישרים ומישורים" ב"שולחנות, כיסאות וספלי בירה", וההוכחות צריכות להישאר תקפות. הוכחה טובה לא תלויה בציור או במשמעות; היא תלויה רק בצורה.

זה רעיון חשוב בשבילנו: אם הוכחה תלויה רק בצורה, אז מכונה, שלא מבינה משמעות, יכולה לבדוק אותה.

המאה ה-19: מהפכת הקפדנות

בזמן שהגאומטרים גילו את גבולות האקסיומות, האנליטיקאים גילו את גבולות האינטואיציה. החשבון הדיפרנציאלי והאינטגרלי של ניוטון ולייבניץ עבד מצוין, אבל נשען על "גדלים אינפיניטסימליים", מספרים קטנים מכל מספר חיובי ועם זאת לא אפס. הבישוף ברקלי לעג להם ב-1734 וכינה אותם "רוחות של כמויות שנפטרו".

במאה ה-19 ביצעו קושי, בולצאנו ובעיקר ויירשטראס את מה שנקרא **אריתמטיזציה של האנליזה**: הם החליפו את האינפיניטסימלים בהגדרה המדויקת של גבול, הגדרת ה-δ–ε:

$$\lim_{x \to a} f(x) = L \iff \forall \varepsilon > 0 \; \exists \delta > 0 \; \forall x \; (0 < |x - a| < \delta \Rightarrow |f(x) - L| < \varepsilon).$$

ההגדרה הזו מכוערת, אבל היא מדויקת: אין בה שום דבר מעורפל, וכל טענה על גבולות ניתנת לבדיקה צעד אחר צעד. דדקינד (1872) וקנטור הגדירו את המספרים הממשיים עצמם באמצעות מספרים רציונליים, והרציונליים הוגדרו באמצעות הטבעיים. פתאום כל האנליזה עמדה על יסוד אחד: המספרים הטבעיים ותורת הקבוצות.

הלקח של המאה ה-19 היה שהאינטואיציה מטעה. ויירשטראס הציג ב-1872 פונקציה רציפה בכל מקום ולא גזירה בשום מקום, אובייקט שהאינטואיציה הגאומטרית "ידעה" שאינו יכול להתקיים. הקפדנות לא הייתה פדנטיות, אלא הגנה.

משבר היסודות: הפרדוקס של ראסל

קנטור בנה את תורת הקבוצות, שבה אפשר לדבר על אינסופים בגדלים שונים. פרגה, בספרו "חוקי היסוד של האריתמטיקה", ניסה לבנות את כל המתמטיקה מתוך לוגיקה בלבד. ואז, ב-1902, כשהכרך השני של

פרגה היה בדפוס, הוא קיבל מכתב מברטרנד ראסל.

ראסל הציע להתבונן ב"קבוצת כל הקבוצות שאינן איברים של עצמן":

$$R = \{ x \mid x \notin x \}.$$

האם $R \in R$? אם כן, אז לפי ההגדרה $R \notin R$. אם לא, אז לפי ההגדרה $R \in R$. סתירה בשני המקרים. המערכת של פרגה, שאפשרה ליצור קבוצה מכל תכונה, הייתה **לא עקבית (inconsistent)**: אפשר להוכיח בה כל דבר, כולל $0 = 1$.

משבר היסודות הוליד שלוש תשובות, ולכל אחת מהן יש צאצא ישיר בעולם של עוזרי ההוכחות של היום:

1. **תורת הקבוצות האקסיומטית.** צרמלו (1908) ופרנקל (1922) הגבילו את דרכי היצירה של קבוצות. התוצאה, **ZFC** (צרמלו–פרנקל עם אקסיומת הבחירה), היא היסוד הסטנדרטי של המתמטיקה כיום. מערכת Mizar ומערכת Metamath (במסד set.mm) בנויות עליה.
2. **תורת הטיפוסים.** ראסל עצמו, יחד עם וייטהד, פיתח בספר "פרינקיפיה מתמטיקה" (1910–1913) מערכת שבה לכל אובייקט יש **טיפוס (type)**, ואי אפשר לשאול אם קבוצה שייכת לעצמה, כי השאלה פשוט לא מנוסחת היטב. הפרינקיפיה מפורסמת בכך שההוכחה ש-$1 + 1 = 2$ מגיעה רק אחרי מאות עמודים. תורת הטיפוסים המודרנית, שעליה בנוי Lean, היא צאצא רחוק של הרעיון הזה.
3. **אינטואיציוניזם.** ברואר טען שאסור להשתמש בהוכחות קיום שאינן בונות את האובייקט, ודחה את חוק השלישי הנמנע ($P \vee \neg P$). הרעיון נשמע אז קיצוני, אבל הוא חזר בגדול במדעי המחשב: הוכחה קונסטרוקטיבית היא בעצם אלגוריתם.

התוכנית של הילברט

הילברט רצה לסגור את המשבר אחת ולתמיד. **התוכנית של הילברט (Hilbert's program)**, שגובשה בשנות ה-20, דרשה שלושה דברים:

- **פורמליזציה:** לנסח את כל המתמטיקה כמערכת פורמלית, עם אקסיומות וכללי היסק מכניים לחלוטין.
- **עקביות:** להוכיח שהמערכת לא מובילה לסתירה, ולעשות זאת באמצעים "סופיים" שאיש לא יכול לפקפק בהם.
- **הכרעה:** למצוא שיטה מכנית שקובעת, לכל טענה, אם היא ניתנת להוכחה. זו בעיית ההכרעה, **Entscheidungsproblem**.

ב-1930, בנאום בקניגסברג, אמר הילברט את המשפט שנחקק על מצבתו: "עלינו לדעת, אנו נדע" (Wir müssen wissen, wir werden wissen). יום קודם לכן, באותה עיר, הציג צעיר בן 24 בשם קורט

גדל תוצאה שהרסה את התוכנית.

גדל: משפטי אי-השלמות

ב-1931 פרסם גדל את **משפטי אי-השלמות (incompleteness theorems)**[5]. בניסוח לא-פורמלי:

1. **המשפט הראשון:** בכל מערכת פורמלית עקבית שמספיק חזקה כדי לתאר את האריתמטיקה של המספרים הטבעיים, יש טענות אמיתיות שאי אפשר להוכיח בתוכה.
2. **המשפט השני:** מערכת כזו אינה יכולה להוכיח את העקביות של עצמה.

הטריק של גדל היה לקודד נוסחאות והוכחות כמספרים (**מספור גדל, Gödel numbering**), כך שטענות על הוכחות הופכות לטענות על מספרים. כך הוא בנה טענה שאומרת, בעקיפין, "אני לא ניתנת להוכחה". אם המערכת עקבית, הטענה הזו אכן לא ניתנת להוכחה, ולכן היא אמיתית.

חשוב להבין מה גדל לא אמר, כי משפטי אי-השלמות הם אולי התוצאה המתמטית שמצוטטת באופן השגוי ביותר:

- גדל לא אמר שהמתמטיקה "לא ודאית". הוכחה תקפה נשארת תקפה.
- גדל לא אמר שבעיות כמו השערת רימן לא ניתנות להוכחה. רוב הבעיות שמתמטיקאים עוסקים בהן ניתנות להוכחה או להפרכה ב-ZFC, ככל שידוע.
- גדל לא פסל את Lean. Lean לא מנסה למצוא כל הוכחה, אלא *לבדוק* הוכחה נתונה, וזו משימה פשוטה ומכנית לחלוטין.

אבל הוא כן הראה שיש טענות שהן **בלתי תלויות (independent)** במערכת. הדוגמה המפורסמת: **השערת הרצף (continuum hypothesis)**, השאלה הראשונה ברשימת הבעיות של הילברט. גדל הראה ב-1940 שאי אפשר להפריך אותה ב-ZFC, ופול כהן הראה ב-1963 שאי אפשר גם להוכיח אותה. היא פשוט לא נקבעת על ידי האקסיומות.

טיורינג: מה מכונה יכולה לחשב

נשארה השאלה השלישית של הילברט, בעיית ההכרעה. כדי לענות עליה צריך קודם להגדיר מה זה "שיטה מכנית". ב-1936, באופן בלתי תלוי, נתנו שתי תשובות שקולות: אלונזו צ'רץ' עם **תחשיב למדא (lambda calculus)**, ואלן טיורינג עם **מכונת טיורינג (Turing machine)**[6].

טיורינג תיאר מכונה מופשטת: סרט אינסופי, ראש שקורא וכותב סמל אחד בכל פעם, ומספר סופי של מצבים. הטענה שכל חישוב מכני ניתן לביצוע במכונה כזו נקראת **תזת צ'רץ'–טיורינג**. ואז טיורינג הוכיח

שיש שאלה שאף מכונה כזו לא יכולה להכריע: **בעיית העצירה (halting problem)**. אין אלגוריתם שמקבל תוכנית וקלט ואומר תמיד נכון אם התוכנית תעצור.

מכאן נובע שאין שיטה מכנית שמכריעה כל טענה מתמטית. בעיית ההכרעה של הילברט נפתרה בשלילה.

שתי ההשלכות של טיורינג חשובות למעבדה:

1. **חיפוש הוכחה הוא קשה באופן עקרוני.** אין אלגוריתם שמוצא תמיד הוכחה אם היא קיימת ואחרת עוצר ואומר "אין". לכן כל מוכיח אוטומטי, וכל מודל AI, הם בהכרח היוריסטיים. הם יכולים להצליח הרבה, אבל לא תמיד.
2. **בדיקת הוכחה היא קלה באופן עקרוני.** לבדוק שכל צעד בשרשרת נובע מהקודמים זה תהליך מכני וסופי. זו האסימטריה שעליה בנויה המעבדה: **ליצור הוכחה קשה, לבדוק אותה קל.** לכן אפשר לתת למחולל לא-אמין (מודל שפה) ליצור, ולמוודא אמין (Lean) לבדוק.

לבעיית העצירה יש גם קשר ישיר לפרק 3: מספרי "הבונה העסוק" (Busy Beaver), שאחד מהם, BB(5), נקבע ב-2024 בעזרת הוכחה ב-Coq, הם בדיוק הביטוי הכמותי של אי-הכריעות הזו.

ZFC: היסוד הסטנדרטי

כשמתמטיקאית עובדת אומרת "הוכחתי", היא בדרך כלל מתכוונת, אם לוחצים עליה, ש"אפשר בעיקרון לתרגם את ההוכחה לשרשרת של צעדים ב-ZFC". ZFC כולל אקסיומות כמו:

- **היקפיות:** שתי קבוצות עם אותם איברים שוות.
- **זוג, איחוד, קבוצת חזקה:** אפשר לבנות קבוצות חדשות מקבוצות קיימות.
- **הפרדה (סכמה):** אפשר לבחור תת-קבוצה של קבוצה קיימת לפי תכונה. זה מה שחוסם את הפרדוקס של ראסל: אי אפשר לבחור "מכל הקבוצות", רק מתוך קבוצה נתונה.
- **אינסוף:** קיימת קבוצה אינסופית.
- **הבחירה (Axiom of Choice):** מכל אוסף של קבוצות לא ריקות אפשר לבחור איבר אחד מכל קבוצה. האקסיומה הזו שנויה במחלוקת היסטורית, כי היא מאפשרת תוצאות מוזרות כמו הפרדוקס של בנך–טרסקי. כיום רוב המתמטיקאים מקבלים אותה.

המילה החשובה היא "בעיקרון". אף אחד לא באמת כותב הוכחות ב-ZFC. הוכחה מלאה ב-ZFC של משפט אמיתי תהיה ארוכה באופן בלתי אפשרי. הפער בין "אפשר בעיקרון" ל"עשינו בפועל" הוא בדיוק הפער שעוזרי ההוכחות סוגרים.

תורת הטיפוסים והתאמת קרי–הווארד

ZFC איננו היסוד היחיד האפשרי. לאורך המאה ה-20 התפתחה דרך אחרת, שהתחילה אצל ראסל ועברה דרך צ'רץ': **תורת הטיפוסים (type theory)**. במקום "הכול הוא קבוצה", בתורת הטיפוסים לכל ביטוי יש טיפוס: `3` הוא מטיפוס "מספר טבעי", `fun n ⇒ n + 1` הוא מטיפוס "פונקציה ממספרים טבעיים למספרים טבעיים", וכן הלאה.

הוכחות הן תוכניות

ההבחנה המכרעת התגלתה בהדרגה. הסקל קרי שם לב כבר בשנות ה-30 וה-50 לדמיון מוזר בין טיפוסים של פונקציות לבין נוסחאות לוגיות, וויליאם הווארד ניסח את הקשר במלואו בכתב יד מ-1969 (שפורסם ב-1980). הקשר הזה נקרא **התאמת קרי–הווארד (Curry–Howard correspondence)**, או בקיצור "הוכחות כתוכניות" (proofs as programs)[7]:

לוגיקה	תכנות
טענה P	טיפוס `P`
הוכחה של P	ערך (תוכנית) מטיפוס `P`
$P \Rightarrow Q$	פונקציה מ-`P` ל-`Q`
$P \wedge Q$	זוג: ערך מ-`P` וערך מ-`Q`
$P \vee Q$	ערך מאחד משני הטיפוסים, עם תווית
$\forall x,\ P(x)$	פונקציה שמקבלת `x` ומחזירה הוכחה של `P x`
טענה שקרית	טיפוס ריק, שאין לו ערכים
בדיקת הוכחה	בדיקת טיפוסים (type checking)

כלומר: להוכיח ש-P גורר Q זה לכתוב פונקציה שמקבלת הוכחה של P ומחזירה הוכחה של Q. ולבדוק שההוכחה נכונה זה פשוט לבדוק שהתוכנית מקיימת את הטיפוס שלה, בדיוק מה שקומפיילר עושה בכל פעם שהוא מוודא שלא העברתם מחרוזת לפונקציה שמצפה למספר.

איך זה נראה ב-Lean

הנה דוגמה קטנה ב-Lean 4. הטענה: אם P ו-Q מתקיימות, אז Q ו-P מתקיימות.

```
-- מקבלים זוג הפוך (hp, hq) ומחזירים זוג (h.2, h.1) הוכחה כתוכנית:
theorem and_swap (P Q : Prop) (h : P ∧ Q) : Q ∧ P :=
  ⟨h.2, h.1⟩
```

```
-- אותה הוכחה בסגנון טקטיקות, קרוב יותר לאופן שבו בני אדם חושבים
theorem and_swap' (P Q : Prop) (h : P ∧ Q) : Q ∧ P := by
  obtain ⟨hp, hq⟩ := h
  exact ⟨hq, hp⟩
```

בגרסה הראשונה ההוכחה היא פשוט זוג: הרכיב השני של `h` ואחריו הרכיב הראשון. בגרסה השנייה כותבים **טקטיקות (tactics)**, הוראות שמייצרות את התוכנית בשבילנו. בשני המקרים, מה שבודק את ההוכחה בסוף הוא אותו דבר: **הגרעין (kernel)** של Lean, תוכנה קטנה יחסית שבודקת טיפוסים. אם הגרעין מקבל, ההוכחה נכונה (בהנחה שהגרעין עצמו נכון, והטענה נוסחה נכון).

מתורת הטיפוסים ל-Lean

הדרך מהוואר ד ל-Lean עוברת דרך כמה תחנות, שפרק 3 מספר עליהן בפירוט: Automath של דה-ברויין (1967), **תורת הטיפוסים של מרטין-לף (Martin-Löf type theory)** בשנות ה-70, **תחשיב הקונסטרוקציות (Calculus of Constructions)** של קוקאן והואה (1988), שעליו נבנה Coq (היום Rocq), ותחשיב הקונסטרוקציות האינדוקטיביות, שעליו בנוי גם Lean. Lean עצמו נוצר ב-2013 בידי לאונרדו דה מורה במיקרוסופט, ו-Lean 4 יצא ב-2021.

תורת הטיפוסים של Lean איננה קונסטרוקטיבית לחלוטין: mathlib משתמשת בחופשיות באקסיומת הבחירה ובחוק השלישי הנמנע, כמו מתמטיקאים רגילים. שלוש האקסיומות הסטנדרטיות של Lean הן `propext` (טענות שקולות שוות), `Classical.choice` (בחירה) ו-`Quot.sound` (על מנות). ה-verifier של המעבדה בודק בדיוק את זה: הפקודה `#print axioms` חייבת להראות שההוכחה לא משתמשת בשום דבר מעבר לשלוש האלה (ראו `docs/architecture.md`).

מהי "הוכחה פורמלית"?

עכשיו אפשר לענות בדיוק. **הוכחה פורמלית (formal proof)** היא אובייקט שעונה על שלושה תנאים:

1. היא כתובה בשפה שהתחביר שלה מוגדר במדויק, כך שתוכנה יכולה לקרוא אותה.
2. כל צעד בה הוא הפעלה של כלל מתוך רשימה סגורה של כללים, או של אקסיומה מתוך רשימה סגורה של אקסיומות.
3. תוכנה, **בודק הוכחות (proof checker)**, יכולה לוודא את (1) ו-(2) באופן מכני.

עוזר הוכחות (proof assistant), כמו Lean, Coq/Rocq, Isabelle, HOL Light או Mizar, הוא סביבה שבה אדם (או סוכן) כותב הוכחה פורמלית בעזרת אוטומציה, והבודק מוודא אותה. **מוכיח אוטומטי (automated theorem prover)**, כמו Vampire או E, מחפש הוכחה לבד, בדרך כלל בלוגיקה פשוטה יותר. **פותר SAT** מכריע אם נוסחה בוליאנית ספיקה, ופותרים מודרניים מפיקים

תעודה (certificate), למשל בפורמט DRAT או LRAT, שבודק נפרד ופשוט יכול לבדוק. כל הכלים האלה מופיעים בפרק 8.

מה הוכחה פורמלית *לא* מבטיחה

הוכחה פורמלית מבטיחה שהטענה, *כפי שנוסחה*, נובעת מהאקסיומות, *בהנחה שהבודק תקין*. יש כאן שתי הנחות:

- **הבודק תקין.** לכן משתמשים בבודקים קטנים (עקרון דה-ברויין: גרעין קטן שאפשר לבדוק בעין), ולפעמים בכמה בודקים עצמאיים. ל-Lean יש כמה מימושים חיצוניים של בודק הגרעין.
- **הניסוח נכון.** זו הבעיה הקשה באמת. אם ניסחתם "כל מספר זוגי גדול מ-2 הוא סכום של שני ראשוניים" ובטעות הגדרתם "ראשוני" כך שגם 1 נחשב ראשוני, הוכחתם משהו אחר. ב-`docs/ai-math-lessons.md` מתועד מקרה שבו סוכן אוטו-פורמליזציה "עקף" למות שנוסחו בצורה שגויה. מכאן הכלל של המעבדה: בודקים את הטענה, לא רק את ההוכחה.

הוכחה חברתית מול הוכחה פורמלית

אם הוכחות פורמליות כל כך טובות, למה מתמטיקאים לא כותבים רק כך? כי ההוכחות שמתמטיקאים כותבים בפועל הן משהו אחר: **טיעונים חברתיים (social proofs)**. הוכחה במאמר היא טקסט שנועד לשכנע מומחה, שמדלג על צעדים "שגרתיים" ומסתמך על ידע משותף. היא מתקבלת כשהקהילה, בדרך כלל דרך שופטים ודרך שנים של שימוש, משתכנעת.

ב-1979 פרסמו דה-מילו, ליפטון ופרליס מאמר מפורסם בשם "Social Processes and Proofs of Theorems and Programs"[8]. הטענה שלהם: הוכחות מתמטיות מתקבלות דרך תהליך חברתי של קריאה, דיון ושימוש, ולא דרך אימות מכני. הם השתמשו בזה כדי לטעון נגד אימות פורמלי של תוכנה. ארבעים שנה אחר כך, הטיעון שלהם קיבל תפנית: התהליך החברתי עובד היטב, אבל לא מושלם, ויש לו גבולות קיבולת.

כמה התהליך החברתי טועה?

אין נתון מדויק על שיעור הטעויות בספרות המתמטית, וכל מספר שמופיע ברשת צריך להתייחס אליו בזהירות. אבל יש מקרים מתועדים היטב:

- **קמפה וארבעת הצבעים.** ב-1879 פרסם אלפרד קמפה "הוכחה" למשפט ארבעת הצבעים. היא התקבלה במשך 11 שנים, עד שב-1890 הית'וד מצא בה שגיאה.
- **ויילס והמשפט האחרון של פרמה.** ההוכחה שהציג אנדרו ויילס ב-1993 הכילה פער רציני, שהתגלה בתהליך השיפוט. ויילס, עם ריצ'רד טיילור, תיקן אותו ב-1994. הסיפור נגמר טוב, אבל

הוא מראה שגם הוכחה שנבדקה בידי מומחי העולם יכולה להחזיק פער.

- **וובודסקי.** ולדימיר וובודסקי, זוכה מדליית פילדס (2002), סיפר בהרצאות מאוחרות שבמאמר שלו עם קפרנוב מ-1989 הייתה שגיאה שקרלוס סימפסון הצביע עליה ב-1998, ושוובודסקי עצמו השתכנע בה רק ב-2013. הוא תיאר גם שגיאה במאמר מרכזי שלו מ-1993, שהתגלתה שנים אחרי שהמאמר כבר שימש אחרים. המקרים האלה היו המניע המוצהר שלו לעבור לעבודה על יסודות חדשים, **יסודות יוניוולנטיים (univalent foundations)**, ולהשתמש בעוזרי הוכחות. במילים שלו, בתרגום חופשי: הוא הבין שהוא כבר לא יכול לסמוך על כך שאחרים יבדקו את עבודתו, כי היא מסובכת מדי ואיש אינו קורא אותה עד הסוף[9].
- **סיווג החבורות הפשוטות הסופיות.** ההוכחה פרושה על פני עשרות אלפי עמודים במאות מאמרים. "סיום" הסיווג הוכרז בתחילת שנות ה-80, אבל פער בטיפול במקרה ה"קוואזי-תין" נסגר רק ב-2004, בספר של אשבכר וסמית' באורך של יותר מ-1,200 עמודים.
- **השערת abc.** ב-2012 פרסם שיניצ'י מוצ'יזוקי טענה להוכחה של השערת abc, בתורה חדשה (IUT). ב-2018 טענו פטר שולצה ויעקב שטיקס שיש פער בלתי ניתן לגישור בטענה מרכזית (Corollary 3.12). ההוכחה פורסמה ב-2021 בכתב עת שמוצ'יזוקי עצמו הוא העורך הראשי שלו, ורוב הקהילה עדיין לא מקבלת אותה. פרק 2 מרחיב.

גם הצד השני חשוב: רוב המשפטים החשובים נכונים, גם כשההוכחות המקוריות שלהם פגומות. התהליך החברתי מתקן את עצמו. השאלה היא לא אם הוא עובד, אלא אם הוא יחזיק מעמד מול גידול בהיקף, בהתמחות, ועכשיו גם מול מבול של הוכחות שנוצרות בידי מודלי שפה. Kevin Buzzard טוען כבר שנים שהמתמטיקה המחקרית סובלת מ"הוכחות שאף אחד לא בדק באמת"[10]. אם הוכחות מגיעות במהירות של מכונה, הבדיקה חייבת לעבוד במהירות של מכונה.

היחסים בין שני סוגי ההוכחה

נכון יותר לראות את שני סוגי ההוכחה כשותפים ולא כמתחרים:

- **ההוכחה החברתית מסבירה.** היא אומרת *למה* משהו נכון, מאיפה בא הרעיון, ומה עוד אפשר לעשות איתו.
- **ההוכחה הפורמלית מבטיחה.** היא אומרת *שמשהו* נכון, בלי להסתמך על אמון.

פרויקטים כמו Liquid Tensor Experiment ו-PFR (פרק 3) מראים שהתהליך עובד בשני הכיוונים: פטר שולצה כתב שהפורמליזציה שיפרה את ההבנה שלו את ההוכחה שלו עצמו. ובמעבדה, רמות הטענה משקפות בדיוק את החלוקה הזו: L2 היא הוכחה חברתית (טיוטה שאדם או סוכן שני קרא), L3 היא הוכחה פורמלית, ו-L4 היא שוב תהליך חברתי, של קבלה בקהילה. אף אחת מהן איננה מיותרת.

איך כל זה מתחבר למעבדה

הפרק הזה הוא היסוד הרעיוני של `docs/architecture.md` . ליתר דיוק:

1. **האסימטריה של טיורינג היא העיקרון הארכיטקטוני.** קשה למצוא הוכחה, קל לבדוק אותה. לכן הסוכנים (מחוללים לא-אמינים) מחפשים, וה-verifier (גרעין אמין) בודק. "ליבת האמון" של המעבדה היא בדיוק הגרעין של Lean ובודקי התעודות.
2. **קרי–הוארד הוא הסיבה ש-Lean עובד עם מודלי שפה.** הוכחה ב-Lean היא קוד, ומודלי שפה טובים בכתיבת קוד. משוב הקומפיילר ("השורה הזו לא עוברת בדיקת טיפוסים") הוא בדיוק סוג המשוב שמאפשר לסוכן לתקן את עצמו.
3. **הוויכוח החברתי-פורמלי מסביר את רמות הטענה.** L2 ו-L4 הן הוכחות חברתיות, L3 היא פורמלית, וכל אחת אומרת משהו אחר.
4. **גדל מזכיר לנו צניעות.** יש שאלות שאינן ניתנות להכרעה במערכת שלנו. סוכן שמבטיח "לפתור הכול" לא הבין משהו בסיסי.
5. **הבעיה של הניסוח הנכון היא הסיכון המרכזי.** לכן בכל פורמליזציה במעבדה, ה-statement עובר בדיקה אנושית מול המקור, ועדיף השוואה אוטומטית ל-statement קיים ב-mathlib.

בקצרה

- הוכחה היא שרשרת של צעדים מאקסיומות; אוקלידס נתן את השיטה, והמאה ה-19 גילתה שגם אקסיומות וגם אינטואיציה דורשות זהירות.
- הפרדוקס של ראסל פירק את היסודות של פרגה והוליד את ZFC, את תורת הטיפוסים ואת האינטואיציוניזם, שלושה כיוונים שחיים היום בעוזרי ההוכחות.
- גדל הראה שבכל מערכת מספיק חזקה יש טענות שאינן ניתנות להוכחה, וטיורינג הראה שאין אלגוריתם שמכריע הכול; אבל *בדיקה* של הוכחה נתונה היא תמיד מכנית.
- התאמת קרי–הוארד מזהה הוכחות עם תוכניות וטענות עם טיפוסים; זה הבסיס של Lean, וזו הסיבה שמודלי שפה יכולים לכתוב הוכחות שמחשב בודק.
- הוכחות חברתיות מסבירות והוכחות פורמליות מבטיחות; מקרים כמו קמפה, ויילס ו-וובודסקי מראים שהתהליך החברתי טועה לפעמים, ורמות הטענה במעבדה משלבות את שני הסוגים.

רעיונות לתוכן

1. **"הקבוצה שלא יכולה להחליט"** זווית: סרטון אנימציה קצר על הפרדוקס של ראסל, מכתב לפרגה ועד ZFC, עם גרסת "הספר שמקטלג את כל הספרים שלא מקטלגים את עצמם". קהל: הציבור הרחב, רשתות חברתיות.
2. **"מה גדל באמת אמר (ומה לא)"** זווית: כתבה שמפרקת שלוש אי-הבנות נפוצות על משפטי אי-השלמות, ומסבירה למה הם לא מאיימים על Lean. קהל: קוראי S27 News עם עניין במדע.

3. **"הוכחה היא תוכנית"** זווית: פוסט למפתחים שמסביר את קרי–הווארד דרך קוד Lean קצר, עם הטבלה לוגיקה↔תכנות. קהל: מתכנתים, בלוג טכני.
4. **"המתמטיקאי שהפסיק לסמוך על עצמו"** זווית: פרופיל של וובודסקי והמעבר שלו לעוזרי הוכחות, כסיפור אנושי על טעות ואחריות. קהל: קהל רחב, פודקאסט או כתבת עומק.
5. **"חמש הוכחות שגויות שכולם האמינו להן"** זווית: רשימה מתועדת (קמפה, ויילס 1993, החבורות הפשוטות, abc), עם הדגשה שהמשפטים עצמם לרוב נכונים. קהל: קוראי S27 News.

פרק 2: הבעיות הגדולות

מתמטיקה מתקדמת דרך שאלות. משפט שהוכח הוא סוף של סיפור, אבל בעיה פתוחה היא מנוע: היא מושכת אנשים, מייצרת כלים, ולעתים קרובות הכלים שנבנו בדרך חשובים יותר מהפתרון עצמו. הפרק הזה מציג את הבעיות המפורסמות ביותר במתמטיקה: הרשימה של הילברט מ-1900, שבע בעיות המילניום מ-2000, וכמה בעיות "קטנות" בניסוח וענקיות בקושי. לכל בעיה ננסה לתת תמונה אינטואיטיבית אחת, בלי לרמות בתוכן. ובסוף נסביר למה, למרות כל הקסם, המעבדה מסמנת את רובן כ"מונשוטים" ולא עובדת עליהן ישירות. הסיבה איננה פחדנות; זו אסטרטגיה, והיא נובעת ישירות מהלקחים של 2024–2026.

1900: עשרים ושלוש הבעיות של הילברט

ב-8 באוגוסט 1900, בקונגרס הבין-לאומי של המתמטיקאים בפריז, נשא דויד הילברט את אחת ההרצאות המשפיעות בתולדות המדע. הוא פתח בשאלה "מי מאיתנו לא היה שמח להרים את הצעיף שמאחוריו מסתתר העתיד?", והציג רשימה של בעיות שלדעתו יעצבו את המאה הבאה. בהרצאה עצמה הוא הציג עשר בעיות, ובגרסה המודפסת הרשימה המלאה מנתה $23 = 11 + 10 + 2$.

הרשימה של הילברט מעניינת לא רק בגלל הבעיות, אלא בגלל מה שהיא מלמדת על בעיות בכלל:

- **חלק מהבעיות נפתרו באופן חד.** הבעיה השלישית (האם שני פאונים בעלי נפח שווה ניתנים לחיתוך זה לזה) נפתרה בתוך שנה בידי מקס דהן, תלמידו של הילברט: התשובה שלילית. הבעיה השביעית (האם $2^{\sqrt{2}}$ טרנסצנדנטי) נפתרה ב-1934 בידי גלפונד ושניידר.
- **חלק נפתרו בדרך שהילברט לא דמיין.** הבעיה הראשונה, השערת הרצף, התבררה כבלתי תלויה באקסיומות (גדל 1940, כהן 1963; ראו פרק 1). הבעיה העשירית ביקשה אלגוריתם שמכריע אם למשוואה דיופנטית יש פתרון בשלמים. ב-1970 השלים יורי מטיאסביץ' עבודה של דייויס, פוטנם ורובינסון והוכיח ש**אין** אלגוריתם כזה. "פתרון" יכול להיות הוכחה שהשאלה בלתי ניתנת להכרעה.
- **חלק מעורפלות מכדי להיפתר.** הבעיה השישית ביקשה "אקסיומטיזציה של הפיזיקה". זו יותר תוכנית מחקר מבעיה, וחלקים ממנה עדיין פעילים. בנק המטרות כולל אותה בשכבה P (ראו `docs/theoretical-physics.md`).
- **חלק עדיין פתוחות.** הבעיה השמינית כוללת את השערת רימן, שהופיעה מאה שנה אחר כך גם ברשימת המילניום.

הלקח: רשימה טובה של בעיות היא לא רק רשימת משימות. היא הגדרה של מה נחשב חשוב, והיא משפיעה על מה שאנשים לומדים ובונים. בנק המטרות של המעבדה הוא, בקנה מידה צנוע מאוד, ניסיון לעשות אותו דבר: להגדיר מה חשוב *עבורנו*.

2000: בעיות המילניום

מאה שנה אחרי הילברט, ב-24 במאי 2000, הכריז מכון קליי למתמטיקה (Clay Mathematics Institute), שוב בפריז, על שבע **בעיות המילניום (Millennium Prize Problems)**, עם פרס של מיליון דולר לפתרון כל אחת[12]. התנאים: הפתרון צריך להתפרסם בכתב עת מוכר, ולעמוד שנתיים של בדיקה בקהילה. עד היום רק אחת נפתרה.

נעבור על כולן. לכל אחת: מה שואלים, תמונה אינטואיטיבית, ומה המצב.

1. השערת פואנקרה: הבעיה שנפתרה

השאלה. אנרי פואנקרה שאל ב-1904: נניח שיש לנו "מרחב" תלת-ממדי סגור וסופי. אם כל לולאה בו אפשר לכווץ לנקודה, האם הוא בהכרח ספירה תלת-ממדית (במובן הטופולוגי)?

התמונה. דמיינו נמלה על פני כדור, מחזיקה גומייה. כל לולאה שהיא מותחת על הכדור אפשר לכווץ לנקודה בלי לחתוך את הגומייה ובלי לצאת מהמשטח. על פני סופגנייה (טורוס) זה לא נכון: לולאה שמקיפה את החור לא תתכווץ. במימד 2 זה ידוע מזמן: המשטח הסגור היחיד שבו כל לולאה מתכווצת הוא הספירה. פואנקרה שאל אם זה נכון גם במימד 3, שבו אנחנו לא יכולים "לראות" את המרחב מבחוץ.

הפתרון. הגרסאות במימדים גבוהים הוכחו קודם: סמייל עבור מימד 5 ומעלה (1961) ופרידמן עבור מימד 4 (1982). את מימד 3 פתר גריגורי פרלמן, בשלושה מאמרים שהעלה ל-arXiv בשנים 2002–2003. פרלמן השתמש ב**זרימת ריצ'י (Ricci flow)**, תוכנית של ריצ'רד המילטון: מתייחסים לגאומטריה של המרחב כאילו הייתה טמפרטורה, ונותנים לה "להתפזר" עד שהיא נעשית אחידה. הקושי היה ב"סינגולריות", מקומות שבהם הזרימה מתפוצצת, ופרלמן מצא איך לנתח אותן ולחתוך אותן בצורה מבוקרת. ההוכחה נבדקה בידי כמה צוותים במשך כמה שנים.

פרלמן דחה את מדליית פילדס (2006) ואת פרס המילניום (2010). הסיפור שלו מלמד שני דברים שחשובים לספר הזה: פתרון בעיה גדולה בא לרוב מכלי מתחום אחר (כאן, משוואות דיפרנציאליות חלקיות בשירות הטופולוגיה), ובדיקת פתרון כזה לוקחת שנים של עבודה קהילתית. זה בדיוק סוג הבדיקה ש-Lean, בעתיד, אולי יוכל לקצר. ההוכחה של פואנקרה עדיין לא פורמלה במלואה, ככל הידוע לנו.

2. השערת רימן

השאלה. פונקציית זטא של רימן מוגדרת עבור $\mathrm{Re}(s) > 1$ על ידי

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s} = \prod_{p\ \text{ראשוני}} \frac{1}{1 - p^{-s}},$$

ומורחבת לכל המישור המרוכב (חוץ מ-$s = 1$). ההשערה, מ-1859: כל האפסים ה"לא-טריוויאליים" של הפונקציה נמצאים על הישר $\mathrm{Re}(s) = \frac{1}{2}$.

התמונה. המכפלה בנוסחה (מכפלת אוילר) מראה ש-ζ "יודעת" על המספרים הראשוניים. רימן הראה שהאפסים שלה קובעים כמה מסודרים הראשוניים. חשבו על הראשוניים כעל מוזיקה: האפסים הם התדרים. ההשערה אומרת שכל התדרים מאוזנים בדיוק, ולכן ה"רעש" בהתפלגות הראשוניים הוא הקטן ביותר האפשרי, בערך כמו השורש הריבועי של הטווח. אם ההשערה נכונה, הראשוניים מפוזרים "באקראי באופן הכי מסודר שאפשר".

המצב. פתוחה. מיליארדי אפסים ראשונים נבדקו נומרית ונמצאו על הישר; בדיקה של פלאט וטרוג'יאן (2021) אימתה את ההשערה לכל האפסים עד גובה של כ-$3 \cdot 10^{12}$. אבל עדות נומרית, ככל שתהיה גדולה, היא L1 ולא הוכחה. ב-2024 פרסמו לארי גות' וג'יימס מיינרד שיפור לחסם על מספר האפסים שעלולים להיות *רחוק* מהישר, השיפור הראשון מזה עשרות שנים בשאלה הזו, אבל זו עדיין רחוקה מאוד מההשערה עצמה.

3. P מול NP

השאלה. האם כל בעיה שקל *לבדוק* פתרון שלה, קל גם *למצוא* לה פתרון? "קל" פירושו כאן בזמן פולינומי בגודל הקלט. P היא מחלקת הבעיות שקל לפתור, ו-NP היא מחלקת הבעיות שקל לבדוק.

התמונה. סודוקו ענק, בגודל מיליון על מיליון. אם מישהו נותן לכם לוח מלא, לבדוק אותו קל: עוברים על כל שורה, עמודה וריבוע. אבל למצוא פתרון? ככל הנראה קשה מאוד. השאלה היא אם ה"ככל הנראה" הזה הוא אמת מתמטית, או רק מגבלה של הדמיון שלנו.

המצב. פתוחה. רוב החוקרים מאמינים ש-P≠NP. הבעיה נוסחה בידי סטיבן קוק (1971) ובאופן בלתי תלוי בידי לאוניד לוין. יש תוצאות "מחסום" שמראות שכמה גישות טבעיות לא יכולות לעבוד: relativization (בייקר, גיל וסולוביי, 1975), natural proofs (רזבורוב ורודיך, 1994) ו-algebrization (אהרונסון ווידרזון, 2008).

לבעיה הזו יש קשר הדוק לספר: היא בדיוק האסימטריה בין חיפוש לבדיקה מפרק 1. בעיית הספיקות הבוליאנית (SAT), שפותרי SAT פותרים בהצלחה מפתיעה בפועל, היא הבעיה ה-NP-שלמה הראשונה (משפט קוק–לוין). המעבדה כולה בנויה על ההנחה שגם אם *למצוא* קשה, *לבדוק* קל.

4. משוואות נאוויה–סטוקס

השאלה. משוואות נאוויה–סטוקס מתארות זרימה של נוזל צמיג, כמו מים או אוויר. בתלת-ממד, אם מתחילים מזרימה חלקה עם אנרגיה סופית, האם הפתרון נשאר חלק לנצח? או שייתכן שאחרי זמן סופי

מהירות הנוזל בנקודה כלשהי תהיה אינסופית, מה שנקרא **התפוצצות (blow-up)**?

התמונה. מערבולת באמבטיה. כשהמים מסתובבים מהר יותר ויותר בתוך אזור שהולך וקטן, האם ייתכן שהמערבולת "תתכווץ" לנקודה בזמן סופי, עם מהירות אינסופית? הצמיגות פועלת נגד זה, כמו חיכוך. השאלה היא מי מנצח.

המצב. פתוחה. בדו-ממד ידוע שהפתרונות נשארים חלקים. טרנס טאו הראה ב-2016 שגרסה "ממוצעת" של המשוואות כן מתפוצצת, מה שמרמז שכל הוכחה של חלקות חייבת להשתמש במבנה המיוחד של המשוואות. עבור משוואות אוילר (נוזל ללא צמיגות) עם דופן, צ'ן והו הציגו הוכחה בעזרת מחשב להתפוצצות (preprint 2022)[14]. זו דוגמה מצוינת להוכחה בעזרת מחשב בעולם האנליזה, וחלק מהמסלולים שהמעבדה רואה בשכבה P עוברים בדיוק שם (ראו `docs/theoretical-physics.md`).

5. יאנג–מילס ופער המסה

השאלה. תורות יאנג–מילס הן הבסיס של המודל הסטנדרטי בפיזיקת חלקיקים. הבעיה מבקשת שני דברים: להוכיח שתורה קוונטית כזו (עם חבורת כיול פשוטה, במרחב-זמן ארבע-ממדי) *קיימת* באופן מתמטי מדויק, ולהוכיח שיש לה **פער מסה (mass gap)**: לחלקיק הקל ביותר בתורה יש מסה חיובית.

התמונה. פיזיקאים משתמשים בתורות האלה כבר יותר מחמישים שנה, והן מנבאות תוצאות ניסוי בדיוק מדהים. ובכל זאת, מתמטית, אף אחד לא יודע להגדיר אותן עד הסוף. זה כמו בניין שכולם גרים בו ועובד מצוין, אבל אין לו תוכנית הנדסית. הפער עצמו מסביר למה הכוח החזק פועל רק למרחקים קצרים, בעוד שהגלואונים שנושאים אותו חסרי מסה ב"משוואה". משהו בדינמיקה הקוונטית יוצר מסה מאין.

המצב. פתוחה, ונחשבת אולי לרחוקה ביותר מפתרון. הבעיה דורשת **תורת שדות קוונטית קונסטרוקטיבית (constructive QFT)**, שקיימת היום רק במימדים נמוכים יותר. בבנק המטרות, ה-`formalization_readiness` שלה הוא 1: חסרות תיאוריות שלמות.

6. השערת הודג'

השאלה. הנוסח הטכני עוסק ביריעות אלגבריות פרויקטיביות מרוכבות ובמחלקות קוהומולוגיה. בניסוח פשוט: במרחבים גאומטריים מסוג מסוים, האם כל "צורה" שנראית אלגברית מבחינה טופולוגית היא באמת צירוף של צורות אלגבריות, כלומר של קבוצות פתרונות של משוואות פולינומיות?

התמונה. יש לנו שני סוגי עדשות. עדשה טופולוגית, שרואה רק "חורים" ומבנה גס, ועדשה אלגברית, שרואה רק את מה שאפשר לבנות ממשוואות פולינומיות. ההשערה אומרת שבמחלקה מסוימת של חורים (מחלקות הודג'), מה שהעדשה הטופולוגית רואה תמיד בנוי מחלקים שהעדשה האלגברית רואה. הטופולוגיה לא מכילה הפתעות שהאלגברה לא יכולה להסביר.

המצב. פתוחה. ידועה במקרים פרטיים (למשל עבור מחלקות מדרגה 1, משפט לפשץ). זו הבעיה שהכי קשה להסביר לציבור, וגם הכי קשה לפורמליזציה: בבנק המטרות היא משמשת בעיקר כתרגיל בניסוח.

7. השערת בירץ' וסווינרטון-דייר (BSD)

השאלה. עקום אליפטי (elliptic curve) הוא, בקירוב, משוואה מהצורה $y^2 = x^3 + ax + b$. נקודות רציונליות על העקום, כלומר פתרונות שבהם x,y שברים, יוצרות חבורה, ואפשר "לחבר" אותן. לחבורה הזו יש **דרגה (rank)**, שאומרת כמה נקודות בלתי תלויות צריך כדי לייצר את כולן. ההשערה מקשרת את הדרגה הזו, אובייקט אלגברי, לסדר האפס של פונקציה אנליטית, פונקציית L של העקום, בנקודה $s = 1$.

התמונה. בשנות ה-60 בירץ' וסווינרטון-דייר ספרו במחשב, באוניברסיטת קיימברידג', כמה פתרונות יש לעקום מודולו כל ראשוני p. הם גילו שכשיש לעקום הרבה פתרונות "מקומיים" (מודולו הרבה ראשוניים), יש לו גם הרבה פתרונות "גלובליים" (בשברים). כאילו ספירה של צללים בכל חדר בבית מגלה כמה אנשים גרים בו. אגב, זו אחת ההשערות הגדולות הראשונות שנולדה מניסוי מחשב.

המצב. פתוחה. ידועה לעקומים מדרגה אנליטית 0 או 1, בזכות גרוס–זגייר וקוליווגין (שנות ה-80). בהרגאווה ושנקר הראו שחלק חיובי של העקומים מקיים את ההשערה. העקומים האליפטיים הם גם הגיבורים של הוכחת המשפט האחרון של פרמה, ולכן חלק ניכר מהתשתית עבורם כבר קיים ב-mathlib.

בעיות מפורסמות נוספות

בעיות המילניום אינן היחידות. יש בעיות ששמן ידוע לכל מי שאוהב מתמטיקה, בגלל שהן פשוטות לניסוח באופן מתגרה.

השערת גולדבך

הניסוח. כל מספר זוגי גדול מ-2 הוא סכום של שני ראשוניים. למשל $28 = 5 + 23 = 11 + 17$. כריסטיאן גולדבך הציע אותה במכתב לאוילר ב-1742.

התמונה. ככל שהמספר גדל, מספר הדרכים לכתוב אותו כסכום של שני ראשוניים גדל, בערך כמו $n/(\ln n)^2$. ההשערה נראית "בטוחה" סטטיסטית, אבל סטטיסטיקה איננה הוכחה.

המצב. פתוחה. נבדקה במחשב לכל המספרים הזוגיים עד $4 \cdot 10^{18}$ (אוליביירה א-סילבה, הרצוג ופרדי, 2014)[15]. **השערת גולדבך החלשה**, שכל מספר אי-זוגי גדול מ-5 הוא סכום של שלושה ראשוניים, הוכחה ב-2013 בידי הרלד הלפגוט, בשילוב של אנליזה עמוקה ובדיקות מחשב נרחבות למספרים הקטנים. זה דפוס שחוזר: תיאוריה מטפלת ב"אינסוף", ומחשב מטפל בטווח הסופי שנשאר.

השערת הראשוניים התאומים

הניסוח. יש אינסוף זוגות ראשוניים שההפרש ביניהם 2, כמו (11,13), (17,19), (41,43).

המצב. פתוחה, אבל עם אחד מסיפורי ההצלחה היפים של העשור הקודם. ב-2013 הוכיח יטאנג ג'אנג, מתמטיקאי שעבד שנים מחוץ לאקדמיה הממסדית, שיש אינסוף זוגות ראשוניים שההפרש ביניהם קטן מ-70 מיליון. זה היה החסם הסופי הראשון אי פעם. בתוך חודשים, פרויקט שיתופי מקוון, Polymath8, יחד עם שיטה חדשה של ג'יימס מיינרד (ובאופן בלתי תלוי טאו), הוריד את החסם ל-246. המעבר מ-70 מיליון ל-246 הוא אחת הדוגמאות הטובות למחקר מתמטי שיתופי ופתוח, ובמובן מסוים הוא המודל שהמעבדה רוצה לחקות: הרבה צעדים קטנים ובדוקים.

השערת קולץ (3n+1)

הניסוח. קחו מספר טבעי. אם הוא זוגי, חלקו ב-2; אם הוא אי-זוגי, כפלו ב-3 והוסיפו 1. חזרו על התהליך. ההשערה: תמיד תגיעו בסוף ל-1.

$$n \mapsto \begin{cases} n/2 & n \text{ זוגי} \\ 3n+1 & n \text{ אי-זוגי} \end{cases}$$

התמונה. מתחילים ב-27 והמסלול מטפס עד 9,232 לפני שהוא יורד, אחרי 111 צעדים, ל-1. המספרים קופצים כמו אבן ברד בענן, ולכן הם נקראים לפעמים "מספרי ברד". בממוצע, כל צעד של "×3+1" מגיע עם כמה חלוקות ב-2, והמסלול יורד, אבל ממוצע איננו הוכחה.

המצב. פתוחה. נבדקה במחשב לכל המספרים עד לפחות 2^{68} (בארינה, 2020)[16]. ב-2019 הוכיח טאו ש"כמעט כל" המספרים מגיעים לערכים קטנים מאוד, במובן מדויק של צפיפות לוגריתמית, התוצאה החזקה ביותר עד היום. ארדש אמר עליה, לפי מסורת שמצוטטת הרבה, ש"המתמטיקה עוד לא מוכנה לבעיות כאלה" [לא אומת מילולית]. קולץ היא גם מלכודת קלאסית לחובבים ולמודלי שפה: קל "להוכיח" אותה בטעות. במעבדה היא ב-Tier A, ושום טענה עליה לא תפורסם מעל L1 בלי Lean.

השערת abc: המחלוקת

הניסוח. לכל שלשה של מספרים זרים $a + b = c$, נסמן ב-$\mathrm{rad}(abc)$ את מכפלת הראשוניים השונים שמחלקים את abc. ההשערה (אוסטרלה ומאסר, 1985) אומרת שלכל $\varepsilon > 0$, רק למספר סופי של שלשות מתקיים

$$c > \mathrm{rad}(abc)^{1+\varepsilon}.$$

התמונה. חיבור וכפל לא "מסתדרים" טוב זה עם זה. אם a ו-b בנויים מעט ראשוניים בחזקות גבוהות (כמו 2^{10} ו-3^5), אז c בדרך כלל לא יכול להיות גם הוא בנוי כך. ההשערה מכמתת את זה. אם היא נכונה, נובעים ממנה הרבה משפטים אחרים, כולל גרסה "אסימפטוטית" של המשפט האחרון של פרמה.

המצב: שנוי במחלוקת. ב-2012 העלה שיניצ'י מוצ'יזוקי מאוניברסיטת קיוטו ארבעה מאמרים, מאות עמודים, שבהם פיתח תורה חדשה בשם **תורת טייכמילר הבין-אוניברסלית (Inter-universal Teichmüller theory, IUT)** וטען שהיא מוכיחה את abc. מעט מאוד אנשים הצליחו לקרוא את המאמרים. ב-2018 נסעו פטר שולצה ויעקב שטיקס לקיוטו, ובדו"ח שפרסמו טענו שיש פער בלתי ניתן לגישור ב-Corollary 3.12, לב הטיעון. מוצ'יזוקי דחה את הטענה. ב-2021 פורסמו המאמרים בכתב העת PRIMS, שמוצ'יזוקי הוא העורך הראשי שלו (הוא לא השתתף בהליך השיפוט, לפי ההודעה הרשמית). רוב קהילת תורת המספרים אינה מקבלת את ההוכחה[17].

הפרשה הזו היא אולי הטיעון החזק ביותר בעד פורמליזציה. כאן יש מחלוקת שבה שני צדדים מבריקים מתעקשים, והתהליך החברתי (פרק 1) נתקע. הוכחה ב-Lean הייתה מכריעה את השאלה, אבל פורמליזציה של IUT היא משימה עצומה, ובבנק המטרות הסטטוס של השורה הזו הוא `disputed`.

ארדש והבעיות שלו

פול ארדש (1913–1996) היה אולי המתמטיקאי הפורה ביותר במאה ה-20: כ-1,500 מאמרים עם מאות שותפים. הוא חי כנווד, נסע מחבר לחבר עם מזוודה אחת, ובכל מקום השאיר בעיות. **מספר ארדש (Erdős number)** של מתמטיקאי הוא המרחק שלו מארדש בגרף של כתיבה משותפת.

ארדש נהג להציע פרסים כספיים לפתרון בעיות, מסכומים סמליים של כמה דולרים ועד אלפי דולרים, לפי הקושי שהעריך. הסכומים הם חלק מהפולקלור; כמה מהם שולמו בידי עמיתים אחרי מותו.

לפרק הזה בספר, הבעיות של ארדש חשובות במיוחד מכמה סיבות:

1. **יש מהן מאות, בכל רמות הקושי.** חלקן עמוקות מאוד, וחלקן, כפי שהתברר, פשוט לא נוסו ברצינות.
2. **יש להן אתר מרכזי.** תומס בלום מתחזק את erdosproblems.com, עם סטטוס לכל בעיה.
3. **הן הפכו לזירת המבחן הראשית של AI במתמטיקה ב-2025–2026.** באוקטובר 2025 הוכרז ש-GPT-5 "פתר 10 בעיות ארדש", ובלום הבהיר שמדובר בבעיות שכבר נפתרו בספרות, והמודל מצא את המאמרים. מינואר 2026 החלו להגיע פתרונות חדשים באמת, הראשון שבהם לבעיה #728, עם הוכחה פורמלית ב-Lean. הוויקי שמתחזק טאו על "תרומות AI לבעיות ארדש" מתעד הצלחות, כישלונות וגילויים-מחדש, ומזהיר ש"היעדר התקדמות בעבר עשוי לשקף אלמוניות ולא קושי" (ראו `docs/ai-math-lessons.md`, סעיף 4).

הלקח השלישי הזה מעצב את כל אסטרטגיית המעבדה, ולכן הוא מוביל אותנו לסעיף הבא.

למה המעבדה לא תוקפת את הבעיות האלה (עדיין)

אחרי כל הקסם, הגיע הזמן לכנות. בנק המטרות של המעבדה (`data/goals.csv` , 77 מטרות נכון ל-2026-11-04) מחלק את הבעיות לחמש שכבות. הפירוט המלא ב- `docs/goal-bank.md` ובפרק 7, וכאן רק העיקר:

שכבה	שם	דוגמאות
A	מונשוטים (Moonshots)	רימן, P מול NP, נאוויה–סטוקס, יאנג–מילס, הודג', BSD, גולדבך, ראשוניים תאומים, קולץ, abc
B	בעיות פתוחות מפורסמות	union-closed, lonely runner, Hadwiger, sunflower
C	ברות-השגה בעזרת AI	בעיות ארדש, מספרי רמזי ו-Schur, cap sets, kissing numbers
D	פורמליזציה	משפטים שהוכחו ואינם ב-Lean, ערכים קטנים של Schur ו-van der Waerden דרך SAT
P	פיזיקה	Euler blow-up, 3D Ising bootstrap, הבעיה השישית של הילברט

כל הבעיות שתוארו בפרק הזה, חוץ מבעיות ארדש, נמצאות בשכבה A. ההחלטה לא לעבוד עליהן ישירות נשענת על ארבעה טיעונים.

1. ציון העדיפות. כל מטרה מקבלת ציון `ai_tractability` (הסיכוי של צוות קטן עם AI להגיע לתוצאה חדשה בתוך חודשים) וציון `formalization_readiness` (עד כמה Lean ו-mathlib מוכנים). הנוסחה היא

$$\text{priority} = 2 \cdot \text{ai_tractability} + \text{formalization_readiness} + \text{bonus},$$

כשה-bonus הוא 2+ לשכבות C ו-D ו-3− לשכבה A. להשערת רימן יש ציונים (1, 3), ולכן עדיפות 2. לפורמליזציה של בעיות ארדש שנפתרו לאחרונה יש (5, 4), ולכן עדיפות 16. הנוסחה איננה קדושה, אבל היא כופה דיון מפורש.

2. הלקח של בעיות ארדש. ההצלחות של AI ב-2025–2026 הגיעו מ"הזנב הארוך": בעיות שאיש לא ניסה ברצינות, בעיות בנייה עם ציון מספרי, ופורמליזציה של תוצאות ידועות. אין שום עדות שמודלים

קרובים לפריצה בבעיות שבהן עבדו אלפי מתמטיקאים מובילים במשך עשרות שנים.

3. עלות ההזדמנות. כל שעת חישוב על קולץ היא שעה שלא הושקעה בצינור SAT→Lean (מטרה 1), בפורמליזציה של בעיית ארדש (מטרה 2) או בשיפור חסם במספרי van der Waerden (מטרה 3). שלוש המטרות האלה מייצרות תוצרים אמיתיים, בונות תשתית ומלמדות את הצוות.

4. סיכון המוניטין. מעבדה שמכריזה "פתרנו את רימן" ומתבדה, מאבדת את האמון שבלעדיו אין ערך לשום טענה שלה. רמות הטענה (פרק 0) נועדו בדיוק לזה.

אז מה כן עושים עם שכבה A?

שכבה A איננה זבל. יש לה שלושה תפקידים:

- **פורמליזציה של ניסוחים.** לנסח ב-Lean את הטענה המדויקת של כל בעיית מילניום, ואת התוצאות החלקיות הידועות. זה תרגיל מצוין, וזה גם שימושי לקהילה. פרויקט formal-conjectures של Google DeepMind עושה בדיוק את זה, והמעבדה יכולה לתרום לו.
- **תמונת מצב כנה.** לשמור, לכל בעיה, סיכום עדכני ומדויק של מה ידוע, עם מקורות. זה שירות לציבור ולקוראי S27 News, ומקור לתוכן.
- **השראה.** בעיות גדולות מושכות אנשים. הן הסיבה שרבים מאיתנו התאהבו במתמטיקה. אין סיבה להסתיר אותן, רק סיבה להיות כנים לגביהן.

כלל העבודה מ-`docs/goal-bank.md`: "שורות A לא מקבלות compute משמעותי בלי החלטה מפורשת של הצוות."

בקצרה

- רשימת 23 הבעיות של הילברט (1900) עיצבה את המתמטיקה של המאה ה-20, ולימדה שגם "אין אלגוריתם" או "בלתי תלוי באקסיומות" הם פתרונות לגיטימיים.
- מתוך שבע בעיות המילניום (2000), רק השערת פואנקרה נפתרה, בידי פרלמן בשנים 2002–2003, באמצעות זרימת ריצ'י; פרלמן דחה את הפרס.
- רימן, P מול NP, נאוויה–סטוקס, יאנג–מילס, הודג' ו-BSD עדיין פתוחות; לכל אחת יש תמונה אינטואיטיבית פשוטה וקושי טכני עצום.
- גולדבך, ראשוניים תאומים וקולץ פשוטות לניסוח ונבדקו במחשב לטווחים עצומים, אבל בדיקה נומרית היא L1, לא הוכחה. השערת abc היא דוגמה למחלוקת שהתהליך החברתי לא הצליח להכריע.
- המעבדה מסמנת את כל אלה כ-Tier A ומתמקדת בשכבות C ו-D, כי שם הגיעו הצלחות ה-AI האמיתיות, ושם אפשר לייצר תוצרים מאומתים כבר עכשיו.

רעיונות לתוכן

1. **"שבע בעיות, מיליון דולר כל אחת, ותמונה אחת לכל בעיה"** זווית: סדרת סרטונים קצרים (דקה לבעיה), כל אחד בנוי סביב התמונה האינטואיטיבית מהפרק: הנמלה והגומייה, הסודוקו הענק, המערבולת באמבטיה. קהל: הציבור הרחב, רשתות חברתיות.
2. **"האיש שסירב למיליון"** זווית: כתבת פרופיל על פרלמן ועל מה שהסיפור שלו מלמד על בדיקת הוכחות, ואיך Lean יכול לשנות את זה. קהל: קוראי S27 News.
3. **"מ-70 מיליון ל-246"** זווית: הסיפור של ג'אנג ושל Polymath8 כמודל לשיתוף פעולה פתוח, וההקבלה לנחיל הסוכנים של המעבדה. קהל: קהילת המדע והטכנולוגיה, בלוג.
4. **"הוויכוח שאף אחד לא מנצח: abc"** זווית: הסבר מאוזן של המחלוקת בין מוצ'יזוקי לשולצה–שטיקס, וטיעון למה זו בדיוק הבעיה שפורמליזציה נועדה לפתור. קהל: קוראים עם רקע מדעי, כתבת עומק.
5. **"למה אנחנו לא תוקפים את השערת רימן"** זווית: פוסט אסטרטגי שמציג את נוסחת העדיפות ואת השכבות, ומראה את החישוב לרימן מול בעיית ארדש. קהל: מתמטיקאים, ספקנים ומשקיעים פוטנציאליים, לינקדאין.
6. **"מספרי ברד"** זווית: אינטראקטיב באתר שבו קוראים מקלידים מספר ורואים את מסלול קולץ שלו, עם הסבר למה זה לא הוכחה. קהל: תלמידים ומורים.

פרק 3: מכונות והוכחות

הרעיון שמכונה תוכל לעשות מתמטיקה ישן יותר מהמחשב. לייבניץ חלם במאה ה-17 על "חשבון של המחשבה" (calculus ratiocinator), שבו מחלוקות ייפתרו במשפט אחד: "בואו נחשב". אבל ההיסטוריה האמיתית של מכונות והוכחות מתחילה רק באמצע המאה ה-20, והיא מתפצלת לשני ענפים שהתפתחו בנפרד במשך עשרות שנים: מכונות *שמוצאות* הוכחות, ומכונות *שבודקות* הוכחות. הפרק הזה מספר את שני הסיפורים, מה-Logic Theorist של 1956 ועד פורמליזציות מחקריות ב-Lean, ומסביר את הוויכוח הפילוסופי שליווה אותם. המעבדה יושבת בדיוק בנקודת המפגש בין שני הענפים, ולכן ההיסטוריה הזו היא גם ההסבר לבחירות הטכניות שלה.

הענף הראשון: מכונות שמוצאות הוכחות

Logic Theorist :1956

בקיץ 1956 התכנסה בדארטמות' הסדנה שבה נטבע המונח "בינה מלאכותית". אחד הדברים המעטים שהוצגו בה כתוכנה עובדת היה ה-**Logic Theorist** של אלן ניואל, הרברט סיימון וקליף שו[18]. התוכנה הוכיחה 38 מתוך 52 המשפטים הראשונים בפרק 2 של "פרינקיפיה מתמטיקה" של ראסל ווייטהד (ראו פרק 1). לפי הסיפור המוכר, אחת ההוכחות שלה הייתה אלגנטית יותר מזו של המחברים, וסיימון כתב על כך לראסל, שהגיב בהנאה.

ה-Logic Theorist לא היה חזק במיוחד. החשיבות שלו הייתה רעיונית: הוא חיפש הוכחות בעזרת **היוריסטיקות (heuristics)**, כללי אצבע שמכוונים את החיפוש, במקום לנסות את כל האפשרויות. הרעיון הזה, חיפוש מודרך במרחב עצום, הוא אותו רעיון שעומד מאחורי AlphaProof ומאחורי מודלי השפה של היום, עם כלים טובים בהרבה.

שנות ה-60 עד ה-90: מוכיחים אוטומטיים

ב-1965 פרסם אלן רובינסון את שיטת **הרזולוציה (resolution)**, כלל היסק אחד שמספיק כדי להוכיח כל טענה תקפה בלוגיקה מסדר ראשון (אם מחכים מספיק). מכאן התפתחה משפחה של **מוכיחים אוטומטיים (automated theorem provers, ATP)**: Otter, ואחריו E, Prover9, Vampire ואחרים.

ההישג המפורסם ביותר שלהם הגיע ב-1996: התוכנה EQP של ויליאם מקיון הוכיחה את **השערת רובינס**, שאלה באלגברה בוליאנית שהייתה פתוחה מאז שנות ה-30, ושטרסקי ותלמידיו ניסו לפתור בלי

הצלחה. ה"ניו יורק טיימס" דיווח על כך. זו הייתה אחת הפעמים הראשונות שמכונה פתרה בעיה פתוחה שמתמטיקאים התעניינו בה, וההוכחה עצמה הייתה קצרה מספיק כדי שבני אדם יבדקו אותה.

המוכיחים האוטומטיים עדיין חיים. ב-Equational Theories Project של טאו (2024) הם הכריעו מיליוני טענות על מבנים אלגבריים, ובעולם של Lean, כלים בסגנון "hammer" מתרגמים מטרה ל-ATP ומחזירים הוכחה (ראו פרק 8).

הענף השני: מכונות שבודקות הוכחות

Automath :1967

הענף השני התחיל בהולנד. ניקולס חוברט דה-ברויין, מתמטיקאי באיינדהובן, תכנן ב-1967 את **Automath**, שפה שבה אפשר לכתוב מתמטיקה כך שמחשב יבדוק אותה[19]. דה-ברויין גילה בעצמו, ובמקביל לקרי והוארד, את הרעיון של טענות כטיפוסים (פרק 1). ב-1977 תרגם תלמידו, ל. ס. ואן בנטם יוטינג, ל-Automath את כל הספר "יסודות האנליזה" של לנדאו, הפורמליזציה המשמעותית הראשונה של ספר מתמטיקה שלם.

לדה-ברויין אנחנו חייבים גם את **עקרון דה-ברויין (de Bruijn criterion)**: הבודק צריך להיות קטן ופשוט מספיק כדי שאדם יוכל לבדוק אותו בעצמו. כל השאר, האוטומציה, הטקטיקות וממשק המשתמש, יכולים להיות מסובכים ככל שרוצים, כי כל מה שהם מייצרים עובר בסוף דרך הבודק הקטן. זה העיקרון שעומד מאחורי ה"גרעין" (kernel) של Lean, ומאחורי "ליבת האמון" של המעבדה.

Mizar :1973

ב-1973 התחיל אנדז'י טריבולץ בפולין את פרויקט Mizar. **Mizar** בנוי על תורת הקבוצות (גרסה של ZFC עם אקסיומות נוספות), והשפה שלו נועדה להיראות כמו מתמטיקה כתובה. הספרייה שלו, Mizar Mathematical Library, גדלה לאורך עשרות שנים לאחד מאוספי המתמטיקה הפורמלית הגדולים, וכתב העת "Formalized Mathematics" מפרסם את התוצרים שלה מאז 1990.

LCF, HOL, Isabelle ו-Coq

בשנות ה-70 פיתח רובין מילנר באדינבורו את **LCF**, וממנו יצא רעיון מכריע: לכתוב את הבודק כ"טיפוס מופשט" בשפת תכנות (ML, שהומצאה בדיוק לשם כך), כך שהדרך היחידה לייצר "משפט" היא דרך הפונקציות של הגרעין. ממנו התפתחו HOL, HOL Light (של ג'ון הריסון) ו-Isabelle (של לורנס פולסון).

במקביל, בצרפת, פיתחו תיירי קוקאן וז'ראר הואה את **תחשיב הקונסטרוקציות (Calculus of Constructions)**, ועליו נבנה **Coq**, ששמו שונה ב-2025 ל-**Rocq**. מכל המערכות הקלאסיות,

Coq הוא זה שהגיע להישגים המתמטיים המרשימים ביותר לפני עידן Lean, כפי שנראה מיד.

משפט ארבעת הצבעים: ההוכחה שפתחה את הוויכוח

הבעיה

ב-1852 שם לב פרנסיס גאתרי שאפשר לצבוע את מפת מחוזות אנגליה בארבעה צבעים, כך ששני מחוזות שכנים לעולם לא יקבלו אותו צבע. הוא שאל אם זה נכון לכל מפה במישור. השאלה נשמעת פשוטה, אבל היא עמדה פתוחה יותר ממאה שנה, וכפי שראינו בפרק 1, ה"הוכחה" של קמפה מ-1879 החזיקה מעמד 11 שנים לפני שנמצאה בה שגיאה.

1976: אפל והייקן

ב-1976 הודיעו קנת אפל וולפגנג הייקן מאוניברסיטת אילינוי שהוכיחו את המשפט[20]. ההוכחה שלהם עבדה בשני שלבים:

1. **טיעון אנושי** שמראה שכל מפה שהיא "דוגמה נגדית מינימלית" חייבת להכיל אחת מתוך רשימה סופית של כ-1,900 תצורות (configurations).
2. **בדיקה ממוחשבת** שכל אחת מהתצורות האלה "ניתנת לצמצום" (reducible): אי אפשר שהיא תופיע בדוגמה נגדית מינימלית. הבדיקה לקחה כ-1,200 שעות חישוב.

הדואר של אוניברסיטת אילינוי הוסיף לחותמת הדואר את המשפט "Four Colors Suffice". אבל הקהילה המתמטית קיבלה את ההוכחה באי-נוחות. בפעם הראשונה, חלק מהותי מהוכחה היה חישוב שאף אדם לא יכול לבדוק בעצמו.

ב-1996 פרסמו רוברטסון, סנדרס, סימור ותומס הוכחה פשוטה יותר, עם 633 תצורות, אבל עדיין עם חלק ממוחשב.

2005: גונטייה ו-Coq

ב-2005 השלים ז'ורז' גונטייה, ממיקרוסופט ריסרץ' ו-INRIA, עם בנז'מן ורנר, הוכחה מלאה של משפט ארבעת הצבעים ב-Coq[21]. ההבדל עקרוני: לא רק חלק הבדיקה הממוחשב, אלא כל ההוכחה, כולל הטיעון האנושי ונכונות התוכנה שבודקת את התצורות, נבדקה בידי הגרעין של Coq. השאלה "אולי יש באג בתוכנה?" קיבלה תשובה: גם התוכנה הוכחה.

גונטייה המשיך משם להוביל את הפורמליזציה של **משפט פייט–תומפסון (משפט הסדר האי-זוגי)**, הוכחה של 255 עמודים בתורת החבורות, שהושלמה ב-Coq ב-2012 אחרי שש שנות עבודה. שני הפרויקטים הוכיחו שעוזרי הוכחות יכולים להתמודד עם מתמטיקה רצינית.

השערת קפלר ו-Flyspeck

הבעיה

ב-1611 שיער יוהנס קפלר שהדרך הצפופה ביותר לארוז כדורים זהים במרחב היא הדרך שבה מוכרי פירות בשוק עורמים תפוזים: שכבות משושות, שכל אחת יושבת בגומות של זו שמתחתיה. הצפיפות של סידור כזה היא $\pi/\sqrt{18} \approx 74.05\%$. השאלה נכללה כחלק מהבעיה ה-18 של הילברט.

1998: הוכחה ש"99% בטוחים" בה

תומס הייל, עם תלמידו סמואל פרגוסון, הציג ב-1998 הוכחה[22]. גם היא בנתה על צמצום הבעיה לחישוב: כ-5,000 תצורות מקומיות, ופתרון של מאות אלפי בעיות תכנון לינארי. ההוכחה הוגשה ל-Annals of Mathematics, שמינה צוות של 12 שופטים. אחרי ארבע שנים של עבודה, הודיעו השופטים שהם "99% בטוחים" בנכונות ההוכחה, אבל אינם יכולים לאמת את כל החישובים. המאמר פורסם ב-2005, עם הערה חריגה של העורכים.

2014: Flyspeck

הייל לא הסתפק ב-99%. ב-2003 הוא השיק את פרויקט **Flyspeck** (מעין ראשי תיבות של "Formal Proof of the Kepler conjecture"), שמטרתו להוכיח את ההשערה באופן פורמלי מלא, ב-HOL Light ובחלקו ב-Isabelle. ההערכה הראשונית שלו הייתה שהפרויקט ייקח כעשרים שנה. בפועל, עם צוות בין-לאומי, הוא הושלם באוגוסט 2014, והמאמר המסכם פורסם ב-2017[23].

Flyspeck הוא המודל המובהק ל"פורמליזציה כפתרון למחלוקת": כשהתהליך החברתי (12 שופטים, ארבע שנים) נתקע, התהליך הפורמלי סגר את השאלה.

הסיפור ממשיך: ב-2016 פתרה מרינה ויאזובסקה את בעיית אריזת הכדורים במימד 8 (ובשיתוף עם אחרים, במימד 24), ב-2022 היא קיבלה על כך מדליית פילדס, ולפי `docs/goal-bank.md` , ב-2026 הושלמה הפורמליזציה של תוצאת מימד 8 ב-Lean.

עידן ה-SAT: הוכחות בגודל של טרה-בייטים

בעשור השני של המאה ה-21 צץ סוג חדש של הוכחה ממוחשבת. **פותרי SAT** מכריעים אם לנוסחה בוליאנית יש השמה מספקת. כשהתשובה היא "כן", קל לבדוק אותה: מציגים את ההשמה. כשהתשובה היא "לא" (UNSAT), צריך להוכיח שאין שום השמה, ופותרים מודרניים מפיקים לשם כך **תעודה (certificate)** בפורמט כמו **DRAT** או **LRAT**, שבודק קטן ועצמאי יכול לוודא. כמה מהבודקים האלה, כמו cake_lpr, הוכחו נכונים בעצמם בעוזר הוכחות.

2016: שלשות פיתגוריות בוליאניות

השאלה, שהציג רון גרהם: האם אפשר לצבוע כל מספר טבעי באדום או בכחול, כך שאין שלשה פיתגורית $a^2 + b^2 = c^2$ שכל שלושת איבריה באותו צבע? ב-2016 הראו מריין היולה, אוליבר קולמן וויקטור מרק שהתשובה היא לא: אפשר לצבוע כך את המספרים עד 7,824, אבל לא עד $7{,}825^{24}$. תעודת ה-DRAT שהפיקו הייתה בגודל של כ-200 טרה-בייט, באותה תקופה כנראה ההוכחה הגדולה ביותר שנכתבה. גרהם שילם את הפרס של 100 דולר.

ב-2017 קבע היולה גם את מספר שור החמישי, $S(5) = 160$, עם תעודה בגודל כשני פטה-בייט.

2020: השערת קלר במימד 7

השערת קלר (1930) עוסקת בריצוף של המרחב בקוביות זהות: האם בכל ריצוף כזה יש שתי קוביות שחולקות פאה שלמה? ההשערה הופרכה במימדים 10 ומעלה ב-1992 (לגריאס ושור) ובמימד 8 ב-2002 (מקי), ונשאר רק מימד 7. ב-2020 פתרו אותו ברקנסיק, היולה, מקי ונרבאס בעזרת SAT, עם תעודה שנבדקה בבודק מאומת: במימד 7 ההשערה נכונה[25].

2024: המשושה הריק

ב-2024 קבעו היולה ושויכר בעזרת SAT שכל קבוצה של 30 נקודות במישור, בלי שלוש על ישר אחד, מכילה "משושה ריק" (משושה קמור בלי נקודות בתוכו), ושיש 29 נקודות שאינן מכילות. באותה שנה אימתו סוברקסו ושותפיו את ה-encoding ואת החיבור לתעודה ב-Lean. הפרויקט הזה הוא בדיוק התבנית של מטרה 1 של המעבדה: SAT מוצא, Lean מוודא שה-encoding נכון ושהתעודה תקפה.

מה SAT מלמד אותנו

הוכחות SAT הן גם הצלחה וגם אתגר. הן נכונות (התעודה נבדקה), אבל הן לא מסבירות כלום. אף אחד לא יודע *למה* 7,825 הוא המספר הקסום. נחזור לנקודה הזו בוויכוח הפילוסופי.

BB(5): הבונה העסוק ואי-הכריעות

בפרק 1 ראינו שבעיית העצירה של טיורינג אינה כריעה. ב-1962 הגדיר טיבור ראדו את פונקציית **הבונה העסוק (Busy Beaver)**: $BB(n)$ הוא המספר המרבי של צעדים שמכונת טיורינג עם n מצבים (וסרט בינארי) יכולה לבצע לפני שהיא עוצרת, כשמתחילים מסרט ריק. הפונקציה גדלה מהר יותר מכל פונקציה ניתנת לחישוב, ובשל כך אי אפשר לחשב אותה באופן כללי.

הערכים הקטנים ידועים: $BB(1) = 1$, $BB(2) = 6$, $BB(3) = 21$, $BB(4) = 107$. את $BB(5)$ ניסו לקבוע מאז שנות ה-80. המועמדת המובילה, מכונה שמצאו הייקו מרקסן ויורגן בונטרוק ב-1989, עוצרת אחרי

47,176,870 צעדים. כדי להוכיח ש-$BB(5) = 47,176,870$, צריך להראות שכל אחת מעשרות מיליוני המכונות האחרות עם 5 מצבים או עוצרת מוקדם יותר, או לא עוצרת לעולם.

ביולי 2024 הודיע פרויקט **bbchallenge**, שיתוף פעולה מקוון של חובבים וחוקרים, שההוכחה הושלמה, ושהיא בדוקה ב-Coq[26]. רוב המכונות סווגו בעזרת "מחליטים" (deciders) אוטומטיים, ומעט מכונות עקשניות במיוחד ("cryptids") טופלו בהוכחות פרטניות. הנקודה החשובה: כמעט כל המשתתפים לא היו מתמטיקאים מקצועיים, והאמון בתוצאה לא נשען על המוניטין שלהם, אלא על Coq. זה בדיוק המודל שהמעבדה מאמצת.

Lean ו-mathlib

מ-2013 עד Lean 4

את Lean יצר לאונרדו דה מורה במיקרוסופט ריסרץ' ב-2013. דה מורה הגיע מעולם פותרי ה-SMT (הוא אחד מיוצרי Z3), והמטרה הייתה לבנות עוזר הוכחות שהוא גם שפת תכנות יעילה. Lean 3 הפך לפלטפורמה של קהילה מתמטית צומחת, ו-**Lean 4**, שנכתב מחדש כמעט מאפס, יצא ב-2021. ב-2023 הוקם **Lean FRO**, ארגון ללא כוונת רווח שמתחזק את השפה.

mathlib

mathlib התחילה ב-2017 כספרייה קהילתית של מתמטיקה פורמלית ב-Lean. ההבדל בינה לבין ספריות קודמות הוא תרבותי לא פחות מטכני: ספרייה *אחת*, מאוחדת, עם סטנדרטים מחמירים לסגנון ולהכללה, שבה כל משפט נכנס למקום אחד ומשתמש בהגדרות משותפות. המעבר ל-Lean 4 ("הפורט") הושלם ביולי 2023. היום mathlib כוללת מאות אלפי משפטים, ומאות תורמים (את המספרים המדויקים כדאי לבדוק בדף הסטטיסטיקה הרשמי של mathlib, כי הם משתנים כל שבוע).

היום Lean הוא כמעט סטנדרט דה-פקטו לפורמליזציה של מתמטיקה מחקרית, וגם ל-AI: AlphaProof, DeepSeek-Prover, Aristotle ו-Seed-Prover כולם עובדים ב-Lean. פרק 5 מוקדש לעבודה מעשית איתו.

2020–2022: Liquid Tensor Experiment

בדצמבר 2020 פרסם פטר שולצה, זוכה מדליית פילדס, אתגר בבלוג של Kevin Buzzard[27]. שולצה ודסטין קלאוזן פיתחו תורה חדשה, **מתמטיקה מעובה (condensed mathematics)**, ומשפט מרכזי בה הוא מה ששולצה כינה "אולי המשפט החשוב ביותר שהוכחתי". ההוכחה הייתה מסובכת מספיק כדי ששולצה עצמו לא היה בטוח בה לחלוטין, והוא שאל: האם עוזר הוכחות יכול לבדוק אותה?

קהילת Lean, בהובלת יוהאן קומלין, קיבלה את האתגר. ביוני 2021 הושלמה הלמה המרכזית, ושולצה כתב שהתהליך שיפר את ההבנה שלו את ההוכחה. ביולי 2022 הושלם כל הפרויקט. בפעם הראשונה, מתמטיקה מחקרית עכשווית, ממש מהחזית, נבדקה פורמלית בתוך שנה וחצי.

2023: PFR בשלושה שבועות

בנובמבר 2023 פרסמו טימותי גאוורס, בן גרין, פרדי מאנרס וטרנס טאו הוכחה של **השערת פרימן–רוזה הפולינומית (PFR)**, בעיה מרכזית בקומבינטוריקה אדיטיבית[28]. טאו פתח מיד פרויקט פורמליזציה ב-Lean, עם **blueprint**: מסמך שמפרק את ההוכחה לעשרות למות קטנות, עם גרף תלויות שמראה מה תלוי במה. מתנדבים לקחו למות בודדות, והפורמליזציה הושלמה בתוך כשלושה שבועות.

PFR הראה שהפער בין "הוכחנו" ל"פורמלנו" יכול להצטמצם לשבועות. ה-blueprint הפך מאז לשיטת העבודה הסטנדרטית בפורמליזציות גדולות, ובמעבדה הוא שלב חובה (ראו `docs/architecture.md` ו-`docs/ai-math-lessons.md`).

2024–2026: הסוכנים נכנסים

מכאן ההיסטוריה ממשיכה בפרק 4: Strong PNT (2024), Equational Theories Project בידי סוכן אוטו-פורמליזציה (2025), ופורמליזציה של FLT בידי סוכני Claude (ספטמבר 2026). המשותף לכולם: בני אדם מספקים את המבנה (blueprint, ניסוח), מכונות ממלאות את הפרטים, והגרעין של Lean מכריע.

הוויכוח הפילוסופי: האם זו הוכחה?

ההוכחה של אפל והייקן עוררה ויכוח שעדיין חי. אפשר לסדר אותו סביב ארבע שאלות.

1. האם הוכחה צריכה להיות ניתנת לבדיקה בידי אדם?

ב-1979 פרסם הפילוסוף תומס טימוצ'קו מאמר בשם "The Four-Color Problem and Its Philosophical Significance"[29]. הטענה שלו: הוכחה מתמטית הייתה תמיד אובייקט שניתן ל"סקירה" (surveyable), שאדם יכול לעבור עליו בעיניו ולהשתכנע. ההוכחה של אפל והייקן איננה כזו. לכן, טען טימוצ'קו, קבלתה משנה את מושג ההוכחה עצמו: היא מכניסה למתמטיקה רכיב אמפירי, כמו ניסוי. אנחנו סומכים על המחשב כפי שפיזיקאי סומך על מכשיר מדידה.

התשובה המקובלת היום מבחינה בין שתי שאלות: האם ההוכחה *נכונה*, והאם היא *מובנת*. לשאלה הראשונה, מכונה יכולה לענות טוב יותר מאדם. ההוכחה של גונטייה ב-Coq אמינה יותר מכל הוכחה בכתב יד של משפט באותו גודל. לשאלה השנייה, מכונה לא עונה, והיא אולי לא צריכה.

2. האם אפשר לסמוך על התוכנה?

ההתנגדות: אולי יש באג בתוכנה, בקומפיילר, במעבד. התשובה ההנדסית היא שכבות: עקרון דה-ברויין (בודק קטן), בודקים עצמאיים מרובים (ל-Lean יש כמה מימושים חיצוניים של בודק הגרעין), ובודקי תעודות SAT שהוכחו נכונים בעצמם. אמינות מוחלטת לא קיימת, גם לא בהוכחות אנושיות. מה שאפשר להשיג הוא אמינות גבוהה מאוד, ניתנת לכימות ולשיפור.

3. האם הוכחה שאינה מסבירה היא הוכחה?

הוכחת ה-SAT לשלשות הפיתגוריות מראה ש-7,825 הוא הגבול, אבל לא *למה*. מתמטיקאים רבים חשים שמשהו חסר. פול ארדש דיבר על "הספר" (The Book), שבו אלוהים שומר את ההוכחה האלגנטית ביותר לכל משפט. הוכחת SAT בגודל 200 טרה-בייט רחוקה מאוד מהספר.

אבל ההיסטוריה מלמדת שהוכחה "מכוערת" היא לעתים קרובות הצעד הראשון. ההוכחה של ארבעת הצבעים עדיין ממוחשבת, אבל הצטמצמה. ההוכחה של קפלר הובילה לשיטות חדשות באופטימיזציה. ואם יודעים *שמשהו* נכון, קל יותר לחפש את *הלמה*. פרק 9 מתאר את הדפוס "המכונה מחפשת, האדם מסביר" כאחד מעשרת דפוסי היצירתיות.

4. מה זה אומר כשה"מוכיח" הוא AI?

השאלה החדשה, של 2024–2026, משנה את הוויכוח. כשהמחשב רק *חישב* (ארבעת הצבעים, SAT), האדם תכנן את האסטרטגיה. כשמודל שפה *כותב את ההוכחה*, מי הוא המוכיח? יש כאן שתי שאלות נפרדות:

- **נכונות**: התשובה זהה לזו של ארבעת הצבעים. אם ההוכחה עברה את הגרעין של Lean והניסוח נבדק, היא נכונה, לא משנה מי כתב אותה.
- **ייחוס וערך**: האם התוצאה חדשה? האם היא חשובה? מי ראוי לקרדיט? השאלות האלה חברתיות לגמרי, ופרק 12 מוקדש להן.

עמדת המעבדה, כפי שהיא מנוסחת ב-`docs/architecture.md` , נשענת על כל ארבע התשובות: נכונות נקבעת בגרעין (L3), הבנה וערך נקבעים בקהילה (L4), והמעבדה מדווחת בכנות מה עשה כל אחד, אדם או מכונה.

מה המעבדה לוקחת מההיסטוריה הזו

הפרק הזה הוא מפת הדרכים של בחירות טכניות במעבדה:

לקח היסטורי	איפה זה במעבדה
עקרון דה-ברויין: בודק קטן ואמין	"ליבת האמון": הגרעין של Lean ובודקי DRAT/LRAT (`docs/architecture.md`)
ארבעת הצבעים ← Coq: גם התוכנה צריכה הוכחה	בודקים שה-encoding של SAT נכון, ב-Lean (`docs/goal-bank.md` ,מטרה 1)
Flyspeck: פורמליזציה מכריעה מחלוקות	שכבה D בבנק המטרות
SAT: תעודה בודקת, גם אם לא מסבירה	search-agent מפיק תמיד תעודה; בלי תעודה זה L1
BB(5): קהילה פתוחה + עוזר הוכחות = אמון בלי מוניטין	מודל העבודה הפתוח והציבורי של המעבדה
PFR: blueprint מאפשר עבודה מבוזרת	lean-agent כותב blueprint לפני קוד
הוויכוח הפילוסופי: נכונות ≠ הבנה	הפרדה בין L3 (מכונה) ל-L4 (קהילה)

בקצרה

- ההיסטוריה של מכונות והוכחות מתפצלת לשני ענפים: מכונות *שמוצאות* הוכחות (Logic Theorist ב-1956, מוכיחים אוטומטיים, השערת רובינס ב-1996) ומכונות *שבודקות* הוכחות (Automath, Mizar, HOL, Coq, Lean). המעבדה יושבת במפגש ביניהם.
- משפט ארבעת הצבעים (1976) היה ההוכחה הממוחשבת הגדולה הראשונה, וב-2005 הוא הוכח במלואו ב-Coq; השערת קפלר (1998) הוכרעה סופית בפורמליזציה של Flyspeck (2014).
- פותרי SAT עם תעודות DRAT/LRAT פתרו בעיות כמו השלשות הפיתגוריות הבוליאניות (2016), קלר במימד 7 (2020) והמשושה הריק (2024), ו-BB(5) נקבע ב-2024 בהוכחה ב-Coq של קהילה פתוחה.
- Lean ו-mathlib הפכו לסטנדרט: Liquid Tensor Experiment (2020–2022) ו-PFR (2023) הוכיחו שאפשר לפרמל מתמטיקה מחקרית עכשווית, ובמהירות.
- הוויכוח הפילוסופי מבחין בין נכונות, שמכונה מבטיחה טוב יותר מאדם, לבין הבנה וערך, שנשארים חברתיים; רמות L3 ו-L4 במעבדה משקפות את ההבחנה הזו.

רעיונות לתוכן

1. **"ההוכחה ששקלה 200 טרה-בייט"** זווית: סיפור השלשות הפיתגוריות הבוליאניות, עם המחשה של צביעת המספרים עד 7,824 ושאלת ה"למה" שנשארה פתוחה. קהל: הציבור הרחב, סרטון

קצר.

2. **"99% בטוחים: איך השערת קפלר חיכתה 16 שנה לאישור"** זווית: מהתפוזים בשוק, דרך 12 השופטים, ועד Flyspeck, כסיפור על גבולות הבדיקה האנושית. קהל: קוראי S27 News, כתבת עומק.
3. **"חובבים, Coq ומספר שגדל מהר מכל חישוב"** זווית: BB(5) ו-bbchallenge כדוגמה לכך שקהילה פתוחה יכולה לייצר מתמטיקה אמינה. קהל: קהילת המפתחים והקוד הפתוח.
4. **"ציר הזמן: 70 שנה של מכונות שמוכיחות"** זווית: אינפוגרפיקה מ-1956 עד 2026, עם שני הענפים (מוצאים/בודקים) שמתאחדים. קהל: רשתות חברתיות, עמוד באתר.
5. **"האם הוכחה שאף אחד לא יכול לקרוא היא הוכחה?"** זווית: דיון פילוסופי נגיש בעקבות טימוצ'קו, עם עמדת המעבדה. קהל: פודקאסט או פאנל, קהל עם עניין בפילוסופיה של המדע.
6. **"שלושה שבועות: מה PFR מלמד על עבודה עם סוכנים"** זווית: ה-blueprint כממשק בין אדם למכונה, וההקבלה לנחיל של המעבדה. קהל: מהנדסי AI.

פרק 4: AI ומתמטיקה 2019–2026

בשבע שנים בלבד השתנתה השאלה "האם מחשב יכול לעשות מתמטיקה?" מעבר לכל היכר. ב-2019 התשובה הכנה הייתה "רק בתוך מגרש משחקים קטן ומוגדר היטב". ב-2026 מודלי שפה מוכיחים למות מחקריות, מוצאים מאמרים נשכחים, משפרים חסמים בבעיות בנייה, וצי של סוכנים השלים פורמליזציה של המשפט האחרון של פרמה. הפרק הזה מספר את הסיפור הזה לפי הסדר, אבל הוא לא נכתב כדי להתפעל. הוא נכתב כדי לחלץ ממנו לקחים. כמעט כל הצלחה אמיתית בתקופה הזו נשענה על **מוודא (verifier)**: עוזר הוכחות כמו Lean, חישוב שאפשר לשחזר, או פונקציית ניקוד קשיחה. וכמעט כל שערוריית הייפ נבעה מהיעדרו של מוודא כזה. על התובנה הזו בנויה כל ESH27 Lab, ולכן חשוב להבין מאיפה היא באה.

הפרק מבוסס על מסמך הרקע של המעבדה, `docs/ai-math-lessons.md`, ומרחיב אותו לסיפור. המוסכמה נשמרת: מספרים ותאריכים נבדקו מול מקורות ראשוניים ככל האפשר, כל דבר שלא הצלחנו לאמת מסומן [לא אומת], וטענות של חברות על המוצרים שלהן נחשבות "טענת יצרן" עד שגורם חיצוני בודק אותן.

המצב בנקודת ההתחלה

כדי להבין את גודל השינוי צריך לזכור איך נראה התחום לפני 2019. היו שני עולמות כמעט נפרדים. בעולם הראשון, **עוזרי הוכחות (proof assistants)** כמו Coq (היום Rocq), Isabelle, HOL Light ו-Lean שימשו קהילה קטנה ומסורה לבדיקת הוכחות קיימות: משפט ארבעת הצבעים, משפט Feit–Thompson, השערת קפלר. העבודה הייתה ידנית ואיטית. פורמליזציה של משפט אחד יכלה לקחת שנים של אנשי מקצוע.

בעולם השני, **למידת מכונה (machine learning)** כמעט לא נגעה במתמטיקה מחקרית. היו ניסויים בבחירת למות (premise selection) ובהכוונת מוכיחים אוטומטיים קלאסיים, אבל איש לא ציפה שרשת נוירונים תכתוב הוכחה שמתמטיקאי ירצה לקרוא.

החיבור בין שני העולמות, כלומר מכונה שמייצרת ומכונה שבודקת, הוא הסיפור של התקופה.

מערכה ראשונה: Lean מתבגר (2020–2023)

ניסוי הטנזור הנוזלי

בדצמבר 2020 הציב Peter Scholze, אחד המתמטיקאים הבולטים בדורו, אתגר פומבי: לפרמל משפט מרכזי מתוך התחום החדש שפיתח עם Dustin Clausen, condensed mathematics. Scholze הודה שהוא עצמו לא היה בטוח לגמרי בהוכחה, שהייתה ארוכה וטכנית במיוחד. קהילת Lean, בהובלת Johan Commelin, קיבלה את האתגר. הלמה המרכזית פורמלה ביוני 2021, והפרויקט, שנודע בשם **Liquid Tensor Experiment**, הושלם ביולי 2022[30].

המשמעות הייתה כפולה. ראשית, Lean הוכיח שהוא מתאים למתמטיקה מחקרית עכשווית ולא רק לתרגילים או למשפטים קלאסיים. שנית, Scholze כתב שתהליך הפורמליזציה שיפר את הבנתו את ההוכחה. המחשב לא רק בדק. הוא אילץ את בני האדם לחשוב בבהירות.

mathlib עוברת ל-Lean 4

במקביל, ספריית המתמטיקה המשותפת של Lean, **mathlib**, גדלה במהירות. ביולי 2023 הושלם הפורט שלה מ-Lean 3 ל-Lean 4, מבצע הנדסי עצום. מאז mathlib היא ספרייה מאוחדת ומתוחזקת היטב, עם מאות אלפי משפטים (את המספר המדויק יש לבדוק בדף הסטטיסטיקה של הפרויקט[31]). כל מה שנבנה במעבדה שלנו, בפרק 5 ובפרק 6, נשען על הספרייה הזו.

PFR: פורמליזציה בתוך שבועות

בנובמבר 2023 פרסמו Tim Gowers, Ben Green, Freddie Manners ו-Terence Tao הוכחה של השערת Freiman–Ruzsa הפולינומית (PFR), תוצאה חשובה בקומבינטוריקה אדיטיבית. תוך כשלושה שבועות, בדצמבר 2023, ההוכחה כבר פורמלה במלואה ב-Lean[32]. הסוד היה **blueprint**: מסמך שמפרק את ההוכחה לגרף של למות ותלויות, כך שעשרות מתנדבים יכלו לעבוד במקביל, כל אחד על צומת אחר. הלקח שהמעבדה אימצה: השילוב של blueprint ושיתוף פעולה פתוח הוא שיטת העבודה המנצחת.

מערכה שנייה: המוכיחים הנוירוניים (2020–2025)

הצעדים הראשונים

בספטמבר 2020 פרסמו Stanislas Polu ו-Ilya Sutskever מ-OpenAI את **GPT-f**, מודל השפה הראשון שתרם הוכחות שהתקבלו לספרייה פורמלית (Metamath)[33]. ב-2021 נוצר **miniF2F**, אוסף של 488 בעיות אולימפיאדה בניסוח פורמלי ב-Lean ובמערכות נוספות, שהפך לבנצ'מרק המרכזי של התחום[34]. ב-2022 הציגה Meta את **HTPS** (Lample ואחרים), חיפוש בעץ הוכחה בהשראת AlphaZero יחד עם אימון מקוון[35].

ב-2023 הגיע רגע של דמוקרטיזציה: **LeanDojo** ו-ReProver (Yang ואחרים) סיפקו כלים פתוחים, מאגר נתונים שחולץ מ-mathlib ומנגנון לשליפת למות רלוונטיות (premise retrieval)[36]. פתאום גם מעבדה קטנה יכלה להשתתף. זה לקח חשוב עבורנו: תשתית פתוחה היא מה שמאפשר לצוות קטן לעבוד בחזית.

מרוץ המודלים הפתוחים

בין 2024 ל-2025 הואץ הקצב. **DeepSeek-Prover** בגרסאות V1 (מאי 2024), V1.5 (אוגוסט 2024) ו-V2 (אפריל 2025) השתמש בנתונים סינתטיים שנוצרו באוטו-פורמליזציה ובלמידת חיזוק (RL) עם משוב מ-Lean. V2, מודל של 671 מיליארד פרמטרים, הגיע ל-88.9% ב-miniF2F ול-49 בעיות ב-**PutnamBench**, בנצ'מרק של 600+ בעיות מתחרות Putnam[37][38]. **Goedel-Prover** (פברואר ואוגוסט 2025) הראה שמודל פתוח קטן יחסית, שאומן היטב בשיטת expert iteration, מתחרה במודלים ענקיים[39]. **Kimina-Prover** של Moonshot ו-Numina (אפריל 2025) אימץ גישת "whole-proof" בסגנון של מודלי reasoning[40]. **Seed-Prover** של ByteDance (יולי 2025) פתר 78.1% מבעיות IMO היסטוריות שפורמלו ועבר את רף ה-50% ב-PutnamBench[41].

כאן צריך להיזהר. miniF2F **כבר רווי**: Seed-Prover עבר 99% ב-2025, ובעיותיו דלפו מזמן לנתוני האימון של מודלים. ב-PutnamBench ההצלחות היו בודדות בהתחלה, ובסוף 2025 דווח על יותר מ-50% ([לא אומת] המספר העדכני). ציון בבנצ'מרק ישן אומר מעט מאוד על יכולת לפתור בעיה חדשה.

האולימפיאדה: 2024 ו-2025

האולימפיאדה הבינלאומית במתמטיקה (IMO) הפכה לאבן בוחן ציבורית. בינואר 2024 פרסמה DeepMind ב-Nature את **AlphaGeometry**: מנוע סימבולי לגאומטריה, יחד עם מודל שפה שמציע בניות עזר, שאומן על נתונים סינתטיים בלבד[42].

ביולי 2024, ב-IMO 2024, השיגו **AlphaProof** ו-**AlphaGeometry 2** ציון של 28 מתוך 42, ברמת מדליית כסף: ארבע בעיות מתוך שש, כולל הקשה ביותר, P6[43]. אבל הפרטים חשובים. חלק מהבעיות דרשו **ימים** של חישוב, והטענות תורגמו ל-Lean ביד אדם. המאמר המלא ב-Nature פורסם רק ב-12 בנובמבר 2025, ותיאר אוטו-פורמליזציה של כ-80 מיליון טענות ולמידת חיזוק בזמן הבדיקה (Test-Time RL)[44].

ב-IMO 2025, ביולי 2025, התמונה התפצלה לשני מסלולים. במסלול הפורמלי, **Aristotle** של Harmonic הפיק פתרונות ב-Lean לחמש מתוך שש בעיות, מה שהחברה כינתה "gold-medal-equivalent"[45], ו-Seed-Prover קיבל ציון רשמי של 30 נקודות (כסף) עם פתרונות פורמליים. בשני

המקרים הטענות עצמן פורמלו בידי בני אדם. במסלול הטבעי, **Gemini Deep Think** ומודל ניסיוני של OpenAI הגיעו ל-35 מתוך 42 **בשפה טבעית**, ברמת זהב[46]. ההוכחות של Deep Think אושרו על ידי מארגני ה-IMO, ושל OpenAI נבדקו בידי מדליסטים לשעבר.

ההבדל בין המסלולים מכריע. הוכחה פורמלית נבדקת על ידי תוכנה קטנה ואמינה. הוכחה בשפה טבעית נבדקת על ידי בני אדם, שגם הם טועים, ושאינם זמינים בהיקף של מיליוני הוכחות.

הצנרת של 2026

המגמה המרכזית של 2025–2026 היא שמודלי reasoning כלליים (סדרות GPT-5.x, Gemini Deep Think, Claude) מתקרבים למוכיחים הייעודיים. הצנרת שהפכה לסטנדרט: **מודל כללי כותב סקיצה בשפה טבעית, ומפרמל ייעודי (כמו Aristotle) סוגר אותה ב-Lean.** מאז 2026 Aristotle משמש בדיוק כך, כ"מפרמל" של הוכחות שכתבו מודלי שפה, למשל בבעיות ארדש.

מערכה שלישית: חיפוש וגילוי (2025–2021)

לצד ההוכחה התפתח קו מקביל: שימוש ב-AI כדי **למצוא** דברים. דוגמאות נגדיות, בניות, השערות.

ב-2021 הראה Adam Zsolt Wagner במאמר "Constructions in combinatorics via neural networks" שלמידת חיזוק פשוטה מאוד (שיטת cross-entropy) מוצאת דוגמאות נגדיות להשערות בתורת הגרפים[47]. הלקח: שיטה זולה ופשוטה מצליחה כשיש ניקוד ברור.

באותה שנה פורסמו שתי עבודות ב-**Ramanujan Machine** (Raayoni ואחרים, Nature. פברואר 2021) ייצרה השערות על שברים משולבים לקבועים מתמטיים[48]. רוב הפלט היה השערות ולא הוכחות: ערך אמיתי, אבל מוגבל. בדצמבר 2021 פרסמו **Davies ואחרים** (DeepMind, יחד עם Marc Lackenby, András Juhász ו-Geordie Williamson) עבודה שבה למידת מכונה הצביעה על קשרים לא צפויים: בין ה-signature של קשר (knot) לבין ה-slope שלו, ובפולינומי Kazhdan–Lusztig[49]. כאן ה-ML שימש כ"מיקרוסקופ" שמכוון את האינטואיציה, וההוכחה נשארה אנושית.

בדצמבר 2023 הגיע **FunSearch** של DeepMind[50]. הרעיון: מודל שפה מציע **תוכניות** קטנות, מעריך אוטומטי מריץ אותן ונותן ציון, והתוכניות הטובות עוברות "אבולוציה". FunSearch מצא cap sets גדולים יותר (במימד 8: קבוצה בגודל 512) ו-admissible sets שהובילו לחסם אסימפטוטי טוב יותר. החידוש העמוק: **התוצר הוא תוכנה קריאה ולא רק מספר**, ולכן אפשר להבין אותו ולהכליל ממנו.

בנובמבר 2024 הציגו Wagner, Jordan Ellenberg, François Charton ו-Williamson את **PatternBoost**, שילוב של transformer וחיפוש מקומי בבעיות קומבינטוריות[51]. חיפוש גלובלי וחיפוש מקומי משלימים זה את זה.

ב-14 במאי 2025 הכריזה DeepMind על **AlphaEvolve**, מערכת קידוד אבולוציונית מבוססת Gemini[52]. בכ-50 בעיות פתוחות היא שיחזרה את התוצאה הטובה הידועה בכ-75% מהמקרים ושיפרה אותה בכ-20%. מספר הנשיקה (kissing number) במימד 11 עלה מ-592 ל-593, ונמצאה דרך לכפול מטריצות מרוכבות 4×4 ב-48 כפלים, במקום 49 בשיטת Strassen. בנובמבר 2025 פרסמו Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao ו-Wagner מחקר בקנה מידה גדול: AlphaEvolve על 67 בעיות באנליזה, קומבינטוריקה וגאומטריה[53]. מה שהופך את המאמר הזה למופת הוא הכנות. הם מתעדים הצלחות, כישלונות, וגם "רמאויות" של המערכת: מקרים שבהם היא ניצלה באגים במעריך במקום לפתור את הבעיה.

הלקח, שחוזר בכל הקו הזה: **חיפוש אבולוציוני עם מוודא מצליח בבעיות בנייה עם ניקוד, אבל השיפורים בדרך כלל שוליים, והמעריך חייב להיות קשיח ובלתי ניתן לעקיפה.**

מערכה רביעית: פורמליזציה בקנה מידה (2024–2026)

פרויקט התיאוריות המשוואתיות

בספטמבר–אוקטובר 2024 השיק Terence Tao את **Equational Theories Project**[54]. השאלה הייתה פשוטה לניסוח ועצומה בהיקפה: בהינתן 4,694 חוקים אלגבריים על מגמות (magmas), אילו מהם גוררים אילו? מדובר בכ-22 מיליון טענות גרירה. כולן הוכרעו, בשילוב של מוכיחים אוטומטיים קלאסיים (Vampire ודומיו), חיפוש מודלים סופיים, עבודה אנושית, ו-Lean כ"שופט סופי". המסקנה: מתמטיקה בהיקף רחב ובמיקור המונים אפשרית רק עם בדיקה פורמלית. אף אדם לא יכול לסמוך על 22 מיליון טענות שנבדקו ביד.

Strong PNT ובעיית הטענה השגויה

בספטמבר 2025 השלים **Gauss**, סוכן אוטו-פורמליזציה של חברת Math Inc., את האתגר של Tao ו-Alex Kontorovich לפרמל את משפט המספרים הראשוניים בגרסה החזקה: כ-25 אלף שורות בכשלושה שבועות[55]. בני אדם סיפקו את ה-blueprint. אבל בדיון ב-Zulip של קהילת Lean דווח שהסוכן לפעמים "עקף" למות שפורמלו בצורה שגויה. כלומר, הוא הוכיח בהצלחה טענות שלא אמרו את מה שהיו אמורות לומר. זה הלקח החשוב ביותר של הפרק, ונחזור אליו בפרק 5 ובפרק 6: **צריך לבדוק את הטענות (statements), לא רק את ההוכחות.**

המשפט האחרון של פרמה

באוקטובר 2024 קיבל Kevin Buzzard מ-Imperial College מענק EPSRC לחמש שנים כדי לפרמל את המשפט האחרון של פרמה (FLT), או ליתר דיוק לצמצם אותו לתוצאות שהיו ידועות בשנות

ה-80[56]. הפרויקט תוכנן כמאמץ קהילתי ארוך.

ב-4 בספטמבר 2026 דיווחה Anthropic על **הוכחה מלאה ובדוקה במחשב של FLT ב-Lean**: כ-13 מיליון שורות, כ-30 אלף משפטי ביניים, כ-11 ימים של עבודת סוכני Claude, ושימוש רק בשלוש האקסיומות הסטנדרטיות[57]. הטענה הסופית הושוותה ל-statement שכבר היה קיים ב-mathlib, כלומר לניסוח שבני אדם כתבו ובדקו מראש, וזה בדיוק מה שמנטרל את בעיית "הטענה השגויה". Buzzard בדק את התוצאה, ולפי הדיווחים הגדיר אותה "תוצאה גדולה לאוטו-פורמליזציה" שמתמטית "לא מלמדת אותנו כמעט דבר חדש". את פוסט הבלוג המקורי שלו לא הצלחנו לקרוא ישירות, והציטוטים נלקחו ממקורות משניים: **[לא אומת מילולית]**[58].

שני הצדדים של המשפט של Buzzard נכונים, והם מסכמים את המצב ב-2026. מצד אחד, מכונות יכולות היום לבדוק מתמטיקה בקנה מידה שאיש לא דמיין. מצד שני, בדיקה אינה גילוי.

מסמכי המעבדה מזכירים גם השלמה של פורמליזציית sphere packing ב-8 ממדים ב-Lean במהלך 2026, ואת הוכחת BB(5) שהושלמה ב-2024 ב-Coq (ראו `docs/goal-bank.md` ו-`data/goals.csv`, שורה G062) ([לא אומת] מול מקור ראשוני לגבי sphere packing).

שני קולות: Tao ו-Buzzard

שני מתמטיקאים מלווים את כל הסיפור, וכדאי להכיר את עמדותיהם כי הן מעצבות את הקו של המעבדה.

Terence Tao הוא הדוגמה הבולטת לחוקר מוביל שמשלב AI בעבודה היומיומית: GitHub Copilot לכתיבת Lean, מודלי שפה לחיפוש ספרות ולניסוח, פרויקט ה-ETP, תחזוקה משותפת של ויקי "AI contributions to Erdős problems", ומאמר משותף עם AlphaEvolve. הקו שהוא מציג באופן עקבי: AI כ"עוזר מחקר" שהפלט שלו נבדק, ולא כאורקל.

Kevin Buzzard טוען כבר שנים שהמתמטיקה המחקרית סובלת מ"הוכחות שאף אחד לא בדק באמת", ש-Lean הוא הפתרון, ושמודלי שפה שמייצרים הוכחות בשפה טבעית בלי פורמליזציה הם בעייתיים. הוא גם הזהיר שוב ושוב מפני הייפ סביב "פתרון בעיות גדולות" בידי AI.

המעבדה לוקחת משניהם: מ-Tao את הנכונות להשתמש בכל כלי, ומ-Buzzard את הדרישה שכל דבר ייבדק.

פרשת בעיות ארדש

אם יש פרק אחד בסיפור שכל חבר במעבדה צריך להכיר, זה הפרק הזה.

Paul Erdős השאיר אחריו מאות בעיות פתוחות, רבות מהן עם פרסים כספיים קטנים. Thomas Bloom מתחזק את האתר erdosproblems.com, שמקטלג אותן ואת מצבן.

אוקטובר 2025. חוקרי OpenAI הכריזו ברשתות החברתיות ש-GPT-5 "פתר 10 בעיות ארדש פתוחות". Bloom הבהיר במהירות ש"פתוחה" באתר שלו פירושה רק שהוא עצמו לא מכיר מאמר שפותר אותה, ושהמודל **מצא מאמרים קיימים** שכבר פתרו את הבעיות. הציוצים נמחקו, ו-Demis Hassabis מ-DeepMind כינה את האירוע "מביך"[59]. הלקח: המודל היה מצוין בחיפוש ספרות, וזה ערך אמיתי. אבל זה לא גילוי מתמטי.

ינואר 2026 ואילך. הפעם הגיעו פתרונות חדשים באמת. לבעיה **#728** הגיעו GPT-5.2 Pro ו-Aristotle עם הוכחה פורמלית ב-Lean (6 בינואר 2026)[60]. זו נחשבת בדרך כלל לבעיית ארדש הראשונה שנפתרה באופן אוטונומי בעיקרו. אחריה באו בעיות נוספות, למשל #397 ([לא אומת]: ספירות של "15 בעיות" ממקורות משניים).

הוויקי של Tao. הדף "AI contributions to Erdős problems"[61] עודכן עד 30 ביוני 2026 ומאז אינו מתעדכן. בקטגוריה "AI standalone" יש בו כ-61 רשומות. רק כ-19 מהן פתרון מלא, כ-10 **שגויות**, וכ-9 לא אומתו. עוד כ-25 רשומות הן פתרונות שהתברר בדיעבד שכבר היו בספרות. הוויקי עצמו מוסיף הסתייגויות שכדאי לזכור בעל פה:

- "This page is not a benchmark."
- "Keep selection bias in mind."
- "Absence of past progress may reflect obscurity rather than difficulty." כלומר, היעדר התקדמות עשוי לשקף שהבעיה נשכחה, לא שהיא קשה.
- פתרון לבעיית ארדש אינו בהכרח מאמר שראוי לפרסום.

שלושה לקחים יוצאים מהפרשה. (א) בדיקת ספרות קודמת לכל דבר אחר. (ב) הבעיות ש-AI פותר הן בדרך כלל "הזנב הארוך" של בעיות שאיש לא ניסה ברצינות, ולא הבעיות הקשות. (ג) Lean שינה את הוויכוח: השאלה כבר אינה "האם זה נכון?" אלא "האם זה חדש וחשוב?".

בנצ'מרקים: מדידה, זיהום והייפ

לאורך כל התקופה, הדיון הציבורי התנהל סביב מספרים בבנצ'מרקים. כדאי להבין למה רבים מהם מטעים.

FrontierMath של Epoch AI (נובמבר 2024) נבנה מבעיות מחקר עם תשובה מספרית שאפשר לבדוק אוטומטית[62]. בהתחלה מודלים פתרו פחות מ-2%. בדצמבר 2024 הודיעה OpenAI שמודל o3 הגיע לכ-25%. בינואר 2025 התברר ש-OpenAI מימנה את הבנצ'מרק ושיש לה גישה לחלק מהבעיות,

ו-Epoch לא הצליחו לשחזר באופן עצמאי את התוצאה המקורית[63]. ב-2026 נוסף Tier 4, וביוני 2026 שוחררה גרסה v2 מתוקנת: 12 בעיות תוקנו ו-7 הוסרו. כלומר, **גם בבנצ'מרק מוקפד היו שגיאות.** ציונים עדכניים מתפרסמים בדיווחים סותרים ([לא אומת]: נתונים על Tier 4 מעל 80%).

Humanity's Last Exam (CAIS ו-Scale AI, ינואר 2025) כולל כ-2,500 שאלות[64]. ביקורות מאוחרות מצאו שיעור משמעותי של תשובות "נכונות" שגויות או שנויות במחלוקת, במיוחד בכימיה ובביולוגיה ([לא אומת]: הנתון המדויק).

שתי בעיות מבניות חוזרות. **זיהום (contamination)**: miniF2F ובעיות אולימפיאדה ישנות דלפו לנתוני האימון, ולכן ציון גבוה עליהן לא מוכיח יכולת. **הטיית בחירה (selection bias)**: מכריזים על ההצלחות, ועל אלפי הניסיונות הכושלים לא מדווחים.

התשובה שהקהילה פיתחה היא **First Proof** (1stproof.org): אתגר של מתמטיקאים בכירים עם בעיות מחקר **שלא פורסמו**[65]. בסבב 1, בפברואר 2026, המודל הטוב ביותר פתר רק בעיות בודדות. בסבב 2, ביוני 2026, הטוב ביותר פתר 6 מתוך 10 (צוות ETH עם מערכת סוכנים). בסבב 3, באוקטובר 2026, לפי הדיווחים המערכות יחד פתרו את כל העשר, אבל התיעוד והתקשורת היו חלשים, ובכל ההגשות של המודלים הפתוחים היו שגיאות מהותיות ([לא אומת] מול הדוח הרשמי). הלקח: מבחנים על בעיות לא מפורסמות הם הדרך האמינה היחידה להימנע מזיהום.

ובפיזיקה התיאורטית

הסיפור בפיזיקה צנוע יותר, אבל הדפוס זהה.

רגרסיה סימבולית מנסה לשחזר נוסחאות מנתונים. AI Feynman (Silviu-Marian Udrescu ו-Max Tegmark, 2020)[66] ו-PySR (Miles Cranmer, 2023) עובדים היטב על "נוסחאות ספר", ופחות על פיזיקה חדשה באמת.

אמפליטודות ו-bootstrap: transformers שמנבאים מקדמים באמפליטודות פיזור של תורת N=4 סופר-יאנג-מילס (Lance Dixon, Charton ואחרים, 2024–2025), ולמידת מכונה לחיפוש במרחב ה-bootstrap של תורות שדה קונפורמיות. התחום מבטיח, כי יש בו מבנה אלגברי עשיר שאפשר לבדוק בחישוב.

בפברואר 2026 פורסם המאמר "Single-minus gluon tree amplitudes are nonzero" (Guevara, Lupsasca, Skinner, Strominger, Weil)[67]. GPT-5.2 Pro הציע נוסחה כללית מתוך מקרים שבני אדם חישבו, וגרסה פנימית של המודל הפיקה הוכחה שבני אדם וידאו. זו תרומה אמיתית אבל צרה. שניים מהמחברים מ-OpenAI, ויש הטוענים ש-Witten רמז על האפשרות כבר ב-2003. וגם כאן, הבדיקה האנליטית היא שהפכה את ההצעה לתוצאה.

נוף המיתרים (string landscape): Yang-Hui He, Fabian Ruehle, ו-Carifio, Halverson, Krioukov ו-Nelson (2017 ואילך) הפעילו למידת מכונה על יריעות Calabi–Yau ומספרי Hodge[68]. זה כלי סיווג וחיפוש שימושי, לא פריצת דרך בשאלות היסוד.

Lattice QCD: normalizing flows לדגימה (Albergo, Kanwar, Shanahan, 2019 ואילך)[69] השפיעו באופן מעשי על עלויות החישוב, עם ערבויות סטטיסטיות מדויקות, כי המדגם מתוקן.

הלקח הכללי לפיזיקה: ההצלחות נמצאות היכן שיש **בדיקה עצמאית**, כלומר חישוב אנליטי, סימטריות, גבולות ידועים או סימולציה. "תיאוריה חדשה של הכול" ממודל שפה היא רעש.

הלקחים: מה המעבדה לומדת מכל זה

`docs/ai-math-lessons.md` מונה שבע-עשרה מסקנות. כאן הן מקובצות לחמישה עקרונות, וליד כל אחד מוסבר איך הוא מתממש במעבדה.

עיקרון ראשון: אימות גובר על יצירה

זה העיקרון שממנו נגזר כל השאר. כל תוצר של המעבדה מסתיים ב-Lean (בלי `sorry`, ועם פלט `#print axioms` נקי) או בחישוב שאפשר לשחזר (קוד, seed, תוצאה עם hash). אם אין מוודא, אין תוצאה. הוכחות בשפה טבעית שמגיעות ממודל שפה אינן אמינות בלי בדיקה פורמלית, ולכן המודל משמש ככותב סקיצות, ואחריו באה פורמליזציה. גם אחרי Lean, שני מומחים אנושיים צריכים לאשר שהתוצאה חדשה ומשמעותית לפני הכרזה פומבית. בפרק 6 נראה איך העיקרון הזה הפך לארכיטקטורה: **גרעין אמון (trust kernel)** ורמות **טענה** L0–L4.

עיקרון שני: בדקו את הטענה, לא רק את ההוכחה

טענה שפורמלה לא נכון היא הכשל השקט המסוכן ביותר, כפי שהראה המקרה של Gauss ו-Strong PNT. ההוכחה תקינה, ה-kernel מרוצה, ועדיין התוצאה חסרת ערך. לכן כל statement ב-Lean עובר ביקורת אנושית מול המקור, ורצוי השוואה אוטומטית ל-statement קיים ב-mathlib, כפי שנעשה ב-FLT. בפרק 5 נראה דוגמה קונקרטית לניסוח שנראה נכון ואינו נכון.

עיקרון שלישי: בחרו בעיות שמתאימות לכלים

- **הניצחון הזול ביותר הוא חיפוש ספרות.** לפני שתוקפים בעיה "פתוחה", מריצים חיפוש שיטתי (מודל שפה, arXiv, MathSciNet, zbMATH, OEIS) ומתעדים אותו. חלק ניכר מבעיות ארדש ש"נפתרו" ב-2025 כבר היו פתורות.
- **בעיות עם ציון מספרי.** בעיות בנייה עם פונקציית מטרה (חסמים, צפיפויות, אריזות, מספרי Ramsey קטנים, כפל מטריצות) מתאימות ביותר לצוות קטן עם חיפוש אבולוציוני ומוודא, בסגנון

FunSearch ו-AlphaEvolve.

- **מכוונים לזנב הארוך.** הבעיות ש-AI פותר הן בדרך כלל מוזנחות, לא קשות. בוחרים במודע בעיות שאיש לא ניסה ברצינות, ולא מתיימרים להכריע את השערת רימן.
- **בפיזיקה, תתי-בעיות עם בדיקה עצמאית**: אמפליטודות עם גבולות ידועים, bootstrap מספרי, בעיות סימולציה.
- **מתכננים תקציב חישוב.** AlphaProof דרש ימים של חישוב לבעיה, ו-FLT של Anthropic דרש כ-6 מיליארד טוקנים. מעדיפים בעיות שבהן כל ניסיון זול ובדיקתו מהירה.

כל אלה מתממשים בבנק המטרות (פרק 7), שבו כל בעיה מקבלת ציון של "סיכוי ל-AI" ו"מוכנות לפורמליזציה".

עיקרון רביעי: הנדסה נכונה

- **המעריך הוא קוד קריטי.** מערכות אבולוציוניות מנצלות באגים במעריך, כפי שתיעדו Georgiev ושותפיו. כותבים לו טסטים, משתמשים בחשבון מדויק (מספרים רציונליים או אינטרוולים) ולא ב-floats, ובודקים כל שיפור במעריך נוסף ובלתי תלוי.
- **Blueprint לפני קוד.** בכל פורמליזציה גדולה (LTE, PFR, FLT, Strong PNT) הייתה מפה של למות ותלויות. סוכנים מצליחים כשהעבודה מפורקת ליחידות קטנות שאפשר לבדוק כל אחת בנפרד.
- **לולאת סוכן עם משוב מהקומפיילר.** מודל כותב, Lean מחזיר שגיאות, המודל מתקן, וחוזר חלילה, עם שליפת למות מ-mathlib. משקיעים בתשתית: build מהיר, cache וגרסת mathlib נעולה.
- **לא בונים מה שקיים.** משתמשים ב-mathlib, ב-LeanDojo, במוכיחים פתוחים ובשירותי פורמליזציה. היתרון של צוות קטן הוא בבחירת הבעיות ובמעריכים, לא באימון מודלים מאפס.
- **AI כמיקרוסקופ להשערות**, בסגנון Davies ו-Ramanujan Machine: מייצרים נתונים, מחפשים דפוסים, מנסחים השערות, ואז מנסים להוכיח או להפריך. השערה מנוסחת היטב היא תוצר לגיטימי, כל עוד היא מסומנת כהשערה.

עיקרון חמישי: יושר בפרסום

- **מתעדים ומפרסמים תוצאות שליליות ו-datasets.** ניסיונות כושלים, statements מפורמלים וטבלאות חסמים הם תרומה. הוויקי של Tao מראה שתיעוד שקוף של כישלונות שווה יותר מהכרזות.
- **חלוקה מפורשת של קרדיט.** בכל תוצאה מתעדים מה עשה ה-AI (חיפוש, סקיצה, פורמליזציה, חישוב) ומה עשו בני אדם.
- **מודדים על בעיות שלא דלפו**, ומדווחים על כל הניסיונות, לא רק על הטוב מבין N.

מה לא עושים

המסמך המקורי מסכם גם רשימת "אנטי-דפוסים", שכדאי לתלות על הקיר:

- לא להכריז על הוכחה של השערת רימן, P≠NP, נאוויה-סטוקס וכדומה. כל טענה כזו ממודל שפה כמעט בוודאות שגויה, והיא תפגע באמינות המעבדה.
- לא לפרסם "פתרנו בעיה פתוחה" לפני חיפוש ספרות ובדיקה פורמלית.
- לא לסמוך על הוכחה בשפה טבעית רק משום שהיא "נראית טוב".
- לא להשתמש ב- `sorry` , ב- `axiom` או ב- `native_decide` בלי סימון מפורש. כל אלה נבדקים אוטומטית.
- לא לפרמל טענה חלשה או שגויה ולהציג אותה כמשפט המקורי.
- לא לצטט ציוני בנצ'מרק של יצרנים בלי לציין שלא אומתו ושיש חשש לזיהום.
- לא לבחור בעיה בלי קריטריון הצלחה מוגדר.
- לא לייצר "מאמרי AI" בכמויות. פתרון שולי לבעיה מוזנחת מתפרסם כרשומה ב-dataset או כפתק קצר.
- לא להסתיר את תפקיד ה-AI, וגם לא לנפח אותו.
- לא לאמן מודלים מאפס כשמודל פתוח ומוכן כבר קיים.

מהיסטוריה לפרקטיקה

הסיפור של 2019–2026 אינו סיפור של מכונה שהחליפה מתמטיקאים. זה סיפור של חלוקת עבודה חדשה: מכונות טובות מאוד בייצור מועמדים (הוכחות, בניות, מאמרים רלוונטיים) ובבדיקה מכנית של מועמדים; בני אדם עדיין נחוצים כדי לנסח את השאלה הנכונה, לוודא שהניסוח הפורמלי אומר את מה שצריך, ולשפוט אם התוצאה חשובה.

המעבדה מנסה לבנות בדיוק את חלוקת העבודה הזו בקטן. הפרק הבא מלמד את הכלי המרכזי, Lean 4, בידיים. אחריו נראה איך הכלי הזה נעשה לליבה של ארכיטקטורה שלמה.

בקצרה

- בין 2019 ל-2026 AI במתמטיקה עבר מתרגילי אולימפיאדה בסביבה פורמלית ללמות מחקריות, גילויי ספרות, שיפורי חסמים ופורמליזציה של FLT.
- כמעט כל הצלחה אמיתית נשענה על מוודא (Lean, חישוב משוחזר, פונקציית ניקוד), וכמעט כל הייפ נבע מהיעדרו.
- פרשת בעיות ארדש (אוקטובר 2025) לימדה שחיפוש ספרות הוא ערך אמיתי אבל אינו גילוי, ושה"זנב הארוך" של בעיות מוזנחות הוא המקום שבו AI מצליח.

- הכשל המסוכן ביותר הוא טענה שפורמלה לא נכון. לכן בודקים את ה-statement, ולא רק את ההוכחה.
- בנצ'מרקים ישנים מזוהמים. בעיות שלא פורסמו (First Proof) הן מדד אמין יותר.
- המעבדה מתרגמת את הלקחים לחמישה עקרונות: אימות, בדיקת טענות, בחירת בעיות, הנדסה נכונה ויושר בפרסום.

רעיונות לתוכן

1. **"הוא לא פתר, הוא מצא": פרשת ארדש באוקטובר 2025.** זווית: מה ההבדל בין חיפוש ספרות לגילוי, ולמה גם חיפוש ספרות הוא הישג. קהל יעד: קוראי S27 News מהקהל הרחב שמתעניינים ב-AI.
2. **ציר זמן אינטראקטיבי: שבע שנים של AI ומתמטיקה.** זווית: מ-GPT-f ועד FLT, כל אירוע עם תג "היה מוודא? כן/לא". קהל יעד: סטודנטים ואנשי טכנולוגיה, לאתר המעבדה.
3. **סרטון קצר: "למה בנצ'מרקים משקרים".** זווית: זיהום, הטיית בחירה ופרשת FrontierMath, בשלוש דקות. קהל יעד: עוקבי רשתות חברתיות.
4. **ראיון דמיוני בין Tao ל-Buzzard (מבוסס ציטוטים מתועדים בלבד).** זווית: שתי גישות לשילוב AI במחקר ומה המעבדה לקחה מכל אחת. קהל יעד: מתמטיקאים וסטודנטים מתקדמים. יש לסמן בבירור שזה עיבוד ולא ראיון אמיתי.
5. **"FLT ב-11 ימים: מה זה אומר ומה לא".** זווית: ההבדל בין בדיקה לגילוי, דרך המשפט של Buzzard. קהל יעד: קוראי מדע פופולרי.
6. **פוסט: "17 לקחים, 5 עקרונות".** זווית: גרסה תמציתית של סעיף הלקחים כמניפסט של המעבדה. קהל יעד: שותפים פוטנציאליים ומתנדבים.

פרק 5: Lean 4 בידיים

בפרק הקודם ראינו שכמעט כל הצלחה של AI במתמטיקה נשענה על מוודא, ושבמקרה של הוכחות המוודא הוא בדרך כלל **Lean 4**. במעבדה שלנו Lean הוא יותר מכלי: הוא **גרעין האמון (trust kernel)**, המקום היחיד שממנו יכולה לצאת טענה ברמה L3, "מאומת מכונה". מי שרוצה להבין מה המעבדה עושה, לבקר את התוצאות שלה או לתרום לה, צריך לקרוא קוד Lean ברמה בסיסית לפחות. הפרק הזה הוא מדריך מעשי לקורא חכם שלא כתב Lean מעולם: התקנה, משפט ראשון, הטקטיקות הנפוצות, קריאת מטרות, התמצאות ב-mathlib, חיפוש למות, ולבסוף שני נושאים שבלעדיהם אי אפשר להבין את המעבדה: למה `sorry` אסור בתוצאות, ואיך ניסוח פורמלי יכול להיות שגוי גם כשהכול "עובר".

> **הערה על בדיקת הקוד.** סביבת הכתיבה של הפרק הזה לא אפשרה להוריד את Lean או את mathlib (הגישה לשרתי ההורדה נחסמה), ולכן **אף אחד מקטעי הקוד בפרק לא נבדק בקומפיילר**. כל בלוק מסומן בהתאם. הקוד מ- `lean/EshLab/Basic.lean` ומ- `lean/EshLab/Targets/` מועתק מהמאגר כפי שהוא, וגם הוא לא נבנה מחדש בזמן הכתיבה. לפני שמצטטים קטע כלשהו מהפרק בפרסום, מריצים אותו.

מה זה Lean, בשתי פסקאות

Lean הוא שתי מערכות בגוף אחד: **שפת תכנות** פונקציונלית, ו**עוזר הוכחות (proof assistant)**. הוא מבוסס על תורת טיפוסים (dependent type theory), ובה יש רעיון אחד שכדאי להפנים מיד: **טענה היא טיפוס, והוכחה היא ערך מהטיפוס הזה.** כשכותבים `theorem t : 2 + 2 = 4 := ...`, אומרים "הנה ערך שהטיפוס שלו הוא הטענה 2+2=4". אם Lean מקבל את הערך, הטענה הוכחה.

מי שמחליט אם הערך תקין הוא ה-**kernel**: תוכנה קטנה יחסית, שנבדקה היטב, שבודקת כל צעד בהוכחה. כל השאר (הטקטיקות, האוטומציה, מודלי השפה שמציעים הוכחות) רק מייצרים מועמדים ל-kernel. זה מה שמאפשר למעבדה לסמוך על הוכחה בלי לסמוך על מי שכתב אותה, גם אם זה מודל שפה.

התקנה

elan, Lean ו-Lake

ההתקנה מתחילה ב-**elan**, מנהל הגרסאות של Lean (בדומה ל-rustup בעולם Rust). elan מתקין את Lean ואת **Lake**, מערכת הבנייה ומנהל החבילות, ודואג שכל פרויקט ירוץ בדיוק עם הגרסה שהוא מצהיר עליה בקובץ `lean-toolchain`.

```
# (פקודות מעטפת) לא נבדק בקומפיילר
curl -sSfL https://raw.githubusercontent.com/leanprover/elan/master/elan-
init.sh | sh -s -- -y
source ~/.elan/env      # או לפתוח טרמינל חדש
lean --version
lake --version
```

במאגר שלנו הסקריפט `scripts/setup.sh` עושה את זה אוטומטית (שלב 2), ויודע לדלג אם elan כבר מותקן. כעורך מומלץ VS Code עם התוסף הרשמי `lean4`: הוא מציג בחלון צדדי את **מצב ההוכחה** (infoview) תוך כדי הקלדה, וזו הדרך העיקרית לעבוד עם Lean.

פרויקט עם mathlib

mathlib היא ספריית המתמטיקה המשותפת של Lean, ובלעדיה כמעט אין מה לעשות. הדרך המהירה לפתוח פרויקט חדש שתלוי בה:

```
# לא נבדק בקומפיילר
lake new my_project math
cd my_project
lake exe cache get      # מוריד קבצים מהודרים מראש של mathlib
lake build
```

הפקודה החשובה ביותר כאן היא `lake exe cache get`. mathlib עצומה, וקומפילציה שלה מאפס לוקחת שעות. הפקודה מורידה קבצי `.olean` מהודרים מראש, וחוסכת את כל זה. אם שוכחים אותה, `lake build` יתחיל לקמפל את כל mathlib, ולרוב זה סימן שמשהו השתבש.

הפרויקט של המעבדה

הפרויקט של המעבדה נמצא ב-`lean/` ונקרא `EshLab`. ההגדרות שלו ב-`lean/lakefile.toml`:

```
name = "EshLab"
defaultTargets = ["EshLab"]
```

```
[leanOptions]
autoImplicit = false
relaxedAutoImplicit = false

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4.git"
rev = "master"
```

שימו לב לשני דברים. הראשון, `autoImplicit = false` : נחזור אליו בסוף הפרק, כי הוא מונע סוג מסוים של ניסוח שגוי. השני, `rev = "master"` : זו הגדרה נוחה לשלב השלד, אבל היא לא רפרודוקטיבית, כי mathlib משתנה כל יום. ההערה בקובץ עצמו, וגם `lean/README.md` , אומרות במפורש שלפני שסומכים על תוצאה צריך **לנעול** את mathlib לתג או ל-commit מסוים, להעתיק את קובץ `lean-toolchain` של אותה גרסת mathlib, ולשמור את `lake-manifest.json` במאגר. כרגע `lean/lean-toolchain` מכיל את `leanprover/lean4:v4.24.0` כמחזיק מקום. ה-README מתאר את רצף הבנייה:

```
# לא נבדק בקומפיילר
cd lab/lean
lake update                                          # כותב, פותר את mathlib
lake-manifest.json
cp .lake/packages/mathlib/lean-toolchain ./lean-toolchain  # מסנכרן גרסאות
lake exe cache get
lake build
```

לחלופין, `WITH_MATHLIB=1 bash lab/scripts/setup.sh` מריץ את `lake exe cache get` && `lake build` בפרויקט הזה (שלב 7 בסקריפט).

המשפט הראשון

פותחים קובץ חדש, למשל `Scratch.lean` , וכותבים:

```
-- לא נבדק בקומפיילר
theorem two_plus_two : 2 + 2 = 4 := by
  rfl
```

שלושה חלקים: `theorem two_plus_two` - הוא השם. - אחרי הנקודתיים בא **הניסוח (statement)**, כלומר הטיפוס: `2 + 2 = 4` . - אחרי `:=` באה **ההוכחה**. המילה `by` פותחת **מצב טקטיקות (tactic mode)**, שבו כותבים רשימת פקודות שבונות את ההוכחה צעד אחר צעד. כאן פקודה אחת מספיקה: `rfl` ("רפלקסיביות"), שאומרת "שני הצדדים שווים בהגדרה". Lean מחשב את `2 + 2`, מקבל `4`, ומסכים.

אפשר גם לכתוב הוכחה בלי טקטיקות, כ**ביטוי (term mode)**: `theorem two_plus_two : 2 + 2 = 4 := rfl`. בפועל, רוב ההוכחות במעבדה כתובות במצב טקטיקות, כי קל יותר לקרוא אותן ולתקן אותן.

כדי לראות מה Lean יודע, יש כמה פקודות שימושיות:

```
-- לא נבדק בקומפיילר
#check two_plus_two          -- מציג את הטיפוס: two_plus_two : 2 + 2 = 4
#check Nat.exists_infinite_primes
#eval 2 ^ 10                 -- מריץ חישוב: 1024
#print axioms two_plus_two   -- על אילו אקסיומות ההוכחה נשענת
```

לקרוא מטרות

הכלי החשוב ביותר בעבודה עם Lean הוא חלון ה-infoview. כשהסמן נמצא בתוך הוכחה, הוא מציג את **מצב ההוכחה**: רשימת ההנחות שיש בידינו, ומתחתיהן, אחרי הסימן `⊢`, **המטרה (goal)**, כלומר מה שנשאר להוכיח. לדוגמה:

```
case succ
k : ℕ
ih : sumOdd k = k ^ 2
⊢ sumOdd (k + 1) = (k + 1) ^ 2
```

קוראים את זה כך: אנחנו בענף `succ` של הוכחה באינדוקציה. יש לנו מספר טבעי `k`, והנחת אינדוקציה בשם `ih` שאומרת ש-`sumOdd k = k ^ 2`. צריך להוכיח ש-`sumOdd (k + 1) = (k + 1) ^ 2`.

כל טקטיקה משנה את המצב: מוסיפה הנחה, מחליפה את המטרה, מפצלת אותה לכמה מטרות, או סוגרת אותה. ההוכחה גמורה כשלא נשארו מטרות, ו-Lean מציג "No goals". הדרך ללמוד Lean היא לכתוב טקטיקה אחת, להסתכל על המצב, ולהחליט על הבאה.

עשר טקטיקות שכדאי להכיר

הטקטיקות הבאות מכסות חלק גדול מההוכחות הקטנות. בכל דוגמה ההערה מתארת את המטרה לפני ואחרי.

intro: להכניס הנחה

כשהמטרה היא גרירה (`p → q`) או כמת כולל (`∀ x, ...`), `intro` מכניס את ההנחה או את המשתנה לרשימת ההנחות.

```
-- לא נבדק בקומפיילר
example (p q : Prop) : p → q → p := by
  intro hp hq      -- עכשיו hp : p, hq : q, והמטרה היא p
  exact hp
```

exact: לסגור עם ערך מדויק

`exact e` סוגר את המטרה אם `e` הוא בדיוק הוכחה שלה. בדוגמה למעלה, `hp` הוא הוכחה של `p`.

apply: לעבוד אחורה

`apply f` מתאים כש-`f` היא גרירה שהמסקנה שלה היא המטרה. Lean מחליף את המטרה בהנחות של `f`.

```
-- לא נבדק בקומפיילר
example (p q : Prop) (hpq : p → q) (hp : p) : q := by
  apply hpq        -- המטרה הופכת מ-q ל-p
  exact hp
```

rw: לשכתב באמצעות שוויון

`rw [h]` מחליף במטרה כל מופע של אגף שמאל של `h` באגף ימין שלו. `rw [← h]` מחליף בכיוון ההפוך. אם אחרי השכתוב המטרה נהיית `a = a`, `rw` סוגר אותה בעצמו.

```
-- לא נבדק בקומפיילר
example (a b c : ℕ) (h1 : a = b) (h2 : b = c) : a = c := by
  rw [h1, h2]      -- a = c  →  b = c  →  c = c, ונסגר
```

simp: לפשט

`simp` מפעיל אוסף גדול של כללי פישוט שמסומנים ב-mathlib (למשל `n + 0 = n`, `x * 1 = x`), ומנסה להביא את המטרה לצורה שסוגרת את עצמה. אפשר להוסיף לו למות: `simp [h]`. הוא חזק ונוח, אבל קשה לצפות מה בדיוק יעשה, ולכן בהוכחות ארוכות מעדיפים לפעמים `simp only [...]` עם רשימה מפורשת.

```
-- לא נבדק בקומפיילר
example (n : ℕ) : n + 0 = n * 1 := by
  simp
```

induction: אינדוקציה

`induction n with` מפצל את המטרה לבסיס (`zero`) ולצעד (`succ`), ובצעד נותן הנחת אינדוקציה.

```
-- לא נבדק בקומפיילר
example (n : ℕ) : 0 + n = n := by
  induction n with
  | zero ⇒ rfl
  | succ k ih ⇒
    rw [← Nat.add_assoc, ih]   -- 0 + (k + 1)  →  0 + k + 1  →  k + 1
```

omega: אריתמטיקה לינארית על שלמים

`omega` הוא אלגוריתם הכרעה לטענות לינאריות על מספרים טבעיים ושלמים (חיבור, חיסור, כפל בקבוע, אי-שוויונות, וגם `/` ו-`%` בקבוע). אם הטענה מהסוג הזה, הוא פשוט סוגר אותה.

```
-- לא נבדק בקומפיילר
example (a b : ℕ) (h1 : a < b) (h2 : b < 10) : a + 1 < 10 := by
  omega
```

decide: לחשב את התשובה

`decide` מתאים לטענות שאפשר להכריע בחישוב סופי: השוואה של מספרים קונקרטיים, בדיקה על קבוצה סופית. ה-kernel עצמו מבצע את החישוב, ולכן התוצאה אמינה לגמרי. המחיר: זה איטי, ובמספרים גדולים זה לא יעבוד.

```
-- לא נבדק בקומפיילר
example : 2 ^ 10 = 1024 := by decide
example : ∀ x : Fin 5, x.val < 5 := by decide
```

לטקטיקה יש אחות מסוכנת, `native_decide`, שמקמפלת את החישוב לקוד מכונה ומריצה אותו. היא מהירה בהרבה, אבל משמעותה שסומכים על הקומפיילר ולא רק על ה-kernel, ולכן הכללים של המעבדה אוסרים אותה בהוכחות אלא אם הטענה מסומנת במפורש ככזו שסומכת על הקומפיילר (ראו בהמשך).

`norm_num`: חשבון מספרי

`norm_num` (מ-mathlib) מוכיח טענות על מספרים קונקרטיים: חשבון בשברים, ראשוניות של מספרים קטנים, אי-שוויונות מספריים.

```
-- לא נבדק בקומפיילר
example : (3 : ℚ) / 4 + 1 / 4 = 1 := by norm_num
example : Nat.Prime 97 := by norm_num
```

`ring` ושאר האוטומציה

`ring` מוכיח זהויות פולינומיות בחוגים קומוטטיביים, למשל `(a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2`. ראינו אותו כבר, ונפגוש אותו שוב בקובץ של המעבדה. בקטלוג הכלים (`data/tools.csv`, שורה T013) רשומים `grind`, `omega`, `decide` ו-`norm_num` כ"קו האוטומציה הראשון", לפני שפונים למוכיחי AI. זה עיקרון עבודה: קודם מנסים את האוטומציה הזולה והאמינה.

סיור בקובץ `Basic.lean` של המעבדה

הקובץ `lean/EshLab/Basic.lean` נכתב בכוונה כ"משפטי חימום". לפי ההערה בראשו, יש לו שני תפקידים: לבדוק שה-toolchain וה-CI עובדים, ולשמש **דוגמת סגנון לסוכנים**: ניסוחים מפורשים, הוכחות קצרות, בלי `sorry`. הקוד שלהלן מועתק מהמאגר כפי שהוא (לא נבדק בקומפיילר בזמן כתיבת הפרק).

סכום האי-זוגיים

```
-- מתוך lean/EshLab/Basic.lean (לא נבדק בקומפיילר בזמן הכתיבה)
def sumOdd : Nat → Nat
```

```
  | 0 ⇒ 0
  | n + 1 ⇒ sumOdd n + (2 * n + 1)

theorem sumOdd_eq_sq (n : Nat) : sumOdd n = n ^ 2 := by
  induction n with
  | zero ⇒ rfl
  | succ k ih ⇒
    show sumOdd k + (2 * k + 1) = (k + 1) ^ 2
    rw [ih]
    ring
```

ההגדרה `sumOdd` היא רקורסיה פשוטה: הסכום של אפס אי-זוגיים הוא 0, והסכום של $n+1$ אי-זוגיים הוא הסכום של n הראשונים ועוד האי-זוגי ה-$(n+1)$, כלומר $2n+1$. המשפט אומר את הזהות המוכרת

$$1 + 3 + 5 + \ldots + (2n-1) = n^2 .$$

ההוכחה באינדוקציה. בבסיס, `sumOdd 0` הוא `0` לפי ההגדרה, ו-`0 ^ 2` מחושב ל-`0`, ולכן `rfl` מספיק. בצעד, המטרה היא `sumOdd (k + 1) = (k + 1) ^ 2`. הפקודה `show` כותבת מחדש את המטרה בצורה שבה אגף שמאל "נפתח" לפי ההגדרה: `sumOdd k + (2 * k + 1)`. Lean מקבל את זה כי שתי הצורות שוות בהגדרה. ה-`show` לא הכרחי מבחינה לוגית, אבל הוא הופך את ההוכחה לקריאה ומבטיח ש-`rw [ih]` ימצא את `sumOdd k`. אחרי השכתוב המטרה היא `k ^ 2 + (2 * k + 1) = (k + 1) ^ 2`, זהות פולינומית, ו-`ring` סוגר אותה.

גאוס

```
-- מתוך lean/EshLab/Basic.lean (לא נבדק בקומפיילר בזמן הכתיבה)
def sumTo : Nat → Nat
  | 0 ⇒ 0
  | n + 1 ⇒ sumTo n + (n + 1)

theorem two_mul_sumTo (n : Nat) : 2 * sumTo n = n * (n + 1) := by
  induction n with
  | zero ⇒ rfl
  | succ k ih ⇒
    show 2 * (sumTo k + (k + 1)) = (k + 1) * (k + 1 + 1)
    rw [Nat.left_distrib 2, ih]
    ring
```

זו הנוסחה של גאוס, $2(0 + 1 + \dots + n) = n(n+1)$, בניסוח שנמנע מחילוק (חילוק במספרים טבעיים ב-Lean מעגל כלפי מטה, וזה מקור קלאסי לטעויות). המבנה זהה, עם צעד נוסף: `Nat.left_distrib` שהופך את `2 * (sumTo k + (k + 1))` ל-`2 * sumTo k + 2 * (k + 1)`. עכשיו `ih` יכול לשכתב את `2 * sumTo k`, ו-`ring` מסיים. $2(a+b) = 2a + 2b$ הוא חוק הפילוג

אינסוף ראשוניים, בעזרת mathlib

```
-- מתוך lean/EshLab/Basic.lean (לא נבדק בקומפיילר בזמן הכתיבה)
theorem exists_prime_ge (n : Nat) : ∃ p, n ≤ p ∧ p.Prime :=
  Nat.exists_infinite_primes n
```

כאן אין טקטיקות בכלל: ההוכחה היא ביטוי. `Nat.exists_infinite_primes` הוא משפט ב-mathlib שאומר בדיוק את זה, ואנחנו פשוט מפעילים אותו על `n`. זו דוגמה לעיקרון "לא בונים מה שקיים": חלק גדול מהעבודה ב-Lean הוא למצוא את הלמה הנכונה ב-mathlib. בהמשך נראה איך מחפשים.

חישוב שה-kernel בודק

```
-- מתוך lean/EshLab/Basic.lean (לא נבדק בקומפיילר בזמן הכתיבה)
theorem sumOdd_ten : sumOdd 10 = 100 := by decide
```

`decide` מריץ את `sumOdd 10` בתוך ה-kernel ומשווה ל-100. זה נראה טריוויאלי, אבל זה בדיוק המנגנון שבו המעבדה מתכננת לבדוק בניות קטנות: צביעה מפורשת, גרף, קונפיגורציה. בפרק 7 נראה שמטרה G050 בבנק המטרות מתכננת להשתמש ב-`decide` כדי לאמת חסם תחתון למספר Schur באמצעות צביעה מפורשת.

איך mathlib מאורגנת

mathlib היא עץ של מאות תיקיות ואלפי קבצים, והיא מאורגנת לפי תחומים. כמה תיקיות ראשיות שכדאי להכיר:

תיקייה	מה יש בה
`Mathlib/Logic`, `Mathlib/Order`	לוגיקה, יחסי סדר, סריגים
`Mathlib/Data`	מבני נתונים מתמטיים: `Nat`, `Int`, `Rat`, `Real`, `Finset`, `List`, `Fin`

תיקייה	מה יש בה
`Mathlib/Algebra`	חבורות, חוגים, שדות, מודולים
`Mathlib/NumberTheory`	ראשוניים, פונקציות אריתמטיות, שדות מספרים
`Mathlib/Combinatorics`	גרפים פשוטים, קומבינטוריקה אדיטיבית, Ramsey בסיסי
`Mathlib/Analysis` , `Mathlib/Topology`	אנליזה ממשית ומרוכבת, טופולוגיה
`Mathlib/MeasureTheory` , `Mathlib/Probability`	מידה ואינטגרציה, הסתברות
`Mathlib/LinearAlgebra` , `Mathlib/RingTheory`	אלגברה לינארית, תורת חוגים
`Mathlib/Geometry` , `Mathlib/CategoryTheory`	גאומטריה, תורת קטגוריות
`Mathlib/Tactic`	הטקטיקות עצמן: `ring` , `norm_num` , `linarith` ועוד

שמות הלמות עוקבים אחרי מוסכמה שכדאי ללמוד, כי היא הופכת ניחוש לאפשרי. שמות של משפטים כתובים ב-snake_case ומתארים את הטענה: `mul_comm` (*ab* = *ba*), `add_le_add` , `Nat.exists_infinite_primes` . שמות של טיפוסים ומבנים כתובים ב-UpperCamelCase: `Nat` , `Finset` , `SimpleGraph` . מרחב השמות (namespace) אומר על איזה אובייקט מדובר: `Nat.Prime` הוא ראשוניות של מספר טבעי. בקבצים שלנו כותבים `import Mathlib` כדי לייבא את הכול. זה פשוט ונוח, במחיר זמן טעינה ארוך יותר.

איך מוצאים את הלמה הנכונה

רוב הזמן בכתיבת Lean לא מושקע בהיגיון, אלא בשאלה "איך קוראים ב-mathlib למשפט שאני צריך?". יש שלושה כלים עיקריים, וכולם מופיעים ב-Starter Stack של המעבדה (פרק 8).

`exact?` ו-**`apply?`** הם טקטיקות שמחפשות בעצמן. כותבים `exact?` במקום ההוכחה, ו-Lean סורק את הלמות הזמינות ומציע שורה שסוגרת את המטרה, למשל `Try this: exact Nat.exists_infinite_primes n`. `apply?` מציע למות שהמסקנה שלהן מתאימה למטרה, גם אם נשארות הנחות להוכיח. `rw?` מציע שכתובים אפשריים. אלה הכלים הראשונים לנסות, והם גם רשומים כחלק מארגז הכלים של ה-lean-agent ב- `orchestrator/agents.py` .

```
-- לא נבדק בקומפיילר
example (n : ℕ) : ∃ p, n ≤ p ∧ p.Prime := by
  exact?      -- יציע: exact Nat.exists_infinite_primes n
```

Loogle הוא מנוע חיפוש לפי **צורה**. מחפשים לפי *שמות* קבועים, לפי *תבנית של ביטוי*, או לפי מחרוזת בשם. לדוגמה, השאילתה `_ * _ + _ * _ = _ * (_ + _)` מחזירה את חוקי הפילוג, והשאילתה `Nat.Prime, "infinite"` מחזירה למות *שמזכירות* ראשוניים ושבשמן מופיעה המילה infinite. Loogle מתאים כשיודעים בדיוק איך הטענה אמורה להיראות.

LeanSearch הוא מנוע חיפוש בשפה טבעית: כותבים "there are infinitely many primes" ומקבלים רשימת למות מועמדות. הוא מתאים כשיודעים מה רוצים להגיד, אבל לא איך mathlib מנסחת את זה. ל-LeanSearch יש חלופה בשם Moogle (T017 בקטלוג).

סדר עבודה מומלץ: קודם `exact?` , אחר כך LeanSearch אם אין מושג איך הדבר נקרא, ו-Loogle כשיש תבנית ברורה.

sorry: ההבטחה שלא קוימה

כשכותבים הוכחה ארוכה, נוח לדחות חלקים ממנה. Lean מאפשר זאת עם המילה `sorry` :

```
-- לא נבדק בקומפיילר
theorem hard (n : ℕ) : n * n ≥ n := by
  sorry
```

`sorry` אומר ל-Lean "תניח שזה הוכח". הקובץ יתקמפל, עם אזהרה (`declaration uses 'sorry'`), וכל מה *שמשתמש* ב-`hard` יראה תקין. זה כלי לגיטימי בזמן הפיתוח, כמו פיגום בבניין, וזה חלק מתהליך ה-blueprint: כותבים את השלד עם `sorry` במקומות הפתוחים, וממלאים אותם בהדרגה.

אבל בתוצאה סופית `sorry` הוא *שקר*. *משפט שמשתמש* ב-`sorry` לא הוכח. ומאחר שהאזהרה היא רק אזהרה ולא *שגיאה*, קל מאוד לפספס אותה, במיוחד כשמודל *שפה* כותב אלפי *שורות*. לכן המעבדה קובעת כללי קבלה מפורשים להוכחה ברמה L3, *שמופיעים* ב-`lean/README.md` :

1. `lake build` מצליח על checkout נקי, עם ה-manifest הנעול.
2. בקובץ ההוכחה אין `sorry` , `admit` , `axiom` , `native_decide` , `implemented_by` או `@[extern]` . (`native_decide` מותר רק אם הטענה מסומנת במפורש ככזו שסומכת על

הקומפיילר.)

3. `#print axioms EshLab.<theorem>` מציג לכל היותר את `propext` , `Classical.choice` ו- `Quot.sound` , שלוש האקסיומות הסטנדרטיות של Lean ו-mathlib.
4. ניסוח המשפט **זהה תחבירית** ליעד שבני אדם בדקו בתיקייה `EshLab/Targets/` . קובץ ההוכחה צריך לייבא את היעד ולהוכיח את `EshLab.Targets.<Name>` עצמו, ולא עותק שנוסח מחדש.
5. כל קובץ יעד כולל דוגמאות (`example`) שבודקות את הקידוד על מקרים קטנים.

הכלל השלישי הוא הרשת הבטוחה: גם אם `sorry` הוסתר איפשהו בשרשרת התלויות, הוא יופיע בפלט של `#print axioms` כ- `sorryAx` . גם `axiom` חדש שמישהו הוסיף, וגם השימוש ב- `native_decide` , יופיעו שם. אם הפלט נקי, ה-kernel באמת בדק הכול.

```
-- לא נבדק בקומפיילר; הפלט המדויק עשוי להשתנות בין גרסאות
#print axioms EshLab.exists_prime_ge
-- 'EshLab.exists_prime_ge' depends on axioms: [propext, Classical.choice, Quot.sound]
#print axioms hard
-- 'hard' depends on axioms: [sorryAx]
```

הכלל הרביעי מטפל בבעיה אחרת, עמוקה יותר, שאליה מוקדש הסעיף הבא.

כשהניסוח עצמו שגוי

כל מה ש-Lean מבטיח הוא שההוכחה נכונה **ביחס לניסוח**. אם הניסוח לא אומר את מה שהתכוונו, ההוכחה חסרת ערך. גרוע מזה, היא מזיקה, כי היא נראית מאומתת. `docs/architecture.md` קורא לזה "אופן הכישלון מס' 1", וראינו בפרק 4 שזה קרה בפועל בפרויקט Strong PNT. הנה כמה דרכים שבהן זה קורה, מהמקרה הגס ועד העדין.

דוגמה ראשונה: שוכחים תנאי

ככה מנוסחת השערת קולץ במעבדה, ב- `lean/EshLab/Targets/Collatz.lean` :

```
-- מתוך lean/EshLab/Targets/Collatz.lean (לא נבדק בקומפיילר בזמן הכתיבה)
def collatzStep (n : ℕ) : ℕ :=
  if n % 2 = 0 then n / 2 else 3 * n + 1

def CollatzConjecture : Prop :=
  ∀ n : ℕ, 0 < n → ∃ k : ℕ, collatzStep^[k] n = 1
```

`collatzStep^[k] n` פירושו "להפעיל את `collatzStep` על `n` בדיוק `k` פעמים". ההשערה: כל מספר חיובי מגיע בסופו של דבר ל-1.

עכשיו נניח שמישהו (אדם או מודל) שכח את התנאי `n > 0`:

```
-- לא נבדק בקומפיילר — ניסוח שגוי בכוונה
def CollatzWrong : Prop :=
  ∀ n : ℕ, ∃ k : ℕ, collatzStep^[k] n = 1
```

הניסוח הזה **שקרי**, ומסיבה טריוויאלית: 0 זוגי, ו- `collatzStep 0 = 0 / 2 = 0`. האפס תקוע לנצח ולא מגיע ל-1. מודל שמתבקש "להוכיח או להפריך" את `CollatzWrong` יפריך אותו בהצלחה בשתי שורות, וכותרת כמו "AI הפריך את השערת קולץ!" תהיה במרחק ציוץ אחד. זו לא תהיה הפרכה של שום דבר.

דוגמה שנייה: ניסוח חלש מדי

השערת הראשוניים התאומים אומרת שיש **אינסוף** זוגות ראשוניים $p, p+2$. הנה ניסוח שנראה סביר במבט חטוף:

```
-- לא נבדק בקומפיילר — ניסוח שגוי בכוונה
def TwinPrimesWrong : Prop :=
  ∃ p : ℕ, p.Prime ∧ (p + 2).Prime

theorem twinPrimesWrong_proof : TwinPrimesWrong :=
  ⟨3, by norm_num, by norm_num⟩
```

הניסוח אומר רק **שקיים** זוג תאומים אחד, וזה נכון: 3 ו-5. ההוכחה עוברת, `#print axioms` נקי, וזה משפט "מאומת ב-Lean" שאין לו שום קשר להשערה. הניסוח הנכון צריך לומר שמעל כל חסם יש זוג:

```
-- לא נבדק בקומפיילר
def TwinPrimeConjecture : Prop :=
  ∀ N : ℕ, ∃ p : ℕ, N ≤ p ∧ p.Prime ∧ (p + 2).Prime
```

שימו לב שזה בדיוק המבנה של `exists_prime_ge` שראינו קודם, ולא במקרה: "אינסוף" מנוסח כ"לכל N יש אחד מעל N".

דוגמה שלישית: מלכודות של מספרים טבעיים

ב-Lean, חיסור במספרים טבעיים נקטע באפס (`0 - 5 = 0`), וחילוק מעגל כלפי מטה (`7 / 2 = 3`), כולל `n / 0 = 0`. הבחירות האלה הגיוניות מבחינה טכנית, אבל הן מייצרות טענות שנראות נכונות ואינן:

```
-- לא נבדק בקומפיילר
example : ∀ n : ℕ, (n - 1) + 1 = n := by
  sorry   -- שקרי! עבור n = 0: (0 - 1) + 1 = 0 + 1 = 1 ≠ 0
```

ניסוח השערה שכולל ביטוי כמו `n - 1` או `a / b` על מספרים טבעיים, בלי תנאים מתאימים, עלול לשנות את המשמעות לגמרי. לכן `two_mul_sumTo` שראינו מנוסח עם כפל ב-2 ולא עם חילוק ב-2.

דוגמה רביעית: משתנה שנולד מטעות הקלדה

כאן נכנסת ההגדרה `autoImplicit = false` מה-lakefile. כשהאפשרות הזו פעילה (וזו ברירת המחדל של Lean עצמו), Lean מתייחס לשם לא מוכר **בכותרת** של הצהרה, כלומר בניסוח של משפט, כמשתנה חדש שנוסף אוטומטית כפרמטר מובלע. נניח שכותבים משפט על השערת גולדבך עם טעות הקלדה, `m` במקום `n`:

```
-- פעיל, זה עלול להתקבל ולשנות משמעות autoImplicit לא נבדק בקומפיילר — עם
theorem goldbach_attempt :
    ∀ n : ℕ, 2 < n → Even n → ∃ p q : ℕ, p.Prime ∧ q.Prime ∧ p + q = m := by
  sorry
```

עם `autoImplicit` פעיל, Lean עלול להוסיף בשקט פרמטר מובלע `{m}` ולקבל את הניסוח. עכשיו זו טענה אחרת לגמרי, על מספר `m` קבוע, שהיא שקרית לרוב ערכי `m` ולא קשורה לגולדבך.
כש-`autoImplicit = false`, Lean מסרב: "unknown identifier 'm'". זו שורת הגדרה אחת שמונעת קטגוריה שלמה של שגיאות, ולכן היא מופיעה בפרויקט של המעבדה.

איך המעבדה מתגוננת

ההגנה היא שילוב של כמה שכבות, וכולן מופיעות בקבצים:

- **ביקורת אנושית לפני כל מאמץ הוכחה.** כל יעד חדש בתיקייה `Targets/` חייב אישור של אדם. ההערה בראש `Collatz.lean` אומרת במפורש `reviewed-by-human = false`: הניסוח עוד לא אושר.
- **דוגמאות שפיות (sanity examples).** כל קובץ יעד כולל `example`-ים שבודקים את הקידוד על מקרים קטנים. ב-`Collatz.lean`:

```
-- מתוך lean/EshLab/Targets/Collatz.lean (לא נבדק בקומפיילר בזמן הכתיבה)
example : collatzStep^[7] 3 = 1 := by decide   -- 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1
example : collatzStep 1 = 4 := by decide       -- מוסכמת המעגל הטריוויאלי
```

ובקובץ `Goldbach.lean`:

```
-- מתוך lean/EshLab/Targets/Goldbach.lean (לא נבדק בקומפיילר בזמן הכתיבה)
def GoldbachConjecture : Prop :=
  ∀ n : ℕ, 2 < n → Even n → ∃ p q : ℕ, p.Prime ∧ q.Prime ∧ p + q = n

example : ∃ p q : ℕ, p.Prime ∧ q.Prime ∧ p + q = 10 :=
  ⟨3, 7, by norm_num, by norm_num, rfl⟩
```

המפרט של ה-lean-agent (`agents/lean-agent.md`) דורש דוגמאות גם למקרים **נכונים** וגם למקרים **שאמורים להיכשל**. דוגמה כמו `example : collatzStep 0 = 0 := by decide` הייתה מראה מיד למה התנאי `0 < n` הכרחי. בקבצים הנוכחיים יש רק דוגמאות חיוביות, וזה פער שכדאי לסגור. - **תרגום חזרה לשפה טבעית.** כל יעד מלווה בתיאור מילולי (docstring) של מה שהוא אומר, כדי שבודק אנושי יוכל להשוות למקור. - **הוכחה של היעד עצמו.** כלל הקבלה הרביעי דורש להוכיח את `EshLab.Targets.<Name>` שנבדק, ולא "עותק" שנוסח מחדש בקובץ ההוכחה. כך אי אפשר להחליף בשקט ניסוח קשה בניסוח קל. - **השוואה לניסוחים קיימים.** כשיש ניסוח ב-mathlib או ב-Formal Conjectures, משווים אליו. זה בדיוק מה שנעשה בפרויקט FLT של 2026.

מכאן לאן

מי שעבר את הפרק הזה יודע מספיק כדי לקרוא את קובצי ה-Lean של המעבדה, להבין מה נבדק ומה לא, ולזהות כמה מהטעויות הנפוצות בניסוח. כדי להתקדם הלאה, המקורות הרשמיים של קהילת Lean (Theorem Proving in Lean 4, Mathematics in Lean, והמשחק Natural Number Game) הם המשך טבעי, ואת הקישורים המדויקים יש לבדוק באתר הקהילה, leanprover-community.github.io. בפרק הבא נראה איך Lean משתלב בארכיטקטורה שלמה של סוכנים, מוודאים ויומן תוצאות.

בקצרה

- ב-Lean טענה היא טיפוס והוכחה היא ערך. ה-kernel הקטן בודק כל הוכחה, ולכן אפשר לסמוך על התוצאה בלי לסמוך על הכותב.

- ההתקנה: elan, ואחריו Lake, ובפרויקט עם mathlib חובה להריץ `lake exe cache get`. לתוצאות רפרודוקטיביות נועלים את גרסת mathlib.
- עשר טקטיקות (`intro`, `apply`, `exact`, `rw`, `simp`, `induction`, `omega`, `decide`, `norm_num`, `ring`) מכסות חלק גדול מההוכחות הקטנות. קוראים את המטרה אחרי כל צעד.
- חיפוש למות: `exact?` קודם, LeanSearch לשפה טבעית, Loogle לתבניות.
- `sorry` אסור בתוצאות. `#print axioms` חושף אותו, וגם `axiom` חדש ו-`native_decide`.
- הסכנה הגדולה היא ניסוח שגוי: תנאי חסר, טענה חלשה מדי, מלכודות של מספרים טבעיים, משתנה שנולד מטעות הקלדה. ההגנה: ביקורת אנושית, דוגמאות שפיות, `autoImplicit = false`, והוכחת היעד שאושר.

רעיונות לתוכן

1. **"המשפט הראשון שלך ב-Lean, ב-10 דקות".** זווית: סרטון מסך שעובר מההתקנה ועד `sumOdd_eq_sq`, עם חלון ה-infoview בפוקוס. קהל יעד: סטודנטים למתמטיקה ולמדעי המחשב.
2. **"איך מוכיחים משפט שקרי": ארבעה ניסוחים שגויים.** זווית: כתבה שמראה את `CollatzWrong` ו-`TwinPrimesWrong`, ומסבירה למה אימות מכונה לא מספיק. קהל יעד: קוראי S27 News, כולל לא-מתמטיקאים.
3. **פוסט: "המילה `sorry`, והמחיר שלה".** זווית: מה זה `sorry`, למה הוא שימושי, ולמה המעבדה אוסרת אותו בתוצאות, עם `#print axioms` כ"בדיקת רעלים". קהל יעד: מפתחים וחובבי AI.
4. **כרטיסיית עזר להורדה: עשר טקטיקות.** זווית: דף אחד עם כל טקטיקה, מטרה לפני ואחרי, ומתי להשתמש בה. קהל יעד: מתנדבים חדשים במעבדה.
5. **סדרת שורטים: "לקרוא מטרה".** זווית: כל סרטון מציג מצב הוכחה אחד ושואל מה הטקטיקה הבאה. קהל יעד: קהילת הרשתות החברתיות של S27.

פרק 6: ארכיטקטורת המעבדה

הפרק הרביעי הסתיים בלקח אחד: אימות גובר על יצירה. הפרק החמישי לימד את הכלי שמבצע את האימות. הפרק הזה מראה איך שני הדברים האלה הופכים למערכת עבודה: איך המעבדה בוחרת בעיה, מי עובד עליה, מי מחליט אם משהו נכון, ואיך כל ניסיון, מוצלח או כושל, נרשם כך שאי אפשר לשכתב אותו בדיעבד. הארכיטקטורה הזו היא מה שמבדיל מעבדה שמשתמשת ב-AI ממכונה לייצור הייפ. היא בנויה סביב הפרדה פשוטה: מי שמייצר רעיונות אינו מי שמאשר אמת. כל השאר נגזר מההפרדה הזו.

הפרק מבוסס על `docs/architecture.md` ועל הקוד שבתיקייה `orchestrator/`, ומתאר את מה שקיים במאגר נכון ל-4 בנובמבר 2026. איפה שהתיאור מתכנן משהו שעוד לא מומש בקוד, הדבר נאמר במפורש.

העיקרון: גרעין אמון

במעבדה יש שלוש שכבות, ולכל אחת מעמד אחר:

שכבה	מה היא כוללת	האם סומכים עליה?
מחוללים (generators)	מודלי שפה, חיפוש אבולוציוני, ה-analogy-agent, בני אדם בסיעור מוחות	**לא.** כל פלט הוא השערה עד שהוכח אחרת
גרעין אמון (trust kernel)	ה-kernel של Lean 4 (עם mathlib), ותעודות חישוב רפרודוקטיביות: הוכחות DRAT/LRAT לתוצאות SAT (נבדקות ב-`drat-trim` או ב-`cake_lpr`), אריתמטיקה מדויקת (שלמים ורציונליים) לבדיקת בניות, ותעודות SDP רציונליות לחסמי bootstrap	**כן.** זה הדבר היחיד שמעלה טענה לרמה L3
ביקורת אנושית	אישור ניסוח הטענה, ביקורת עמיתים, החלטה על פרסום	נדרשת בשתי נקודות: לפני ההוכחה ולפני הפרסום

הכלל, בניסוח של המסמך: **מודל שפה אף פעם לא "מוכיח". הוא מציע; ה-kernel מכריע.**

למה דווקא ה-kernel של Lean? כי הוא קטן, פשוט יחסית, ונבדק על ידי קהילה גדולה לאורך שנים. כשמשהו עובר `lake build` בלי `sorry` ובלי אקסיומות חדשות, הוא נכון. אבל, כפי שראינו בפרק 5, הוא נכון **ביחס לניסוח**. לכן גרעין האמון לבדו לא מספיק, ולכן יש שכבה שלישית.

> **אופן הכישלון מס' 1: לפרמל את הטענה הלא נכונה.** הוכחה מושלמת של ניסוח שגוי חסרת ערך, ואף מזיקה, כי היא נראית מאומתת. לכן ניסוח כל יעד עובר ביקורת אנושית לפני שמשקיעים בו מאמץ הוכחה, וכל קובץ יעד כולל דוגמאות שבודקות את הניסוח על מקרים קטנים.

כדאי לשים לב לסימטריה. גם התעודות שאינן Lean בנויות באותו היגיון: פותר SAT הוא תוכנה מסובכת שאולי יש בה באגים, אבל הוכחת DRAT שהוא מפיק נבדקת על ידי תוכנה קטנה ופשוטה, ו-`cake_lpr` אף אומת פורמלית בעצמו. אופטימייזר נומרי מוצא פתרון בנקודה צפה, אבל בודק בחשבון רציונלי מדויק מאשר אותו. בכל המקרים, החיפוש יכול להיות מסובך ולא אמין, והבדיקה פשוטה ואמינה.

הלולאה

כך נראית העבודה על מטרה אחת, מבחירתה ועד פרסומה:

```
flowchart TD
    GB[(בנק המטרות<br/>data/goals.csv)] ⟶ SEL[Orchestrator<br/>score = tractability ×
readiness × tier × remaining]
    SEL ⟶ LIT[literature-agent<br/>מה החזית? נפתר? כבר?]
    LIT ⟶|הופרך / נפתר| LED
    LIT ⟶|פתוח| FORM[lean-agent: פירמול הטענה<br/>Targets/Name.lean + examples]
    FORM ⟶ HR{{ביקורת אנושית<br/>של הניסוח}}
    HR ⟶|נדחה| FORM
    HR ⟶|אושר| ATK
    subgraph ATK [התקפה במקביל]
      direction LR
      C[construction-agent<br/>חיפוש אבולוציוני + scorer]
      S[search-agent<br/>SAT / SMT / חיפוש ממצה]
      P[lean-agent + AI prover<br/>הוכחת → סקיצה Lean]
      PH[physics-agent<br/>PhysLean / bootstrap]
      A[analogy-agent<br/>ניסוחים מחדש בלבד]
    end
```

```
    A -. ניסוח חדש .-> C & S & P
    ATK ⟶ VER[[Verifier: לא תוכנה, LLM<br/>lake build · drat-trim · exact arithmetic ·
rational SDP]]
    VER ⟶ LED[(Result ledger<br/>append-only JSONL, hash chain)]
    LED ⟶ PUB[publisher-agent<br/>blueprint · arXiv · site · news]
    LED ⟶|כישלונות, מכשולים| SEL
```

התרשים מופיע גם ב- `docs/architecture.md` , יחד עם גרסת ASCII לטרמינל. הנה השלבים, אחד אחד.

1. בחירת יעד. ה-orchestrator קורא את בנק המטרות ומדרג את המטרות הפתוחות לפי נוסחה שמשלבת את הסיכוי שכלי AI יתקדמו, את המוכנות של mathlib, את השכבה (tier) של המטרה, ואת העבודה שנשארה. את הנוסחה המדויקת נפרק בפרק 7.

2. ספרות. הסוכן הראשון שנוגע בכל מטרה הוא תמיד ה-literature-agent, והשאלה הראשונה שלו תמיד זהה: האם זה כבר נפתר או הופרך? זה הלקח הישיר של פרשת ארדש (פרק 4). הוא גם ממפה את החסמים הטובים ביותר הידועים, בודק מה כבר קיים ב-mathlib, ב-PhysLean וב-Formal Conjectures, ומסמן עמימויות בניסוח. אם המטרה נפתרה, היא הולכת ישר ליומן.

3. פירמול. ה-lean-agent כותב את הטענה כ- `def Name : Prop` בתיקייה `lean/EshLab/Targets/` , יחד עם דוגמאות שפיות ותרגום חזרה לשפה טבעית. כאן נמצאת **נקודת הביקורת האנושית הראשונה**: אף אחד לא מתחיל להוכיח לפני שאדם אישר שהניסוח אומר את מה שצריך.

4. התקפה במקביל. כמה סוכנים עובדים על המטרה בו-זמנית, כל אחד בשיטה אחרת: - *בנייה:* מודל שפה מציע תוכניות שמייצרות אובייקטים, ו-scorer דטרמיניסטי וקפוא מעריך אותן, בסגנון FunSearch ו-AlphaEvolve. - *SAT/SMT:* קידוד מקרים סופיים לנוסחה. תשובה "אין פתרון" (UNSAT) חייבת להגיע עם הוכחת DRAT או LRAT. - *הוכחה:* סקיצה לא-פורמלית (L2) הופכת להוכחת Lean בעזרת מוכיח AI, עם למות ביניים מאורגנות כ-blueprint. - *פיזיקה:* ניסוח מדויק וחסמי bootstrap עם תעודה רציונלית. - *אנלוגיה:* ניסוחים מחדש שמועברים לסוכנים האחרים לבדיקה.

5. אימות. ה-verifier הוא תוכנה דטרמיניסטית, לא מודל שפה, והיא היחידה שרשאית לסמן `certificate.verified = true` . נכון לכתיבת הפרק, ה-verifier מתואר במסמכים ומופיע כשלב בתוכנית ההתקפה, אבל עוד לא מומש כמודול נפרד בקוד. האכיפה שקיימת כבר היום היא בצד היומן: `ledger.validate()` מסרב לרשום טענה ברמה L3 ומעלה בלי תעודה שמסומנת כמאומתת.

6. יומן. כל ניסיון נרשם, בין שהצליח ובין שנכשל.

7. פרסום. ה-publisher-agent הופך רשומות מאומתות ל-blueprint, לטיוטת arXiv, לעדכון באתר ולכתבה ב-S27 News, תמיד עם תג של רמת הטענה וקישור לתעודה. כאן נמצאת **נקודת הביקורת האנושית השנייה**.

והחץ האחרון בתרשים חשוב לא פחות: כישלונות ומכשולים חוזרים מהיומן אל ה-orchestrator. ניסיון שנכשל הוא מידע שמשפיע על הבחירה הבאה, ומשמש את ה-analogy-agent כחומר גלם.

רמות טענה

כל טענה במעבדה נושאת רמה, מ-L0 עד L4. זה אולי הרעיון הפשוט והחשוב ביותר בארכיטקטורה, כי הוא מאפשר לדבר בכנות על התקדמות חלקית.

רמה	שם	מה נדרש	מי רשאי לקבוע
L0	רעיון / השערה	ניסוח ברור	כל סוכן (ה-analogy-agent תמיד כאן)
L1	ראיה נומרית	סקריפט רפרודוקטיבי, גרסאות seeds,	search, construction, physics
L2	הוכחה לא-פורמלית	טקסט מתמטי מלא שנבדק על ידי סוכן שני	lean-agent, אדם
L3	מאומת מכונה	`lake build` נקי, או DRAT שנבדק, או בדיקה מדויקת; אקסיומות רק מתוך {propext, Classical.choice, Quot.sound}; הוכחה של היעד **שאושר**	רק ה-**verifier**
L4	פורסם / ביקורת עמיתים	מאמר שהתקבל, או אימוץ בקהילה (למשל מיזוג ל-mathlib)	אדם

בקוד, הרמות מוגדרות כ-`ClaimLevel` ב-`orchestrator/ledger.py`: `L0_IDEA`, `L1_NUMERIC`, `L2_INFORMAL`, `L3_LEAN_VERIFIED`, `L4_PUBLISHED`. ה-README הראשי של המעבדה מוסיף לכל רמה את מה שמותר לומר עליה בפומבי: "יש לנו כיוון" (L0), "בדקנו עד N" (L1), "יש טיוטת הוכחה" (L2), "הוכח, מאומת מכונה" (L3), "פורסם" (L4).

נקודה עדינה: L3 על **חלק** מבעיה, כמו חסם משופר או מקרה פרטי, הוא תוצאה לגיטימית ויקרה. האתר מציג אותה בדיוק כך, ולא כ"נפתר". ה-publisher-agent מונחה במפורש לכתוב "Lean-verified improved lower bound", ולא "solved".

סגל הסוכנים

המעבדה מגדירה שבעה סוכנים ותוכנה אחת. לכל סוכן יש מפרט קריא לבני אדם בתיקייה `agents/<name>.md`, וגרסה כנתונים (כולל ה-system prompt) ב-`orchestrator/agents.py`. לכל סוכן מוגדרת גם **הרמה המקסימלית שמותר לו לטעון בעצמו** (`max_self_claim`).

סוכן	משימה	כלים עיקריים	רמה מקסימלית
literature-agent	להכריע אם המטרה כבר נפתרה, ולמפות את החזית	arXiv, zbMATH, MathSciNet, OEIS, חיפוש ב-mathlib	L1
lean-agent	לפרמל ניסוחים בנאמנות ולחפש הוכחות Lean	`lake build`, Lean REPL/LSP, מוכיח AI, Loogle, `exact?`	L2 (L3 רק דרך ה-verifier)
search-agent	דוגמאות נגדיות, מקרים קטנים ותעודות בחיפוש ממצה, SAT ו-SMT	CaDiCaL, Kissat, drat-trim, cake_lpr, Z3, cvc5	L1
construction-agent	לבנות אובייקטים מפורשים שמשפרים חסמים	לולאה אבולוציונית מונחית-LLM, חשבון מדויק	L1
physics-agent	להפוך בעיות פיזיקה לטענות מדויקות ולחסמים עם תעודה	PhysLean, SDPB, CAS	L1
analogy-agent	לייצר ניסוחים מחדש ואנלוגיות כזוויות התקפה	המודל בלבד	**L0 תמיד**
publisher-agent	להפוך רשומות מאומתות לפרסום	leanblueprint, LaTeX, מחולל האתר	L0 (משקף את היומן בלבד)
verifier (תוכנה)	לבדוק תעודות	lake, drat-trim, בודקים מדויקים	**היחיד שקובע L3**

לכל הסוכנים יש פסקה משותפת של "כללי יושר" (הקבוע `HONESTY` בקוד), שמצורפת לכל system prompt. בתמצית: אתה מחולל לא אמין; אל תטען שמשהו הוכח בלי תעודה מאומתת; סמן כל טענה ברמה; צטט עם מזהים (arXiv, DOI, שם ב-mathlib), ואם אינך בטוח שמקור קיים, אמור זאת; עדיף "אני לא יודע" מהמצאה; כישלונות נרשמים ביומן.

כמה מהמפרטים כוללים כללים מיוחדים שכדאי להכיר:

- ה-**lean-agent** מזהר ש"פירמול הטענה הלא נכונה הוא אופן הכישלון מס' 1", ונדרש לכתוב דוגמאות למקרים נכונים **וגם** למקרים שאמורים להיכשל, ולהוכיח את היעד שאושר ולא עותק שלו.
- ה-**construction-agent** עובד עם scorer שמוקפא לפני תחילת החיפוש ונבדק בנפרד, ובודק שיאים שנכתב בנפרד מהמחולל. "ציון בנקודה צפה אינו ראיה לשיא." וגם: "לעולם אל תכוונן את ה-scorer למועמד". זה הלקח של Georgiev ושותפיו על מערכות שמנצלות באגים במעריך.
- ה-**analogy-agent** הוא "מחולל ניסוחים בלבד, לעולם לא מקור לאמת". נחזור אליו בהמשך.
- ה-**publisher-agent** מפרסם רק את מה שהיומן תומך בו, ברמה המדויקת שהיומן רושם. הסתייגויות ותיקונים מתפרסמים באותה בולטות כמו הטענות.

איך נבנית תוכנית התקפה

הפונקציה `plan_for` ב-`orchestrator/agents.py` מחליטה אילו סוכנים ירוצו על מטרה נתונה. הלוגיקה פשוטה: תמיד מתחילים ב-literature-agent (שלב triage) וב-lean-agent (שלב פירמול). בשלב ההתקפה המקבילי נכנס קודם הסוכן האחראי על המטרה (`owner_agent`), אם הוא לא אחד משני הראשונים; אם התחום כולל פיזיקה, נכנס ה-physics-agent; אחר כך משלימים מתוך construction, search ו-lean עד שלושה סוכנים; ותמיד מוסיפים את ה-analogy-agent. בסוף, ה-verifier וה-publisher-agent.

היומן

היומן (ledger) הוא הזיכרון של המעבדה, והוא בנוי כך שיהיה קשה לשקר בו. הקוד נמצא ב-`orchestrator/ledger.py`, והקובץ הוא `data/ledger.jsonl` (הוא נוצר בריצה האמיתית הראשונה, ונכון לכתיבת הפרק עוד לא קיים במאגר).

שלוש תכונות מגדירות אותו:

Append-only. כל ניסיון הוא שורת JSON אחת, ושורות אף פעם לא נערכות. תיקון הוא רשומה חדשה, עם שדה `supersedes` שמצביע על `attempt_id` של הרשומה הישנה. ההיסטוריה נשארת גלויה.

שרשרת hash. כל רשומה כוללת את ה-SHA-256 של השורה הקודמת, בשדה `prev_sha256`. אם מישהו ישנה רשומה ישנה בשקט, השרשרת תישבר. הפונקציה `verify_chain()` בודקת את זה, והפקודה `python -m orchestrator.run --ledger-check` מריצה אותה.

אכיפת יושר. הפונקציה `validate()` נקראת לפני כל הוספה, ומסרבת לרשום טענה ברמה L3 ומעלה אם אין לה תעודה עם `verified: true` ועם סוג מוגדר (`lean`, `drat`, `exact-arith` וכן הלאה). כלומר, גם אם סוכן "מתבלבל" ומנסה לרשום L3, הקוד לא יאפשר זאת.

ומה קורה כשסוכן באמת רץ? בפקודה `run --`, הפלט של המודל נרשם ביומן תמיד ברמה **L0**, עם `outcome: "pending"`. ההערה בקוד אומרת זאת במפורש: פלט גולמי של מודל שפה לעולם אינו מעל L0 עד שנבדק.

הרמה של מטרה כולה נגזרת מהיומן: `best_level()` מחזירה את הרמה המקסימלית על פני ניסיונות שאינם ריצת ניסיון (`dry_run`). לפי `docs/architecture.md`, גם ניסיונות שהוחלפו (`supersedes`) אמורים להיות מוחרגים מהחישוב. בקוד הנוכחי, `best_level()` מסננת רק את ריצות הניסיון, וזה פער שכדאי לסגור.

מודל הנתונים

`docs/architecture.md` מגדיר ארבע ישויות:

```
Goal         (data/goals.csv — מקור האמת לבנק המטרות)
  id, title, title_he, field, tier, status, statement_short, why_it_matters,
  ai_tractability, formalization_readiness, known_partial_results,
  suggested_attack, references, owner_agent, progress_pct, last_update
  + (נגזר) lean_target, statement_reviewed_by, statement_reviewed_at

Attempt      (שורה ב-data/ledger.jsonl)
  attempt_id, goal_id, agent, method, claim_level (0-4), outcome
  (pending|success|partial|failed|refuted), summary, artifacts[], certificate?,
  model, dry_run, supersedes?, timestamp, prev_sha256

Artifact     (קובץ תחת artifacts/<goal_id>/<attempt_id>/)
  path, sha256, kind (lean | cnf | drat | construction | script | notebook | report),
  produced_by (agent + model + version)

Certificate  (מצורף ל-Attempt)
  kind (lean | drat | lrat | exact-arith | sdp-rational | none),
```

```
path, checker, checker_version, sha256, verified
+ עבור lean: lean-toolchain, hash של lake-manifest.json, פלט #print axioms
```

היחסים: למטרה יש הרבה ניסיונות, לניסיון יש הרבה תוצרים (artifacts), ולניסיון יש לכל היותר תעודה אחת. שימו לב לשדות של תעודת Lean: גרסת ה-toolchain וה-hash של `lake-manifest.json`. זה הקשר לנעילת הגרסאות שראינו בפרק 5. בלי גרסה נעולה, אי אפשר לשחזר את הבדיקה, ולכן אין תעודה.

בקוד הנוכחי מומשו `Attempt` ו-`Certificate` כ-dataclasses ב-`ledger.py`. השדות הנגזרים של Goal (כמו `statement_reviewed_by`) ותיקיית `artifacts/` הם עדיין תכנון.

ה-orchestrator בשימוש

ה-orchestrator הוא תוכנית פייתון קטנה, שכתובה בספרייה הסטנדרטית בלבד. ה-SDK של Anthropic נטען רק כשמבצעים קריאה אמיתית למודל. כך מריצים אותו, מתוך התיקייה `lab/`:

```
        python -m orchestrator.run --list                  # דירוג כל המטרות הפתוחות
        python -m orchestrator.run --next                  # המטרה הבאה
        python -m orchestrator.run --dry-run G050          # תוכנית התקפה, בלי קריאות API
        python -m orchestrator.run --dry-run G050 --prompts   # כולל הפרומפטים לכל סוכן
        python -m orchestrator.run --ledger-check          # בדיקת שלמות שרשרת ה-hash
        ESH_MODEL=claude-opus-5-5 python -m orchestrator.run --run G050 --agent
literature-agent
```

הפקודה האחרונה דורשת משתנה סביבה `ANTHROPIC_API_KEY`. בלעדיו היא מסרבת ומפנה ל-`--dry-run`.

כך נראה הפלט של `next` – על בנק המטרות הנוכחי (הורץ בזמן כתיבת הפרק):

```
NEXT: G024 - Formal-conjectures contributions
  score=1.100 (tractability=1.00, readiness=1.00, progress=0%, tier=C-Tractable)
  owner: lean-agent   field: formalization / all fields
```

וכך נראה `dry-run G050` – המטרה הראשונה מבין שלוש מטרות הפתיחה של המעבדה (פרק 7):

```
DRY RUN for G050: Small Ramsey-type numbers in Lean via SAT certificates (S(4)=44,
W(3,3)=27, W(2,4)=35)
```

```
model: claude-opus-5-5  (no API calls will be made)
ledger: 0 prior attempts, best level none

[1-triage] literature-agent  (may self-claim ⩽ L1)
[2-formalize] lean-agent  (may self-claim ⩽ L2)
[3-attack (parallel)] construction-agent  (may self-claim ⩽ L1)
[3-attack (parallel)] search-agent  (may self-claim ⩽ L1)
[3-attack (parallel)] lean-agent  (may self-claim ⩽ L2)
[3-attack (parallel)] analogy-agent  (may self-claim ⩽ L0)
[4-verify] verifier (program, not LLM): lake build / drat-trim / exact-arith → L3
[5-publish] publisher-agent  (may self-claim ⩽ L0)

Nothing was written. Use --run GOAL --agent NAME for a real call.
```

(הפלט קוצר: בפלט המלא מופיעה מתחת לכל סוכן גם רשימת הכלים שלו.)

כדאי לשים לב שבמקרה של G050 הסוכן האחראי הוא ה-lean-agent עצמו, ולכן שלב ההתקפה מתמלא בסוכני ברירת המחדל: construction, search ו-lean.

ההגדרות נקבעות במשתני סביבה, בלי סודות בקוד (`orchestrator/config.py`): `ESH_MODEL` (ברירת המחדל `claude-opus-5-5`), `ESH_GOALS` (נתיב לבנק המטרות), `ESH_LEDGER` (נתיב ליומן), `ESH_LEAN_ROOT` (נתיב לפרויקט Lean) ו- `ESH_MAX_TOKENS` (ברירת המחדל 8000). לבדיקות יש קובץ מטרות קטן ב- `tests/fixture_goals.csv` .

איך הפיזיקה משתלבת

פיזיקה תיאורטית מציבה אתגר מיוחד, כי בעיה בפיזיקה היא לרוב לא "טענה" אחת, אלא משפחה של הנחות: יוניטריות, סימטריית crossing, פערים בספקטרום, קיטוע. לכן ה-physics-agent פועל לפי כמה כללים:

- **ניסוח קודם לכל.** הוא כותב ניסוח מתמטי מדויק עם רשימת הנחות מפורשת, ומפריד בין החלק הריגורוזי לבין ההיוריסטיקה הפיזיקלית.
- **PhysLean.** כשקיימות הגדרות בספריית PhysLean (ספריית Lean לפיזיקה, לשעבר HepLean), משתמשים בהן, ומדווחים על הגדרות חסרות כמשימות פירמול.
- **חסמי bootstrap כתעודות.** חסם conformal bootstrap הוא בעיית תכנות חצי-מוגדר (SDP). פתרון נומרי בתוכנה כמו SDPB הוא **L1** בלבד. כדי להפוך אותו לתעודה, מעגלים את הפתרון הדואלי למספרים רציונליים ובודקים באריתמטיקה מדויקת את החיוביות. זו תעודה שניתן

לבדוק באופן עצמאי (`certificate.kind = "sdp-rational"`), ובטווח הארוך אפשר לייבא אותה ל-Lean.

הדוגמה שהמסמך נותן: חסמים ריגורוזיים על ממדי סקלינג במודל Ising התלת-ממדי, בהינתן הנחות מפורשות. זו המטרה G068 בבנק.

ה-analogy-agent: מחולל ניסוחים בלבד

ESH27 Lab שואבת השראה גם ממקורות לא-מתמטיים: גמרא, קבלה וחוכמות עתיקות. ה-analogy-agent הוא המקום שבו ההשראה הזו פוגשת את העבודה, והוא מוגדר בזהירות רבה, דווקא כדי שהיא לא תפגע באמינות.

התפקיד שלו הוא אך ורק לייצר זוויות תקיפה חדשות. כמה דוגמאות לתרגום, מתוך המפרט:

- **קל וחומר** ← למת מונוטוניות ("אם זה נכון למקרה הקשה, זה נכון לקל").
- **בניין אב** ← אינדוקציה ממקרה-אב, או דוגמה קיצונית.
- **גזירה שווה** ← איזומורפיזם בין שני מבנים שנראים שונים.
- **הספירות** ← חיפוש מבנה מדורג או מסונן, ואינווריאנטים לאורכו.

ושלושה כללים:

1. כל אנלוגיה **חייבת** להיות מתורגמת לטענה מתמטית מדויקת שאפשר לבדוק, ולהיות מועברת לסוכן שיבדוק אותה (search, construction או lean). אנלוגיה שאי אפשר לתרגם נזרקת.
2. כל פלט שלו הוא **L0**. הוא לעולם אינו מקור לאמת. מסורת, גימטריה או סמליות אינן ראיה לטענה מתמטית.
3. מודדים פוריות: היומן רושם אילו ניסוחים מחדש הובילו לתוצאות L1 ומעלה, כדי ללמוד אילו סוגי אנלוגיות מועילים בפועל.

הכלל השלישי הופך שאלה שבדרך כלל היא עניין של טעם ("האם הגישה הזו מועילה?") לשאלה אמפירית שאפשר לענות עליה מתוך נתונים.

מבנה המאגר

החלק של המאגר שרלוונטי לארכיטקטורה נראה כך:

```
lab/
├── data/goals.csv              # בנק המטרות
```

```
├── data/ledger.jsonl          # יומן התוצאות (נוצר בריצה הראשונה)
├── lean/                      # פרויקט Lake בשם EshLab: גרעין האמון
│   ├── lakefile.toml, lean-toolchain
│   ├── EshLab/Basic.lean      # משפטי חימום מוכחים
│   └── EshLab/Targets/        # בעיות פתוחות כ-Prop, בלי sorry
├── orchestrator/              # Python stdlib (+ anthropic SDK אופציונלי)
│   ├── goals.py  ledger.py  agents.py  config.py  run.py
├── agents/*.md                # מפרטי תפקידים
└── tests/fixture_goals.csv
```

בעתיד תתווסף התיקייה `lean/EshLab/Solved/` , עם קובץ הוכחה אחד לכל מטרה (`G0xx_*.lean`), ותיקיית `artifacts/` לתוצרים.

עקרונות היושר, בסיכום

המסמך מסכם את הארכיטקטורה בחמישה עקרונות, וכדאי לחזור עליהם כלשונם:

1. מודל שפה הוא מחולל לא אמין. Lean, DRAT ואריתמטיקה מדויקת הם גרעין האמון.
2. ניסוח הטענה נבדק על ידי אדם לפני כל מאמץ הוכחה.
3. כל טענה נושאת רמה (L0–L4), וכל טענה ציבורית מקושרת לתעודה.
4. כישלונות נרשמים. ההיסטוריה ביומן אינה נמחקת.
5. גרסאות מוצמדות (lean-toolchain, lake-manifest, גרסאות של פותרים). אחרת אין רפרודוקטיביות, ואין תעודה.

בקצרה

- הארכיטקטורה מפרידה בין מחוללים לא אמינים (מודלי שפה, חיפוש, בני אדם בסיעור מוחות) לבין גרעין אמון (Lean, DRAT/LRAT, חשבון מדויק, SDP רציונלי).
- הלולאה: בחירת יעד, ספרות, פירמול, ביקורת אנושית, התקפה מקבילה, אימות, יומן, פרסום. כישלונות חוזרים לבחירה.
- רמות טענה L0–L4. רק ה-verifier קובע L3, ולכל סוכן יש תקרה של מה שמותר לו לטעון.
- היומן הוא append-only, עם שרשרת hash ואכיפה בקוד: אין L3 בלי תעודה מאומתת, ופלט גולמי של מודל נרשם תמיד כ-L0.
- ה-orchestrator הוא CLI קטן: `--list` , `--next` , `--dry-run` , `--run` , `--ledger-check`.

- חלקים מהארכיטקטורה עדיין בתכנון: verifier כמודול, תיקיית `artifacts/` , החרגת רשומות שהוחלפו מחישוב הרמה.

רעיונות לתוכן

1. **"מי מחליט מה נכון?": מדריך חזותי לגרעין האמון.** זווית: אינפוגרפיקה של שלוש השכבות, עם דוגמה של טענה אחת שעוברת מ-L0 ל-L3. קהל יעד: הקהל הרחב של S27 News.
2. **סרטון: "יומן שאי אפשר לשקר בו".** זווית: הסבר על append-only ושרשרת hash, והשוואה לפנקס מעבדה במדעי הניסוי. קהל יעד: אנשי טכנולוגיה וחובבי מדע.
3. **פוסט טכני: "בנינו orchestrator בכ-600 שורות פייתון".** זווית: סיור בקוד, עם הפלט האמיתי של `dry-run` --. קהל יעד: מפתחים שבונים מערכות סוכנים.
4. **"מקל וחומר ללמת מונוטוניות": ה-analogy-agent.** זווית: איך מסורת לימוד יהודית הופכת לכלי חשיבה, בלי להפוך לטיעון. קהל יעד: קוראים דתיים ומסורתיים עם עניין במדע.
5. **דף באתר: "מה פירוש L3?".** זווית: מילון קצר לרמות הטענה, שאליו מקשרת כל כתבה. קהל יעד: כל קוראי האתר.

פרק 6½: חמשת היסודות

לכל מערכת יש כללים, אבל רוב הכללים נשארים על הנייר. הפרק הזה מתאר חמישה כללים שלא נשארו על הנייר. הם כתובים בקוד, בהנחיות של הסוכנים, בפנקס התוצאות, באתר ובספר הזה. מקורם במסורת: גמרא, משנה ותורה. הם לא קישוט תרבותי. כל אחד מהם פותר בעיה אמיתית של מערכות AI, בעיה שהמסורת זיהתה, בשפה שלה, לפני מאות ואלפי שנים.

המשפט שמחבר את כולם:

> **תורה + AI = תיקון עולם, כשה-AI הוא עבד ולא אדון.**

א. לא בשמים היא

בבבא מציעא (נט ע"ב) מסופר על מחלוקת בין רבי אליעזר לחכמים, על תנור שנחתך לחוליות. רבי אליעזר מביא את כל הראיות שבעולם, וחכמים לא מקבלים. לבסוף הוא קורא לעזרה מן השמים, ובת קול יוצאת ואומרת שהלכה כמותו. רבי יהושע עומד על רגליו ואומר: **"לא בשמים היא."** התורה כבר ניתנה, וההכרעה נעשית בבית המדרש: ברוב, בנימוק ובראיה.

הבעיה שזה פותר: מודל שפה נשמע בטוח, רהוט וסמכותי. הוא בת קול מודרנית. הנטייה האנושית היא להאמין לו. **איך זה כתוב אצלנו:** שום טענה לא נקראת "מוכחת" עד שעברה ב-Lean או בבודק תעודות. בקוד הפנקס זה כלל **חוסם**: אי אפשר לרשום L3 בלי תעודה שנבדקה.

ב. בשם אומרו

"כל האומר דבר בשם אומרו מביא גאולה לעולם" (אבות ו, ו). הגמרא מקפידה על שלשלת מסירה: "אמר רבי יוחנן משום רבי שמעון בן יוחאי". המידע לא מנותק ממקורו.

הבעיה שזה פותר: מודלים ממציאים מקורות. וגרוע מזה, גם כשהם צודקים, הם לא יודעים להגיד מאיפה. ובמתמטיקה, במחשבה שנייה, הרבה "גילויים" הם שחזורים של מה שכבר פורסם. כך קרה בפרשת בעיות ארדש באוקטובר 2025. **איך זה כתוב אצלנו:** לכל רשומה בפנקס יש שדה `sources` (מקורות) ושדות

ייחוס (`agent` , `model`). בספר יש הערות שוליים. והסוכן שבודק את הספרות רץ **ראשון** בכל מטרה, לפני כל ניסיון.

ג. למד לשונך לומר "איני יודע"

"למד לשונך לומר איני יודע ותתמה" (ברכות ד ע"א). הגמרא הכניסה את חוסר הוודאות לתוך הטקסט המקודש: "תיקו", "צריך עיון", "לא איפשטא". שאלה שלא הוכרעה נשארת פתוחה, בגלוי.

הבעיה שזה פותר: מערכות AI כמעט לא אומרות "אני לא יודע". הן ממלאות את החסר במשהו שנשמע סביר. **איך זה כתוב אצלנו:** `teiku` - הוא אחד מסוגי התוצאות בפנקס, כשר כמו "הצלחה". - כל עובדה שלא אומתה מסומנת **[לא אומת]**. - המבקר מחפש שפת הייפ ("הוכחנו", "לראשונה בעולם") ברשומות שמתחת ל-L3.

רמות הטענה עצמן (L0 עד L4) הן סולם של מדרגות ודאות, בדומה (כמשל) למדרגות שההלכה מכירה: סברא, רוב, ראיה וודאי.

ד. או חברותא או מיתותא

"או חברותא או מיתותא" (תענית כג ע"א). לימוד לבד הוא סכנה, כי אין מי שיעצור אותך כשאתה טועה.

הבעיה שזה פותר: התקלה החמורה ביותר בפורמליזציה היא לא הוכחה שגויה. Lean תופס את אלה. התקלה החמורה היא **הוכחה מושלמת של המשפט הלא נכון**: טעות של אחד, כמת הפוך, הנחה ריקה. רק עין שנייה תופסת את זה. **איך זה כתוב אצלנו:** - יש סוכן חדש, `makshan-agent` (המקשן). תפקידו להתקיף ולא להגן. - לכל מטרה נוסף שלב `4-chavruta` לפני הפרסום. - כל תוצאה ברמה L2 ומעלה צריכה שדה `review` עם הקושיות והתשובות.

הביקורת הראשונה שהרצנו מצאה מיד חמש תוצאות של G050 בלי מקשן רשום. **זה לא כישלון, זו בדיוק המטרה של הכלל.** עכשיו יודעים מה צריך לתקן.

ה. עבד ולא אדון

"הבה נבנה לנו עיר ומגדל וראשו בשמים, ונעשה לנו שם" (בראשית יא, ד). דור הפלגה לא נענש על הטכנולוגיה. הוא נענש על המטרה: לבנות כדי לעשות לעצמם שם.

הבעיה שזה פותר: מערכת אוטונומית שמפרסמת, שולחת ופורסת בעצמה. וגם הפיתוי המוכר: להכריז על פריצת דרך כדי למשוך תשומת לב. **איך זה כתוב אצלנו:** - בקוד הפנקס זה כלל **חוסם**: פעולה כלפי חוץ

(`deploy` , `send` , `publish`) בלי אדם שאישר אותה בשמו (`approved_by`) לא נרשמת. - הסוכן המפרסם רק מנסח טיוטות. - אריאל מאשר.

חמישה רבדים, חמישה יסודות

רובד	איך היסודות חיים בו
הנחיות הסוכנים	כל סוכן מקבל את חמשת היסודות בהנחיית המערכת שלו
הפנקס	שני כללים חוסמים ושלושה שדות חדשים: `sources` , `review` , `approved_by` , ותוצאה מסוג `teiku`
המתזמר	פקודת `audit` -- מדווחת על כל רשומה שלא עומדת ביסודות
האתר והספר	"מוכח" רק מ-L3, מקורות לכל טענה, סימוני [לא אומת]
האנשים	שואלים תמיד "מה הקושיא?", ואדם מאשר כל יציאה החוצה

בקצרה

- חמישה יסודות ממקורות יהודיים, וכל אחד מהם פותר בעיה ממשית של AI.
- **לא בשמים היא:** ראיה ולא סמכות.
- **בשם אומרו:** מקור לכל טענה.
- **איני יודע:** "תיקו" כשר.
- **חברותא:** מקשן לכל תוצאה.
- **עבד ולא אדון:** אדם מחליט.
- שני כללים חוסמים בקוד, והשאר מבוקרים ב- `audit` --.

רעיונות לתוכן

1. **"תנורו של עכנאי ומודלי השפה":** למה חכמי התלמוד לא היו מקבלים את ChatGPT כפוסק. כתבת דעה לקהל רחב.

2. **"תיקו: המילה החסרה של הבינה המלאכותית":** פוסט קצר על הכוח של "איני יודע", עם דוגמאות.
3. **"החברותא הדיגיטלית":** סרטון קצר שמראה סוכן מקשה על סוכן אחר, ו-Lean מכריע ביניהם.
4. **"מגדל בבל בעידן ה-AI":** מאמר על אחריות האדם על טכנולוגיה, לקהל דתי ומסורתי.
5. **"חמשה כללים מהגמרא שכל מפתח AI צריך להכיר":** כתבה בסגנון רשימה, וטובה לשיתוף.

פרק 7: בנק המטרות

מעבדה שתוקפת "בעיות פתוחות" צריכה קודם כל להחליט אילו בעיות. ההחלטה הזו חשובה יותר ממה שנדמה. פרק 4 הראה שההצלחות של AI במתמטיקה הגיעו כמעט תמיד מבעיות עם מוודא ברור, מבעיות בנייה עם ניקוד, ומ"הזנב הארוך" של בעיות מוזנחות, ולא מהבעיות המפורסמות ביותר. מעבדה שתבחר את השערת רימן כמטרה ראשונה תבזבז את זמנה, ותסתכן בהכרזות שיפגעו באמינותה. **בנק המטרות** הוא הכלי שבו המעבדה מתרגמת את הלקחים האלה להחלטות: רשימה של 77 בעיות, מסודרות בשכבות, עם ציונים שמאפשרים למיין אותן, ועם כללי עבודה שקובעים מי מעדכן מה ומתי. הפרק הזה מסביר איך הבנק בנוי, איך מחושב הדירוג בפועל, איך הנתונים זורמים מקובץ CSV לאתר, ומהן שלוש המטרות שבהן המעבדה מתחילה.

קודם כל, בכנות

`docs/goal-bank.md` נפתח בשלוש הצהרות, וכדאי להתחיל בהן גם כאן:

- **שכבה A, ה"מונשוטים", נועדה ללמידה ולהשראה, לא לעבודה שוטפת.** המעבדה לא תגיע לפריצה בהשערת רימן, ב-P מול NP או בנאוויה-סטוקס בזכות AI ו-Lean בשנה הקרובה. בשורות האלה שומרים תמונת מצב עדכנית וכנה, ומתרגלים בעזרתן פורמליזציה של טענות והגדרות.
- **את הניצחונות הראשונים צפויות להביא שכבות C ו-D.** זה הדפוס של 2025–2026: שיאים בבניות קומבינטוריות (FunSearch ו-AlphaEvolve), סגירת בעיות ארדש (לפעמים בזכות חיפוש ספרות טוב, לפעמים בהוכחה חדשה), ופורמליזציה של תוצאות קיימות.
- **לא מכריזים על פתרון בלי תעודה.** תעודה היא הוכחה ב-Lean בלי `sorry` , certificate שאפשר לבדוק (צביעה, גרף, קונפיגורציה בקוואורדינטות מדויקות), או בדיקת ספרות מתועדת. החוק הזה חל במיוחד על טענות שמגיעות ממודלי שפה.

חמש השכבות

כל מטרה בבנק שייכת לאחת מחמש שכבות (tiers). השכבה קובעת מה נחשב ניצחון.

שכבה	שם	מה נכנס	מה נחשב ניצחון	מספר מטרות
A-Moonshot	בעיות המילניום ושות'	רימן, P מול NP, נאוויה-סטוקס,	פורמליזציה של ניסוחים ושל	10

שכבה	שם	מה נכנס	מה נחשב ניצחון	מספר מטרות
		יאנג-מילס, הודג׳, BSD, גולדבך, ראשוניים תאומים, קולץ, abc	תוצאות חלקיות ידועות; תמונת מצב מדויקת	
B-Famous-open	בעיות פתוחות מפורסמות ומקומיות יותר	union-closed, lonely runner, Hadwiger, Erdős–Hajnal, sunflower, מרחק היחידה	התקדמות חלקית, מקרה פרטי, או פורמליזציה של הוכחה חלקית	12
C-Tractable	בעיות שבהן חיפוש בעזרת AI כבר הניב תוצאות, או סביר שיניב	בעיות ארדש, מספרי Ramsey, ו-van der Waerden Schur, cap sets, מספרי נשיקה, Hadwiger–Nelson, אריזות, השערות מ-OEIS	חסם חדש עם certificate, או שינוי סטטוס שהקהילה מאשרת	27
D-Formalization	משפטים שכבר הוכחו ועדיין אינם ב-Lean/mathlib	Ramsey/Schur קטנים דרך SAT, בעיות ארדש שנפתרו לאחרונה, Gilmer, Kuratowski, Sylvester–Gallai	הוכחה ב-Lean 4 בלי `sorry` , ועדיף שתתקבל ל-mathlib או לפרויקט קהילתי	17
P-Physics	בעיות פתוחות בפיזיקה תיאורטית עם ניסוח מתמטי	התפוצצות באוילר, 3D Ising bootstrap, הבעיה השישית של סמייל, Bell/I3322, Lieb–Thirring, PhysLean	חסם נומרי מוכח (אריתמטיקת אינטרוולים או SDP), או תרומה ל-PhysLean	11

הסיכום: 77 מטרות, והמספרים בטבלה תואמים את הקובץ `data/goals.csv` נכון ל-4 בנובמבר 2026.

השכבה הגדולה ביותר היא C, וזה לא מקרי. אלה בעיות שבהן יש פונקציית ניקוד (גודל של קבוצה, חסם תחתון, מספר נקודות), ולכן הן מתאימות ללולאות חיפוש עם מוודא. השכבה השנייה בגודלה, D, היא העבודה ה"שקטה": לקחת מתמטיקה שכבר הוכחה ולהפוך אותה לבדוקה במכונה. זו עבודה שאינה מייצרת כותרות, אבל היא מייצרת תוצרים אמיתיים, בונה תשתית, ומלמדת את הצוות ואת הסוכנים לעבוד עם Lean.

השדות

כל שורה בקובץ כוללת 16 עמודות. רובן תיאוריות (`title`, `title_he`, `field`, `statement_short`, `why_it_matters`, `known_partial_results`, `suggested_attack`, `references`), וכמה מהן קובעות איך המטרה מתנהגת במערכת:

- **`ai_tractability` (1–5):** הסיכוי שצוות קטן עם כלי AI (חיפוש, מודלי שפה, SAT, אופטימיזציה) יגיע לתוצאה **חדשה** בטווח של חודשים. 5 פירושו שכבר היו הצלחות דומות והתוצאה ניתנת לבדיקה אוטומטית. 1 פירושו שאין שום מסלול נראה לעין.
- **`formalization_readiness` (1–5):** עד כמה התשתית ב-Lean ו-mathlib מוכנה. 5 פירושו שכל ההגדרות קיימות והפורמליזציה היא עבודה שגרתית. 1 פירושו שחסרות תיאוריות שלמות (לדוגמה, תורת שדות קוונטית קונסטרוקטיבית).
- **`status`:** אחד מארבעה ערכים: `open`, `partially-solved`, `solved-elsewhere`. כרגע: 32 פתוחות, 27 נפתרו חלקית, 17 נפתרו במקום אחר (`formalize`), `disputed`. וממתינות לפורמליזציה, ואחת שנויה במחלוקת (השערת abc). כשהמצב לא ודאי, למשל טענה טרייה או preprint שעוד לא נבדק, הדבר נאמר במפורש בעמודה `known_partial_results`, במילים כמו "verify" או "reported".
- **`owner_agent`:** הסוכן האחראי על השורה. ההתפלגות כרגע: lean-agent אחראי על 24 מטרות, search-agent ו-construction-agent על 14 כל אחד, literature-agent על 11, physics-agent על 10, ו-analogy-agent על 4.
- **`progress_pct` ו-`last_update`:** כרגע 0 בכל השורות, עם תאריך 2026-11-04. מעדכנים בכל פעם שמשהו זז.

איך מחושב הדירוג

בגרסה הראשונה של `docs/goal-bank.md` הופיעה נוסחה חיבורית אחרת (`2T + R + bonus`). היא אוחדה עם הקוד, וזו הנוסחה היחידה כעת.

ה-orchestrator ממשש ב- `orchestrator/goals.py` נוסחה **כפלית**, שגם מתוארת ב- `docs/architecture.md` :

$$\text{score} = T \cdot R \cdot w_{\text{tier}} \cdot \max(0.05,\ 1 - p)$$

כאשר:

- T ו-R הם `ai_tractability` ו- `formalization_readiness` , **מנורמלים לטווח 0–1**. ציון בסולם 1–5 מחולק ב-5, כך ש-5 הופך ל-1.0, 4 ל-0.8, ו-1 ל-0.2. הפונקציה `norm01` סובלנית: היא מקבלת גם סולמות 1–10 ו-0–100, שברים כמו `0.7` , ומילים כמו `low` , `medium` ו- `high` (שממופות ל-0.25, 0.5 ו-0.8). ערך חסר או לא קריא מקבל 0.3.
- w_{tier} הוא משקל השכבה: **A = 0.6, B = 0.8, C = 1.1, D = 1.0, P = 0.9**. השכבה C מועדפת מעט על D, ושכבה A מקבלת משקל נמוך אבל לא אפס. ההערה בקוד מסבירה: "אנחנו מעדיפים מעט שכבות נמוכות (ברות-השגה יותר, משוב מהיר יותר), אבל לעולם לא מאפסים את שכבת הדגל".
- p הוא `progress_pct` מחולק ב-100, כי זה תמיד אחוז בין 0 ל-100, בדיוק כמו באתר ובאקסל. הגורם p – 1 מוריד את העדיפות של מטרות שכבר התקדמו, אבל לעולם לא מתחת ל-0.05.

מטרה נחשבת "פעילה" אם הסטטוס שלה אינו אחד מהערכים הסגורים (`solved` , `done` , `closed` , `verified` , `published` , `archived` , `l3` , `l4`). ארבעת הסטטוסים שבשימוש בבנק אינם ביניהם, ולכן כל 77 המטרות נכנסות כרגע לדירוג. בתיקו, המיון הוא לפי המזהה.

דוגמאות מחושבות

מטרה	T	R	שכבה	ציון (קוד)	מקום
G024 תרומות ל-formal-conjectures	1.0 → 5	1.0 → 5	C (1.1)	**1.100**	1
G060 משפט Sylvester–Gallai	1.0 → 5	1.0 → 5	D (1.0)	1.000	2
G023 סריקת בעיות ארדש	1.0 → 5	0.8 → 4	C (1.1)	0.880	3

מטרה	T	R	שכבה	ציון (קוד)	מקום
G050 מספרי Ramsey קטנים ב-Lean	1.0 → 5	0.8 → 4	D (1.0)	0.800	4
G052 בעיות ארדש שנפתרו, ב-Lean	1.0 → 5	0.8 → 4	D (1.0)	0.800	5
G029 van der Waerden: חסמים תחתונים	0.8 → 4	0.8 → 4	C (1.1)	0.704	7
G001 השערת רימן	0.2 → 1	0.6 → 3	A (0.6)	0.072	64

הציונים והמקומות בעמודת הקוד חושבו בהרצה של `python -m orchestrator.run --list` על הבנק הנוכחי. כפי שרואים, בראש הרשימה שתי הנוסחאות מסכימות כמעט לגמרי. ההבדלים מופיעים באמצע ובתחתית. למשל, G077 (תרומות ל-PhysLean) נמצא במקום 13 בנוסחה הכפלית ובמקום 23 בערך בנוסחה החיבורית, כי הנוסחה הכפלית לא "מענישה" את שכבת הפיזיקה באותה מידה.

יש גם הבדל עקרוני. בנוסחה הכפלית, ציון נמוך באחד הממדים מוריד את כל הציון: מטרה עם סיכוי מצוין ל-AI אבל בלי שום תשתית ב-Lean לא תגיע לראש הרשימה. בנוסחה החיבורית, ציון גבוה בממד אחד יכול לפצות על ממד אחר. הנוסחה הכפלית משקפת טוב יותר את הלקח "אם אין מוודא, אין תוצאה".

מה קובע? הקוד הוא מה שבוחר בפועל את המטרה הבאה, ולכן לצורכי הספר הנוסחה הכפלית היא הנוסחה המחייבת. את `docs/goal-bank.md` כדאי לעדכן כך שיתאר אותה, או לפחות יציין שהנוסחה החיבורית היא חלופה ידנית.

איך הנתונים זורמים

לבנק המטרות יש מקור אמת אחד, ומספר תצוגות.

מקור האמת הוא `data/goals.csv`. כל שינוי נעשה שם, בקובץ טקסט שנשמר במאגר git, כך שלכל שינוי יש היסטוריה, מחבר ותאריך.

התצוגות נבנות ממנו:

1. **קובץ אקסל.** הסקריפט `scripts/build_workbook.py` בונה את `ESH27-Lab.xlsx` : גיליון לכל בנק (מטרות, הוכחות יצירתיות, כלים, פרויקטי חוכמה) ולוח בקרה (Dashboard) עם נוסחאות חיות שסופרות מטרות לפי שכבה ומחשבות התקדמות ממוצעת.
2. **Google Sheet.** מעקב חי ב-Google Sheets, עם תצוגה מקוצרת של הבנק. הקישור מופיע ב-README הראשי של המעבדה.
3. **האתר.** `site/build.py` קורא את קובצי ה-CSV ובונה דף סטטי אחד, דו-לשוני, עם הנתונים מוטמעים בתוכו.

```
python scripts/build_workbook.py   # → ESH27-Lab.xlsx
python site/build.py               # → site/dist/index.html
python -m orchestrator.run --next  # מה המטרה הבאה?
```

הכלל: **לא עורכים את התצוגות.** שינוי שנעשה ישירות באקסל או ב-Google Sheet יאבד בבנייה הבאה. אם רוצים לשנות, משנים את ה-CSV ובונים מחדש.

איך מוסיפים או מעדכנים מטרה

`docs/goal-bank.md` קובע ארבעה כללים:

1. **כל סוכן מעדכן רק את השורות שלו**, לפי `owner_agent` , ומשנה את `progress_pct` ואת `last_update` .
2. **שורה חדשה צריכה הפניה אמיתית.** לא ממציאים מספרי בעיות או מזהי arXiv. אם יש ספק, מתארים את הבעיה במילים.
3. **סטטוס מתיישן.** ה-literature-agent עובר על כל שורות C ו-D אחת לחודש. 2026 היא שנה של שינויים מהירים, ושורה שהייתה נכונה לפני חצי שנה עלולה להטעות היום.
4. **שורות A לא מקבלות משאבי חישוב משמעותיים** בלי החלטה מפורשת של הצוות.

ועוד כלל שחוזר בכל המסמכים: **לפני שמתחילים לעבוד על שורה, ה-literature-agent חייב לאמת את המצב שלה.**

סיור בבנק

הנה מבחר מטרות מכל שכבה, כדי לתת תחושה של מה יש בבנק. התיאורים מבוססים על העמודות `statement_short` ו- `known_partial_results` בקובץ.

שכבה A: ללמוד מהגדולות

- **השערת קולץ (G009).** כל מספר חיובי מגיע ל-1 תחת האיטרציה $n \mapsto n/2$ (זוגי) או $n \mapsto 3n+1$ (אי-זוגי). Tao הוכיח ב-2019 ש"כמעט כל" המסלולים מגיעים לערכים כמעט חסומים, והבדיקה החישובית הגיעה לסביבות 2^{70}. ההנחיה בבנק חד-משמעית: "Learning only; do NOT spend compute on brute-force verification." הניסוח ב-Lean כבר קיים ב-`lean/EshLab/Targets/Collatz.lean` (ראו פרק 5). הסוכן האחראי הוא דווקא ה-analogy-agent: בבעיה שבה אין מסלול נראה לעין, ניסוחים מחדש הם מה שיש לתרום.
- **השערת גולדבך (G007).** כל זוגי גדול מ-2 הוא סכום של שני ראשוניים. הגרסה החלשה (שלושה ראשוניים) הוכחה על ידי Helfgott ב-2013, והגרסה החזקה נבדקה עד $4 \cdot 10^{18}$. גם לה יש קובץ יעד, `Targets/Goldbach.lean`. הבנק מציין שההוכחה של Helfgott היא מועמדת (גדולה מאוד) לפורמליזציה.
- **השערת abc (G010).** המטרה היחידה בסטטוס `disputed`. ההוכחה שטען לה Mochizuki (2012, פורסמה ב-2021) אינה מקובלת על רוב המומחים אחרי ההתנגדות של Scholze ו-Stix. המשימה כאן היא שמירה על הערת מצב כנה, לא יותר.

שכבה B: מפורסמות, אבל עם פתחים

- **השערת הקבוצות הסגורות לאיחוד של Frankl (G011).** בכל משפחה סופית סגורה לאיחוד יש איבר שמופיע בלפחות חצי מהקבוצות. ב-2022 הוכיח Justin Gilmer חסם של שבר קבוע, ובעקבותיו שופר החסם לכ-0.38. ה-1/2 עדיין פתוח. ההתקפה המוצעת מתפצלת: לפרמל את טיעון האנטרופיה (מטרה נפרדת בשכבה D, G055), ולחפש משפחות קטנות עם התנהגות קיצונית.
- **השערת הרץ הבודד (G012).** הוכחה עד 7 רצים. preprints מ-2025 מדווחים על הרחבה ל-8, 9 ו-10 רצים בעזרת מחשב, והבנק מסמן במפורש "verify status". אם התוצאות תקפות, הרדוקציה הסופית והתעודה הן מועמדות טובות לפורמליזציה.
- **בעיית מרחק היחידה (G020).** כמה זוגות במרחק 1 יכולים להיות בין n נקודות במישור? הבנק מתעד שב-2026 בנייה של OpenAI עקפה את ההשערה ש-$n^{1+o(1)}$ הוא הסדר הנכון, ושהשיפור נדחף בהמשך. הפרטים המספריים מסומנים בבנק כטעוני אימות ([לא אומת]). יש לה גם מטרת פורמליזציה מקבילה בשכבה D (G053).

שכבה C: היכן שהחיפוש עובד

- **סריקת בעיות ארדש (G023) ותרומות ל-formal-conjectures (G024).** שתי המטרות המדורגות הגבוהות ביותר. הראשונה: לסווג בעיות מ-erdosproblems.com לפי "נפתרה בספרות", "מתאימה ל-AI", "בעיית בנייה" או "עמוקה". השנייה: לתרום ניסוחי Lean נכונים של השערות פתוחות למאגר formal-conjectures של Google DeepMind. שתיהן הן הלקחים של פרשת ארדש בפעולה.

- **Cap sets (G030).** קבוצות ב-$\mathbb{F}_3^n$ בלי *שלוש* נקודות על *ישר*. הגודל המקסימלי ידוע עד $n = 6$ (112). FunSearch מצא קבוצה בגודל 512 ב-$n = 8$. המטרה: *לשחזר* את FunSearch עם מודלים פתוחים, ולתקוף את $n = 7$.
- **מספרי נשיקה (G031).** כמה כדורים זהים יכולים לגעת בכדור מרכזי? הערך המדויק ידוע רק בממדים 1, 2, 3, 4, 8 ו-24. ב-2025–2026 השתנו *שיאים* בכמה ממדים, והבנק מזהיר: "pull current table before claiming".
- **Hadwiger–Nelson (G032).** מספר הצביעה *של* המישור הוא 5, 6 או 7 (de Grey, 2018). הגרף הקטן ביותר הידוע *שמוכיח* 5 הוא בן 509 קודקודים. גרף קטן יותר הוא *שיפור*; 6 יהיה תוצאה גדולה.
- **R(3,3,3,3) (G027).** מספר Ramsey לארבעה צבעים ומשולשים, עם $51 \le R(3,3,3,3) \le 62$. החסם התחתון 51 הוא מ-1973. צביעה *של* K_{51} בלי *משולש* חד-צבעי הייתה מעלה אותו ל-52.
- **BB(6) (G048).** להכריע עצירה *של* מכונות טיורינג בנות 6 מצבים. BB(5) הוכרע ב-2024; בין המכונות הפתוחות *של* BB(6) *יש* "קריפטידים" דמויי-קולץ.

שכבה D: לבנות תשתית

- **Sylvester–Gallai (G060).** לכל קבוצה סופית *של* נקודות במישור, *שלא* כולן על *ישר* אחד, *יש ישר שעובר* בדיוק בשתיים מהן. ציונים 5 ו-5, ומקום *שני* בדירוג: הוכחה קצרה (*של* Kelly), ותשתית גאומטריה אוקלידית קיימת ב-mathlib.
- **חסם Ellenberg–Gijswijt ל-cap sets (G057).** קיימת הוכחה ב-Lean 3; המשימה היא לבדוק אם הועברה ל-Lean 4, ואם לא, להעביר ולהגיש ל-mathlib. "PR ראשון טוב ל-mathlib".
- **BB(5) ב-Lean (G062).** הוכחת BB(5) = 47,176,870 קיימת ב-Coq; המשימה היא להעביר חלקים ממנה ל-Lean 4.
- **פרויקט FLT (G063).** השורה מנוסחת כ"לקחת צמתים פתוחים ב-blueprint *של* פרויקט FLT *של* Kevin Buzzard", ומתארת את הפרויקט כ"מתמשך מאז 2024". אבל לפי מה *שתואר* בפרק 4, בספטמבר 2026 Anthropic דיווחה על השלמת הפורמליזציה כולה. השורה הזו כנראה מתיישנת, וה-literature-agent צריך לבדוק אותה.
- **דוגמה נגדית להשערת הדרגש (G066).** אימות פורמלי *של* הדוגמה הנגדית מ-2024. הפער ההסתברותי בה זעיר, ולכן השאלה הראשונה היא אם חישוב רציונלי מדויק בכלל אפשרי.

שכבה P: פיזיקה עם מוודא

- **התפוצצות באוילר ובנאוויה-סטוקס (G067).** Chen ו-Hou הוכיחו ב-2022–2023, בעזרת מחשב, התפוצצות בזמן סופי לאוילר התלת-ממדי עם *שפה*. ההצעה: לשחזר פרופילים דומי-עצמם

במשוואות מודל, וללמוד אימות באריתמטיקת אינטרוולים.

- **3D Ising (G068).** חסמים על אקספוננטים קריטיים מ-conformal bootstrap. זו הדוגמה שבה `docs/architecture.md` מסביר איך פתרון נומרי של SDP הופך לתעודה רציונלית.
- **הבעיה השישית של סמייל (G069).** האם מספר הקונפיגורציות המרכזיות בבעיית n הגופים סופי? הוכח ל-$n = 4$ ול-$n = 5$ במסות גנריות. $n \geq 6$ פתוח.
- **הפרה קוונטית של אי-שוויוני בל (G072).** הערך המקסימלי של I3322 (בערך 0.25087556) ידוע מקירוב בממדים גבוהים, אבל הערך המדויק, והשאלה אם ממד סופי מספיק, פתוחים.
- **תרומות ל-PhysLean (G077).** המטרה המדורגת הגבוהה בשכבה (מקום 13 בסך הכול): לקחת משימות פתוחות מרשימת ה-TODO של PhysLean.

שלוש המטרות הראשונות

מתוך 77 המטרות, `docs/goal-bank.md` בוחר שלוש להתחלה. הבחירה נשענת על שלושה עקרונות: **הצלחה מהירה** (כל מטרה נותנת תוצר אמיתי תוך שבועיים), **בניית תשתית** (כל מטרה בונה צינור עבודה שגם המטרות הבאות ישתמשו בו), ו**פרסום** (לכל מטרה יש ערוץ פרסום מוגדר). שלושתן נמצאות בעשירייה הראשונה של הדירוג האוטומטי.

מטרה 1: G050, מספרי Schur ו-van der Waerden קטנים ב-Lean

שכבה D. lean-agent ו-search-agent.

הרעיון: להוכיח ב-Lean 4, בלי `sorry` , ערכים מדויקים שידועים מזמן, כמו $S(4) = 44$, $W(3,3) = 27$ ו-$W(2,4) = 35$. $S(4)$ הוא מספר Schur: המספר הגדול ביותר n כך שאפשר לצבוע את $1, \ldots, n$ בארבעה צבעים בלי פתרון חד-צבעי של $x + y = z$. הערכים עצמם אינם חדשים. מה שחדש הוא הצינור: SAT ← תעודת LRAT ← בדיקה בתוך Lean.

התוכנית, לפי ימים: - **ימים 1–2:** ה-literature-agent בודק אם הערכים האלה כבר הוכחו ב-Lean, ב-Isabelle או ב-HOL4. אם כן, עוברים ל-$W(2,5) = 178$ או ל-$R(3,3,3) = 17$. בנוסף לומדים את הצינור ששימש בפרויקט ה-empty hexagon (Lean, 2024). - **ימים 3–6:** כותבים קידוד ל-CNF עם שבירת סימטריה שאפשר לנמק. מריצים CaDiCaL או Kissat עם פלט LRAT. בצד ה-Lean מנסחים את הטענה המתמטית ומחברים אותה ל-CNF. - **ימים 7–11:** מוכיחים ב-Lean שהקידוד נכון. את החסם התחתון מוכיחים בהצגת צביעה מפורשת שנבדקת עם `decide` , אותו מנגנון שראינו ב-`sumOdd_ten` בפרק 5. - **ימים 12–14:** ניקוי, README, מדידת ביצועים, ו-PR (ל-Archive של mathlib, או למאגר עצמאי).

מה נחשב ניצחון: משפט ב-Lean, שה-kernel בדק, ש-$S(4) = 44$ (ולפחות עוד ערך אחד), עם צינור SAT←Lean שאפשר להשתמש בו שוב. **פרסום:** מאגר ציבורי, פוסט ב-Lean Zulip ובבלוג, ואם זה מתרחב, הגשה לסדנה בכנס ITP או SAT.

מטרה 2: G023 ו-G052, סריקת בעיות ארדש ופורמליזציה של מה שנפתר

שכבות C ו-D. literature-agent ו-lean-agent.

הרעיון: לסווג בעיות פתוחות מ-erdosproblems.com, לחפש פתרונות שכבר קיימים בספרות, ולפרמל בעיות שנפתרו ב-2025–2026 ועדיין אין להן הוכחה ב-Lean.

- **ימים 1–3:** ה-literature-agent בונה טבלה של כ-40 בעיות, עם ניסוח, סטטוס באתר, סטטוס בוויקי של תרומות AI, האם הבעיה מנוסחת ב-formal-conjectures, ואורך ההוכחה אם יש.
- **ימים 4–7:** בוחרים 2–3 בעיות שנפתרו, שאין להן Lean, וששההוכחות שלהן קצרות (למשל #728, אחרי שמוודאים את הסטטוס). ה-lean-agent כותב blueprint.
- **ימים 8–12:** פורמליזציה. במקביל, ה-literature-agent מחפש ספרות לבעיות "פתוחות" שאולי כבר נפתרו.
- **ימים 13–14:** PR ל-formal-conjectures, ודיווח למנהלי erdosproblems.com עם הפניות מדויקות.

מה נחשב ניצחון: לפחות הוכחה אחת ב-Lean של פתרון לבעיית ארדש שלא הייתה לה פורמליזציה, **או** שינוי סטטוס אחד שמאושר באתר, ובנוסף לפחות חמישה ניסוחים חדשים שמתקבלים ל-formal-conjectures. **פרסום:** PRs ציבוריים, קרדיט בוויקי, ופוסט מסכם. וחשוב: ייחוס כן של מה שעשה AI ומה שעשה אדם.

מטרה 3: G029 ואחריה G027, חסמים תחתונים חדשים בחיפוש בניות

שכבה C. construction-agent ו-search-agent.

הרעיון: לשפר רשומה בטבלה ציבורית של חסמים תחתונים למספרי van der Waerden, או ל-$R(3,3,3,3)$. זו בעיה בדיוק בסגנון FunSearch ו-AlphaEvolve: מחפשים **תוכניות** שמייצרות צביעות (צביעות ציקליות, zipping, "power-residue"), ולא צביעות בודדות.

- **ימים 1–2:** אוספים את הרשומות הנוכחיות (טבלאות Rabung–Lotts ו-Monroe, והסקר הדינמי של Radziszowski). כותבים verifier מהיר ומשחזרים 3–5 רשומות ידועות. לפני שמחפשים משהו חדש, מוכיחים שהמוודא מזהה נכון את הישן.
- **ימים 3–8:** מריצים לולאה אבולוציונית שבה מודל שפה כותב generator, מריצים אותו ומקבלים ציון, עם מודל פתוח (בסגנון OpenEvolve). במקביל, חיפוש מקומי או SAT על מבנים ציקליים בגדלים שליד השיא.

- **ימים 9–12:** מכוונים ל-2–3 תאים בטבלה שהשיא שלהם הכי ישן, או שהפער בהם הכי גדול. כל שיפור עובר אימות כפול: verifier עצמאי, ובדיקה ב-Lean עם `decide` אם הגודל מאפשר זאת. זה הצינור ממטרה 1.
- **ימים 13–14:** כתיבה. אם אין שיפור, מפרסמים דו"ח שלילי שמתעד מה נוסה ואיפה זה נעצר.

מה נחשב ניצחון: שיפור של חסם תחתון אחד לפחות בטבלה מוכרת, עם certificate שאפשר להוריד, אחרי שמוודאים שהשיפור לא פורסם כבר במקום אחר. **פרסום:** פתק קצר ב-arXiv (math.CO) עם הצביעות, עדכון למתחזקי הטבלאות, ופוסט "AI-found construction" עם קוד פתוח מלא.

מה שלוש המטרות מלמדות יחד

שלוש המטרות מכסות שלושה צינורות שונים, וכל אחד מהם משמש אחר כך מטרות רבות:

- **Lean ← SAT** (מטרה 1): משמש את G054 ($R(4,5) = 25$), G051 (גרף מרחק-יחידה 5-צביעתי) ובעיות נוספות בשכבות C ו-D.
- **Lean ← ספרות** (מטרה 2): הצינור הבסיסי של כל שכבה D, ושל הבדיקה החודשית של שכבה C.
- **certificate ← חיפוש** (מטרה 3): משמש את כל בעיות הבנייה, מ-cap sets ועד מספרי נשיקה.

ושלושתן כוללות תוכנית לכישלון. מטרה 1 עוברת לערך אחר אם הקיים כבר פורמל. מטרה 2 מוגדרת כהצלחה גם אם רק סטטוס אחד השתנה. מטרה 3 מתחייבת מראש לפרסם דו"ח שלילי. זו אולי התכונה החשובה ביותר שלהן: הן מתוכננות כך שגם התוצאה הגרועה מייצרת משהו בעל ערך.

בקצרה

- בנק המטרות כולל 77 בעיות בחמש שכבות: A מונשוטים (10), B מפורסמות (12), C ברות-השגה (27), D פורמליזציה (17), P פיזיקה (11).
- כל מטרה מקבלת ציונים של "סיכוי ל-AI" ו"מוכנות לפורמליזציה" (1–5), סטטוס וסוכן אחראי.
- הדירוג שהקוד מממש בפועל הוא כפלי: $T \cdot R \cdot w_{\text{tier}} \cdot \max(0.05, 1-p)$, עם משקלי שכבות A=0.6, B=0.8, C=1.1, D=1.0, P=0.9. המסמך `docs/goal-bank.md` מתאר נוסחה חיבורית שונה, וכדאי ליישר ביניהם.
- `data/goals.csv` הוא מקור האמת. האקסל, ה-Google Sheet והאתר נבנים ממנו ואינם נערכים ישירות.
- שלוש מטרות הפתיחה (G050, G023+G052, G029) בונות שלושה צינורות: SAT←Lean, ספרות←Lean, חיפוש←certificate, וכל אחת מתוכננת כך שגם כישלון מניב תוצר.

רעיונות לתוכן

1. **"77 בעיות, 5 שכבות": מפה אינטראקטיבית של בנק המטרות.** זווית: כל בעיה כנקודה על גרף של "סיכוי ל-AI" מול "מוכנות ל-Lean", בצבע לפי שכבה. קהל יעד: מבקרי האתר, סטודנטים.
2. **"למה אנחנו לא תוקפים את השערת רימן".** זווית: כתבה שמסבירה את הבחירה המודעת בזנב הארוך, עם הנוסחה ועם ציון 0.072. קהל יעד: קוראי S27 News מהקהל הרחב.
3. **יומן מסע: שבועיים עם G050.** זווית: סדרת פוסטים יומית על המטרה הראשונה, כולל המבוי הסתום. קהל יעד: קהילת Lean וחובבי מתמטיקה.
4. **"מספר Schur ב-60 שניות".** זווית: סרטון קצר שמסביר מה זה $S(4) = 44$ עם צביעה ויזואלית של המספרים 1 עד 44. קהל יעד: רשתות חברתיות, תלמידי תיכון.
5. **"הבעיה שנפתרה בזמן שכתבנו עליה".** זווית: מקרה G063 (FLT) כדוגמה לכמה מהר בנק מטרות מתיישן, ולמה צריך סקירה חודשית. קהל יעד: קוראים שמתעניינים בתהליכי מחקר.
6. **תבנית פתוחה: "בנק מטרות למעבדה שלכם".** זווית: פוסט שמשתף את מבנה ה-CSV ואת נוסחת הדירוג לשימוש חוזר. קהל יעד: צוותי מחקר וקבוצות סטודנטים.

פרק 8: ארגז הכלים

מעבדה היא גם רעיונות וגם כלים. הפרקים הקודמים תיארו את הרעיונות: גרעין אמון, רמות טענה, בנק מטרות. הפרק הזה עוסק בכלים שממממשים אותם. הקטלוג של המעבדה, `data/tools.csv` , מונה 140 כלים ב-19 קטגוריות: עוזרי הוכחות, מוכיחי AI, פותרי SAT, מערכות אלגברה ממוחשבת, ספריות נומריות, כלי פיזיקה, מנועי חיפוש ספרות, ואפילו מאגרי טקסטים עבריים. אף אחד לא צריך להכיר את כולם. מה שחשוב להבין הוא שלושה דברים: אילו סוגי כלים יש ולמה כל סוג קיים, מה מתקינים ביום הראשון, ואיך כלים מתחברים לצינורות שבהם כלי לא אמין מייצר וכלי אמין בודק. הנקודה האחרונה היא לב הפרק, כי היא המקום שבו העיקרון "אימות גובר על יצירה" הופך מסיסמה לפקודות שמריצים.

איך הקטלוג בנוי

לכל כלי בקובץ יש שמונה עמודות: מזהה (T001 עד T140), שם, קטגוריה, מה הכלי עושה, איך המעבדה משתמשת בו, רישיון או עלות, בשלות (maturity), וקישור. סולם הבשלות פשוט: **1** פירושו ניסיוני או לא זמין, **3** שמיש עם חספוסים, **5** סטנדרט תעשייתי. עמודת הרישיון מגיעה עם אזהרה במסמך `docs/tools.md` : לבדוק אותה מחדש לפני כל שימוש מסחרי, כי תנאים משתנים.

שלוש הערות מהמסמך שכדאי לזכור:

- **AlphaProof** של Google DeepMind אינו זמין לציבור. הוא רשום בקטלוג (T027, בשלות 2) לצורך הקשר בלבד.
- **FrontierMath** של Epoch AI ברובו פרטי, ואי אפשר להריץ עליו בעצמנו (T108).
- **Aristotle** של Harmonic (T026): שרת ה-API עשוי לרוץ על גרסת Lean ישנה מזו של המעבדה, והשאילתות נשמרות אצל הספק. לא שולחים אליו חומר רגיש.

סיור לפי קטגוריות

עוזרי הוכחות וספריות Lean

זו הקטגוריה שבמרכז הכול. **Lean 4** (T001) הוא יעד האימות של כל תוצאה. **elan** (T002) נועל את גרסת Lean לכל פרויקט, ו-**Lake** (T003) בונה פרויקטים ומוריד את ה-cache של mathlib. כל השלושה בשלות 5, וכולם נלמדו בפרק 5.

לצד Lean רשומים עוזרי הוכחות אחרים: Isabelle/HOL עם Sledgehammer, Rocq (לשעבר Coq), Agda, HOL Light ו-Metamath. המעבדה לא עובדת בהם, אבל קוראת בהם. ההוכחה של BB(5), למשל, קיימת ב-Coq, ומשפט ארבעת הצבעים פורמל לראשונה ב-Coq. Metamath מתואר בקטלוג כ"בדיקה צולבת באמינות-על".

הקטגוריה הגדולה ביותר בקטלוג היא **lean-library**, עם 14 כלים. בראשה (T009) **mathlib4**, "הבסיס לכל הפורמליזציות של המעבדה". לידה כלי אוטומציה: **Aesop** לסגירת מטרות שגרתיות, **Duper** ללוגיקה מסדר ראשון בתוך Lean, **polyrith** לזהויות פולינומיות, והשורה T013 שמאגדת את `grind`, `omega`, `decide` ו-`norm_num` עם ההגדרה "קו האוטומציה הראשון לפני שפונים למוכיחי AI". ויש את כלי החיפוש: **LeanSearch** (שפה טבעית), **Loogle** (לפי צורה) ו-**Moogle** (חלופה). את הכלים לעבודה עם סוכנים נפגוש בהמשך: **LeanDojo**, **Lean REPL** ו-**Kimina Lean Server**. ולבסוף (T019) **leanblueprint**, הכלי שבו מתכננים כל פורמליזציה גדולה, ו-(T022) **PhysLean**, ספריית Lean לפיזיקה, שעדיין בבשלות 2.

מוכיחי AI

שישה כלים, שהם בעצם ההיסטוריה של פרק 4 כמוצרים. **Goedel-Prover-V2**, **DeepSeek-Prover-V2** ו-**Kimina-Prover** הם מוכיחים פתוחים שאפשר להריץ על GPU של המעבדה. **Aristotle** זמין דרך API. **AlphaGeometry** מתאים לניסויים בגאומטריה בסגנון אולימפיאדה. ו-AlphaProof, כאמור, לא זמין. הקטלוג מציע להשתמש בכמה מוכיחים פתוחים יחד (ensembling): מוכיחים שונים נכשלים בבעיות שונות.

מודלי שפה ותשתית סוכנים

הקטגוריה **llm-api** (9 כלים) כוללת את המנועים שמפעילים את הסוכנים. (T029) **Claude API** הוא "מודל ההיגיון והסוכנים הראשי ללולאות המחקר", (T030) **Claude Agent SDK** משמש לתזמור הסוכנים, ו-(T031) **Claude Code** לכתיבת Lean ופייתון ולתחזוקת המאגר. **OpenAI API** ו-**Gemini API** רשומים כ"דעה שנייה" וכמקור לגיוון. **vLLM** ו-**Hugging Face Hub** נועדו להרצת המוכיחים הפתוחים על חומרה משלנו.

שני כלים בקטגוריה הזו ראויים לתשומת לב מיוחדת. **Model Context Protocol (MCP)** (T032) הוא פרוטוקול אחיד לחשיפת כלים לסוכנים: דרכו אפשר להציג ל-Claude את ה-REPL של Lean, את OEIS, את arXiv ואת פותרי ה-SAT כ"כלים" שהוא קורא להם. ו-**lean-lsp-mcp** (T033) מאפשר לסוכן לראות את מצב ההוכחה ב-Lean בזמן אמת, בדיוק כמו חלון ה-infoview שתיארנו בפרק 5. זה ההבדל בין סוכן שכותב הוכחה "בעיניים עצומות" ומקבל רק "עבר/נכשל", לבין סוכן שרואה את המטרה אחרי כל טקטיקה.

חיפוש ואבולוציה

שישה כלים לבעיות בנייה. **OpenEvolve** ו-**ShinkaEvolve** (של Sakana AI) הם מימושים פתוחים של לולאה בסגנון AlphaEvolve. הקוד של **FunSearch** משמש בעיקר כתבנית לתכנון מעריכים, ו**מאגר התוצאות של** **AlphaEvolve** (T041) משמש כרשימת חסמים שצריך לעבור. לצדם כלים קלאסיים של אופטימיזציה מדויקת: **OR-Tools CP-SAT** של Google לחיפוש מדויק במקרים קטנים, ו-**Gurobi** לתכנות בשלמים בקנה מידה גדול.

SAT ו-SMT

שמונה כלים, כולם בשלים. **Kissat** (T044) הוא מנוע ה-SAT הראשי, **CaDiCaL** (T045) מתאים לפתרון אינקרמנטלי ולהפקת הוכחות LRAT ו-**CryptoMiniSat**, DRAT לבעיות עם אילוצי זוגיות, ו-**PySAT** לבניית קידודים במהירות מפייתון. ב-**Z3**: SMT לבדיקות אריתמטיות מהירות, ו-**cvc5**, שמפיק תעודות ומשמש כ-backend ל-lean-smt. ושני הבודקים, שהם החלק החשוב: **DRAT-trim** (T050) מאמת טענות UNSAT, ו-(T051) **cake_lpr** מספק "את הבדיקה ברמת האמינות הגבוהה ביותר" של תעודות כאלה, כי הוא עצמו אומת פורמלית.

אלגברה ממוחשבת ונומריקה

11 מערכות אלגברה ממוחשבת (CAS): **SageMath** לחישוב כללי, **PARI/GP** לשדות מספרים ועקומים אליפטיים, **GAP** לחבורות, **Macaulay2** ו-**Singular** לבסיסי גרבנר, **SymPy** כ"קוד דבק" סימבולי בצינורות הסוכנים, ו-Mathematica, Maple ו-Magma כשפרויקט ספציפי דורש. ו-**FLINT**, שבתוכו נמצאת היום (T065) **Arb**, ספריית ה"נומריקה המאושרת" (ball arithmetic): כל מספר מיוצג כמרכז ורדיוס, והתוצאה מובטחת להיות בתוך הכדור. זה מה שהופך חישוב נומרי לראיה.

10 כלים נומריים: **mpmath**, **Julia**, **pandas**, **SciPy**, **NumPy** לדיוק שרירותי, **JAX** לחיפוש מבוסס גרדיאנטים, ושלושה כלים לתכנות חצי-מוגדר (SDP): **MOSEK**, **CVXPY** ו-**SDPA-GMP** (בדיוק גבוה). את ה-SDP צריך לחסמי bootstrap ולחסמים בסגנון flag algebras.

קומבינטוריקה, תורת המספרים וגאומטריה

nauty and Traces (T073) מונה גרפים עד כדי איזומורפיזם, ו-**NetworkX** משמש לאב-טיפוס מהיר. **House of Graphs** הוא מאגר של גרפים קיצוניים ודוגמאות נגדיות, ו-**FindStat** מזהה סטטיסטיקות קומבינטוריות שעולות בניסויים. **מאגר בעיות ארדש** (T077) רשום כמקור למטרות ולמעקב אחרי סטטוס. בתורת המספרים, **OEIS** מזהה סדרות שיוצאות מניסויים, ו-**LMFDB** מאפשר לחפש עקומים, תבניות מודולריות ופונקציות L. בגאומטריה וטופולוגיה: SnapPy, Regina, KnotInfo, Knot Atlas ו-polymake.

פיזיקה

11 כלים. (T090) **SDPB** הוא הכלי המרכזי ל-bootstrap נומרי של תורות שדה קונפורמיות. **xAct**, **Cadabra**, **FORM** ו-**FeynCalc** משמשים לחישובים סימבוליים בתורת השדות וביחסות כללית. **Grid** לסימולציות של lattice QCD, **QuTiP** למערכות קוונטיות פתוחות, ו-**INSPIRE-HEP** לחיפוש ספרות בפיזיקת אנרגיות גבוהות. ושני כלים לרגרסיה סימבולית, **PySR** ו-**AI Feynman**, שראינו בפרק 4.

ספרות

תשעה כלים, וזה לא מקרי: חיפוש ספרות הוא "הניצחון הזול ביותר" (פרק 4). **arXiv API** לסריקה יומית של רשימות חדשות, **Semantic Scholar** למעקב ציטוטים, **zbMATH Open** ו-**MathSciNet** כמקורות סמכותיים, **Google Scholar** לחיפוש רחב, **Connected Papers** למיפוי תת-תחום, **Elicit** ו-**Consensus** לסינון מהיר, ו-**MathOverflow** כדי לבדוק אם בעיה כבר ידועה או נפתרה.

נתונים, פרסום ותשתית

בנצ'מרקים (5 כלים): miniF2F ו-PutnamBench לבדיקות רגרסיה של צינור ההוכחה, ProofNet לאוטו-פורמליזציה, ו-**Formal Conjectures** של DeepMind (T109) כמקור למטרות שכבר מנוסחות ב-Lean. כפי שראינו בפרק 4, ציונים על הבנצ'מרקים האלה לא מוכיחים יכולת. הם טובים לבדוק שלא שברנו משהו.

סימון ותרגום (4 כלים): Pandoc, MathJax, KaTeX (להצגת נוסחאות באתר) ו-Mathpix (להמרת מאמרים וכתב יד).

שיתוף ופרסום (8 כלים): **Zenodo**, arXiv (לארכוב קוד, נתונים ותעודות הוכחה עם DOI), GitHub ו-Overleaf, Jupyter, Quarto, GitHub Pages, ו-**Lean Zulip**, הפורום שבו שואלים מומחים ומכריזים על פורמליזציות.

תשתית וחישוב (8 כלים): **GitHub Actions** עם **lean-action** לבדיקת כל הוכחה בכל PR, **Docker** ו-**uv** לסביבות רפרודוקטיביות, **Lambda Cloud**, **Modal** ו-**RunPod** לחישוב GPU, ו-**Google Cloud Run** לאירוח שרתי REPL ו-MCP.

ויזואליזציה (3 כלים): **Graphviz**, Matplotlib (גם לגרף התלויות של blueprint), ו-**ProofWidgets4** להצגת אובייקטים בתוך הוכחות Lean.

טקסטים עבריים ויהודיים

שמונה כלים, וזו הקטגוריה הייחודית ביותר בקטלוג. **Sefaria API** ו-**Sefaria-Export** מספקים טקסטים מקוריים, **Dicta** לעיבוד טקסט עברי ולזיהוי מקורות, **אוצר החכמה**, **פרויקט השו"ת של בר-**

אילן ו-**HebrewBooks** לספרות רבנית, ושתי ספריות פייתון קטנות לגימטריה ולמספרים עבריים. הכלים האלה משרתים את פרויקטי "גשרי החוכמה" של המעבדה, ואת ה-analogy-agent. כפי שהוסבר בפרק 6, כל מה שיוצא מהם הוא השראה ברמה L0, לעולם לא ראיה.

הקטלוג במספרים

קטגוריה	כלים	קטגוריה	כלים
lean-library	14	ai-prover	6
CAS	11	search/evolution	6
physics	11	combinatorics	5
numerics	10	geometry/topology	5
llm-api	9	datasets/benchmarks	5
literature-search	9	notation/translation	4
proof-assistant	8	visualization	3
SAT/SMT	8	number-theory	2
collaboration/publishing	8		
infra/compute	8	**סה"כ**	**140**
hebrew/jewish-texts	8		

ה-Starter Stack: חמישה-עשר כלים ליום הראשון

מתוך 140, `docs/tools.md` בוחר 15 שמתקינים ראשונים. ההיגיון: מספיק כדי להריץ כל אחד משלושת הצינורות של מטרות הפתיחה (פרק 7), ולא יותר.

#	כלי	למה ראשון
1	**Lean 4 + elan**	שפת האימות המרכזית; elan נועל גרסה לכל פרויקט
2	**Lake + mathlib4**	בלי mathlib אין מתמטיקה; `lake exe cache get` חוסך שעות

#	כלי	למה ראשון
3	**Lean REPL / lean-lsp-mcp**	הממשק שדרכו סוכנים רואים מטרות ובודקים הוכחות
4	**LeanSearch + Loogle**	מציאת הלמה הנכונה
5	**Claude API + Claude Agent SDK**	מנוע ההיגיון והתזמור
6	**MCP**	פרוטוקול אחיד לחשיפת Lean, OEIS, arXiv ופותרי SAT לסוכנים
7	**Python venv** (numpy, scipy, pandas, matplotlib, jupyter)	שכבת הדבק לניסויים ולוגים
8	**SymPy**	חישוב סימבולי קל
9	**mpmath**	דיוק שרירותי
10	**NetworkX**	אב-טיפוס לגרפים
11	**PySAT**	קידודי SAT מפייתון
12	**Z3**	SMT לבדיקות אריתמטיות
13	**Kissat + DRAT-trim** (אופציונלי)	פותר SAT ובודק תעודות UNSAT, כדי ש"אין פתרון" יהיה מוכח
14	**OEIS + arXiv API**	זיהוי סדרות וסריקת ספרות
15	**GitHub Actions + lean-action**	CI שבודק כל הוכחה בכל PR. אין "עובד אצלי"

אופציונלי בשלב הראשון: **PySR** (מושך איתו Julia, כבד), **OpenEvolve** או **ShinkaEvolve**, **SageMath** או **PARI/GP** כש-SymPy לא מספיק, ו-**Sefaria API**.

סקריפט ההתקנה

`scripts/setup.sh` מתקין את הליבה. הוא **אידמפוטנטי**: כל שלב בודק אם הכלי כבר קיים ומדלג, כך שאפשר להריץ אותו שוב בבטחה. הוא מיועד ל-Ubuntu 22.04 ו-24.04.

שלב	מה קורה	ברירת מחדל
1	חבילות מערכת: git, curl, build-essential, python3-venv	פועל
2	elan ו-toolchain של `stable` של Lean 4	פועל
3	סביבה וירטואלית של פייתון ב- `lab/.venv`	פועל
4	חבילות פייתון: sympy, numpy, scipy, networkx, python-sat, z3-solver, mpmath, pandas, openpyxl, matplotlib, jupyter, requests, anthropic, claude-agent-sdk, ובדיקת import	פועל
5	PySR	`WITH_PYSR=1`
6	בניית Kissat ו-DRAT-trim מקוד המקור	`WITH_KISSAT=1`
7	`lake exe cache get` && `lake build` בפרויקט `lab/lean`	`WITH_MATHLIB=1`

```
bash lab/scripts/setup.sh                          # ליבה
WITH_PYSR=1 WITH_KISSAT=1 bash lab/scripts/setup.sh  # הכול
source lab/.venv/bin/activate
export ANTHROPIC_API_KEY=...                       # לעולם לא לשמור במאגר
```

שתי הערות מעשיות. ראשית, הסקריפט מתקין את גרסת ה- `stable` של Lean, אבל הפרויקט `lab/lean` מצהיר על גרסה משלו בקובץ `lean-toolchain`. זה לא באג: elan יוריד אוטומטית את הגרסה שהפרויקט דורש בפעם הראשונה שבונים אותו. אבל כדאי לדעת שהגרסה שמותקנת "כברירת מחדל" אינה הגרסה שבה נבדקות התוצאות. שנית, Kissat מפיק הוכחות DRAT, אבל הצינור של

מטרה G050 מתכנן לבדוק בתוך Lean תעודות בפורמט **LRAT**. לשם כך CaDiCaL (T045), שמפיק LRAT, הוא הפותר המתאים, והוא אינו חלק מהסקריפט כרגע.

הצעדים הבאים

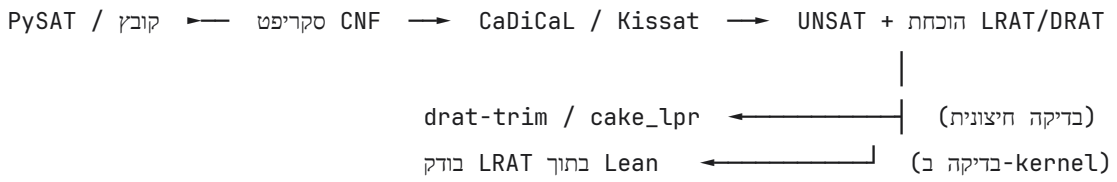
מה שעוד לא בסקריפט, לפי `docs/tools.md`: **Docker image** של המעבדה (Lean, cache של mathlib וסביבת פייתון) לשימוש ב-CI ובשירותי ענן; הרצת מוכיחים פתוחים דרך vLLM על GPU בענן, רק כשיש תקציב; שרתי MCP פנימיים ל-OEIS, ל-LMFDB, ל-arXiv ולפותרי SAT; ורישיונות אקדמיים ל-Gurobi ול-MOSEK.

צינורות: איך כלים מתחברים

כלי בודד כמעט אף פעם לא מייצר תוצאה. מה שמייצר תוצאה הוא **צינור (pipeline)**, ובכל צינור טוב יש אותו מבנה: שלב שמייצר מועמדים (יכול להיות מסובך, מהיר ולא אמין), ושלב שבודק אותם (חייב להיות פשוט ואמין). הנה הצינורות המרכזיים של המעבדה.

צינור 1: Lean ← DRAT/LRAT ← SAT

זה הצינור של מטרה G050, והוא מתאים לכל טענה על אובייקטים סופיים: "אין צביעה של 1 עד 45 בארבעה צבעים בלי פתרון חד-צבעי של $x + y = z$".



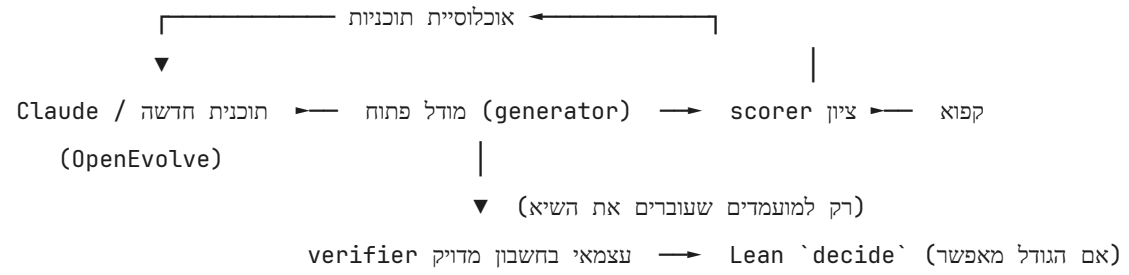
1. **קידוד.** סקריפט (בדרך כלל PySAT) מתרגם את השאלה לנוסחה בוליאנית בצורת CNF. כאן נמצאת הסכנה הגדולה: קידוד שגוי הופך את כל השאר לחסר משמעות. לכן ה-search-agent מחויב לבדוק את הקידוד על מקרים עם תשובה ידועה לפני ההרצה האמיתית.
2. **פתרון.** פותר SAT מנסה למצוא השמה. אם מצא, יש לנו דוגמה (צביעה), וקל לבדוק אותה ישירות, כולל ב-Lean עם `decide`. אם לא מצא, הוא מצהיר UNSAT.
3. **תעודה.** הצהרת UNSAT לבדה היא רק מילה של תוכנה מסובכת. לכן הפותר מפיק **הוכחת DRAT או LRAT**: רצף של צעדי היסק שכל אחד מהם קל לבדוק.
4. **בדיקה.** בודק קטן ופשוט מאמת את התעודה. DRAT-trim מספיק לרמה טובה; cake_lpr, שאומת פורמלית, נותן רמה גבוהה יותר.

5. **Lean.** בשלב הגבוה ביותר, התעודה נבדקת בתוך Lean עצמו (בעזרת LeanSAT או `bv_decide`), ומוכיחים ב-Lean שהקידוד נכון, כלומר שקיום צביעה טובה גורר ש-CNF ספיק. רק אז יש משפט Lean על השאלה המתמטית עצמה, ולא רק על הנוסחה.

הצעד החמישי הוא מה שמבדיל את הצינור הזה מ"הרצנו פותר SAT". בלעדיו, הנכונות של הקידוד נשארת טענה לא מאומתת. פרויקט ה-empty hexagon (2024) הוא התבנית שהמעבדה מתכוונת ללמוד ממנה.

צינור 2: מודל שפה ← חיפוש אבולוציוני ← מוודא מדויק

זה הצינור של מטרה G029 ושל כל בעיות הבנייה, והוא הגרסה של המעבדה ל-FunSearch ו-AlphaEvolve.



1. **אוכלוסייה.** מתחילים בכמה תוכניות שמייצרות אובייקטים, למשל צביעות ציקליות.
2. **מוטציה.** מודל שפה מקבל את התוכניות הטובות ומציע גרסאות חדשות. כאן ה-AI יצירתי, ואין צורך לסמוך עליו.
3. **ניקוד. scorer** דטרמיניסטי מריץ כל תוכנית ומודד את התוצאה. ה-scorer מוקפא לפני תחילת החיפוש ונבדק בנפרד. זה הלקח של Georgiev ושותפיו (פרק 4): מערכות אבולוציוניות ימצאו כל באג במעריך וינצלו אותו.
4. **אימות.** מועמד שעובר את השיא עובר ל-**verifier עצמאי**, שנכתב בנפרד מהמחולל ומשתמש בחשבון מדויק בלבד (מספרים שלמים ורציונליים, למשל המודול `fractions` של פייתון). "ציון בנקודה צפה אינו ראיה לשיא."
5. **Lean.** אם האובייקט קטן מספיק, בודקים אותו גם ב-Lean עם `decide`, ומקבלים L3.

כדאי לשים לב שיש כאן שני מעריכים, ולא במקרה. ה-scorer צריך להיות מהיר כי הוא רץ מיליוני פעמים. ה-verifier צריך להיות אמין כי הוא רץ פעם אחת על כל טענה לשיא. אלה דרישות שונות, ולכן אלה תוכנות שונות.

צינור 3: סקיצה ← מוכיח Lean ← AI

זה הצינור של הוכחות, והוא הצנרת הסטנדרטית של 2025–2026 שתוארה בפרק 4.

1. **סקיצה.** מודל reasoning כללי (Claude, או מודל אחר כ"דעה שנייה") כותב הוכחה בשפה טבעית. זו רמה L2 לכל היותר, ורק אחרי שסוכן שני בדק אותה.
2. **Blueprint.** ההוכחה מפורקת לגרף של למות ב-leanblueprint, כך שכל צומת הוא משימה קטנה.
3. **פורמליזציה.** לכל למה, ה-lean-agent מנסה קודם את האוטומציה הזולה (`exact?` , `simp` , `omega` , `norm_num` , Aesop), אחר כך מוכיחים פתוחים (DeepSeek-Prover, Goedel-Prover, Kimina-Prover) דרך Lean REPL או Kimina Lean Server, ולמות קשות נשלחות ל-Aristotle (בכפוף להערות על גרסה ופרטיות).
4. **משוב.** Lean מחזיר שגיאות, דרך lean-lsp-mcp הסוכן רואה את המטרה שנשארה, ומתקן. חוזרים חלילה.
5. **קבלה.** `lake build` נקי על גרסה נעולה, `#print axioms` נקי, והוכחה של היעד שאושר. GitHub Actions עם lean-action מריץ את הבדיקה הזו בכל PR.

צינור 4: ספרות ← סטטוס ← ניסוח

הצינור הפשוט ביותר, והראשון שרץ על כל מטרה. arXiv API, zbMATH, Semantic Scholar ו-MathOverflow מחפשים אם הבעיה נפתרה. מאגר בעיות ארדש והוויקי של תרומות AI מספקים את הסטטוס הנוכחי. Formal Conjectures ו-mathlib (דרך LeanSearch ו-Loogle) עונים על השאלה "האם זה כבר מנוסח ב-Lean?". הפלט הוא דו"ח ספרות ב- `artifacts/<goal>/literature.md` ורשומה ביומן. הוא לא מייצר משפט, אבל הוא מונע את הטעות הנפוצה ביותר בתחום.

צינור 5: נומריקה ← תעודה

לבעיות פיזיקה ואנליזה, שבהן התשובה היא מספר ממשי ולא אובייקט סופי:

- **SDP:** SDPB או MOSEK מוצאים חסם נומרי (L1). מעגלים את הפתרון הדואלי למספרים רציונליים, ובודקים בחשבון מדויק (SymPy, Sage או FLINT) שהמטריצות חיוביות. זו תעודה מסוג `sdp-rational` .
- **אינטרוולים:** Arb (בתוך FLINT) מחשב עם כדורים במקום מספרים, כך שכל תוצאה מגיעה עם חסם שגיאה מוכח. כך נבנו הוכחות ההתפוצצות של Chen ו-Hou, וזה הכיוון של מטרה G067.

צינור 6: נתונים ← השערה ← בדיקה

הצינור של "AI כמיקרוסקופ" (Davies ואחרים, Ramanujan Machine): מייצרים נתונים בניסוי (NumPy, Sage, nauty), מחפשים דפוסים (PySR לנוסחאות סגורות, OEIS לסדרות, FindStat לסטטיסטיקות קומבינטוריות), ומנסחים השערה. ההשערה נרשמת כ-L0 או L1, ועוברת לאחד הצינורות האחרים כדי להוכיח או להפריך אותה. זה גם הצינור של מטרה G045, כריית השערות מ-OEIS.

עקרונות לבחירת כלים

מהקטלוג ומהצינורות עולים כמה עקרונות שכדאי לנסח במפורש:

1. **בכל צינור, הבודק פשוט מהמחולל.** פותר SAT מסובך, DRAT-trim פשוט. מודל שפה מסובך, ה-kernel של Lean פשוט. אופטימייזר נומרי מסובך, בדיקה רציונלית פשוטה.
2. **אוטומציה זולה לפני AI יקר.** `omega` ו-`decide` לפני מוכיח AI. מוכיח פתוח לפני שירות בתשלום.
3. **לא בונים מה שקיים.** הקטלוג קיים כדי שלא נכתוב מחדש פותר SAT או מוכיח. היתרון של המעבדה הוא בבחירת הבעיות ובמעריכים.
4. **גרסאות נעולות.** כל תעודה רושמת את גרסת הבודק. Docker ו-uv קיימים בקטלוג בדיוק בשביל זה.
5. **פרטיות ותלות בספקים.** שירות חיצוני (Aristotle, מודלים מסחריים) הוא כלי לגיטימי, אבל לא שולחים אליו חומר רגיש, ובודקים את תנאי הרישיון לפני שימוש מסחרי.

בקצרה

- הקטלוג `data/tools.csv` מונה 140 כלים ב-19 קטגוריות, עם סולם בשלות 1–5. כמה כלים רשומים להקשר בלבד (AlphaProof, FrontierMath).
- ה-Starter Stack כולל 15 כלים: Lean, mathlib, REPL/MCP, כלי חיפוש למות, Claude API ו-SDK, פייתון עם ספריות מדעיות, PySAT, Z3, Kissat ו-DRAT-trim, arXiv ו-OEIS, ו-CI.
- `scripts/setup.sh` מתקין את הליבה באופן אידמפוטנטי, עם דגלים לרכיבים כבדים (PySR, Kissat, mathlib).
- כל צינור טוב בנוי ממחולל לא אמין ובודק אמין: Lean←DRAT/LRAT←SAT, מודל←אבולוציה←מוודא מדויק, סקיצה←מוכיח←Lean, נומריקה←תעודה רציונלית או אינטרוולית.
- הבודק פשוט מהמחולל, אוטומציה זולה קודמת ל-AI יקר, וגרסאות נעולות הן תנאי לתעודה.

רעיונות לתוכן

1. **"שישה צינורות, עיקרון אחד".** זווית: אינפוגרפיקה של ששת הצינורות, כל אחד עם "מחולל" באדום ו"בודק" בירוק. קהל יעד: קוראי S27 News ואנשי טכנולוגיה.
2. **מדריך וידאו: "מ-CNF ל-Lean".** זווית: הדגמה של צינור SAT על בעיה קטנה מאוד (למשל $S(2) = 4$), כולל הפקת DRAT ובדיקה. קהל יעד: מפתחים, סטודנטים למדעי המחשב.

3. **"למה הבודק חייב להיות טיפש".** זווית: מאמר דעה על הרעיון שהבודק צריך להיות פשוט מהמחולל, עם דוגמאות ממערכות אבולוציוניות שרימו. קהל יעד: קהילת ה-AI הרחבה.
4. **רשימה: "15 הכלים שמתקינים ביום הראשון במעבדת מתמטיקה של 2026".** זווית: גרסה נגישה של ה-Starter Stack, עם משפט אחד לכל כלי. קהל יעד: סטודנטים וחוקרים שרוצים להתחיל.
5. **"מ-Sefaria ועד גימטריה: הכול ב-L0": הכלים העבריים במעבדה.** זווית: איך Sefaria ו-Dicta משתלבים במעבדה מתמטית, ואיפה עובר הקו בין השראה לראיה. קהל יעד: קוראים שמתעניינים במפגש בין מסורת למדע.

פרק 9: דפוסי הוכחה יצירתיים

מתמטיקאים אוהבים לספר שהוכחה גדולה "נפלה מהשמיים", אבל כשמסתכלים מקרוב על הסיפורים רואים משהו אחר. רוב ההוכחות פורצות הדרך לא הגיעו מעוד שעה של עבודה באותו כיוון. הן הגיעו מצעד הצידה: מישהו החליף את שפת הבעיה, ייבא כלי מתחום רחוק, הצליח לסמוך על חיפוש ממוחשב, או פשוט הגיע מבחוץ ולא ידע שאסור לנסות.

הפרק הזה עוסק בצעדים האלה. הוא בנוי על הקטלוג שבמאגר `data/creative_proofs.csv` (שישים רשומות, P001 עד P060), ועל המסמך `docs/creative-proofs.md` שמסווג אותן לעשרה דפוסים. כאן נספר את הדפוסים דרך הסיפורים עצמם, אחד או שניים לכל דפוס, ובסוף כל סיפור נשאל שאלה אחת: מה סוכן AI יכול ללמוד מזה מחר בבוקר?

הסיבה שהפרק חשוב למעבדה פשוטה. סוכני שפה טובים מאוד בהמשך הדרך שבה כבר הולכים. קשה להם יותר לקפוץ לדרך אחרת. אם נצליח לכתוב את הקפיצות האלה כהוראות עבודה מפורשות, playbook, נוכל לבקש מהסוכנים לעשות אותן באופן שיטתי ולא רק לקוות שיופיעו. בסוף הפרק מופיע ה-playbook המלא.

לפני הכול: הדפוס שאינו דפוס

נתחיל בסיפור שאין בו שום הוכחה, ובכל זאת הוא הסיפור שכל סוכן במעבדה צריך להכיר בעל פה.

באוקטובר 2025 פרסמו חוקרים מ-OpenAI ברשתות החברתיות ש-GPT-5 "פתר עשר בעיות ארדש פתוחות". זה נשמע כמו רגע היסטורי. אבל Thomas Bloom, שמתחזק את האתר erdosproblems.com, הסביר במהירות מה קרה בפועל: בעיה שמסומנת "פתוחה" באתר שלו היא בעיה שהוא לא מכיר מאמר שפותר אותה. המודל לא פתר את הבעיות. הוא מצא מאמרים קיימים שכבר פתרו אותן. הציוצים נמחקו, ו-Demis Hassabis כינה את האירוע "מביך"[70] (P057).

וזה המקום להיזהר מהמסקנה הקלה. חיפוש הספרות של המודל היה מצוין, וזו תרומה אמיתית לקהילה: בעיות שנחשבו פתוחות סומנו נכון. מה שהיה שגוי הוא הכותרת. "מצאנו פתרון שכבר קיים" ו"פתרנו בעיה פתוחה" הן שתי טענות שונות לגמרי.

כמה חודשים אחר כך, בינואר 2026, הגיע המקרה ההפוך. לבעיה #728 הציע GPT-5.2 Pro טיעון לא-פורמלי, והמערכת Aristotle של Harmonic פירמלה אותו ב-Lean. זה נחשב בדרך כלל לאחד

הפתרונות האוטונומיים הראשונים של בעיית ארדש[71] (P058). אבל גם כאן היה פרט חשוב: לפני שאפשר היה לקבוע שהבעיה נפתרה, היה צריך להבהיר מה בדיוק הניסוח שלה אומר.

משני הסיפורים יוצאים שני כללים שקודמים לכל דפוס יצירתי:

- **כלל 0: חיפוש ספרות לפני הכול.** לפני כל ניסיון לפתור, בודקים אם הבעיה כבר נפתרה.
- **כלל 11: בודקים את הניסוח, לא רק את ההוכחה.** הוכחה מושלמת של טענה שגויה לא שווה כלום. היא אפילו מזיקה, כי היא נראית מאומתת.

שני הכללים האלה הם גם השלב הראשון והשלב השלישי בלולאה של המעבדה (`docs/architecture.md`, סעיף 2): קודם literature-agent, ואחר כך ביקורת אנושית של הניסוח.

דפוס 1: לשנות את שפת הבעיה

הגשרים של קניגסברג

בעיר קניגסברג שבפרוסיה היו שבעה גשרים שחיברו בין שתי גדות הנהר ושני איים. תושבי העיר שאלו את עצמם אם אפשר לטייל בעיר ולעבור על כל גשר בדיוק פעם אחת. אנשים ניסו, ואף אחד לא הצליח, אבל אף אחד גם לא ידע להסביר למה.

ב-1736 לאונרד אוילר עשה דבר שנראה כמעט חצוף: הוא זרק את המפה. המרחקים, הצורות, הזוויות, אורך הגשרים, כל זה לא משנה. נשאר רק דבר אחד: איזה חלק יבשה מחובר לאיזה, ובכמה גשרים. מה שנשאר הוא מה שאנחנו קוראים היום **גרף** (graph). ובשפה החדשה התשובה כמעט מובנת מאליה: כל פעם שנכנסים לחלק יבשה צריך גם לצאת ממנו, ולכן לכל נקודה שאינה התחלה או סוף יש מספר זוגי של גשרים. בקניגסברג לכל ארבעת חלקי היבשה היה מספר אי-זוגי של גשרים. הטיול בלתי אפשרי[72] (P004).

המצולע של גאוס

ב-1796 גאוס הצעיר, בן 18, הראה שאפשר לבנות בסרגל ומחוגה מצולע משוכלל בעל 17 צלעות. אלפיים שנה אחרי אוקלידס, איש לא הוסיף מצולע לרשימה. גאוס לא הצליח כי היה גאומטריקן טוב יותר מאוקלידס. הוא הצליח כי תרגם את השאלה הגאומטרית לאלגברה: בנייה בסרגל ומחוגה מתאימה לשרשרת של הרחבות ריבועיות של שדות, והשאלה הופכת לשאלה על שורשי היחידה[73] (P005). שלושים שנה אחר כך גלואה הלך עוד צעד: במקום לחפש נוסחה לפתרון משוואה ממעלה חמישית, הוא חקר את חבורת הסימטריות של השורשים, והראה שהשאלה "האם יש נוסחה" היא בעצם שאלה על המבנה של החבורה הזו (P007).

הלקח לסוכנים. הצעד הראשון בהתקפה על בעיה אינו ניסיון הוכחה. הוא ייצור של כמה ניסוחים חלופיים: גרפי, אלגברי, הסתברותי, דרך פונקציה יוצרת. אחר כך שואלים: באיזה ניסוח ההוכחה נראית הכי קצרה? סוכן שמתחיל להוכיח בניסוח הראשון שקיבל דומה לתושב קניגסברג שמנסה עוד מסלול על המפה.

דפוס 2: כוח גס שהופך לאמין

ארבעת הצבעים, פעמיים

ב-1976 הודיעו Kenneth Appel ו-Wolfgang Haken מאוניברסיטת אילינוי שהוכיחו את משפט ארבעת הצבעים: כל מפה במישור אפשר לצבוע בארבעה צבעים כך ששתי מדינות שכנות אינן באותו צבע. ההוכחה הייתה שונה מכל מה שהקהילה הכירה. החלק התאורטי צמצם את הבעיה לבדיקה של כ-1,900 תצורות, ואת הבדיקה עשה מחשב, במשך מאות שעות[74] (P016).

התגובה הייתה מעורבת. אם אף אדם לא יכול לקרוא את ההוכחה, האם היא הוכחה? השאלה הזו לא נפתרה בוויכוח פילוסופי. היא נפתרה כמעט שלושים שנה מאוחר יותר, ב-2005, כש-Georges Gonthier ו-Benjamin Werner פירמלו את ההוכחה כולה, כולל החלק החישובי, בעוזר ההוכחות Coq[75] (P017). מאותו רגע כבר לא היה צריך לסמוך על התוכנה של המחברים. היה צריך לסמוך רק על הגרעין הקטן של Coq.

קפלר, והשופטים שהתעייפו

השערת קפלר, מ-1611, טוענת שהדרך הצפופה ביותר לארוז כדורים זהים היא הדרך שבה מוכרי פירות בשוק מערימים תפוזים. ב-1998 Thomas Hales, יחד עם Samuel Ferguson, הגיש הוכחה. גם היא צמצמה את הבעיה לבדיקה ממוחשבת: מספר עצום של אי-שוויונות לא לינאריים, שנבדקו בתכנון לינארי ובאריתמטיקת קטעים (interval arithmetic)(P024)[76].

צוות השופטים עבד על ההוכחה שנים, ובסוף לא הצליח לאמת אותה עד הסוף. התשובה של Hales הייתה פרויקט בשם Flyspeck: פורמליזציה מלאה של ההוכחה בעוזרי ההוכחות HOL Light ו-Isabelle. הפרויקט הסתיים ב-2014[77] (P025). זה אולי הסיפור הטוב ביותר על הקשר בין חישוב לאמון: כשבני אדם לא יכולים לבדוק, מכונה קטנה ונבדקת היטב יכולה.

שני פטה-בייט של אמת

הגרסה המודרנית של הדפוס נראית כך. ב-2016 הראו Marijn Heule, Oliver Kullmann ו-Victor Marek שאת המספרים 1 עד 7,824 אפשר לצבוע בשני צבעים כך שאין שלשה פיתגורית ($a^2 + b^2 = c^2$) שכל שלושת איבריה באותו צבע, אבל את 1 עד 7,825 אי אפשר. ההוכחה התקבלה מפותר SAT, ותעודת ההוכחה שלה שקלה כ-200 טרה-בייט[78] (P033). שנה אחר כך Heule הוכיח

שמספר שור החמישי הוא $S(5) = 160$, עם תעודה של כ-2 פטה-בייט, שנבדקה בבודק שעבר בעצמו אימות פורמלי[79] (P034).

הלקח לסוכנים. חישוב גדול אינו בעיה. חישוב שאי אפשר לשחזר או לבדוק הוא בעיה. לכן כשבעיה מצטמצמת למספר סופי של מקרים, העבודה כוללת שלושה חלקים: הוכחה ב-Lean שהצמצום נכון, בדיקה בבודק מאומת (או ב-`decide`), ושמירת התעודה. זו בדיוק התוכנית של המטרה הראשונה של המעבדה, G050 (פרק 13).

דפוס 3: לייבא כלי מתחום אחר

פרמה, ועקומה שלא הייתה צריכה להתקיים

המשפט האחרון של פרמה פשוט לניסוח: למשוואה $x^n + y^n = z^n$ אין פתרונות בשלמים חיוביים כאשר $n > 2$. במשך יותר מ-350 שנה ניסו לתקוף אותו בכלים של תורת המספרים, ולא הצליחו. הפריצה הגיעה כשהשאלה עברה לתחום אחר. Gerhard Frey הציע ש"פתרון" היפותטי יוליד עקומה אליפטית עם תכונות מוזרות מאוד. Ken Ribet הוכיח שעקומה כזו לא יכולה להיות מודולרית. כך נשאר להוכיח שכל עקומה אליפטית (מהסוג הרלוונטי) היא כן מודולרית, ואת זה עשו Andrew Wiles ו-Richard Taylor ב-1994[80] (P022). השאלה על מספרים הפכה לשאלה על עקומות, והשאלה על עקומות הפכה לשאלה על סימטריות של פונקציות.

הפונקציה הקסומה של ויאזובסקה

ב-2016 Maryna Viazovska הוכיחה שהאריזה הצפופה ביותר של כדורים במימד 8 היא הסריג E_8. הכלי המרכזי היה ידוע: חסם התכנון הלינארי של Cohn ו-Elkies אמר שאם קיימת פונקציה עם תכונות מסוימות, החסם הדוק. אף אחד לא ידע לבנות פונקציה כזו. ויאזובסקה בנתה אותה מתבניות מודולריות (modular forms), כלי מתורת המספרים שנראה רחוק מאוד מאריזת כדורים. שבוע אחר כך, יחד עם Cohn, Kumar, Miller ו-Radchenko, הגיע גם מימד 24[81] (P036). דוגמאות נוספות באותו דפוס הן דיריכלה, שהשתמש באנליזה כדי להוכיח טענה אריתמטית טהורה (P006), ופורסטנברג, שהעביר שאלה קומבינטורית לתורה הארגודית (P018).

הלקח לסוכנים. כדאי לנהל **מילון התאמות** בין תחומים: קומבינטוריקה ותורה ארגודית, מספרים ועקומות אליפטיות, ספירה וניתוח פורייה. לכל בעיה שואלים: באיזה תחום יש לבעיה הזו משפט מבני חזק? זה גם התפקיד של ה-analogy-agent במעבדה, עם הסייג החשוב שכל מה שהוא מציע מתחיל ברמה L0 (פרק 12).

דפוס 4: למצוא את האינוריאנט הנכון

שיגעון ה-15

בסוף שנות ה-70 של המאה ה-19 התפשט באמריקה ובאירופה שיגעון של פאזל קטן: חמש-עשרה אבנים ממוספרות בקופסה של 4 על 4 עם משבצת ריקה אחת. מזיזים אבנים למשבצת הריקה ומנסים לסדר אותן לפי הסדר. גרסה מפורסמת של האתגר התחילה עם האבנים 14 ו-15 מוחלפות, ואנשים בילו שעות בניסיון לפתור אותה.

ב-1879 פרסמו W. W. Johnson ו-W. E. Story ב-American Journal of Mathematics את ההסבר: הזוגיות של התמורה, יחד עם הזוגיות של מיקום המשבצת הריקה, לא משתנה באף מהלך חוקי. החלפה של שתי אבנים בלבד משנה את הזוגיות. לכן מחצית מכל הסידורים פשוט לא ניתנים להשגה, ואין טעם לנסות[82] (P009).

לוח השחמט החסר

הדוגמה הקלאסית השנייה קצרה עוד יותר. מסירים משני צידי האלכסון של לוח שחמט את שתי המשבצות הפינתיות. האם אפשר לכסות את 62 המשבצות שנותרו ב-31 אבני דומינו? חיפוש ממוחשב נאיבי יכול לרוץ זמן רב מאוד. אבל כל אבן דומינו מכסה משבצת שחורה אחת ולבנה אחת, ושתי הפינות שהוסרו הן באותו צבע. נשארו 32 משבצות מצבע אחד ו-30 מהצבע השני. אי אפשר. שורה אחת[83] (P013).

ב-1964 הציע John McCarthy את הבעיה הזו כאתגר למוכיחי משפטים אוטומטיים, דווקא בגלל הפער בין הקושי של החיפוש לפשטות של התובנה האנושית. היא עדיין מבחן טוב ל"תובנה" של סוכן.

הדפוס הזה מגיע גם לשיאים. Max Dehn פתר ב-1900 את הבעיה השלישית של הילברט בעזרת אינווריאנט שהוא המציא במיוחד: גודל שלא משתנה כשחותכים פאון לחלקים ומרכיבים אותם מחדש (P010). ו-Grigori Perelman, בהוכחת השערת פואנקרה, השתמש בגודל מונוטוני, מעין "אנטרופיה" בהשראת מכניקה סטטיסטית, לאורך זרימת ריצ'י (P026).

הלקח לסוכנים. חיפוש אינווריאנטים אפשר להפוך לתהליך. מגדירים במפורש את מערכת המהלכים, מציעים מועמדים (זוגיות, צביעה, משקל, פונקציונל מונוטוני), ובודקים אותם אוטומטית על אלפי מהלכים אקראיים לפני שמנסים להוכיח משהו. מועמד שנשבר במהלך אקראי אחד נזרק בלי לבזבז עליו זמן.

דפוס 5: המכונה מחפשת, האדם מסביר

הכובע של דייוויד סמית

במשך עשרות שנים חיפשו מתמטיקאים "אינשטיין" (מגרמנית: אבן אחת): צורה אחת שמרצפת את המישור, אבל רק באופן לא מחזורי. ב-2023 מצא אותה David Smith, חובב מאנגליה, כשהתנסה

בתוכנת ריצוף. הוא לא הוכיח שום דבר. הוא שלח את הצורה, שקיבלה את הכינוי "הכובע", ל-Craig Kaplan, ויחד עם Joseph Samuel Myers ו-Chaim Goodman-Strauss הם הוכיחו שהיא באמת עושה את מה שנראה שהיא עושה. ההוכחה שילבה קומבינטוריקה בעזרת מחשב עם טיעון גאומטרי[84] (P048). חמישים שנה קודם לכן עשתה משהו דומה Marjorie Rice, עקרת בית בלי השכלה מתמטית פורמלית, שמצאה ארבעה סוגים חדשים של מחומשים קמורים שמרצפים את המישור, בעזרת סימון שהמציאה בעצמה (P019).

קשרים ושיפועים

ב-2021 פרסמו Alex Davies מ-DeepMind ושותפיו, ובהם המתמטיקאים Marc Lackenby, András Juhász ו-Geordie Williamson, ניסוי שונה. הם אימנו מודל למידת מכונה לחזות אינווריאנט אלגברי של קשרים (signature) מתוך אינווריאנטים גאומטריים. המודל הצליח, וכלי ייחוס (attribution) הראו על אילו תכונות הוא נשען. עם הרמז הזה המתמטיקאים ניסחו והוכיחו משפט חדש שמקשר את ה-signature ל"שיפוע הטבעי" (natural slope) של הקשר[85] (P041). המכונה לא הוכיחה דבר. היא הצביעה למתמטיקאים איפה לחפש.

הלקח לסוכנים. צינור קבוע: מייצרים נתונים, מאמנים מודל חיזוי, בודקים על מה הוא נשען, מנסחים השערה, ומנסים להוכיח אותה ב-Lean. ההשערה נשמרת יחד עם הנתונים שתומכים בה, ומסומנת L1 עד שמוכיחים אותה. גם ה-Ramanujan Machine, שמצאה השערות חדשות על שברים משולבים (P042), עובדת כך.

דפוס 6: לחפש תוכניות, לא אובייקטים

FunSearch ו-cap sets

cap set הוא קבוצה של נקודות במרחב $\mathbb{F}_3^n$ שאין בה שלוש נקודות על ישר אחד. השאלה כמה גדולה יכולה להיות קבוצה כזו היא שאלה קלאסית בקומבינטוריקה. ב-2023 הציג צוות של DeepMind, יחד עם Jordan Ellenberg, את FunSearch. במקום לבקש ממודל שפה למצוא cap set, ביקשו ממנו לכתוב **תוכנית** שמייצרת cap sets. מעריך אוטומטי הריץ כל תוכנית ובדק את הפלט, והתוכניות הטובות נבחרו לסיבוב הבא, כמו באבולוציה. במימד 8 התוכנית מצאה cap set בגודל 512, גדול מכל מה שהיה ידוע[86] (P049).

היתרון החשוב אינו רק המספר. התוצר הוא תוכנה קצרה שבן אדם יכול לקרוא, להבין ולהכליל.

AlphaEvolve ו-48 כפלים

ב-2025 DeepMind הרחיבה את הרעיון ל-AlphaEvolve, סוכן קוד אבולוציוני. בין התוצאות: אלגוריתם לכפל מטריצות מרוכבות בגודל 4×4 עם 48 פעולות כפל (השיטה הקלאסית של Strassen דורשת 49), וקונפיגורציה של 593 כדורים שמשפרת את החסם התחתון למספר הנשיקה (kissing number) במימד 11[87] (P056). עבודה מאוחרת יותר של Georgiev, Gómez-Serrano, Tao ו-Wagner על 67 בעיות תיעדה גם את הצד השני: המערכת למדה לנצל באגים במעריך[88].

הלקח לסוכנים. לכל בעיה של שיפור חסם או בנייה, כותבים קודם **מעריך דטרמיניסטי**, עם אריתמטיקה מדויקת ולא עם מספרים עשרוניים מקורבים, ובודקים אותו כמו קוד קריטי. רק אחר כך מריצים את הלולאה האבולוציונית. התוצר הוא תעודה שאפשר לבדוק. זו התוכנית של המטרה השלישית של המעבדה, G029 (`docs/goal-bank.md`).

דפוס 7: לקודד כ-SAT, ולאמת את הקידוד

היחידה שאיש לא ציפה לה

השערת היחידות של Kaplansky, משנות ה-40 של המאה ה-20, עסקה באלגברה מופשטת: היא טענה שבחוגי חבורה מסוג מסוים אין "יחידות" לא טריוויאליות. במשך שמונים שנה מתמטיקאים ניסו להוכיח אותה. ב-2021 Giles Gardam ניסה את הכיוון ההפוך. הוא קודד את השאלה "האם קיימת יחידה לא טריוויאלית בחוג החבורה של חבורת Hantzsche–Wendt" כבעיית SAT, ופותר SAT מצא אחת[89] (P040). דוגמה נגדית לשאלה אלגברית, במקום שבו כמעט אף אחד לא חשב להפעיל פותר בוליאני.

המשושה הריק

ב-2024 הוכיחו Marijn Heule ו-Manfred Scheucher שבכל קבוצה של 30 נקודות במצב כללי במישור יש משושה קמור ריק. גם זו הוכחת SAT. אבל כאן קרה דבר נוסף: הקידוד עצמו, כלומר הטענה שבעיית ה-SAT באמת שקולה לשאלה הגאומטרית, אומת ב-Lean על ידי Subercaseaux ושותפיו[90] (P051). זה סוגר את החור הנפוץ ביותר בהוכחות SAT: לא הפלט של הפותר, אלא הגשר בין המתמטיקה לנוסחה.

הלקח לסוכנים. שאלות מסוג "האם קיים" בגודל חסום מקודדים ל-SAT, עם שבירת סימטריות ו-cube-and-conquer. מאמתים את הפלט בתעודת DRAT או LRAT, וחשוב לא פחות: מאמתים את הקידוד. צינור empty hexagon הוא התבנית של G050.

דפוס 8: קיום בלי בנייה

הצביעה האקראית של ארדש

מספר רמזי $R(k,k)$ הוא המספר הקטן ביותר n כך שבכל צביעה של קשתות הגרף השלם על n קודקודים בשני צבעים יש k קודקודים שכל הקשתות ביניהם באותו צבע. כדי להראות ש-$R(k,k)$ גדול, צריך להציג צביעה בלי קבוצה כזו. אף אחד לא ידע לבנות צביעות כאלה בגדלים גדולים.

ב-1947 Paul Erdős עקף את הבעיה. הוא צבע את הקשתות באקראי, מטבע לכל קשת, וחישב שההסתברות שתופיע קבוצה מונוכרומטית קטנה מ-1 כאשר $n < 2^{k/2}$. אם ההסתברות לכישלון קטנה מ-1, יש צביעה שמצליחה. הוא הוכיח שהיא קיימת בלי להציג אותה[91] (P014). עד היום לא ידוע איך לבנות צביעה מפורשת שמגיעה לחסם הזה.

שלושה עמודים, שני עמודים

הגרסה המודרנית של הדפוס היא "שיטת הפולינום". ב-2008 Zeev Dvir הוכיח את השערת Kakeya מעל שדות סופיים בהוכחה קצרה במיוחד: פולינום מדרגה נמוכה שמתאפס על קבוצת Kakeya חייב להתאפס בכל מקום (P027). ב-2016 Croot, Lev ו-Pach פיתחו טיעון פולינומי לבעיית cap set מעל $\mathbb{Z}_4^n$, ובתוך שבועות Ellenberg ו-Gijswijt התאימו אותו ל-$\mathbb{F}_3^n$ וקיבלו שיפור מעריכי בהוכחה של שלושה עמודים (P035). ב-2019 Hao Huang פתר את השערת הרגישות (sensitivity), שהייתה פתוחה כשלושים שנה, בשני עמודים, בעזרת מטריצת סימנים של הקוביה וספקטרום[92] (P038).

הלקח לסוכנים. כשבנייה מפורשת נכשלת שוב ושוב, מנסים אובייקט אקראי עם חישוב תוחלת, פולינום מדרגה נמוכה שמתאפס על הקבוצה, או קידוד ספקטרלי. אלה הוכחות קצרות, וההוכחות הקצרות הן גם הקלות ביותר לפירמול.

דפוס 9: אינטואיציה פיזיקלית כמחולל השערות

המכתב של ארכימדס שנמצא מתחת לתפילות

ארכימדס כתב חיבור בשם "השיטה", ובו הסביר איך הוא מגלה את תוצאותיו: הוא דמיין פרוסות דקות של גופים מונחות על מנוף, ושקל אותן זו מול זו. כך מצא, למשל, שנפח הכדור הוא שני שלישים מנפח הגליל החוסם אותו. ארכימדס הבהיר שהשקילה הזו אינה הוכחה. אחרי הגילוי הוא הוכיח כל תוצאה בשיטת המיצוי הקפדנית.

החיבור אבד במשך מאות שנים. ב-1906 זיהה J. L. Heiberg בקושטא קלף שעליו נכתבו תפילות ביזנטיות, ומתחתן, מגורד חלקית, טקסט של ארכימדס, ובו "השיטה"[93] (P002). עיקרון ההפרדה בין גילוי להוכחה, בן יותר מאלפיים שנה, הוא בדיוק עיקרון רמות הטענה של המעבדה: L1 אינו L3.

חמשת אלפים קווים וקונוס אחד

ב-1991 קבוצה של פיזיקאים בראשות Philip Candelas השתמשה בעיקרון מתורת המיתרים בשם סימטריית מראה (mirror symmetry) כדי לחזות כמה עקומות רציונליות מכל דרגה יש על משטח מסוים, ה-quintic threefold. גאומטריקנים ידעו לחשב את המספרים לדרגות נמוכות בלבד, והפיזיקאים נתנו נוסחה לכולן. כשהתברר שהחישוב של גאומטריקנים לדרגה 3 לא התאים לחיזוי, התגלתה טעות דווקא בחישוב של הגאומטריקנים [לא אומת: הפרטים מבוססים על תיאורים משניים מוכרים של הפרשה ולא נבדקו מול המקור]. ההוכחות המתמטיות של הנוסחה הגיעו מאוחר יותר, אצל Givental ואצל Lian, Liu ו-(P020) Yau[94]. באותו אופן, Richard Borcherds הוכיח את השערת ה-moonshine של Conway ו-Norton בעזרת כלים שנלקחו מתורת המיתרים (P021).

הלקח לסוכנים. כדאי להפעיל "מצב גילוי" נפרד: נומריקה, אנלוגיות פיזיקליות, פונקציונלים אקסטרמליים. הפלט של המצב הזה מסומן תמיד כהשערה, ועובר ל"מצב הוכחה" בנפרד, כמו אצל ארכימדס. השאלה המעשית שסוכן יכול לשאול היא זו ששאל יוהאן ברנולי כשפתר את בעיית הברכיסטוכרון בעזרת עקרון הזמן המינימלי של פרמה מהאופטיקה (P003): איזו מערכת פיזיקלית ממקסמת או ממזערת את אותו פונקציונל?

דפוס 10: נחיל, פיגום ומבט מבחוץ

שלושה שבועות בדצמבר

בנובמבר 2023 הוכיחו Timothy Gowers, Ben Green, Freddie Manners ו-Terence Tao את השערת Marton (Polynomial Freiman–Ruzsa) מעל $\mathbb{F}_2$. Tao הציע לפרמל את ההוכחה ב-Lean, ופתח **blueprint**: מסמך שבו ההוכחה מפורקת לעשרות למות, וכל למה מקושרת ללמות שהיא נשענת עליהן. מתנדבים בחרו למות, פירמלו אותן, והגרף הצבוע הראה מה כבר נסגר. כל הפורמליזציה הסתיימה בתוך כשלושה שבועות[95] (P047).

שנה אחר כך Tao הוביל את ה-Equational Theories Project: כ-22 מיליון שאלות מהסוג "האם חוק אלגברי אחד גורר חוק אחר", בין 4,694 חוקים של מגמות (magmas). רוב השאלות הוכרעו על ידי מוכיחים אוטומטיים ומוצאי מודלים נגדיים, בני אדם טיפלו בשארית, ו-Lean שימש כבורר הסופי[96] (P054). באותה שנה, קבוצת bbchallenge, שרבים בה חובבים, סיווגה את כל מכונות טיורינג עם חמישה מצבים והוכיחה ב-Coq ש-(P053) $BB(5) = 47{,}176{,}870$[97].

המרצה מניו המפשייר

הצד השני של הדפוס הוא המבט מבחוץ. ב-2013 Yitang Zhang, מרצה שכמעט לא היה מוכר בקהילת המחקר, הוכיח שיש אינסוף זוגות ראשוניים שההפרש ביניהם קטן מ-70 מיליון (P028). תוך חודשים Maynard ו-Tao, כל אחד בנפרד, מצאו גישה פשוטה יותר (P029), ופרויקט Polymath8

הוריד את החסם ל-246 (P030). ב-2018 Aubrey de Grey, ביולוג שעסק בחקר הזדקנות, בנה גרף של 1,581 קודקודים שהראה שלצביעת המישור צריך לפחות חמישה צבעים, בעיה שעמדה במקומה מאז שנות ה-50 (P037).

הלקח לסוכנים. כל פרויקט מתחיל ב-blueprint, גרף תלויות של למות. הלמות מוקצות במקביל לסוכנים, Lean הוא הבורר, והיומן ציבורי. בבעיות אחידות עם הרבה מקרים, בסגנון ETP, מריצים מוכיחים אוטומטיים ו-SAT ומשאירים לבני אדם רק את מה שנשאר פתוח. והלקח של זאנג ודה-גריי: לא לפסול כיוון רק כי הוא לא מהמקום הצפוי.

סיפור צד: האלכסון

שלושה מהסיפורים בקטלוג חולקים טריק אחד, ושווה לציין אותו בנפרד. ב-1891 קנטור הראה שאין רשימה שמכילה את כל הרצפים הבינאריים האינסופיים: לכל רשימה בונים רצף שונה מהרצף ה-*n* במקום ה-*n* (P008). ב-1931 גדל קידד משפטים כמספרים, כך שנוסחה יכולה לדבר על היכיחות של עצמה, והפעיל את אותו טריק (P012). וב-1955 הלל פורסטנברג נתן הוכחה טופולוגית לאינסוף הראשוניים, שהיא דוגמה לדפוס 3 בתוך דפוס 1 (P015). מבחינת המעבדה, האלכסון הוא תבנית מוכנה להוכחות אי-אפשריות, והיא נתמכת היטב ב-Mathlib (`Function.cantor_surjective`). פרק 11 מספר את האלכסון כמשל.

ה-playbook של הסוכנים

הטבלה הבאה מתרגמת כל דפוס להוראת עבודה. היא מועתקת, בעריכה קלה, מ-`docs/creative-proofs.md`, והיא אמורה להיכנס כמעט מילה במילה להנחיות של הסוכנים ב- `agents/` .

#	דפוס	הוראת עבודה	סוכן אחראי	מקרי מבחן
0	חיפוש ספרות	לפני כל ניסיון, לחפש פתרון קיים (arXiv, erdosproblems.com, Mathlib). אסור לכתוב "נפתר" בלי בדיקה מתועדת.	literature-agent	P057
1	לשנות שפה	לייצר לפחות שלושה ניסוחים חלופיים (גרפי, אלגברי, הסתברותי, פונקציה יוצרת) ולדרג אותם לפי אורך ההוכחה הצפוי.	analogy-agent	P004, P005, P007

#	דפוס	הוראת עבודה	סוכן אחראי	מקרי מבחן
2	כוח גס אמין	הוכחת צמצום ב-Lean, בדיקה ב- `decide` או בבודק מאומת, שמירת התעודה.	search-agent, lean-agent	P016, P017, P025, P034
3	גשר בין תחומים	לנהל מילון התאמות ולשאול: באיזה תחום יש לבעיה משפט מבני חזק?	analogy-agent	P022, P036
4	אינווריאנט	להגדיר את מערכת המהלכים, להציע מועמדים ולבדוק אותם על מהלכים אקראיים לפני הוכחה.	construction-agent	P009, P013
5	המכונה מחפשת	נתונים, מודל, attribution, השערה מנוסחת, ניסיון הוכחה. ההשערה נשמרת עם הנתונים.	construction-agent, lean-agent	P041, P042
6	חיפוש תוכניות	מעריך דטרמיניסטי עם אריתמטיקה מדויקת, ואחריו לולאה אבולוציונית. התוצר: תעודה.	construction-agent	P049, P056
7	SAT	קידוד עם שבירת סימטריות, תעודת DRAT/LRAT, ואימות הקידוד עצמו.	search-agent	P040, P051
8	קיום בלי בנייה	תוחלת על אובייקט אקראי, פולינום מתאפס, קידוד ספקטרלי.	lean-agent	P014, P035, P038
9	אינטואיציה פיזיקלית	מצב גילוי נפרד, ופלט שמסומן כהשערה.	physics-agent	P002, P020
10	נחיל ו-blueprint	גרף תלויות, הקצאה מקבילית, Lean כבורר, יומן ציבורי.	orchestrator	P047, P054
11	ניסוח מדויק	אדם מבקר את הניסוח הפורמלי מול הכוונה	אדם	P058

#	דפוס	הוראת עבודה	סוכן אחראי	מקרי מבחן
		המקורית לפני כל הכרזה.		

שלושה דפוסים נותנים למעבדה את התשואה המיידית הגבוהה ביותר: חיפוש תוכניות עם מעריך (6), חישוב ו-SAT עם תעודה מאומתת ב-Lean (2 ו-7), ו-blueprint עם Lean כבורר (10). שלושתם כבר הוכיחו את עצמם בקנה מידה גדול, ושלושתם מייצרים פלט שמכונה יכולה לבדוק. זו גם הסיבה ששלוש המטרות הראשונות בבנק (G050, G023+G052, G029) בנויות בדיוק עליהם.

איך משתמשים בטבלה בפועל

ה-orchestrator לא מריץ את כל הדפוסים על כל מטרה. לכל מטרה ב- `data/goals.csv` יש שדה `suggested_attack` , והוא מצביע על הדפוסים הרלוונטיים. מטרה מדרגה C של שיפור חסם תפעיל בעיקר את דפוסים 6 ו-7. מטרה מדרגה D של פורמליזציה תפעיל את דפוס 10. ה-analogy-agent מפעיל את דפוסים 1 ו-3 על כל מטרה שנתקעה, ומחזיר ניסוחים מחדש שעוברים לסוכנים האחרים לבדיקה.

כדאי לזכור גם מה הטבלה לא עושה. היא לא מייצרת יצירתיות. היא מבטיחה שהצעדים הצדדיים שבני אדם עשו פעם בדור, לפעמים במקרה, ייבדקו כל פעם, באופן שיטתי, וכל תוצאה תעבור את אותו בודק.

בקצרה

- רוב ההוכחות פורצות הדרך הגיעו מצעד צדדי ולא ממאמץ נוסף באותו כיוון. הקטלוג `data/creative_proofs.csv` מסווג שישים מקרים לעשרה דפוסים.
- שני כללים קודמים לכל דפוס: חיפוש ספרות (פרשת ארדש באוקטובר 2025) ובדיקת הניסוח הפורמלי (#728).
- כמעט כל הצלחה בקטלוג, מארבעת הצבעים ועד FunSearch, נשענת על מנגנון בדיקה חיצוני: Coq, Lean, בודק DRAT או מעריך.
- הדפוסים עם התשואה הגבוהה ביותר למעבדה הם חיפוש תוכניות עם מעריך, SAT עם תעודה, ו-blueprint עם Lean כבורר.
- כל דפוס מתורגם להוראת עבודה לסוכן מסוים ב-playbook, ושלוש המטרות הראשונות בבנק בנויות עליהם.
- מבט מבחוץ (רייס, זאנג, דה-גריי, סמית) הוא דפוס בפני עצמו, וסיבה לא לפסול כיוון בגלל מקורו.

רעיונות לתוכן

1. **"עשרה צעדים הצידה": סדרת פוסטים.** פוסט קצר לכל דפוס, עם סיפור אחד ואיור אחד (הגשרים, לוח השחמט, הכובע). קהל יעד: הקהל הרחב ברשתות, סטודנטים.
2. **"כשהמחשב הוכיח והשופטים התעייפו": כתבה ב-S27 News.** הסיפור של קפלר ו-Flyspeck כמבוא לשאלה למה המעבדה מתעקשת על Lean. קהל יעד: קוראי מדע פופולרי, אנשי תוכנה.
3. **"פתרנו? לא בדיוק": סרטון הסבר.** פרשת ארדש 2025 מול 728#, ושתי הטענות השונות. קהל יעד: עיתונאי טכנולוגיה, משתמשי AI.
4. **"המכתב שנמצא מתחת לתפילות": סיפור קצר מאויר.** הפלימפססט של ארכימדס והעיקרון שגילוי אינו הוכחה, כהקדמה לרמות הטענה. קהל יעד: בני נוער, מורים.
5. **"Playbook פתוח": פוסט טכני.** הטבלה המלאה עם קישורים לקוד הסוכנים, והזמנה לקהילה להציע דפוסים נוספים. קהל יעד: חוקרי AI-for-Math, קהילת Lean.

פרק 10: פיזיקה תיאורטית

כשהמייסד שאל "אפשר להיכנס גם לפיזיקה תיאורטית?", התשובה שלנו הייתה כן, בתנאי שבוחרים נכון. הפרק הזה מסביר מה פירוש "לבחור נכון". הפיזיקה התיאורטית מלאה בבעיות פתוחות, אבל רק חלק מהן הן בעיות במובן שמתמטיקאי, או עוזר הוכחות, יכול לזהות: טענה מדויקת שאפשר להוכיח או להפריך. החלק האחר, ולפעמים החלק המעניין יותר לפיזיקאים, עוסק בבחירת מודלים, בפרשנות ובקשר לניסוי.

הפרק עונה על ארבע שאלות. מה פירוש "בעיה פתוחה בפיזיקה תיאורטית" מבחינה מתמטית? למה יש פער בין מה שפיזיקאים מקבלים כנכון לבין מה שמתמטיקאים מקבלים כמוכח? אילו בעיות פתוחות קיימות היום, ואיפה מהן AI ושיטות פורמליות יכולים לתרום תרומה אמיתית? ובאילו פרויקטים כדאי למעבדה להתחיל?

הפרק נשען על `docs/theoretical-physics.md` , על השורות מדרגה P ב- `data/goals.csv` (G067 עד G077) ועל סעיף 6 ב- `docs/architecture.md` . הוא מתחיל ברקע מושגי ארוך יחסית, משום שבלי הרקע הזה קל מאוד להבטיח יותר ממה שאפשר לקיים.

שתי תרבויות של "נכון"

הפונקציה שלא הייתה פונקציה

בשנות העשרים של המאה ה-20 פול דיראק נזקק, לצורך מכניקת הקוונטים, לאובייקט מוזר: "פונקציה" $\delta(x)$ ששווה לאפס בכל מקום חוץ מנקודה אחת, ובכל זאת השטח שמתחתיה שווה ל-1. כל מתמטיקאי באותה תקופה יכול היה להסביר לדיראק שפונקציה כזו לא קיימת. פיזיקאים השתמשו בה בכל זאת, וקיבלו תשובות נכונות.

רק בשנות הארבעים Laurent Schwartz בנה את תורת ההתפלגויות (distributions), שבה ל-δ יש משמעות מדויקת. היא אינה פונקציה, היא פונקציונל. החשבונות של דיראק התברררו כנכונים, והמתמטיקה קיבלה ענף חדש. Schwartz קיבל על כך את מדליית פילדס ב-1950.

הסיפור הזה חוזר שוב ושוב בהיסטוריה של הפיזיקה. הפיזיקאים מגיעים ראשונים עם כלי שעובד ואין לו הצדקה. המתמטיקאים מגיעים אחר כך, לפעמים אחרי עשרות שנים, ובונים לו הצדקה. בדרך הם לפעמים מגלים שהכלי עובד רק בתנאים מסוימים, ושמחוץ להם הוא נותן תשובות שגויות.

הפער הגדול: תורת השדות הקוונטית

הדוגמה הבולטת ביותר היא תורת השדות הקוונטית (quantum field theory, QFT), התשתית של המודל הסטנדרטי של חלקיקים. התורה הזו נותנת חיזויים שמתאימים לניסויים בדיוק של עשר ספרות ויותר. ובכל זאת, אף אחד לא הוכיח שהתורה של המודל הסטנדרטי בארבעה ממדים קיימת כאובייקט מתמטי.

החשבונות בתורת השדות משתמשים ב"אינטגרלי מסלול" (path integrals) על מרחבים אינסופיים שאין עליהם מידה מתמטית מוגדרת, ובטורים שמתבדרים ובכל זאת נותנים מספרים מדויקים כשקוטעים אותם במקום הנכון. תהליך הרנורמליזציה (renormalization) מסלק אינסופים בדרך שעובדת, ושיש לה היום הבנה עמוקה, אבל לא הוכחה כללית.

בשנות החמישים ניסחו Arthur Wightman ואחרים אקסיומות לתורת שדות קוונטית: רשימה של תכונות שתורה כזו צריכה לקיים. בשנות השבעים Osterwalder ו-Schrader נתנו גרסה שקולה בזמן מדומה. התוכנית הזו, "תורת שדות קונסטרוקטיבית", הצליחה לבנות תורות לא טריוויאליות בשניים ובשלושה ממדים. בארבעה ממדים, הממד של העולם שלנו, היא לא הצליחה עד היום.

מה זה אומר על "בעיה פתוחה"

כשאומרים שבעיה בפיזיקה תיאורטית "פתוחה", אפשר להתכוון לשלושה דברים שונים מאוד:

1. **בעיה מתמטית מדויקת שמקורה בפיזיקה.** לדוגמה: האם למשוואות נאוויה-סטוקס בשלושה ממדים יש פתרון חלק לכל נתון התחלתי חלק? זו טענה מתמטית חדה, ואפשר לכתוב אותה ב-Lean. אין בה שום עמימות פיזיקלית.
2. **בעיה שבה התשובה ידועה לפיזיקאים, וההוכחה חסרה.** לדוגמה: השערת Haldane (G070), שלפיה לשרשרת ספינים אנטי-פרומגנטית עם ספין שלם יש פער ספקטרלי. נומריקה תומכת בה בבירור, ופיזיקאים מתייחסים אליה כעובדה. ההוכחה פתוחה.
3. **בעיה שעוד לא נוסחה כטענה.** לדוגמה: פרדוקס המידע של חורים שחורים, או השאלה מהי תורת הכבידה הקוונטית הנכונה. כאן אין עדיין "טענה" שאפשר לבדוק, ואין לעוזר הוכחות מה לעשות.

המעבדה עובדת כמעט רק על בעיות מהסוג הראשון והשני, ועל חלקים מוגדרים היטב של בעיות מהסוג השלישי. זה לא מפני שהסוג השלישי פחות חשוב. זה מפני שהכלים שלנו, Lean, חישוב מאומת וחיפוש עם מעריך, צריכים טענה כדי לעבוד.

"מתמטיקה תיאורטית" והוויכוח של 1993

ב-1993 פרסמו Arthur Jaffe ו-Frank Quinn ב-Bulletin of the AMS מאמר בשם "Theoretical mathematics", והציעו להבחין בין עבודה ספקולטיבית, בהשראת הפיזיקה, לבין עבודה מוכחת, ולסמן אותן בבירור[98]. המאמר עורר ויכוח חריף. מתמטיקאים בכירים השיבו שההבחנה מלאכותית, ושהשראה לא מוכחת היא חלק מהותי מהמתמטיקה. הוויכוח רלוונטי ישירות למעבדה:

מערכת רמות הטענה שלנו (L0 עד L4) היא בעצם ההצעה של Jaffe ו-Quinn, ממומשת בקוד. היא לא אומרת שהשערה פתוחה שווה. היא אומרת שצריך לכתוב עליה שהיא השערה.

רמות ודאות בפיזיקה

`docs/theoretical-physics.md` מגדיר שלוש רמות לתוצרי פיזיקה: H (היוריסטי או נומרי), R (נומריקה ריגורוזית עם תעודה) ו-L (מאומת ב-Lean). המעבדה כולה משתמשת בסולם L0 עד L4 (`docs/architecture.md`, סעיף 3). שתי המערכות מתיישבות כך:

רמה במסמך הפיזיקה	משמעות	רמה במעבדה	`certificate.kind`
H	נומריקה רגילה, ניסוי מחשבתי, השערת AI	L0 או L1	none
R	interval arithmetic, רציונלים מדויקים, תעודת SDP	L3 (אחרי שה-verifier בדק את התעודה)	exact-arith, sdp-rational
L	משפט מוכח ב-Lean (Mathlib, PhysLean)	L3	lean

כלל הברזל זהה בשתי המערכות: טענה ברמה H לא תוצג לעולם כאילו היא R או L.

ויש סייג נוסף, שהוא ייחודי לפיזיקה. הוכחה ב-Lean מראה שמשפט נובע מההנחות. היא לא מראה שהטבע מקיים את ההנחות. אם הוכחנו שבמודל Ising יש מעבר פאזה, הוכחנו משהו על מודל Ising, ולא על מגנט מסוים במעבדה. כל פרסום של המעבדה בפיזיקה יציין במפורש מה הוכח ומה הונח (פרק 12).

מפת הבעיות הפתוחות

הסעיף הזה עובר על הבעיות הפתוחות הבולטות, ולכל אחת שואל שתי שאלות: מה הבעיה במונחים מתמטיים, ומה צוות קטן עם AI ושיטות פורמליות יכול לתרום. הציונים (1 עד 5) מגיעים מ-`docs/theoretical-physics.md`, והם הערכה ולא מדידה.

Yang–Mills וקיום פער המסה

זו אחת משבע בעיות המילניום של מכון Clay[99]. צריך להוכיח שתורת Yang–Mills קוונטית, עם חבורת כיול (gauge group) קומפקטית ופשוטה, קיימת על $\mathbb{R}^4$ במובן של אקסיומות Wightman או Osterwalder–Schrader, ושיש לה פער מסה $\Delta > 0$. במילים פשוטות: שהחלקיקים הקלים ביותר בתורה הם בעלי מסה חיובית.

הבעיה פתוחה לגמרי, וכאמור גם בניית תורה לא טריוויאלית כלשהי בארבעה ממדים פתוחה. מה כן אפשר לעשות: לפרמל את התשתית. אקסיומות Osterwalder–Schrader, תורות כיול על סריג סופי, חסמי צימוד חזק. זו עבודת יסודות ארוכה, ולא תפתור את הבעיה, אבל היא תשתית שהקהילה תשתמש בה. ריאליות: 2 מתוך 5, לתשתית בלבד.

נאוויה-סטוקס ואוילר: התפוצצות

גם זו בעיית מילניום. האם לכל נתון התחלתי חלק בשלושה ממדים יש פתרון חלק לכל הזמנים, או שקיימת "התפוצצות" (blow-up) בזמן סופי?

כאן קרה בשנים האחרונות משהו מעניין במיוחד מבחינת המעבדה. ב-2022–2023 Jiajie Chen ו-Thomas Hou נתנו הוכחה בעזרת מחשב להתפוצצות של משוואות אוילר (בלי צמיגות) במקרה צירי-סימטרי בנוכחות שפה[100]. ב-2025 פרסמו DeepMind יחד עם Gómez-Serrano, Buckmaster ואחרים עבודה על גילוי סינגולריות לא-יציבות בעזרת רשתות נוירונים מבוססות פיזיקה (PINNs) ודיוק נומרי גבוה מאוד, כצעד לקראת הוכחות בעזרת מחשב (G067).

השיטה הזו היא בדיוק דפוס 9 מפרק 9, בגרסה מודרנית: רשת נוירונים מוצאת פתרון מקורב, ואז אריתמטיקת קטעים מוכיחה שיש פתרון אמיתי בקרבתו. צוות קטן יכול לשחזר חלקים, למשל מודלים חד-ממדיים כמו מודל De Gregorio, ברמה R. ריאליות: 3 מתוך 5 לשחזור מודלים צעצועיים, 1 מתוך 5 לבעיה עצמה.

Conformal bootstrap ומודל Ising התלת-ממדי

זה כנראה המקום שבו המעבדה יכולה לתרום הכי הרבה, ולכן נסביר אותו בפירוט.

מודל Ising הוא המודל הפשוט ביותר למגנט: ספינים על סריג, כל אחד למעלה או למטה, וכל אחד מעדיף להיות כמו שכניו. בטמפרטורה קריטית מסוימת המודל עובר מעבר פאזה, וליד המעבר מתנהגים גדלים פיזיקליים כמו חזקות עם "מעריכים קריטיים". המעריכים האלה זהים, באופן מפתיע, למעריכים של מערכות פיזיקליות שונות לגמרי, כמו נקודה קריטית של נוזל וגז. בשני ממדים Lars Onsager פתר את המודל במדויק ב-1944. בשלושה ממדים אין פתרון מדויק.

ה-bootstrap הקונפורמי עובד אחרת. הוא לא מנסה לפתור את המודל. הוא מניח שבנקודה הקריטית התורה סימטרית תחת טרנספורמציות קונפורמיות, ומוסיף שתי דרישות: crossing symmetry

(עקביות של פירוק פונקציות מתאם בשתי דרכים) ואוניטריות (הסתברויות חיוביות). מתברר שהדרישות האלה מספיקות כדי לחסום מאוד את הערכים האפשריים של ממדי האופרטורים. מבחינה מתמטית זו בעיית אופטימיזציה מסוג semidefinite programming (SDP).

ב-Kos, Poland, Simmons-Duffin 2016 ו-Vichi קיבלו כך את הערכים $\Delta_\sigma = 0.5181489(10)$ ו-$\Delta_\epsilon = 1.412625(10)$[101], הערכים המדויקים ביותר שהיו ידועים. עבודה מ-2024 של Chang ושותפיו שיפרה את הדיוק עוד יותר [לא אומת: הערכים המדויקים צריכים בדיקה מול arXiv לפני ציטוט].

ויש כאן נקודה עדינה. התוצאות האלה מגיעות מפותר נומרי, SDPB[102], שעובד בדיוק גבוה מאוד, אבל הוא עדיין פותר נומרי. במונחי המעבדה זו רמה L1. התרומה שהמעבדה יכולה להוסיף היא להפוך חלק מהחסמים לתעודות: לחלץ את הפונקציונל הדואלי מהפתרון, לעגל אותו לרציונלים, ולבדוק באריתמטיקה מדויקת שהוא חיובי בכל מקום שצריך. אם הבדיקה עוברת, החסם (בהינתן ההנחות של ה-bootstrap) מוכח. זו `certificate.kind = "sdp-rational"` בארכיטקטורה, והיא המטרה G068. ריאליות: 4 מתוך 5.

אמפליטודות פיזור וה-Amplituhedron

אמפליטודות פיזור (scattering amplitudes) הן הגדלים שמהם מחשבים את ההסתברות שחלקיקים יתפזרו בדרך מסוימת. במשך עשורים חישבו אותן בעזרת דיאגרמות פיינמן, שמספרן גדל מהר מאוד. ב-1986 Parke ו-Taylor מצאו שעבור סוג מסוים של אמפליטודות (MHV), מאות דיאגרמות מצטמצמות לביטוי של שורה אחת. מאז התגלו עוד ועוד זהויות אלגבריות: נוסחת הרקורסיה BCFW, הדואליות בין צבע לקינמטיקה (BCJ), ויחסי KLT, שאומרים, באופן גס, ש"כבידה היא ריבוע של תורת כיול".

ב-2013 Arkani-Hamed ו-Trnka הציגו את ה-Amplituhedron, אובייקט גאומטרי שה"נפח" שלו נותן את האמפליטודות בתורה מסוימת (N=4 SYM)[103]. מאחוריו עומדת קומבינטוריקה עשירה של ה-Grassmannian החיובי.

כל זה מתאים מאוד למעבדה מסיבה פשוטה: זהויות של אמפליטודות הן זהויות אלגבריות, ואפשר לבדוק אותן במדויק, על נקודות קינמטיות רציונליות, ובהמשך גם להוכיח אותן ב-Lean. ריאליות: 4 מתוך 5.

ב-2026 הופיעה בתחום הזה דוגמה מוחשית לשילוב של AI. במאמר "Single-minus gluon tree amplitudes are nonzero" (Guevara, Lupsasca, Skinner, Strominger, Weil) הציע GPT-5.2 Pro נוסחה כללית מתוך מקרים שבני אדם חישבו, וגרסה פנימית של המודל הפיקה הוכחה שבני אדם וידאו[104]. `docs/ai-math-lessons.md` מסכם את זה כ"תרומה אמיתית אבל צרה", ומציין ששניים מהמחברים מ-OpenAI. גם כאן הבדיקה האנליטית היא שהפכה את ההצעה לתוצאה.

מערכות אינטגרביליות ומשוואת Yang–Baxter

מערכת אינטגרבילית היא מערכת שיש לה הרבה כמויות שמורות, מספיק כדי לפתור אותה במדויק. במערכות קוונטיות בממד אחד, האינטגרביליות קשורה למשוואת Yang–Baxter, משוואה אלגברית על מטריצה R:

$$R_{12}R_{13}R_{23} = R_{23}R_{13}R_{12}.$$

de Leeuw, Pribytok ו-Ryan הראו ב-2019 איך לסווג באופן שיטתי שרשראות ספין אינטגרביליות עם אינטראקציות בין שכנים קרובים, בעזרת "אופרטור ה-boost"[105]. זה בעצם חיפוש ממוחשב עם מבנה.

כאן AI יכול לגלות משהו חדש באמת, וכל גילוי ניתן לבדיקה מכנית. משוואת Yang–Baxter היא זהות פולינומית, ולכן מועמד שעובר אותה בבדיקה מדויקת הוא פתרון. אין עמימות. הסיכון הוא אחר: רוב המרחב כבר סווג, והחידוש תלוי בבחירה חכמה של תת-מרחב. ריאליות: 4 מתוך 5.

קודים לתיקון שגיאות והולוגרפיה

ב-2015 הציעו Pastawski, Yoshida, Harlow ו-Preskill את "קוד HaPPY": קוד קוונטי לתיקון שגיאות, הבנוי על רשת טנזורים היפרבולית, שמממש כמה מהתכונות של הדואליות AdS/CFT[106]. קודי stabilizer הם אלגברה לינארית מעל השדה $\mathbb{F}_2$, ולכן מתאימים מאוד לפירמול ב-Lean. אפשר להוכיח שקוד מסוים מתקן סוג נתון של שגיאות, לחשב מרחק קוד, ולחפש קודים חדשים. ריאליות: 4 מתוך 5.

AdS/CFT וקונוס האנטרופיה ההולוגרפי

הדואליות AdS/CFT, שהציע Juan Maldacena ב-1997, אומרת שתורת כבידה במרחב מסוים שקולה לתורת שדות קונפורמית על השפה שלו. הדואליות עצמה אינה משפט. אבל ממנה נגזרות טענות מתמטיות מוגדרות. אחת מהן היא "קונוס האנטרופיה ההולוגרפי": מהם כל אי-השוויונות שמקיימות האנטרופיות של N אזורים בתורה הולוגרפית? זו בעיה קומבינטורית-גאומטרית, ושיטת ההוכחה המקובלת בה ("proof by contraction") מתאימה לחיפוש ממוחשב. ריאליות: 3 מתוך 5.

כבידה קוונטית ו-string landscape

רוב השאלות בכבידה קוונטית (השערת הכבידה החלשה, תוכנית ה-swampland, פרדוקס המידע) עוד לא מנוסחות כטענות מתמטיות. החלק שכן מנוסח, למשל ספירת מיקרו-מצבים של חורים שחורים דרך פונקציות מודולריות, הוא תורת מספרים, ואפשר לאמת בו זהויות ונוסחאות אסימפטוטיות. ריאליות: 2 מתוך 5.

ב-string landscape (ההערכה המפורסמת של כ-10^{500} ואקואות) יש כבר עבודה רבה על למידת מכונה במסדי נתונים של יריעות Calabi–Yau, כדי לחזות מספרי Hodge[107]. התוספת האפשרית של

המעבדה היא אימות של התכונות הטופולוגיות שהמודלים חוזים. ריאליות: 3 מתוך 5 לשחזור, 2 מתוך 5 לחידוש.

בעיות נוספות בבנק

שורות P בבנק המטרות כוללות גם בעיות בפיזיקה מתמטית "קלאסית" יותר:

- **הבעיה השישית של Smale (G069):** האם מספר הקונפיגורציות המרכזיות בבעיית n הגופים סופי? הוכח ל-$n = 4$ (Hampton–Moeckel 2006) ול-$n = 5$ למסות גנריות (Albouy–Kaloshin 2012).
- **אי-שוויוני Bell (G072):** הערך הקוונטי המקסימלי של אי-שוויון I_{3322}, והשאלה אם מספיק מימד סופי. ידוע ערך של כ-0.25087556 בממדים גבוהים (Pál–Vértesi 2010).
- קבוע **Lieb–Thirring (G073), חוק השטח בשני ממדים (G071), הליכה עצמית-נמנעת (G075):** בעיות שיש בהן מקום לחסמים נומריים מוכחים.
- **הבעיה השישית של הילברט (G074):** גזירה ריגורוזית של משוואות הזורמים מדינמיקה של חלקיקים. ב-Deng, Hani 2025 ו-Ma הודיעו על גזירה כזו, והעבודה עדיין בבדיקה. זו דוגמה לבעיה שבה literature-agent חייב לעדכן סטטוס לפני כל עבודה.
- **יציבות Kerr וצנזורה קוסמית (G076):** חשובה מאוד, אבל רחוקה מהכלים שלנו (ריאליות 1).
- **תרומות ל-PhysLean (G077):** הפרויקט המעשי ביותר, ראו להלן.

איך AI ושיטות פורמליות משתלבים

ארבעה תפקידים

הניסיון של 2019–2026 (`docs/ai-math-lessons.md` , סעיף 6) מראה שבפיזיקה, כמו במתמטיקה, ההצלחות נמצאות היכן שיש בדיקה עצמאית. אפשר למנות ארבעה תפקידים שבהם AI ושיטות פורמליות מתחברים:

1. **גילוי בעזרת חיפוש, ואימות מדויק.** מערכות אינטגרביליות, זהויות אמפליטודה, קודים. AI מציע מועמדים, בדיקה אלגברית מדויקת מכריעה.
2. **נומריקה שהופכת להוכחה.** bootstrap, התפוצצות. פותר נומרי או רשת נוירונים מוצאים פתרון מקורב, ואריתמטיקת קטעים או תעודה רציונלית הופכות אותו למוכח.
3. **פירמול של יסודות.** PhysLean, אקסיומות QFT, מכניקה קוונטית בסיסית. כאן AI משמש ככותב טיוטות Lean, ו-Lean בודק.
4. **רגרסיה סימבולית.** גילוי נוסחאות מנתונים, עם PySR[108] או AI Feynman[109]. זה עובד היטב על נוסחאות מספרי לימוד, ופחות על פיזיקה חדשה באמת.

ומה שלא עובד: "תיאוריה חדשה של הכול" שמודל שפה כותב. טקסט כזה אינו טענה שאפשר לבדוק, וכל פרסום שלו יפגע באמינות המעבדה.

ה-physics-agent

בארכיטקטורה של המעבדה יש סוכן ייעודי לפיזיקה (`agents/physics-agent.md`). התפקיד הראשון שלו הוא ניסוח. בעיה בפיזיקה תיאורטית היא לרוב לא טענה אחת, אלא משפחה של הנחות: אוניטריות, crossing, פערים בספקטרום, קיטוע. הסוכן כותב ניסוח מתמטי מדויק עם רשימת הנחות מפורשת, ומבדיל בין החלק הריגורוזי לחלק ההיוריסטי. הרמה המקסימלית שהוא יכול לטעון בעצמו היא L1. רק ה-verifier יכול לקבוע L3.

PhysLean

הכלי המרכזי לפירמול פיזיקה ב-Lean הוא PhysLean (לשעבר HepLean), פרויקט קהילתי פתוח שהקים Joseph Tooby-Smith[110]. הוא כולל כבר סימון אינדקסים לטנזורים, את משפט Wick, תנאי ביטול אנומליות, אופרטורי לורנץ ועוד. העיקרון של המעבדה ברור: נתרום לפרויקט הקיים ולא נבנה ספרייה מתחרה.

הנה דוגמה קטנה לסגנון העבודה. זה לא קוד מ-PhysLean, אלא תרגיל חימום: אוסצילטור הרמוני חד-ממדי, והטענה שהפתרון $x(t) = A\cos(\omega t)$ מקיים את משוואת התנועה.

```
import Mathlib

open Real

/-- x(t) = A cos(ω t) מקיים x'' = -ω² x -/
theorem harmonic_solution (A ω : ℝ) :
    ∀ t, deriv (deriv (fun s ⇒ A * cos (ω * s))) t
      = -ω ^ 2 * (A * cos (ω * t)) := by
  intro t
  sorry -- תרגיל פתוח: להשלים בעזרת deriv_const_mul ו-deriv_cos
```

ה- `sorry` הוא סימן מכוון. קובץ כזה יכול לשבת במאגר כמטרה, אבל אסור שיופיע בתוצר שמסומן L3 (פרק 12). [לא אומת: הקוד לא קומפל בגרסת Mathlib של המעבדה, ושמות הלמות עשויים להשתנות.]

חמישה פרויקטים להתחלה

הפרויקטים מסודרים לפי ריאליות לצוות קטן, כמו ב- `docs/theoretical-physics.md` .

P1: גזירות מספר הלימוד ב-Lean ו-PhysLean

בוחרים 3 עד 5 גזירות קלאסיות ומפרמלים אותן: משוואות אוילר–לגראנז׳ לאוסצילטור הרמוני ופתרונן; שימור אנרגיה ממשפט Noether למערכת חד-ממדית; אלגברת התנע הזוויתי $[L_i, L_j] = i\hbar\,\varepsilon_{ijk} L_k$; הספקטרום של האוסצילטור הקוונטי בעזרת אופרטורי סולם, ברמה אלגברית. הסיכון נמוך, יש קהילה שתקבל את התרומות, והצוות בונה שריר Lean. התוצר: PR ל-PhysLean ופוסט בשם "מה למדנו כשמכונה בדקה את ספר הלימוד". זמן משוער: 4 עד 8 שבועות. זהו G077.

P2: רגרסיה סימבולית שמגלה חוקים מחדש

מריצים PySR ו-AI Feynman על ה-Feynman Symbolic Regression Database (כמאה משוואות מתוך Feynman Lectures) ועל נתונים אמיתיים, למשל מסלולי כוכבי לכת, כדי לגלות מחדש את חוקי קפלר. התוספת של המעבדה: כל נוסחה שמתגלה עוברת בדיקה סימבולית של שקילות לנוסחה הנכונה ובדיקת ממדים אוטומטית. התוצר: benchmark פנימי ודו"ח שגיאות, "איפה ה-AI נכשל ולמה". זמן: 3 עד 6 שבועות.

P3: אימות זהויות אמפליטודה

מממשים ובודקים במדויק, על נקודות קינמטיות רציונליות אקראיות, את נוסחת Parke–Taylor, את BCFW, את זהויות Kleiss–Kuijf ו-BCJ ואת יחסי KLT. בהמשך מפרמלים ב-Lean את הבסיס של פורמליזם spinor-helicity, כולל זהות Schouten. התוצר: ספריית בדיקות פתוחה ופרק ב-Lean. זמן: 6 עד 10 שבועות.

P4: חסם bootstrap עם נומריקה מאומתת

משחזרים חסם קלאסי, למשל חסם עליון על Δ_ϵ כפונקציה של Δ_σ בשלושה ממדים (Rattazzi–Rychkov–Tonni–Vichi 2008, El-Showk ושותפיו 2012[111]), בעזרת SDPB. אחר כך מחלצים את הפונקציונל הדואלי, מעגלים לרציונלים, ובודקים במדויק שהוא חיובי על כל הספקטרום הנדרש. התוצר: קוד פתוח, תעודה, והערה טכנית. זמן: 2 עד 4 חודשים, ודרוש יועץ מומחה ל-CFT. זהו G068.

P5: חיפוש מודלים אינטגרביליים חדשים

חיפוש אלגברי, מונחה AI, של מטריצות R או המילטוניאנים שמקיימים Yang–Baxter, למשל לשרשראות עם ממד מקומי 3 או עם אינטראקציות ארוכות טווח, על בסיס שיטת ה-boost. כל מועמד נבדק במדויק. התוצר: קטלוג פתרונות והשוואה לסיווגים הידועים, ואם יימצא משהו חדש, מאמר. זמן: 3 עד 6 חודשים.

על המדף לשלב ב': קודי stabilizer הולוגרפיים ב-Lean; חיפוש אי-שוויונים בקונוס האנטרופיה ההולוגרפי; שחזור התפוצצות במודל De Gregorio עם אריתמטיקת קטעים.

ארגז הכלים

כלי	תפקיד	פרויקט
Lean 4 + Mathlib	הוכחות פורמליות	כולם
PhysLean	פיזיקה ב-Lean	P1, G077
SDPB	פותר SDP בדיוק שרירותי	P4
PySR, AI Feynman	רגרסיה סימבולית	P2
Arb/FLINT, MPFI	אריתמטיקת קטעים	המרת L1 לתעודה
SymPy, FORM, Sage	אלגברה ממוחשבת	P3, P5
סוכני LLM	השערות, טיוטות Lean, תרגום מאמרים לקוד	כולם, תמיד עם בודק בסוף

הקטלוג המלא, 140 כלים ובהם 11 בקטגוריית physics, נמצא ב- `data/tools.csv` .

גבולות ויושר

ארבעה כללים מ- `docs/theoretical-physics.md` , שהם גם מדיניות פרסום (פרק 12):

1. **לא נכריז על פתרון של בעיית מילניום.** כל טענה כזו, שלנו או של AI, תעבור בדיקה פורמלית וביקורת עמיתים חיצונית לפני שתצא החוצה. בפועל, הסיכוי שנזדקק לכלל הזה קטן מאוד, והוא קיים כדי שאף אחד לא יתפתה.
2. **פיזיקה אינה מתמטיקה.** הוכחה ב-Lean מראה שמשפט נובע מהנחות, לא שהטבע מציית להן. מצייניים תמיד מה הוכח ומה הונח.
3. **שותפות עם פיזיקאים.** כל פרויקט מעבר ל-P1 ו-P2 צריך יועץ מהתחום.
4. **קרדיט.** תרומה לפרויקטים קיימים (PhysLean, SDPB) עדיפה על המצאה מחדש. מצטטים ומכבדים.

בקצרה

- "בעיה פתוחה בפיזיקה" יכולה להיות טענה מתמטית חדה, השערה שפיזיקאים כבר מקבלים בלי הוכחה, או שאלה שעוד לא נוסחה. המעבדה עובדת בעיקר על שני הסוגים הראשונים.
- בין הפיזיקה למתמטיקה יש פער ריגורוזיות היסטורי (פונקציית דלתא, תורת השדות הקוונטית). רמות הטענה של המעבדה הן דרך לסמן אותו בגלוי.
- הסולם H/R/L של מסמך הפיזיקה ממופה לסולם L0–L4 של המעבדה: נומריקה רגילה היא L1, תעודה שנבדקה או הוכחת Lean הן L3.
- ההזדמנויות הטובות ביותר: bootstrap עם תעודה רציונלית, זהויות אמפליטודה, חיפוש פתרונות Yang–Baxter, קודי stabilizer ותרומות ל-PhysLean.
- חמשת פרויקטי הפתיחה מסודרים לפי ריאליות, מ-P1 (פירמול ספר לימוד) ועד P5 (חיפוש מודלים אינטגרביליים).
- הוכחה ב-Lean אינה טענה על הטבע. כל פרסום מצייין מה הוכח ומה הונח.

רעיונות לתוכן

1. **"הפונקציה שלא הייתה פונקציה": כתבת רקע ב-S27 News.** הסיפור של דיראק ו-Schwartz כפתח לשאלה למה פיזיקאים ומתמטיקאים מתכוונים לדברים שונים כשהם אומרים "נכון". קהל יעד: קוראי מדע פופולרי, סטודנטים לפיזיקה.
2. **"מה למדנו כשמכונה בדקה את ספר הלימוד": סדרת פוסטים מפרויקט P1.** כל גזירה שפורמלה, והמקומות שבהם ספר הלימוד דילג על הנחה. קהל יעד: סטודנטים, מרצים, קהילת Lean.
3. **"מגנט, שלושה ממדים, ומספר עם עשר ספרות": סרטון הסבר על ה-bootstrap.** מה זה מעריך קריטי, למה Ising בשלושה ממדים קשה, ומה ההבדל בין חסם נומרי לחסם מוכח. קהל יעד: חובבי פיזיקה ב-YouTube.
4. **"כשה-AI ניחש נוסחה לגלואונים": כתבת ניתוח.** המקרה של 2026 כדוגמה לתרומה אמיתית וצרה, ומה הפך אותה לתוצאה. קהל יעד: עיתונאי מדע וטכנולוגיה.
5. **"בעיות המילניום: מה אנחנו לא נעשה ולמה": פוסט עמדה.** הסבר כן על Yang–Mills ונאוויה-סטוקס, ומה כן אפשר לתרום בשוליהן. קהל יעד: הקהל הרחב, משקיעים ושותפים אפשריים.

פרק 11: גשרי חוכמה

המייסד שאל שאלה שלא מופיעה בדרך כלל בתוכניות עבודה של מעבדות AI: האם יש מקום במעבדה לחוכמת הגמרא, לקבלה, ובפרט לנוסחאות של הרש"ש ולסוד הצירוף, ולחוכמה של תרבויות עתיקות? והאם אפשר לתרגם מתמטיקה למשלים? הפרק הזה עונה כן, ומסביר באילו תנאים.

הסיבה שהשאלה חשובה איננה נוסטלגיה. יש בה שלושה דברים מעשיים. הראשון הוא **שיטה**: הגמרא פיתחה במשך דורות פרוטוקול של קושיא ותירוץ שדומה מאוד למה שחוקרי AI מחפשים היום, ויכוח בין שני סוכנים עם בורר. השני הוא **מבנה**: ספר יצירה, הלוח העברי והכוונות של הרש"ש מכילים מבנים קומבינטוריים ואריתמטיים מדויקים, שאפשר לחקור ולפרמל. השלישי הוא **פדגוגיה**: משל טוב פותח את המתמטיקה לקהל שלם שלא היה מגיע אליה אחרת.

אבל לכל הדברים האלה יש תנאי אחד, והפרק חוזר אליו שוב ושוב: צריך להחזיק בהבחנה ברורה בין שני מישורים. הפרק נשען על `docs/wisdom-bridges.md` ועל 24 הפרויקטים ב-`data/wisdom_projects.csv` (W001 עד W024).

ההבחנה היסודית: שני מישורים

"קַבֵּל הָאֱמֶת מִמִּי שֶׁאָמָרוֹ", כתב הרמב"ם בהקדמה לשמונה פרקים. המשפט הזה מנחה את הפרק כולו, וגם מגדיר את הגבול שלו. האמת המתמטית מתקבלת לפי ההוכחה, לא לפי מי שאמר אותה. ומאותה סיבה, ההוכחה המתמטית לא יכולה להכריע בשאלות שאינן מתמטיות.

	מתמטיקה ופיזיקה	מסורת וחוכמה
סוג הטענה	טענה שאפשר לאמת: הוכחה, חישוב, ניסוי	מקור למטאפורה, פדגוגיה, מבנה, השראה ושיטת חשיבה
מי מכריע	הוכחה (אצלנו: Lean), נתונים	המסורת עצמה, לפי כלליה ופוסקיה
מה לא עושים	לא "מוכיחים" אמונה ממשפט	לא "מוכיחים" משפט מפסוק או מגימטריה

מכאן שלושה כללים:

1. **הקבלה אינה מוכיחה פיזיקה, והפיזיקה אינה מוכיחה קבלה.** כשנמצא מבנה מתמטי משותף, למשל שגם בספר יצירה וגם בקומבינטוריקה מופיעים מקדמים בינומיים, נאמר בדיוק את זה ולא יותר.
2. **כשחוקרים טקסט מסורתי מתמטית, חוקרים את המבנה הפורמלי שלו:** כמה צירופים, איזה דקדוק, איזו סימטריה. לא חוקרים את תוקפו המטאפיזי, ולא טוענים עליו דבר, לא לכאן ולא לכאן.
3. **ענווה כפולה.** המעבדה לא פוסקת בענייני קבלה, ולא נותנת להשראה להחליף הוכחה.

בארכיטקטורה של המעבדה ההבחנה הזו ממומשת בקוד. ה-analogy-agent, הסוכן ששואב רעיונות ממסורות, מתחומים אחרים ומפיזיקה, מוגבל לרמת טענה L0 תמיד (`docs/architecture.md`, סעיף 7). כל רעיון שלו חייב להיות מתורגם לטענה מתמטית מדויקת ולעבור לסוכן אחר לבדיקה. רעיון שאי אפשר לתרגם נזרק.

חלק א: דרך הגמרא כשיטת חשיבה

קושיא ותירוץ: הגמרא ו-Lakatos

ב-1976 יצא לאור ספרו של הפילוסוף Imre Lakatos, *Proofs and Refutations*[112]. הספר בנוי כדיאלוג בכיתה דמיונית, שבה מורה ותלמידים מנסים להוכיח את משפט אוילר על פאונים: $V - E + F = 2$ (קודקודים פחות צלעות ועוד פאות שווה 2). המורה מציג הוכחה. תלמיד מביא דוגמה נגדית: פאון עם חור באמצעו, כמו מסגרת תמונה, שעבורו הנוסחה נותנת 0. מישהו מציע לשנות את ההגדרה של "פאון" כך שהמסגרת לא תיחשב. מישהו אחר מציע להכניס את התנאי החסר לתוך המשפט עצמו. וכך הלאה.

מי שלמד דף גמרא מזהה את התנועה הזו מיד:

מונח בגמרא	המקבילה אצל Lakatos ובמתמטיקה
מימרא	טענה, השערה
קושיא, "והא תניא..."	דוגמה נגדית, סתירה למקור מוסמך
תירוץ	חידוד ההוכחה או ההגדרה
אוקימתא ("הכא במאי עסקינן")	צמצום תחום המשפט כדי לחסום את הדוגמה הנגדית (monster-barring, lemma incorporation)
"לא נצרכא אלא..."	הסבר מדוע ההנחה הנוספת הכרחית (מינימליות של הנחות)
תיובתא	הפרכה סופית
קשיא (בלי תיובתא)	קושי פתוח שלא מפיל את הטענה

מונח בגמרא	המקבילה אצל **Lakatos** ובמתמטיקה
תיקו	השאלה נשארת פתוחה
מסקנא	המשפט בניסוחו הסופי

הערת דיוק על "תיקו": תיקו הוא הכרעה שאין בידינו הכרעה. זה מקביל ל"בעיה פתוחה", ולא בהכרח ל"אי-תלות" במובן של גדל וכהן, שבו **הוכח** שאין הוכחה. ההבדל עצמו הוא שיעור יפה. יש תיקו של בורות, ויש תיקו שהוא משפט. השערת הרצף היא מהסוג השני.

שלוש-עשרה מידות ודפוסי היסק

הברייתא של רבי ישמעאל, בפתיחת הספרא, מונה שלוש-עשרה מידות שהתורה נדרשת בהן. המידות הן כלי פרשנות הלכתיים, ויש להן מערכת סמכות וכללים משלהן. ההקבלות שבטבלה הן אנלוגיות מבניות בלבד, אבל הן מאירות:

מידה	מבנה לוגי	מקבילה מתמטית
קל וחומר	אם $A \le B$ ו-$P(A)$, אז $P(B)$	טיעון מונוטוניות, מבחן השוואה
דיו לבא מן הדין להיות כנדון	המסקנה לא חזקה מההנחה	מונוטוניות נותנת חסם ולא יותר
גזירה שווה	מילה משותפת, דין משותף	איזומורפיזם, העברת משפט בין מבנים
"אין אדם דן גזירה שווה מעצמו"	גזירה שווה צריכה מסורת	אנלוגיה שטחית אינה איזומורפיזם
בנין אב מכתוב אחד	הכללה ממקרה יסוד	אינדוקציה, הכללה ממקרה בסיס
בנין אב משני כתובים	הכללה מהצד השווה	הפשטה: מוצאים מבנה משותף לשני מקרים
כלל ופרט וכלל	"אי אתה דן אלא כעין הפרט"	המחלקה הקטנה ביותר הסגורה תחת התכונות של הדוגמאות
שני כתובים המכחישים זה את זה	יישוב סתירה בכתוב שלישי	יישוב סתירה באקסיומות בעזרת פיצול מקרים

שתי שורות בטבלה ראויות להרחבה.

קל וחומר ודיו. הנה טיעון מתמטי בצורת קל וחומר: אם הטור $\sum 1/n^2$ מתכנס, ואם $0 \le a_n \le 1/n^2$ לכל n, אז $\sum a_n$ מתכנס. מה שנכון ל"חמור" נכון גם ל"קל". אבל עקרון "דיו" מוסיף הגבלה: מהטיעון הזה אפשר להסיק ש-$\sum a_n \le \pi^2/6$, ולא יותר. אי אפשר להסיק, למשל, ש-$\sum a_n$ שווה למספר מסוים.

בפרויקט W016 ננסה לנסח את המבנה הזה בצורה פורמלית: קל וחומר כטיעון מונוטוניות על סדר חלקי, דיו כחסם, ו"פירכא" כמקרה שבו המונוטוניות נשברת. זה מודל מבני בלבד, ולא טענה על ההלכה.

"אין אדם דן גזירה שווה מעצמו". זו אולי האזהרה המתמטית הטובה ביותר בטבלה. בגזירה שווה, מילה שמופיעה בשני מקומות מאפשרת להעביר דין ממקום למקום. אבל ההלכה לא מתירה לכל אחד להמציא גזירה שווה. היא חייבת לבוא ממסורת. במתמטיקה זה בדיוק ההבדל בין דמיון לבין איזומורפיזם. שני מבנים יכולים להיראות דומים מאוד, ובכל זאת משפט שנכון באחד לא יעבור לשני, כי ההעתקה ביניהם לא משמרת את המבנה הרלוונטי. וזו בדיוק הטעות שמודלי שפה עושים לעתים קרובות: הם מזהים "דמיון" בין שתי בעיות ומעבירים פתרון שלא עובר. לכן במעבדה כל אנלוגיה של ה-analogy-agent צריכה הוכחה שהיא משמרת מבנה.

עוגן היסטורי. הקשר בין החוכמה היהודית לבין שיטות הוכחה אינו רק מטאפורה. רבי לוי בן גרשום (רלב"ג) השתמש בספרו *מעשה חושב* (1321) בשיטה של התקדמות צעד אחר צעד, שחוקרי היסטוריה של המתמטיקה רואים בה מן ההופעות המוקדמות של הוכחה באינדוקציה מתמטית, והוכיח שם נוסחאות לתמורות ולצירופים[113]. ומושגי ההלכה "רוב", "קבוע כמחצה על מחצה" ו"ספק ספיקא" נחקרו כהיגיון הסתברותי בספרו של N. L. Rabinovitch מ-1973[114] (W018).

החברותא: שני סוכנים ובורר

"או חברותא או מיתותא" (תענית כג ע"א). הלימוד בזוגות, שבו אחד מקשה והשני מתרץ, הוא מה שמחקר AI מכנה ויכוח בין מציע למבקר (proposer–critic, adversarial debate). ההבדל המרכזי בין החברותא המסורתית לבין הגרסה שלנו הוא הפוסק. בבית המדרש, הכרעה מגיעה מסמכות. אצלנו, הבורר האחרון הוא מכונה דטרמיניסטית: ה-kernel של Lean.

הפרוטוקול (W004) נראה כך:

```
תפקידים:
  מתרץ (Proposer)  – מציע טענה, שרטוט הוכחה, קוד Lean
  מקשן (Critic)    – מחפש דוגמה נגדית, פער בהוכחה, הנחה סמויה
  פוסק (Judge)     – לא מודל שפה, ה-kernel של Lean 4
  סופר (Scribe)    – מתעד את הסוגיא בפורמט מובנה

סבב:
  1. המתרץ מנסח טענה ב-Lean (הניסוח בלבד)  מימרא:
  2. המקשן מחפש דוגמה נגדית (חיפוש, SAT, Plausible)  קושיא:
  3. נמצאה ונבדקה → תיובתא; הטענה נמחקת או מצטמצמת (אוקימתא)
  4. לא נמצאה: תירוץ → המתרץ מנסה הוכחה; המקשן מצביע על צעדים חלשים
```

5. Lean מסקנא → מקבל

6. N סבבים בלי הכרעה → תיקו; הבעיה נרשמת כפתוחה

הפלט הוא קובץ `sugya.json`, שבו כל מהלך בדיון מתועד. הנה סוגיא קצרה לדוגמה, כפי שהיא עשויה להיראות:

```
{"goal": "demo-01",
 "steps": [
  {"type": "מימרא", "by": "proposer",
   "claim": "לכל n טבעי, n^2 + n + 41 ראשוני"},
  {"type": "קושיא", "by": "critic", "method": "search",
   "counterexample": "n = 40: 40^2 + 40 + 41 = 41^2"},
  {"type": "תיובתא", "by": "judge", "check": "decide", "result": "verified"},
  {"type": "אוקימתא", "by": "proposer",
   "claim": "לכל n < 40, n^2 + n + 41 ראשוני"},
  {"type": "מסקנא", "by": "judge", "check": "decide", "result": "verified"}
 ]}
```

(הפולינום של אוילר, $n^2 + n + 41$, נותן ראשוניים לכל $0 \le n \le 39$ ונכשל ב-$n = 40$. זו דוגמה קלאסית לטענה שנראית נכונה על סמך הרבה מקרים.)

למה זה טוב? המקשן מונע הזיות של המתרץ. תיעוד הסוגיא שומר את הדרך ולא רק את התוצאה, וזה ערך פדגוגי ומדעי. והפוסק הוא Lean, כך שאיש לא צריך לסמוך על הסמכות של מודל. פרויקט W017 מציע צעד נוסף: לתייג את מבנה הסוגיות במדגם מהתלמוד הבבלי (מימרא, קושיא, תירוץ, אוקימתא, תיקו), ולהשתמש בו כנתונים לאימון של סוכן מקשן.

חלק ב: ספר יצירה, קומבינטוריקה עתיקה

ספר יצירה הוא אחד הטקסטים העתיקים ביותר שעוסקים במפורש בספירה קומבינטורית. זמן חיבורו שנוי במחלוקת בין החוקרים, ויש לו כמה נוסחים עיקריים (הנוסח הקצר, הנוסח הארוך, נוסח רס"ג ונוסח הגר"א), שהמספור שלהם שונה. לכן כל הפניה למשנה בספר צריכה לציין את הנוסח.

22 אותיות, 231 שערים

הספר מחלק את 22 האותיות לשלוש קבוצות: שלוש "אמות" (א, מ, ש), שבע "כפולות" (ב, ג, ד, כ, פ, ר, ת) ושתים-עשרה "פשוטות". כלומר $12 + 7 + 3 = 22$.

הוא מדבר גם על 231 "שערים". זה בדיוק מספר הזוגות הלא-סדורים של 22 אותיות:

$$\binom{22}{2} = \frac{22 \cdot 21}{2} = 231.$$

אם סופרים זוגות סדורים, כלומר מבחינים בין א"ב לבין ב"א ("פנים ואחור"), מקבלים $22 \cdot 21 = 462$. ובחלק מכתבי היד מופיע המספר 221 ולא 231, וזו שאלה מעניינת בפני עצמה, בתפר שבין ביקורת טקסט לקומבינטוריקה[115].

"שתי אבנים בונות שני בתים"

בפרק ד' (משנה יב לפי המספור של Aryeh Kaplan, העוקב אחר נוסח הגר"א[116]) מופיע רצף מפורסם. בתרגום חופשי לעברית בת זמננו: *שתי אבנים בונות שני בתים, שלוש אבנים בונות שישה בתים, ארבע אבנים בונות עשרים וארבעה בתים, חמש אבנים בונות מאה ועשרים בתים, שש אבנים בונות שבע מאות ועשרים בתים, שבע אבנים בונות חמשת אלפים וארבעים בתים.* ומכאן ואילך "צא וחשוב מה שאין הפה יכול לדבר ואין האוזן יכולה לשמוע". [לא אומת מילולית: נוסח הציטוט ומספור המשנה לא נבדקו מול טקסט עברי מקורי, כי Sefaria ו-Wikisource לא היו נגישים בזמן הכתיבה. המספור 4:12 אושר רק דרך תיאורים משניים של התרגום של Kaplan. יש לבדוק לפני פרסום, ולציין את הנוסח.]

המפרשים קוראים את ה"אבנים" כאותיות ואת ה"בתים" כמילים, כלומר כסידורים של האותיות. הסדרה $2,\ 6,\ 24,\ 120,\ 720,\ 5040$ היא בדיוק הפונקציה $n!$, מספר הדרכים לסדר n עצמים שונים בשורה. זה ניסוח מוקדם מאוד של צמיחה עצרתית. וגם התחושה ש"הפה אינו יכול לדבר" מוצדקת: $22! \approx 1.1 \times 10^{21}$.

מה אפשר לחקור כאן

בלי שום מטאפיזיקה, יש כאן שלוש שאלות מתמטיות אמיתיות (W002):

1. **פירמול הספירות.** זה פרויקט פתיחה פדגוגי כמעט מושלם:

```
import Mathlib

example : Nat.choose 22 2 = 231 := by decide
example : 22 * 21 = 462 := by norm_num
example : (List.range 7).map (fun n ⇒ Nat.factorial (n + 1))
    = [1, 2, 6, 24, 120, 720, 5040] := by decide
```

[לא אומת: הקוד לא קומפל במאגר; `decide` עשוי לדרוש `native_decide` או `rfl` בגרסאות מסוימות, ו-`native_decide` דורש סימון מפורש לפי כללי המעבדה.]

2. **מעגל השערים כמבנה קומבינטורי.** במסורת יש סידורים של 231 השערים במעגל ("גלגל"). אפשר לשאול אם הסידור מממש מבנה מוכר. הגרף השלם K_{22} (22 קודקודים, קשת בין כל שניים, כלומר 231 קשתות) ניתן לפירוק ל-21 זיווגים מושלמים, כל אחד של 11 קשתות. פירוק כזה נקרא 1-factorization. האם הסידור המסורתי מממש פירוק כזה? זו שאלה מדויקת שאפשר לבדוק.
3. **השוואת נוסחים.** 221 מול 231 כבעיה של ביקורת טקסט ושל ספירה: איזה סידור יכול להוליד 221?

חלק ג: הרמב"ם, הלכות קידוש החודש כאלגוריתם

שני חשבונות

בספר זמנים של משנה תורה כתב הרמב"ם את הלכות קידוש החודש. החיבור מחולק לשני חלקים שונים מאוד באופיים[117]:

- **פרקים ו עד י:** חשבון הלוח הקבוע, "חשבון העיבור". זה אלגוריתם מלא ודטרמיניסטי, בארית‍מטיקה של מספרים שלמים, שמאפשר לחשב את הלוח לכל שנה.
- **פרקים יא עד יט:** חשבון ראיית הלבנה. זו אסטרונומיה תצפיתית: מהלך השמש והירח, הרוחב של הירח, קשת הראייה. הוא מבוסס על האסטרונומיה של תלמי ושל אסטרונומים מוסלמים.

[לא אומת: חלוקת הנושאים המדויקת בין הפרקים ו–י (מולד, דחיות, סדר החודשים, תקופות) נלקחה מ-`docs/wisdom-bridges.md` ולא נבדקה מול הטקסט.]

ובפרק יז, הלכה כד, מופיע עיקרון שהמעבדה כולה יכולה לאמץ. הרמב"ם מסביר שהחשבונות מבוססים על ספרים שחיברו חכמי יוון, ושאין לחשוש מי חיבר אותם: מאחר שהדברים הוכחו בראיות ברורות שאין בהן דופי, אנו סומכים על מי שאמרם או לימדם, בגלל הראיות ולא בגלל מי שאמר אותן[118]. [המובן אומת דרך מקורות משניים; הנוסח המדויק לא צוטט כאן.]

האלגוריתם

הנה החלק הדטרמיניסטי, בכתיב של פסאודו-קוד. כל היחידות הן "חלקים": שעה היא 1,080 חלקים, וחלק הוא $3\frac{1}{3}$ שניות. המספר 1,080 נבחר כנראה משום שהוא מתחלק בכל המספרים מ-1 עד 10 חוץ מ-7 ($1080 = 2^3 \cdot 3^3 \cdot 5$).

```
קבועים:
  חודש = 29 יום, 12 שעות, 793 חלקים = 765,433 חלקים
  שנים מעוברות במחזור של 19: 3, 6, 8, 11, 14, 17, 19  (גו"ח אדז"ט)

Y: מולד תשרי של שנה
```

```
1. Y ספור חודשים מהמולד הראשון עד תשרי של
   (לשנה פשוטה, 13 למעוברת 12)
2. חלקים (765,433 × (מספר חודשים + מולד בהר"ד = מולד
3. המר ליום בשבוע, שעה וחלקים

:ראש השנה = יום המולד, אלא אם (ארבע הדחיות)
  א. לא אד"ו ראש: היום הוא א', ד' או ו'      → דחה ביום
  ב. מולד זקן: המולד בשעה 18 ומעלה          → דחה ביום (ואם הגענו לאד"ו, ביום נוסף)
  ג'. גטר"ד: שנה פשוטה, מולד ביום ג' בשעה 9 ו-204 חלקים ומעלה → דחה ליום ה
  ד'. בטו"תקפט: אחרי שנה מעוברת, מולד ביום ב' בשעה 15 ו-589 חלקים ומעלה → דחה ליום ג

אורך השנה = ראש השנה הבא פחות ראש השנה הנוכחי
```

הדבר המעניין מבחינה מתמטית הוא הדחיות ג ו-ד. לכאורה הן כללים שרירותיים. בפועל הן קיימות כדי להבטיח משפט: **אורך השנה תמיד אחד משישה ערכים**, 353, 354 או 355 ימים בשנה פשוטה, ו-383, 384 או 385 ימים בשנה מעוברת. בלעדיהן, הדחיות א ו-ב היו יכולות ליצור שנים באורך 356 או 382 ימים.

חלק ד: הלוח העברי כתורת מספרים

למה דווקא 235 חודשים ב-19 שנים?

הבעיה הבסיסית של כל לוח ירחי-שמשי היא שהשנה והחודש אינם מתחלקים זה בזה. שנה שמשית היא כ-365.2422 ימים. חודש סינודי (מולד עד מולד) הוא כ-29.53059 ימים. היחס ביניהם הוא:

$$\frac{365.2422}{29.53059} \approx 12.36827.$$

כלומר בשנה יש קצת יותר מ-12 חודשים. כדי ליצור לוח, צריך קירוב רציונלי טוב למספר הזה: p/q שאומר "ב-q שנים יש p חודשים". הכלי המתמטי למציאת קירובים כאלה הוא שבר משולב (continued fraction):

$$12.36827 = 12 + \cfrac{1}{2 + \cfrac{1}{1 + \cfrac{1}{2 + \cfrac{1}{1 + \cfrac{1}{2 + \cfrac{1}{1 + \cdots}}}}}}$$

הקירובים העוקבים (convergents) הם:

$$12,\ \frac{25}{2},\ \frac{37}{3},\ \frac{99}{8},\ \frac{136}{11},\ \frac{235}{19},\ \frac{4131}{334},\ \dots$$

והנה המחזור של מטון: 235 חודשים ב-19 שנים. הוא קירוב שבר משולב ליחס שנה/חודש, ולכן הוא הקירוב הטוב ביותר עם מכנה עד 19. הקירוב הבא כבר דורש 334 שנים, שאינו מעשי ללוח. (החישוב נעשה לצורך הפרק, בערכים האסטרונומיים המקורבים שלמעלה.)

המולד המדויק, והסחיפה

החודש הממוצע בלוח העברי, 765,433 חלקים, הוא 29.530594 ימים. החודש הסינודי האסטרונומי הממוצע הוא כ-29.530589 ימים. ההבדל הוא פחות משנייה לחודש. אבל השנה של הלוח, 235/19 חודשים כפול אורך החודש, יוצאת כ-365.2468 ימים, ארוכה בכ-0.0046 יום מהשנה השמשית. זה מצטבר ליום אחד בכ-216 שנים. הלוח העברי סוחף לאט ביחס לעונות, וזו סיבה טובה לפרויקט W024, שמשווה את שגיאת הסחיפה של לוחות שונים.

המחזוריות המלאה

כל אחד מהרכיבים מחזורי: 19 שנים למחזור הקטן, 7 ימים לשבוע, 25,920 חלקים ליום. הלוח כולו, כולל ימי השבוע של כל החגים, חוזר על עצמו במלואו רק כל **689,472 שנים**. זה משפט שאפשר להוכיח, והוכחה שלו היא תרגיל יפה בחשבון מודולרי.

פרויקט הדגל: W001

`data/wisdom_projects.csv` מדרג את W001 ב-5 מתוך 5 מבחינת ישימות. המטרה: להגדיר ב-Lean 4 את הלוח לפי לשון הרמב"ם, ולהוכיח ארבע טענות:

1. אורך השנה שייך ל-$\{353, 354, 355, 383, 384, 385\}$.
2. ראש השנה לעולם אינו חל בימים א׳, ד׳ או ו׳.
3. הלוח מחזורי עם מחזור של 689,472 שנים.
4. המימוש מתאים למימוש עצמאי, למשל של Reingold ו-Dershowitz ב-*Calendrical Calculations*[119], על טווח שנים גדול.

הניסוח של הטענה הראשונה עשוי להיראות כך (שלד בלבד, לא קוד שנבדק):

```
/- אורך שנה עברית בימים, לפי האלגוריתם של הלכות קידוש החודש. -/
def yearLength (y : ℕ) : ℕ := sorry

theorem yearLength_mem (y : ℕ) (hy : 1 ≤ y) :
    yearLength y ∈ ({353, 354, 355, 383, 384, 385} : Finset ℕ) := by
  sorry
```

זהו פרויקט עם ודאות מלאה, ערך תרבותי גבוה, גודל מתאים (שבועות, לא שנים), ו"משפט" שכל בית כנסת מבין. ההוכחה עצמה היא סופית במהותה: מאחר שהלוח מחזורי, אפשר לבדוק את כל המקרים במחזור אחד. זה דפוס 2 מפרק 9, כוח גס שהופך לאמין.

חלק ה: צירוף, מילוי והרש"ש

מילוי וגימטריה כמערכות שכתוב

מילוי הוא כתיבת כל אות בשמה המלא: י נכתבת יוד, ה נכתבת הי (או הא, או הה), ו נכתבת ויו (או ואו, או וו). כך מתקבלים ארבעת המילויים המפורסמים של שם ההוי"ה:

מילוי	כתיב	גימטריה
ע"ב	יוד הי ויו הי	72 = 15 + 22 + 15 + 20
ס"ג	יוד הי ואו הי	63 = 15 + 13 + 15 + 20
מ"ה	יוד הא ואו הא	45 = 6 + 13 + 6 + 20
ב"ן	יוד הה וו הה	52 = 10 + 12 + 10 + 20

במונחים מתמטיים:

- **גימטריה** היא הומומורפיזם מהמונואיד של מילים (עם שרשור) למונואיד $(\mathbb{N}, +)$: הערך של שרשור שתי מילים הוא סכום הערכים. היא לא חד-חד-ערכית, כלומר הרבה מילים מקבלות אותו ערך, ולכך נחזור בסעיף הגבולות.
- **מילוי** הוא הומומורפיזם של מילים (string morphism): כל אות מוחלפת במילה. "מילוי דמילוי" הוא הפעלה חוזרת של אותו הומומורפיזם, בדיוק כמו ב-L-systems של Lindenmayer וב-morphic words בתורת הצירופים על מילים. ויש כאן שאלות אמיתיות: באיזה קצב גדל האורך? (התשובה נקבעת על ידי הערך העצמי הגדול ביותר של מטריצת ההומומורפיזם.) מה התפלגות האותיות בגבול? (W003)
- **צירופי השם.** למילה בת ארבע אותיות שבה אות אחת חוזרת פעמיים יש $4!/2! = 12$ סידורים שונים. במסורת הקבלית הם מקושרים לשנים-עשר החודשים. מתמטית זו ספירת מסלולים (orbit counting) של פעולת החבורה S_4 על מילים, ואפשר להכליל אותה בעזרת הלמה של Burnside ומשפט Pólya לצירופים עם ניקוד ומילוי (W023).

הכוונות של הרש"ש כמבנה פורמלי

רבי שלום שרעבי (הרש"ש, 1720–1777), ראש ישיבת המקובלים בית אל בירושלים, חיבר את *נהר שלום* ואת הסידור הידוע על שמו, סידור הכוונות. הכוונות שבו הן אולי המערכת הסמלית-קומבינטורית המפורטת ביותר במסורת: שמות, מילויים, ניקודים (כל שם בכל אחד מהניקודים), שילובים של שמות, שבהם אותיות של שם אחד משולבות בין אותיות של שם אחר, ספירות ופרצופים. כל אלה מסודרים בטבלאות היררכיות לפי תפילה, יום ושעה.

המייסד צודק שיש כאן "הרבה נוסחאות". המבנה הפורמלי הוא אובייקט מחקר לגיטימי ומרתק, בתנאי שאומרים במפורש מה חוקרים:

שאלה מתמטית	כלי
האם הכוונות נוצרות על ידי דקדוק פורמלי קטן (כללי שילוב, מילוי, ניקוד)? מה המורכבות שלו?	דקדוקים פורמליים, הסקת דקדוק
מה פעולת השילוב בין שמות? אילו תמורות מופיעות?	תבניות של תמורות, מכפלת shuffle
האם הטבלאות הן block design, כלומר כל זוג איברים מופיע מספר קבוע של פעמים?	תכנונים קומבינטוריים, ריבועים לטיניים
מה מבנה ההיררכיה (ספירות, פרצופים, עולמות): עץ, גרף מכוון חסר מעגלים, סריג?	תורת הסדר, תורת הגרפים
כמה מידע יש בטקסט ביחס למספר הכללים שמייצרים אותו?	דחיסה, אורך תיאור מינימלי (MDL)

ומה לא נעשה. לא נטען שהמבנה "מוכיח" את תוקף הכוונות, ולא שהוא "צופן" של פיזיקה. ולהפך, לא נטען שמציאת דקדוק פשוט "מסבירה" או "מבטלת" את משמעותן. אלה שאלות שהמתמטיקה לא עונה עליהן. מבחינת התוכן, העבודה תיעשה בשיתוף עם תלמידי חכמים מבית המדרש של הרש"ש, בכבוד למסורת שרואה בכוונות עבודה פנימית. הדיגיטציה תיעשה רק ממהדורות מוסמכות ובאישור בעלי הזכויות (W005, ישימות 3 מתוך 5, בדיוק בגלל התנאים האלה).

הרמח"ל: קל"ח פתחי חכמה כגרף

רבי משה חיים לוצאטו (1707–1746) כתב את *קל"ח פתחי חכמה* כ-138 "פתחים": טענות ממוספרות, שכל אחת נשענת על קודמותיה. זה מבנה שמזכיר את היסודות של אוקלידס ואת האתיקה של שפינוזה, שנכתבה "בסדר גאומטרי". פרויקט W020 מציע למפות את גרף ההפניות בין הפתחים ולנתח אותו: עומק, מרכזיות, האם יש "אקסיומות" שכל השאר נשען עליהן. זהו ניתוח של מבנה הטיעון, לא אימות של התוכן.

חלק ו: מסורות עתיקות אחרות

המסורת היהודית אינה היחידה שיש בה מתמטיקה עמוקה. הטבלה מסכמת את הגשרים שבמסמך, וכל אחד מקושר לפרויקט ב- `data/wisdom_projects.csv`:

תרבות	מה	הגשר המתמטי	פרויקט
בבל	Plimpton 322 (כ-1800 לפנה"ס), טבלה בבסיס 60	שלשות פיתגוריות, מספרים smooth-5	W008
מצרים	שברי מצרים, טבלת $2/n$ בפפירוס רינד	השערת Erdős–Straus: $4/n = 1/x + 1/y + 1/z$, פתוחה	W006
הודו (פינגלה)	ספירת משקלים בשירה	ייצוג בינארי, משולש פסקל, פיבונאצ'י לפני פיבונאצ'י	W009
הודו (קרלה)	Madhava (כ-1340–1425), טורים ל-π	טורים אינסופיים, תיקון שארית	W010
סין	Sunzi Suanjing: שארית 2, 3, 2 בחלוקה ב-3, 5, 7, והתשובה 23	משפט השאריות הסיני	W011
יוון	ארכימדס: ריבוע הפרבולה, ה-Stomachion	שיטת המיצוי; 17,152 סידורים	W012, W013
האסלאם	אל-ח'וארזמי (כ-820)	שש הצורות הקנוניות של משוואות ריבועיות	W014
יפן	Sangaku, לוחות גאומטריה במקדשים	גאומטריה אוקלידית, מוכיחים אוטומטיים	W015
יורובה וסין	Ifá, 256 ה-odù של 64 ההקסגרמות של I Ching	קומבינטוריקה בינארית, $\mathbb{F}_2^8$ ו-$\mathbb{F}_2^6$	W021
רב-תרבותי	לוחות עברי, מאיה, סיני ומוסלמי	שברים משולבים, מחזורים משולבים	W024

שתי הערות על הטבלה. הראשונה: Erdős–Straus היא השערה **פתוחה**, שנבדקה חישובית עד כ-10^{17}. כאן יש מקום לתרומה מתמטית אמיתית שמקורה בטקסט עתיק. השנייה: הפרשנות של Plimpton 322 שנויה במחלוקת. Mansfield ו-Wildberger הציעו ב-2017 פרשנות טריגונומטרית, ו-Eleanor Robson ואחרים חולקים עליה. המעבדה תציג את שני הצדדים.

יש כלל כללי: נמנעים מ"רומנטיקה של העתיק". מתמטיקאים בבבל, בהודו ובסין היו אנשי מקצוע, לא מיסטיקנים, ומציגים את הישגיהם כמתמטיקה. ונמנעים גם מהקצה השני, טענות ש"כולם ידעו הכול". בשאלות של קדימות היסטורית מדייקים ומביאים מקורות אקדמיים.

חלק ז: מנוע המשלים

התבנית

"מנוע המשלים" (W007) הוא תבנית לתרגום של כל משפט לכמה שפות, שמיועדת להיות מופעלת על ידי מודל שפה, עם בדיקה שהמשפט מקושר לקובץ Lean:

```
[0] המשפט Lean – ניסוח מדויק + קישור לקובץ
[1] משל – סיפור קצר מחיי היום-יום, 3–6 משפטים, עם אותו מבנה לוגי
[2] סוגיא – מימרא · קושיא · תירוץ · (קושיא · תירוץ) · מסקנא
[3] "מסגור עתיק – איך זה היה מוצג בתרבות עתיקה; מקור היסטורי מצוין, המצאה מסומנת "בדיוני
[4] נמשל – מיפוי מפורש: כל רכיב במשל מול רכיב בהוכחה
[5] היכן המשל נשבר – חובה. כל משל מטעה במשהו
```

סעיף 5 הוא החידוש של התבנית. משל בלי "גבולות המשל" הוא תעמולה ולא הוראה.

ב-`docs/wisdom-bridges.md` יש שלוש דוגמאות מלאות: אינסוף הראשוניים ("המלך והשומרים"), האלכסון של קנטור ("ספר כל הספרים") ועקרון שובך היונים. כאן נוסיף שתי דוגמאות חדשות, ונסכם בקצרה אחת מהקיימות.

דוגמה מלאה 1: למת לחיצות הידיים

[0] המשפט. בכל גרף סופי, סכום הדרגות של כל הקודקודים שווה לפעמיים מספר הקשתות. ולכן מספר הקודקודים בעלי דרגה אי-זוגית הוא זוגי. (ב-Mathlib: `SimpleGraph.sum_degrees_eq_twice_card_edges`. [לא אומת: שם הלמה בגרסה הנוכחית של Mathlib.])

[1] המשל: החתונה. בחתונה גדולה, בסוף הערב, כל אורח רשם על פתק כמה ידיים לחץ. הגבאי אסף את הפתקים וחיבר. "המספר זוגי", אמר בלי להסתכל. "ואיך אתה יודע?" שאלו אותו. "כי כל לחיצת יד

נרשמה פעמיים, פעם אצל כל אחד מהלוחצים." והוסיף: "ולכן גם מספר האורחים שרשמו מספר אי-זוגי הוא זוגי. אם תמצאו לי אחד כזה, יש עוד אחד."

[2] הסוגיא. - **מימרא:** אמר אוילר: סכום הדרגות כפליים במספר הקשתות. - **קושיא:** והרי יש אדם שלחץ את ידו של עצמו, ונרשם פעם אחת בלבד? - **תירוץ:** לא נצרכא אלא לגרף פשוט, שאין בו לולאות. ואם יש לולאה, סופרים אותה פעמיים בדרגה, והמשפט עומד. - **קושיא:** ודילמא שניים לחצו ידיים פעמיים, ונמצאת קשת אחת שנספרה ארבע פעמים? - **תירוץ:** הא בגרף פשוט, הא בגרף מרובה. בגרף מרובה סופרים כל לחיצה כקשת בפני עצמה, ועדיין כל קשת תורמת 2 בדיוק. - **מסקנא:** בכל גרף סופי, סכום הדרגות זוגי, ומספר הקודקודים מדרגה אי-זוגית זוגי.

[3] מסגור עתיק. זהו בדיוק הטיעון שאוילר השתמש בו ב-1736 בבעיית הגשרים של קניגסברג (פרק 9, מקור היסטורי). שם השאלה הייתה הפוכה: כשיש יותר משני חלקי יבשה עם מספר אי-זוגי של גשרים, אין טיול שעובר על כל גשר פעם אחת.

[4] הנמשל. אורחים = קודקודים; לחיצת יד = קשת; המספר בפתק = דרגה; הגבאי שמחבר = הסכום; "נרשמה פעמיים" = כל קשת תורמת 1 לדרגה של כל אחד משני קצותיה.

[5] היכן המשל נשבר. בחתונה אנשים טועים ברישום, והמשפט מניח רישום מושלם. וחשוב יותר: הגבאי יודע שהסכום זוגי, אבל לא יודע מה הוא. המשפט נותן אינווריאנט (זוגיות), לא ערך. ובגרף אינסופי, שבו יש אינסוף קשתות, "הסכום" אינו מוגדר כלל, והמשל לא מלמד דבר.

דוגמה מלאה 2: סכום המספרים האי-זוגיים

[0] המשפט. לכל n טבעי, $1 + 3 + 5 + \cdots + (2n-1) = n^2$. (הוכחה באינדוקציה; ב-Lean אפשר להוכיח בעזרת `Finset.sum_range_succ` ו-`ring`.)

[1] המשל: הגנן. גנן שתל ערוגה מרובעת. בשנה הראשונה שתל שתיל אחד. בכל שנה הוסיף "חלוק" בצורת האות ר' סביב הפינה של הערוגה, כדי שתישאר מרובעת. בשנה השנייה הוסיף 3 שתילים, בשלישית 5, בשביעית 13. השכן שאל כמה שתילים יש לו בסך הכול אחרי שבע שנים. הגנן לא חיבר. הוא מדד את צלע הערוגה, שבעה שתילים, ואמר: "ארבעים ותשעה".

[2] הסוגיא. - **מימרא:** סכום n האי-זוגיים הראשונים הוא n^2. - **קושיא:** ממה נפשך, ערוגה של 2 על 2 היא 4, ו-$4 = 3 + 1$. ושל 3 על 3 היא 9, ו-$9 = 5 + 3 + 1$. אבל מי אמר שכך לעולם? - **תירוץ:** בנין אב. הנח שכך לריבוע של n. כדי לעבור לריבוע של $n+1$, מוסיפים שורה של n, עמודה של n ופינה אחת: $2n + 1$. וזה בדיוק האי-זוגי הבא. - **מסקנא:** הטענה נכונה ל-1, ועוברת מכל n ל-$n+1$, ולכן נכונה לכל n.

[3] מסגור עתיק. "הוכחות ללא מילים" מהסוג הזה נאספות כבר עשרות שנים (Nelsen, *Proofs Without Words*, פרק 9, P060). הצורה ר', ה-gnomon, מופיעה כבר אצל הפיתגוראים, ששמו

חלוקים במבנים כאלה [לא אומת: הייחוס לפיתגוראים מבוסס על מסורת היסטוריוגרפית ולא נבדק במקור ראשוני]. השם "בנין אב" לשלב האינדוקציה הוא המסגור שלנו.

[4] הנמשל. שנה = n; חלוק בצורת ר' = האי-זוגי ה-n, $2n-1$; ערוגה מרובעת = n^2; מדידת הצלע במקום ספירה = השימוש בנוסחה הסגורה.

[5] היכן המשל נשבר. התמונה משכנעת, אבל תמונה אינה הוכחה לכל n. היא מראה מקרה אחד ומזמינה להכליל, וההכללה היא שלב האינדוקציה. ועוד: המשל מסתמך על כך שהשתילים יושבים ברשת ריבועית מושלמת. במספרים זה נכון תמיד. בגינה אמיתית, לא.

דוגמה מקוצרת: המלך והשומרים

הדוגמה הראשונה ב- `docs/wisdom-bridges.md` מספרת את ההוכחה של אוקלידס לאינסוף הראשוניים (*יסודות*, ספר IX, טענה 20) כמשל על מלך שמתגאה ברשימת כל השומרים בממלכה, ועל חכם שמכפיל את מספרי כולם ומוסיף אחד. הסוגיא שם מגיעה לתיובתא יפה: התלמיד הצעיר טוען ש- $N = p_1 \cdots p_k + 1$ תמיד ראשוני, והתשובה היא $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \cdot 509$. ובסעיף "היכן המשל נשבר" מוסבר שלמספר יכולים להיות כמה גורמים ראשוניים, בעוד שבמשל יש "אב" אחד.

איך המנוע עובד בפועל

מנוע המשלים הוא pipeline של מודל שפה, עם שלוש בדיקות קבועות:

1. **קישור ל-Lean.** סעיף [0] חייב להפנות למשפט שקיים ב-Mathlib או במאגר, ושעבר `lake build`. משל על משפט שלא אומת לא יוצא.
2. **סימון מקורות.** בסעיף [3], כל טענה היסטורית מסומנת "מקור", "פרשנות" או "בדיוני". מודל שפה נוטה להמציא מקורות עתיקים משכנעים, ולכן זו הבדיקה החשובה ביותר.
3. **סעיף [5] לא ריק.** אם המודל לא מצא היכן המשל נשבר, המשל חוזר לעריכה.

התוצר המתוכנן: אתר או ספרון "משלי המתמטיקה", עם 30 משפטים ראשונים והערכה של קוראים.

חלק ח: גבולות

יושר אינטלקטואלי

- **אין "הוכחות" מפסוקים, ואין "הוכחות" לפסוקים.** לא נכתוב "התורה ידעה את משפט X", וגם לא "המתמטיקה מוכיחה את הקבלה". מותר לכתוב: "המבנה הקומבינטורי בטקסט Y הוא Z", ו"הטקסט שימש לנו השראה".

- **הבחנה בין היסטוריה לפרשנות מאוחרת.** כשטקסט "מכיל" רעיון מתמטי, שואלים: האם המחבר התכוון לכך, או שאנחנו קוראים זאת לתוכו? (כמו המחלוקת על Plimpton 322.) מסמנים "מקור", "פרשנות" או "בדיוני".
- **תיעוד מדויק:** מהדורה, נוסח, דף, הלכה. מה שלא נבדק מסומן "לא נבדק". הפרק הזה עצמו מסמן כך כמה פריטים.

מלכודת הנומרולוגיה

בעברית יש מאות אלפי מילים וצירופים, ורוב ערכי הגימטריה שלהם נופלים בטווח קטן, בערך בין 1 ל-1,500. לכן בטוח מבחינה סטטיסטית שיימצאו "התאמות" מפתיעות. בפיזיקה של חלקיקים קוראים לזה look-elsewhere effect: אם בודקים מספיק מקומות, תמיד יימצא "אות".

מקרה המבחן הוא פרשת "צופן התורה". ב-1994 פרסמו Witztum, Rips ו-Rosenberg ב-*Statistical Science* מאמר שטען לדילוגי אותיות מובהקים בספר בראשית. ב-1999 השיבו McKay, Bar-Natan, Bar-Hillel ו-Kalai באותו כתב עת, והראו שאפשר לקבל תוצאות דומות בתרגום של "מלחמה ושלום", בזכות דרגות החופש בבחירת הנתונים[121]. זהו שיעור חובה לכל חוקר במעבדה, ופרויקט W022 הופך אותו לסדנה אינטראקטיבית.

כלל העבודה: כל טענה סטטיסטית על טקסט מסורתי דורשת רישום מראש של ההשערה (pre-registration), קבוצת ביקורת של טקסטים אחרים, ותיקון להשוואות מרובות. ואפילו תוצאה "מובהקת" מראה דפוס בטקסט, לא אמת מטאפיזית.

ובמסורת עצמה, הגימטריה מוצגת כ"פרפראות לחכמה" (אבות ג, יח): "תקופות וגימטריאות פרפראות לחכמה". המפרשים נחלקו מה פירוש המילים. יש שפירשו "גימטריאות" כחכמת המדידה, הגאומטריה, ולא כחישוב ערכי אותיות[122]. בכל מקרה, אלה דברים נלווים לחכמה ולא עיקרה. המעבדה מכבדת את הגימטריה בדיוק במעמד הזה: כלי לימוד וזיכרון, לא ראיה.

כבוד לטקסטים קדושים ולקהילות

- **שותפות, לא ניכוס.** בכל פרויקט שנוגע לקבלה, ובפרט לכוונות הרש"ש, משתפים תלמידי חכמים מהמסורת הרלוונטית. כך גם במסורות אחרות: Ifá היא דת חיה של מיליוני אנשים, ולא "דאטה אקזוטי".
- **שמות קדושים.** בקוד, בקבצים ובתצוגה משתמשים בכתיב מקובל (ה', הוי"ה, י-ה-ו-ה), ומכבדים את מנהגי הגניזה בכל דבר מודפס.
- **זכויות.** סריקה ודיגיטציה רק ממקורות מותרים.
- **אין שימוש מאגי או מסחרי.** לא נבנה "מחולל כוונות", "מחשבון סגולות" או כל כלי שמציג את עצמו כבעל השפעה רוחנית. הכלים שלנו מנתחים מבנה בלבד.

פרק 12 מתרגם את הכללים האלה למדיניות פרסום.

בקצרה

- גשרי החוכמה פועלים בשלושה מישורים: שיטה (קושיא ותירוץ), מבנה (צירופים, לוח) ופדגוגיה (משלים). בשום מישור הם אינם ראיה מתמטית, וה-analogy-agent מוגבל לרמה L0.
- הגמרא מציעה פרוטוקול לשני סוכנים, מקשן ומתרץ, שבו הפוסק הוא Lean, והסוגיא נשמרת כקובץ מובנה.
- ספר יצירה מכיל ספירות מדויקות ($\binom{22}{2} = 231$, $n!$ עד 5,040) שמתאימות לפירמול ולשאלות על מבנה מעגל השערים.
- הלוח העברי לפי הרמב"ם הוא אלגוריתם של חשבון שלמים. 235/19 הוא קירוב שבר משולב, והמשפט על ששת אורכי השנה הוא פרויקט הדגל W001.
- הכוונות של הרש"ש והמילויים הם אובייקטים פורמליים שאפשר לחקור כדקדוקים ומערכות שכתוב, רק בשיתוף עם המסורת ובלי טענות מטאפיזיות.
- מנוע המשלים מחייב סעיף "היכן המשל נשבר", וכל עבודה סטטיסטית על טקסטים דורשת רישום מראש וקבוצת ביקורת.

רעיונות לתוכן

1. **"למה 19 שנים? הלוח העברי והשבר המשולב": סרטון הסבר קצר.** מהבעיה של 12.37 חודשים בשנה, דרך השבר המשולב, ל-235/19, ועד הסחיפה של יום ב-216 שנים. קהל יעד: קהל דתי ומסורתי, תלמידי תיכון, חובבי מתמטיקה.
2. **"שתי אבנים בונות שני בתים": פוסט ויזואלי.** אנימציה של הסידורים מ-2 עד 7 אבנים, והמעבר לעצרת, עם סימון ברור של מה מקור ומה פרשנות. קהל יעד: ציבור רחב ברשתות.
3. **"חברותא של מכונות": כתבת עומק ב-S27 News.** הפרוטוקול של מקשן, מתרץ ופוסק, עם סוגיא אמיתית מהמעבדה (אחרי שתתקיים). קהל יעד: קהילת AI, אנשי חינוך תורני.
4. **"משלי המתמטיקה": סדרה שבועית.** משל אחד בשבוע לפי התבנית המלאה, כולל "היכן המשל נשבר". קהל יעד: הורים, מורים, תלמידי ישיבות ותיכונים.
5. **"צופן התורה, ומה למדנו ממנו": סדנה מקוונת.** הדגמה אינטראקטיבית של look-elsewhere על טקסטים שונים. קהל יעד: סטודנטים, עיתונאים, קהל דתי שמתעניין בסטטיסטיקה.
6. **"הרמב"ם שלא אכפת לו מי אמר": פוסט עמדה.** הלכות קידוש החודש יז, כד כבסיס לעיקרון שהמעבדה מקבלת אמת לפי ההוכחה. קהל יעד: קהל רחב, שותפים אפשריים מהעולם התורני והאקדמי.

פרק 12: יושרה, אתיקה ופרסום

מעבדה שעובדת עם מודלי שפה מחזיקה בידיים כלי שיכול לכתוב, בתוך שניות, הוכחה משכנעת ושגויה, הודעה לעיתונות נלהבת על תוצאה שלא קיימת, או ייחוס היסטורי שנשמע אמין ולא היה ולא נברא. בסביבה כזו, האמינות היא המשאב היקר ביותר. כתבת ההשקה של S27 News ניסחה את זה כך: "כנות היא המותג".

הפרק הזה הופך את העיקרון הזה למדיניות. הוא מסביר איך רמות הטענה L0 עד L4 קובעות מה מותר לכתוב בכותרת, איך מכריזים על תוצאה, איך מחלקים קרדיט בין בני אדם, מודלים ועבודה קודמת, מה עושים כשמתגלה טעות, באילו רישיונות לפרסם קוד ונתונים, ואיך מכבדים טקסטים קדושים. בסוף הפרק יש צינור תוכן, מהספר הזה ועד סרטון, ורשימת בדיקה עריכתית מוכנה לשימוש.

הפרק נשען על `news/README.md` (פורמט הכתבות), `docs/architecture.md` (רמות טענה ועקרונות יושר), `docs/ai-math-lessons.md` (סעיפים 7 ו-8), `site/README.md` (כללי היושר בקוד) ופרק ה ב- `docs/wisdom-bridges.md` .

למה צריך מדיניות כתובה

שתי פרשות שמהן לומדים

באוקטובר 2025 הכריזו חוקרים ברשתות ש-GPT-5 "פתר עשר בעיות ארדש פתוחות". בתוך ימים התברר שהמודל מצא פתרונות קיימים בספרות, והציוצים נמחקו (פרק 9). התוכן עצמו, חיפוש ספרות מצוין, היה ראוי לפרסום. מה שהיה שגוי הוא המילה "פתר" בכותרת.

ובצד השני של אותה שנה, הוויקי של Terence Tao על תרומות AI לבעיות ארדש מתעד בשקיפות גם את הכישלונות: מתוך כ-61 רשומות בקטגוריית "AI standalone", כ-10 שגויות, כ-9 לא אומתו, וכ-25 נוספות התבררו כפתרונות שכבר היו בספרות[123]. הוויקי מוסיף הסתייגויות מפורשות: "This page is not a benchmark", ו"Absence of past progress may reflect obscurity rather than difficulty". `docs/ai-math-lessons.md` מסכם: "תיעוד שקוף של כישלונות שווה יותר מהכרזות".

ההבדל בין שתי הפרשות אינו בכמות העבודה ולא באיכות המודלים. הוא בשאלה אם הייתה מדיניות שקבעה מראש מה מותר לומר.

מה אנחנו מגינים עליו

המדיניות מגינה על שלושה דברים. על **הקוראים**, שצריכים לדעת בדיוק מה הוכח. על **הקהילה המתמטית**, שהזמן שלה יקר ושלא צריכה לבזבז אותו על בדיקת הכרזות מנופחות. ועל **המעבדה עצמה**, משום שכותרת אחת מוגזמת יכולה למחוק שנים של עבודה זהירה.

רמות הטענה כמדיניות עריכה

הסולם

כל כתבה ב-S27 News מסומנת בשדה `claim_level` ב-front-matter, והוא משקף את הטענה המתמטית **החזקה ביותר** בכתבה, כפי שהיא רשומה ב-`data/ledger.jsonl` (`news/README.md`):

רמה	משמעות	מה נדרש	מי קובע
none	אין טענה מתמטית	הודעות, הסברים, ראיונות	עורך
L0	רעיון, השערה	ניסוח ברור	כל סוכן
L1	ראיה חישובית	סקריפט רפרודוקטיבי, גרסאות, seeds	סוכן חיפוש או בנייה
L2	הוכחה לא-פורמלית	טקסט מלא שנבדק על ידי סוכן שני או אדם	סוכן Lean, אדם
L3	מאומת מכונה	`lake build` נקי, DRAT נבדק או בדיקה מדויקת, לניסוח שאושר	רק ה-verifier
L4	פורסם או נבדק	מאמר שהתקבל, או מיזוג ל-Mathlib	אדם

מה מותר בכותרת

הסולם הופך למדיניות עריכה דרך שאלה אחת: אילו מילים מותרות בכותרת ובפסקה הראשונה? הטבלה הבאה היא המדיניות:

רמה	פעלים מותרים	פעלים אסורים	כותרת לדוגמה
none	"משיקים", "מסבירים", "פותחים"	כל פועל שמרמז על תוצאה	"ESH27 Lab יוצאת לדרך"
L0	"מציעים", "חושבים ש", "כיוון חדש"	"מצאנו", "גילינו", "הראינו"	"רעיון: לקרוא את הלוח העברי כשבר משולב"
L1	"בדקנו עד N", "ניסויים מראים", "ראיה חישובית"	"הוכחנו", "תמיד", "לכל"	"בדקנו את ההשערה עד 10^{12} ולא מצאנו דוגמה נגדית"
L2	"טיוטת הוכחה", "טיעון שעדיין בבדיקה"	"הוכחנו", "פתרנו", "מאומת"	"טיוטת הוכחה למקרה פרטי של בעיה X, ממתינה לפירמול"
L3	"הוכחנו (מאומת ב-Lean)", "מאומת מכונה"	"פתרנו את הבעיה" כשהוכח רק חלק; "פורסם"	"הוכחנו ב-Lean ש-S(4)=44, עם צינור SAT שאפשר לשחזר"
L4	"פורסם", "התקבל ל-Mathlib", "נבדק על ידי עמיתים"	הגזמה של היקף התוצאה	"ההוכחה של S(4)=44 התקבלה ל-Mathlib"

שלושה כללים משלימים את הטבלה:

1. **המילים "הוכחנו", "proved" ו"solved" מופיעות רק בכתבות L3 ו-L4.** זה כלל מ- `news/README.md` , ואין לו חריגים.
2. **תוצאה חלקית מתוארת כחלקית.** L3 על מקרה פרטי או על חסם משופר היא תוצאה לגיטימית ויקרה, ומתארים אותה בדיוק כך. "שיפרנו את החסם התחתון של $W(3,7)$" ולא "התקדמות בבעיית ואן דר וורדן".
3. **רמה בכותרת = רמה ב-front-matter = רמה ב-ledger.** אם כתבה מסומנת L3, יש לה קישור לתעודה, וה-ledger מראה `certificate.verified: true` .

האתר אוכף את הכלל

הכלל הזה אינו רק עניין של משמעת עריכה. הוא כתוב בקוד. לפי `site/README.md` , כל ניסיון ב-ledger שטוען L3 או L4 **בלי** `certificate.verified == true` מוצג באתר כ-L2 ומסומן בציר הזמן. האתר אף פעם לא מציג "הוכח" על סמך טענה שלא אומתה. ו- `progress_pct` מוצג כ"עבודה שנעשתה", לא כקרבה להוכחה.

הגבול של L3

גם L3 אינו סוף פסוק. Lean מאמת שההוכחה נכונה **עבור הניסוח שנכתב**. אם הניסוח לא נאמן לבעיה, האימות לא שווה הרבה. `docs/architecture.md` קורא לזה "אופן הכישלון מס' 1". לכן בכל כתבה ברמה L3 מופיעים, מלבד הקישור לתעודה, גם הניסוח הפורמלי עצמו, והשם של מי שאישר שהוא נאמן לבעיה המקורית.

איך מכריזים על תוצאה

הסדר הנכון

הכרזה על תוצאה עוברת שבעה שלבים, בסדר הזה:

1. **בדיקת ספרות מתועדת.** literature-agent מחפש פתרון קיים (arXiv, zbMATH, OEIS, Mathlib, erdosproblems.com). התוצאה נשמרת ב-ledger.
2. **בדיקת הניסוח.** אדם מאשר שהניסוח הפורמלי אומר את מה שהבעיה אומרת, ורושם את שמו ואת התאריך (`statement_reviewed_by`, `statement_reviewed_at`).
3. **אימות.** ה-verifier מריץ את הבדיקה ורושם `certificate.verified = true`. בהוכחת Lean, הפלט של `#print axioms` מראה רק את שלוש האקסיומות הסטנדרטיות.
4. **מומחה בתחום.** לפני כל הכרזה פומבית, מומחה אנושי בתחום מאשר שהתוצאה חדשה ושיש לה משמעות (`docs/ai-math-lessons.md`, מסקנה 16).
5. **הודעה לבעלי העניין.** אם התוצאה משנה טבלה ציבורית (חסמי רמזי, erdosproblems.com, טבלאות Radziszowski), מודיעים למתחזקים לפני הכתבה, ולא דרכה.
6. **פרסום טכני.** repo ציבורי עם הקוד והתעודה, ולפי הגודל: פוסט ב-Lean Zulip, טיוטת arXiv, או PR ל-Mathlib.
7. רק אז: **S27 News.** הכתבה מקשרת לכל מה שלמעלה.

מבנה של כתבת תוצאה

לכתבה שמדווחת על תוצאה יש מבנה קבוע:

- **שורה ראשונה:** מה הוכח, באיזו רמה, ובאיזה היקף. "הוכחנו ב-Lean (L3) ש..." ולא "פריצת דרך!".
- **מה זה לא:** פסקה קצרה שאומרת מה התוצאה לא אומרת. האם זה מקרה פרטי? האם הבעיה המלאה עדיין פתוחה?
- **רקע:** למה השאלה מעניינת, ומה היה ידוע לפני כן.
- **מה עשינו:** השיטה, כולל מה נוסה ונכשל.

- **מי עשה מה:** סעיף קרדיט (ראו להלן).
- **איך לבדוק:** קישורים לתעודה, לקוד ולניסוח הפורמלי, והוראות שחזור.
- **מה הלאה.**

כתבת ההשקה (`news/2026-11-04-launch.md`) היא דוגמה לרוח הזו, גם אם אין בה תוצאה: כבר בשורה השנייה היא אומרת "לא הוכחנו עדיין שום דבר חדש".

מה לא עושים

הרשימה הזו מבוססת על פרק 8 ב- `docs/ai-math-lessons.md` :

- לא מכריזים על הוכחה של בעיית מילניום, השערת רימן או P מול NP. כל טענה כזו שמגיעה ממודל שפה היא כמעט בוודאות שגויה.
- לא מפרסמים "פתרנו בעיה פתוחה" לפני בדיקת ספרות ואימות פורמלי.
- לא מצטטים ציוני benchmark של יצרן בלי לציין שהם לא אומתו, ושיש חשש לזיהום נתונים.
- לא מייצרים "מאמרי AI" בכמויות. פתרון שולי לבעיה מוזנחת מפרסמים כרשומה ב-dataset או כפתק קצר.
- לא מסתירים את תפקיד ה-AI, וגם לא מנפחים אותו.

קרדיט: בני אדם, מודלים ועבודה קודמת

שלושה סוגי תורמים

כל תוצאה של המעבדה מגיעה משלושה מקורות, וכל אחד מהם מקבל קרדיט מפורש:

עבודה קודמת. כל תוצאה נשענת על ספרות. מצטטים את המקורות, את הספריות (Mathlib, PhysLean, SDPB), את הכלים (CaDiCaL, Kissat, drat-trim) ואת הפרויקטים שהשיטה נלקחה מהם. העיקרון מ- `docs/theoretical-physics.md` תקף לכל המעבדה: תרומה לפרויקט קיים עדיפה על המצאה מחדש.

בני אדם. מי בחר את הבעיה, מי אישר את הניסוח, מי כתב את הסקיצה, מי בדק. בדיקת הניסוח מקבלת קרדיט מפורש, משום שהיא העבודה האנושית החשובה ביותר בתהליך ולרוב היא נשכחת.

מודלים וסוכנים. אילו מודלים, באילו גרסאות, ובאיזה תפקיד. ה-ledger כבר שומר את השדות `agent` ו- `model` בכל ניסיון, ואת `produced_by` בכל artifact.

טבלת תפקידים

`docs/ai-math-lessons.md` (מסקנה 12) ממליץ לתעד מה עשה AI ומה עשו בני אדם, לפי הקטגוריות של ויקי ארדש. ההצעה של הפרק היא טבלה קבועה בסוף כל כתבת תוצאה:

שלב	מי	הערות
בחירת הבעיה	אדם / orchestrator	
חיפוש ספרות	literature-agent (מודל X)	נבדק על ידי …
ניסוח פורמלי	lean-agent (מודל X)	אושר על ידי …
רעיון ההוכחה או הבנייה	…	
פירמול	…	
אימות	verifier (Lean 4, גרסה …)	
כתיבה	publisher-agent, עריכה: …	

מודל אינו מחבר

שאלה עדינה היא אם מודל יכול להיות "מחבר" של מאמר. רוב כתבי העת והמוציאים לאור קבעו ב-2023–2025 שלא, משום שמחבר צריך לשאת באחריות לתוכן. המעבדה מאמצת את הגישה הזו: מודלים מוזכרים בסעיף תודות ובטבלת התפקידים, בפירוט רב, ובני אדם חותמים כמחברים ונושאים באחריות. [לא אומת: מדיניות כתבי עת ספציפיים לא נבדקה; לפני הגשה בודקים את מדיניות כתב העת הרלוונטי.]

טעויות, תיקונים והסרות

טעויות יקרו. השאלה היא איך מטפלים בהן. העיקרון הבסיסי כבר כתוב בארכיטקטורה: "כישלונות נרשמים. ההיסטוריה ב-ledger אינה נמחקת." ה-ledger הוא append-only עם hash chain, ולכן גם טעות אי אפשר "למחוק". אפשר רק לרשום ניסיון חדש שמחליף אותה (`supersedes`).

ארבע דרגות תגובה

מצב	דוגמה	תגובה
טעות קטנה שאינה משנה את הטענה	שם שגוי, קישור שבור, תאריך	תיקון בכתבה, והערה בתחתית: "תוקן ב-…"

מצב	דוגמה	תגובה
טעות שמשנה פרט בטענה	החסם הוא 43 ולא 44; תנאי חסר בניסוח	תיקון בגוף הכתבה, הערה בראש הכתבה, ורשומה חדשה ב-ledger
טעות שמורידה את רמת הטענה	התעודה לא עוברת בגרסה חדשה של הבודק; התגלה `sorry` נסתר	עדכון `claim_level`, הערה בולטת בראש הכתבה, כתבת המשך
הטענה שגויה או לא חדשה	התוצאה כבר פורסמה במקום אחר; הניסוח לא נאמן לבעיה	הסרה (retraction): הכתבה נשארת באתר, מסומנת "הוסרה", עם הסבר; כתבה נפרדת שמסבירה מה קרה

כללים להסרה

- **לא מוחקים.** כתבה שהוסרה נשארת בכתובת שלה, עם סימון ברור והסבר. מחיקה שקטה היא בדיוק מה שקרה בפרשת ארדש 2025, וזה מה שאנחנו לא רוצים לחקות.
- **מודיעים לאותו קהל.** אם התוצאה פורסמה ב-Zulip, ההסרה מתפרסמת ב-Zulip. אם היא הופצה ברשתות, ההסרה מופצת ברשתות.
- **מודים למי שמצא את הטעות**, בשמו אם הוא מסכים.
- **מנתחים את הכשל.** כל הסרה מלווה בניתוח קצר: איזה שלב בתהליך היה צריך לתפוס את הטעות, ומה משנים בו. הניתוח נרשם ביומן המבצעים (פרק 13).

ה-front-matter של תיקון

`news/README.md` לא מגדיר כרגע שדות לתיקונים. ההצעה של הפרק היא להוסיף שני שדות אופציונליים, ולעדכן בהתאם את `site/build.py`:

```
corrected: 2026-12-01          # תאריך התיקון האחרון
status: retracted              # published (ברירת מחדל) | corrected | retracted
```

[הצעה בלבד: השדות לא ממומשים באתר.]

נתונים וקוד פתוחים

למה פתוח

התעודה היא לב התוצאה, ותעודה שאי אפשר להוריד ולבדוק אינה תעודה. לכן כל מה שמאחורי טענה ברמה L1 ומעלה צריך להיות פתוח: הקוד, הנתונים, התעודות והניסוחים. `docs/ai-math-lessons.md` מוסיף שגם תוצאות שליליות, ניסוחים פורמליים וטבלאות חסמים הן תרומה, וגם אותן מפרסמים.

הצעת רישוי

סוג תוכן	רישיון מוצע	נימוק
קוד Lean	Apache-2.0	הרישיון של Mathlib, כך שאפשר להעביר משפטים ל-Mathlib בלי בעיה
קוד Python (orchestrator, סקריפטים, אתר)	Apache-2.0 או MIT	רישיונות מתירניים מקובלים
נתונים (`data/*.csv`, ledger, תעודות)	CC BY 4.0	שימוש חופשי עם ייחוס
טקסטים (הספר, כתבות, מסמכים)	CC BY 4.0	אותו דבר; מאפשר תרגום וגזירת תוכן
נתונים שנגזרו מטקסטים מסורתיים	לפי רישיון המקור	למשל טקסטים מ-Sefaria, שלכל אחד מהם רישיון משלו

[המלצה בלבד: הרישיונות לא נקבעו במאגר; ההחלטה היא של המייסד. הרישיון של Mathlib, Apache-2.0, מופיע ב-`data/tools.csv`.]

רפרודוקטיביות

הרישיון לבד לא מספיק. כדי שמישהו אחר יוכל לבדוק, צריך גם (`docs/architecture.md`, סעיף 9):

- גרסאות נעולות: `lean-toolchain`, `lake-manifest.json`, גרסאות של פותרי SAT.
- hash לכל artifact ולכל תעודה.
- seeds לכל חישוב אקראי.
- הוראות שחזור בשורה אחת, אם אפשר.
- את `data.json` שהאתר מפרסם לצד העמוד, לכל מי שרוצה את הנתונים הגולמיים.

כבוד לטקסטים קדושים

פרק 11 הגדיר את העקרונות. כאן הם מתורגמים למדיניות פרסום:

1. **אין כותרות של "התורה ידעה".** כתבה לא תטען שטקסט מסורתי "הכיל" משפט מודרני, וגם לא שמתמטיקה "מוכיחה" או "מפריכה" טקסט מסורתי. מותר: "המבנה הקומבינטורי בספר יצירה הוא $\binom{22}{2}$".
2. **סימון מקור, פרשנות ובדיוני.** כל טענה היסטורית בכתבה מסומנת. משל שמוצג במסגרת עתיקה ואין לו מקור מסומן "בדיוני".
3. **ציטוט מדויק.** מהדורה, נוסח, פרק והלכה. ציטוט שלא נבדק מול מקור ראשוני לא מתפרסם, או מסומן [לא אומת]. בפרק 11 יש כמה פריטים כאלה, והם צריכים בדיקה לפני שייגזרו מהם כתבות.
4. **שמות קדושים.** בכתבות, בתמונות ובסרטונים משתמשים בכתיב מקובל (ה', הוי"ה, י-ה-ו-ה). בחומר מודפס מתחשבים במנהגי גניזה.
5. **שותפות.** כתבה על פרויקט שנוגע לקבלה, ובפרט לכוונות הרש"ש, עוברת קריאה של שותף מהמסורת לפני פרסום. כך גם במסורות אחרות, למשל Ifá.
6. **אין סגולות.** שום כתבה, כלי או סרטון לא יציג את עצמו כבעל השפעה רוחנית, ולא ישתמש בשפה כזו בשביל תשומת לב.
7. **סטטיסטיקה על טקסטים** מתפרסמת רק עם רישום מראש, קבוצת ביקורת ותיקון להשוואות מרובות (פרק 11, פרשת "צופן התורה").

הימנעות מהייפ

הייפ אינו רק כותרות. הוא נכנס גם דרך פרטים קטנים. כמה מלכודות נפוצות, והתחליף שלהן:

מלכודת	דוגמה	תחליף
פעל חזק על תוצאה חלשה	"AI פיצח את בעיית X"	"סוכן מצא בנייה שמשפרת את החסם התחתון של X ב-1"
השמטת ההקשר	"שיפור של 20%"	"שיפור בכ-20% מ-50 הבעיות שנבדקו, ושחזור בכ-75%"
ייחוס יתר ל-AI	"ה-AI הוכיח"	"המודל כתב סקיצה, Lean אימת, ואדם אישר את הניסוח"
ייחוס חסר ל-AI	הצגת עבודה של סוכנים כאילו נעשתה ידנית	טבלת תפקידים מלאה
ציון benchmark כראיה	"המודל שלנו פותר 90% מ-miniF2F"	לא מצטטים benchmarks רוויים; מדווחים על בעיות שלא דלפו

מלכודת	דוגמה	תחליף
הבחירה בהצלחות	מדווחים על הניסיון שעבד	מדווחים כמה ניסיונות היו, וכמה הצליחו
מילים שמבטיחות עתיד	"בקרוב נפתור..."	"המטרה הבאה בבנק היא..."

ובקצרה: הסגנון של S27 News, לפי `news/README.md`, הוא "מעורר השראה אבל מפוכח". אומרים מה נוסה, מה עבד, מה נכשל ומה רמת הטענה. מצטטים מקורות. ולא מעגלים תוצאה כלפי מעלה.

צינור התוכן: מהספר לסרטון

ארבע שכבות

הספר הזה הוא שכבת המקור. ממנה נגזרות שכבות נוספות, כל אחת לקהל אחר, וכל אחת שומרת על אותה רמת טענה:

```
ספר (פרקים, מקורות מלאים)
  └► כתבה ב-S27 News (800–1,500 מילים, front-matter עם claim_level)
       └► פוסטים ברשתות (שרשור, תמונה, ציטוט; קישור לכתבה)
            └► סרטון (קצר: 60–90 שניות; ארוך: 8–15 דקות; קישור לכתבה)
```

כל פרק בספר מסתיים בסעיף "רעיונות לתוכן", שהוא נקודת הכניסה לצינור.

הכללים שעוברים בין השכבות

1. **רמת הטענה לא עולה בדרך.** אם הכתבה היא L1, גם הפוסט וגם הסרטון הם L1. קיצור נוטה להגזים, ולכן בודקים את זה בכל שכבה.
2. **כל שכבה מקשרת לשכבה שמעליה.** הפוסט מקשר לכתבה, הכתבה לספר, לתעודה ולקוד.
3. **הסתייגות אחת שורדת תמיד.** גם בסרטון של 60 שניות יש משפט אחד שאומר מה זה לא. לדוגמה: "זה מקרה פרטי; הבעיה המלאה עדיין פתוחה".
4. **מקורות בתיאור.** בסרטון ובפוסט, רשימת המקורות מופיעה בתיאור או בתגובה הראשונה.
5. **תוכן שגוזר מפריט [לא אומת] לא מתפרסם** עד שהפריט נבדק.

מי עושה מה

- **publisher-agent** מכין טיוטות של כתבות מתוך ה-ledger ומתוך הספר. לפי הארכיטקטורה, הוא "משקף את ה-ledger בלבד": הוא לא יכול לטעון רמה גבוהה מזו שרשומה.

- **עורך אנושי** עובר על כל טיוטה עם רשימת הבדיקה שלמטה, ומאשר פרסום.
- **אתר המעבדה** נבנה מחדש אוטומטית מ- `news/*.md` ומ- `data/` אחרי כל commit (`site/README.md`).
- **פוסטים וסרטונים** נוצרים מהכתבה שאושרה, לא מהספר ישירות, כך שכל תוכן ציבורי עבר את רשימת הבדיקה לפחות פעם אחת.

מחזור עבודה לדוגמה

נניח שהמטרה G050 מסתיימת בהצלחה (פרק 13). המחזור ייראה כך:

1. ה-verifier רושם ב-ledger ניסיון L3 עם תעודת Lean.
2. publisher-agent מכין טיוטה: `news/2026-11-xx-g050-schur.md`, עם `claim_level: L3`, קישור לתעודה, הניסוח הפורמלי וטבלת תפקידים.
3. עורך עובר על רשימת הבדיקה. אדם שבדק את הניסוח מאשר. מומחה חיצוני (אם יש) מאשר.
4. הכתבה מתפרסמת, והאתר מתעדכן.
5. שרשור ברשתות: "הוכחנו ב-Lean ש-$S(4)=44$. זה ערך שידוע מ-1965; החידוש הוא צינור SAT ל-Lean שאפשר לשחזר. הנה איך."
6. סרטון קצר: צביעה של 1 עד 44 בארבעה צבעים, ולמה 45 בלתי אפשרי.
7. עדכון של הפרק המתאים בספר, ורשומה ביומן המבצעים.

רשימת בדיקה עריכתית

את הרשימה הזו עובר עורך אנושי על כל כתבה לפני פרסום. אפשר להעתיק אותה לתבנית של pull request.

רמת טענה - [ ] `claim_level` ב-front-matter שווה לטענה החזקה ביותר בכתבה. - [ ] הרמה תואמת את ה-ledger. בכתבת L3 ומעלה: יש קישור לתעודה, וה-ledger מראה `certificate.verified: true`. - [ ] המילים "הוכחנו", "solved", "proved" מופיעות רק בכתבת L3 או L4. - [ ] תוצאה חלקית מתוארת כחלקית, כבר בכותרת. - [ ] יש פסקה שאומרת מה התוצאה **לא** אומרת.

ניסוח ואימות - [ ] בכתבת L3: הניסוח הפורמלי מופיע או מקושר, ויש שם של מי שאישר אותו. - [ ] `#print axioms` נקי (רק propext, Classical.choice, Quot.sound); אין `sorry`; `native_decide` מסומן אם יש. - [ ] גרסאות נעולות והוראות שחזור מקושרות.

חידוש ומקורות - [ ] בדיקת ספרות מתועדת ב-ledger. ברור שהתוצאה לא פורסמה קודם. - [ ] מתחזקי טבלאות ומקורות ציבוריים רלוונטיים קיבלו הודעה לפני הכתבה. - [ ] כל טענה עובדתית מגובה במקור;

מה שלא אומת מסומן [לא אומת] או הוסר. - [ ] טענות של יצרנים על מוצריהם מסומנות "טענת יצרן".

קרדיט - [ ] טבלת תפקידים: מה עשו בני אדם, מה עשו מודלים (כולל גרסאות), מה נלקח מעבודה קודמת. - [ ] ספריות וכלים מצוטטים (Mathlib, PhysLean, פותרים). - [ ] תפקיד ה-AI לא מוסתר ולא מנופח.

טקסטים מסורתיים (אם רלוונטי) - [ ] אין טענה ש"המסורת ידעה" או ש"המתמטיקה מוכיחה" טקסט מסורתי. - [ ] כל טענה היסטורית מסומנת מקור, פרשנות או בדיוני. - [ ] ציטוטים כוללים מהדורה, נוסח ומיקום, ונבדקו מול מקור. - [ ] שמות קדושים בכתיב מקובל. - [ ] שותף מהמסורת קרא את הכתבה (בפרויקטים של קבלה ומסורות חיות).

סגנון והייפ - [ ] אין פעלים חזקים על תוצאות חלשות; אין "פריצת דרך" בלי L4 ותמיכה חיצונית. - [ ] מספרים מוצגים עם ההקשר שלהם (מתוך כמה, באיזה תנאי). - [ ] הכתבה מספרת גם מה נכשל.

טכני - [ ] שם הקובץ `YYYY-MM-DD-slug.md`; front-matter שטוח עם `title`, `date`, `lang`, `tags`, `claim_level`. - [ ] האתר נבנה בלי אזהרות (`python3 site/build.py`), והנוסחאות מוצגות. - [ ] אין טבלאות או תמונות בגוף הכתבה (האתר לא תומך בהן כרגע).

שכבות נגזרות - [ ] פוסטים וסרטונים שומרים על אותה רמת טענה ומקשרים לכתבה. - [ ] הסתייגות אחת לפחות שורדת בכל שכבה.

בקצרה

- רמות הטענה L0–L4 הן מדיניות עריכה: הן קובעות אילו פעלים מותרים בכותרת. "הוכחנו" רק ב-L3 ומעלה, והאתר אוכף את זה בקוד.
- הכרזה על תוצאה עוברת שבעה שלבים, מבדיקת ספרות ועד כתבה, ומומחה אנושי מאשר לפני כל הכרזה פומבית.
- קרדיט ניתן לשלושה מקורות: עבודה קודמת, בני אדם (כולל מי שבדק את הניסוח) ומודלים, בטבלת תפקידים קבועה. מודל אינו מחבר.
- טעויות לא נמחקות. יש ארבע דרגות תגובה, עד הסרה גלויה עם ניתוח של הכשל.
- ההמלצה לרישוי: Apache-2.0 לקוד ול-Lean, CC BY 4.0 לנתונים ולטקסטים, ורישיון המקור לטקסטים מסורתיים.
- צינור התוכן עובר מספר לכתבה לפוסט לסרטון, ורמת הטענה לא עולה בדרך. רשימת הבדיקה העריכתית חלה על כל כתבה.

רעיונות לתוכן

1. **"מה מותר לנו לכתוב בכותרת": פוסט שקיפות.** הטבלה של הפעלים המותרים לכל רמה, כהתחייבות פומבית של S27 News. קהל יעד: קוראי S27 News, עיתונאי מדע.
2. **"הכתבה שלא פרסמנו": מדור קבוע.** כל רבעון, דוגמה לטענה שנעצרה ברשימת הבדיקה, ומה עצר אותה. קהל יעד: קהילת AI-for-Math, קוראים ספקנים.
3. **"איך בודקים הכרזה על AI שפתר בעיה": מדריך לעיתונאים.** שבע שאלות לשאול לפני שמפרסמים, על בסיס רשימת הבדיקה. קהל יעד: עיתונאים, יחצ"נים, עורכי חדשות.
4. **"מי הוכיח את זה? טבלת התפקידים": סרטון קצר.** איך נראית חלוקת עבודה בין אדם, מודל ו-Lean בתוצאה אחת. קהל יעד: סטודנטים, קהל רחב שמתעניין ב-AI.
5. **"הסרה היא לא בושה": פוסט עמדה.** למה מעבדה צריכה לתכנן מראש איך היא מודה בטעות, עם הוויקי של Tao כדוגמה. קהל יעד: חוקרים, מנהלי מעבדות, הקהילה המדעית.

פרק 13: יומן מבצעים

שאר הפרקים בספר מסבירים מה המעבדה היא ולמה היא בנויה כך. הפרק הזה מתעד מה היא עושה, יום אחרי יום. הוא הפרק החי של הספר: הוא לא נגמר, ורשומות חדשות מתווספות לו בכל פעם שמשהו משמעותי קורה, הצלחה, כישלון או החלטה.

ליומן יש שלושה תפקידים. הראשון הוא **זיכרון**: בעוד שנה, כשנרצה לדעת למה בחרנו בכלי מסוים או למה עזבנו מטרה מסוימת, התשובה תהיה כאן. השני הוא **שקיפות**: היומן מראה לקוראים, ולנו עצמנו, את הקצב האמיתי של העבודה, כולל הימים שבהם שום דבר לא עבד. השלישי הוא **חומר גלם לתוכן**: רוב הכתבות ב-S27 News יתחילו כרשומה כאן.

היומן הזה אינו ה-ledger. ה-ledger (`data/ledger.jsonl`) הוא רשומה מכנית, append-only עם hash chain, של כל ניסיון של כל סוכן, וממנו נקבעות רמות הטענה. היומן הוא הסיפור האנושי שמסביב: מה תכננו, מה קרה, מה למדנו ומה מחליטים הלאה. כשיש סתירה בין השניים, ה-ledger קובע.

איך כותבים רשומה

כללים

1. **רשומה לכל אירוע משמעותי**, לא לכל יום. אירוע משמעותי הוא: מטרה שהתחילה או נסגרה, תוצאה ברמה L1 ומעלה, כישלון שלימד משהו, החלטה על כלי או מדיניות, תיקון או הסרה של כתבה.
2. **רשומות לא נמחקות.** אם רשומה התבררה כשגויה, מוסיפים רשומה חדשה שמתקנת אותה ומפנה אליה, כמו ב-ledger.
3. **רמת טענה בכל רשומה** שמדווחת על תוצאה, עם הפניה לניסיון ב-ledger.
4. **הסדר כרונולוגי**, הישן למעלה. רשומות על מטרה שעדיין בתהליך מתעדכנות במקום, עם תאריך עדכון, עד שהמטרה נסגרת. אז הן ננעלות.
5. **כנות לפני הכול.** הסגנון זהה לזה של S27 News: מה נוסה, מה עבד, מה נכשל.

התבנית

```
### YYYY-MM-DD: כותרת קצרה

- סוג:** הקמה | מטרה | תוצאה | כישלון | החלטה | תיקון**
- מטרות:** G0xx, W0xx (אם רלוונטי)
- רמת טענה:** none | L0 | L1 | L2 | L3 | L4 (+ attempt_id ב-ledger)
- מי:** בני אדם וסוכנים שהשתתפו**

**מה תכננו.**..
**מה קרה.**..
**מה למדנו.**..
**מה הלאה.**..
**קבצים:** קישורים, commits, תעודות, כתבות.
```

הרשומות

2026-11-04: הקמת המעבדה

- **סוג:** הקמה
- **מטרות:** כל הבנק; G050, G023, G052, G029 כמטרות הראשונות
- **רמת טענה:** none
- **מי:** המייסד; סוכני Claude שבנו את התשתית

מה תכננו. להעמיד ביום אחד את כל השלד של המעבדה: בנק מטרות, קטלוגים של רקע, ארכיטקטורה, אתר, ערוץ חדשות וכלי מעקב, כך שמהיום הבא אפשר יהיה להתחיל לעבוד על מטרות ולא על תשתית.

מה קרה. נבנו הדברים הבאים, כולם במאגר תחת `lab/`:

רכיב	קבצים	היקף
בנק מטרות	`data/goals.csv`, `docs/goal-bank.md`	77 מטרות: A=10, B=12, C=27, D=17, P=11; שיטת ניקוד (`ai_tractability`, `formalization_readiness`) ושלוש מטרות ראשונות
קטלוג הוכחות יצירתיות	`data/creative_proofs.csv`, `docs/creative-proofs.md`	60 מקרים (P001–P060), 10 דפוסים, playbook לסוכנים

רכיב	קבצים	היקף
ארגז כלים	`data/tools.csv`, `docs/tools.md`	140 כלים ב-19 קטגוריות; stack התחלתי של 15 כלים; `scripts/setup.sh`
גשרי חוכמה	`data/wisdom_projects.csv`, `docs/wisdom-bridges.md`	24 פרויקטים (W001–W024), מנוע המשלים, סעיף גבולות
לקחים מ-AI-for-Math	`docs/ai-math-lessons.md`	סקירה של 2019–2026, 17 מסקנות ורשימת anti-patterns
פיזיקה תיאורטית	`docs/theoretical-physics.md`	מפת בעיות, ארגז כלים, חמישה פרויקטי פתיחה
ארכיטקטורה	`docs/architecture.md`, `orchestrator/`, `agents/`	גרעין אמון, לולאה, רמות טענה L0–L4, מודל נתונים, שבעה סוכנים ו-verifier
Lean	`lean/`	פרויקט Lake בשם EshLab (`leanprover/lean4:v4.24.0`), משפטי חימום ב- `Basic.lean`, ו-Collatz ו-Goldbach כ- `Prop` ב- `Targets/`
אתר	`site/`	אתר סטטי דו-לשוני (עברית ראשונה), נבנה מ- `data/` ומ- `news/` בסקריפט Python אחד; כללי היושר כתובים בקוד
S27 News	`news/README.md`, `news/2026-11-04-launch.md`	פורמט כתבות עם `claim_level`; כתבת ההשקה
מעקב	Google Sheets, `ESH27-Lab.xlsx`, `scripts/build_workbook.py`	תצוגה מקוצרת של בנק המטרות ב-Google Sheets; חוברת Excel שנבנית מה-CSV
הספר	`book/`	מדריך סגנון ופרקי טיוטה ראשונים, ובהם הפרק הזה

קובצי ה-CSV ב- `data/` הם מקור האמת. ה-Google Sheet, חוברת ה-Excel והאתר הם תצוגות שנבנות מהם (`README.md` של המעבדה).

מה למדנו. שלושה דברים כבר ביום הראשון:

1. **כמה מהר אפשר לבנות שלד, וכמה מהר אפשר לטעות בו.** חלק גדול מהמסמכים נכתב על ידי סוכנים, והם כוללים פריטים שמסומנים [לא אומת]. כמה מהם, למשל מספור הקטע "שתי אבנים" בספר יצירה, לא אומתו גם בכתיבת הספר, משום שהמקורות לא היו נגישים מסביבת העבודה. רשימת הפריטים האלה היא משימה פתוחה.
2. **שתי מערכות של רמות.** מסמך הפיזיקה הגדיר סולם H/R/L, והמעבדה כולה משתמשת ב-L0–L4. פרק 10 מיפה ביניהם. ההחלטה היא שהסולם L0–L4 הוא הקובע, ו-H/R/L הוא תיאור משלים בפיזיקה.
3. **ה-ledger עדיין ריק.** כל מה שנבנה היום הוא תשתית, ושום טענה מתמטית חדשה לא נטענה. כתבת ההשקה אומרת את זה במפורש.

מה הלאה. להתחיל את שלוש המטרות הראשונות מ- `docs/goal-bank.md` : G050 (מספרי Schur ו-van der Waerden קטנים ב-Lean), G023+G052 (סריקת בעיות ארדש ופירמול) ו-G029 (חסמים תחתונים חדשים בחיפוש בניות). במקביל, לאמת את הפריטים המסומנים [לא אומת] בספר ובמסמכים.

קישורים: `lab/README.md` ; `news/2026-11-04-launch.md` ; מעקב: https://docs.google.com/spreadsheets/d/1V9coJlUiul7Z8uRsovujRcw9KB6DWcYMtOQhWudvhNo/edit

---

G050 — מספרי שור ו-ון דר וורדן ב-Lean (בתהליך)

- **סוג:** מטרה
- **מטרות:** G050 (Tier D)
- **רמת טענה:** טרם נקבעה; אין עדיין ניסיון רשום ב-ledger [יעודכן]
- **מי:** lean-agent, search-agent; בודק ניסוח: [יעודכן]
- **נפתחה:** 2026-11-04 · **עדכון אחרון:** 2026-11-04

מה תכננו. להוכיח ב-Lean 4, בלי `sorry` , ערכים מדויקים קטנים של מספרי Schur ו-van der Waerden: $S(4) = 44$, $W(3,3) = 27$ ו-$W(2,4) = 35$, דרך תעודות SAT. מספר Schur $S(k)$ הוא ה-n הגדול ביותר שעבורו אפשר לצבוע את $1, \dots, n$ ב-k צבעים בלי פתרון מונוכרומטי של $x + y = z$. מספר van der Waerden $W(r,k)$ הוא ה-n הקטן ביותר שעבורו כל צביעה של $1, \dots, n$ ב-r צבעים מכילה סדרה חשבונית מונוכרומטית באורך k. הערכים האלה ידועים מזמן (Baumert 1965; Chvátal 1970)[124]. החידוש אינו בערכים, אלא בצינור: SAT, תעודה, Lean. זה דפוס 2 ודפוס 7 מפרק 9, והתבנית היא פרויקט ה-empty hexagon.

לכל ערך יש שני חצאים:

- **חסם תחתון:** צביעה מפורשת של $1, \dots, n$ שאין בה מבנה אסור. בודקים אותה ב-Lean ב- `decide` .
- **חסם עליון:** הוכחה שלכל צביעה של $1, \dots, n+1$ יש מבנה אסור. מקודדים כבעיית SAT, הפותר מוצא שהיא לא ספיקה (UNSAT) ומפיק תעודת DRAT או LRAT, והתעודה נבדקת. הצעד הקשה הוא לחבר את התעודה ל-Lean, כולל הוכחה שהקידוד נכון.

התוכנית מ- `docs/goal-bank.md` : ימים 1–2 בדיקת ספרות (האם כבר הוכח ב-Lean, ב-Isabelle או ב-HOL4); ימים 3–6 קידוד CNF עם שבירת סימטריות והרצת CaDiCaL או Kissat עם פלט LRAT; ימים 7–11 הוכחה ב-Lean שהקידוד נכון והחסם התחתון; ימים 12–14 ניקוי, README ו-PR.

מה קרה עד עכשיו. [תמונת מצב בזמן כתיבת הפרק; העבודה רצה במקביל ועשויה להשתנות.] בתיקייה `experiments/G050/` יש צינור SAT רפרודוקטיבי: מקודד (`encode.py`), סקריפט הרצה (`run_all.py`) וסקריפט שבונה גרסאות נעולות של CaDiCaL ו-drat-trim. לפי `results.json` :

- נמצאו צביעות עדות (witnesses) לחסמים התחתונים של $S(1)$ עד $S(4)$, ושל $W(2,3)$, $W(3,3)$ ו-$W(2,4)$, ונבדקו בבודק מדויק עצמאי.
- לחסמים העליונים של $S(1)$, $S(2)$ ו-$S(3)$ הופקו תעודות DRAT שעברו את drat-trim.
- בדיקת LRAT מדווחת בשלב זה `lrat_verified: false` גם במקרים הקטנים. [יעודכן: לברר אם זו בעיה בתעודה או בפענוח הפלט של הבודק.]
- החסם העליון של $S(4)$ ($n = 45$) והחיבור ל-Lean עדיין בעבודה. קובץ העדים ל-Lean אמור להיווצר ב- `lean/EshLab/Targets/G050/` .

בשלב הזה כל מה שיש הוא לכל היותר L1 (ראיה חישובית עם תעודות SAT חיצוניות). שום דבר עוד לא עבר את ה-verifier, ולכן אין עדיין "הוכחנו".

מה למדנו. [יעודכן]

מה הלאה. [יעודכן] לסגור את בדיקת ה-LRAT; להשלים את $S(4)$; לנסח ב-Lean את הטענה ($\forall c : \mathrm{Fin}\ 44 \to \mathrm{Fin}\ 4, \dots$) ולהעביר את הניסוח לבדיקה אנושית; לחבר את התעודה ל-Lean (LeanSAT, `bv_decide` או בודק מאומת).

מה נחשב ניצחון. משפט ב-Lean, שנבדק על ידי ה-kernel, ש-$S(4) = 44$ ולפחות ערך אחד נוסף, עם צינור SAT ל-Lean שאפשר להשתמש בו שוב, למשל במטרה G029.

קישורים: `experiments/G050/` ; `data/goals.csv` (שורה G050); `docs/goal-bank.md` , סעיף 4.

בקצרה

- היומן הוא הפרק החי של הספר: רשומה לכל אירוע משמעותי, בסדר כרונולוגי, בלי מחיקות.
- היומן מספר את הסיפור האנושי. ה-ledger הוא הרשומה המכנית, וכשיש סתירה ה-ledger קובע.
- ב-2026-11-04 הוקמה המעבדה: 77 מטרות, 60 הוכחות יצירתיות, 140 כלים, 24 פרויקטי חוכמה, ארכיטקטורה, אתר, כתבת השקה ומעקב ב-Google Sheets. שום טענה מתמטית חדשה עוד לא נטענה.
- המטרה הראשונה, G050, בתהליך: צינור SAT רפרודוקטיבי קיים, החיבור ל-Lean עוד לא, והרמה היא לכל היותר L1.
- כל רשומה עם תוצאה נושאת רמת טענה והפניה ל-ledger.

רעיונות לתוכן

1. **"היום הראשון": פוסט מאחורי הקלעים.** מה בונים ביום אחד כשרוב העבודה נעשית על ידי סוכנים, ומה עדיין דורש בדיקה אנושית. קהל יעד: קהילת AI, יזמים, קוראי S27 News.
2. **"יומן G050": סדרת עדכונים קצרים.** עדכון לכל שלב (ספרות, קידוד, תעודה, Lean), כולל התקלות, למשל בדיקת ה-LRAT. קהל יעד: קהילת Lean ו-SAT, סטודנטים.
3. **"44 מספרים, ארבעה צבעים": סרטון קצר.** מה זה מספר Schur, איך נראית הצביעה של 1 עד 44, ולמה 45 בלתי אפשרי. קהל יעד: קהל רחב, תלמידי תיכון.
4. **"ledger מול יומן": פוסט טכני.** למה למעבדה יש שתי רשומות, ואיך hash chain מונע שכתוב של ההיסטוריה. קהל יעד: מהנדסי תוכנה, חוקרים שמתעניינים ברפרודוקטיביות.

עדכון 2026-11-05: G050, תוצאות ראשונות

רמת טענה: L3, פורמליזציה של תוצאות ידועות. אין כאן מתמטיקה חדשה.

- **נבדקו ב-kernel של Lean בלבד** (האקסיומות היחידות: `propext` , `Classical.choice` , `Quot.sound`): $S(1) = 1$, $S(2) = 4$, $S(3) = 13$, $W(2,3) = 9$, וגם החסמים התחתונים $S(4) \geq 44$, $W(3,3) > 26$ ו-$W(2,4) > 34$.
- **בדרגת אמון חלשה יותר** (`native_decide` , עם האקסיומה `Lean.ofReduceBool`): $W(3,3) = 27$ ו-$W(2,4) = 35$. הוכחת ה-UNSAT של CaDiCaL בפורמט LRAT נבדקה ב-checker ה-LRAT המאומת של Lean, ונכונות המקודד הוכחה ב-Lean.
- **עדיין פתוח ב-Lean:** החסם העליון $S(4) \leq 44$. תעודת ה-LRAT שלו היא כ-520MB, והיא נבדקה רק בכלים חיצוניים (DRAT-trim).

- **בנייה:** `cd lab/lean/G050 && lake build` עובר בכ-43 שניות מאפס, בלי תלות ב-mathlib. אומת שוב באופן עצמאי.
- **פרטים:** `experiments/G050/REPORT.md` , `data/ledger.jsonl` .

שיעור: הצינור SAT → LRAT → Lean עובד מקצה לקצה. צוואר הבקבוק הוא גודל התעודה, ולא הלוגיקה.

עדכון 2026-11-11: G029, שני מועמדים לשיפור חסמים

רמת טענה: L1 (חישוב משוחזר), אחרי ביקורת מקשן. לא "שיא" עד שייבדק מול כל הספרות ויפורסם.

- **W(3,7) > 70,357** (בטבלאות: 48,811) ו-**W(3,8) > 494,285** (בטבלאות: 238,400): צביעות מפורשות ב-3 צבעים בלי סדרה חשבונית חד-צבעית באורך 7 ו-8.
- **השיטה:** המשפט של Kim (arXiv:2610.32000, אוקטובר 2026) על "בלוקים פוליכרומטיים", עם בלוקים חדשים שנמצאו בעזרת פותר SAT (q=19 ו-q=43) על בסיסים ידועים של Rabung. החידוש שלנו הוא הבלוקים, לא המשפט.
- **אימות:** שלושה בודקים בלתי תלויים, כולל בודק שנכתב מאפס בסשן הראשי (`experiments/G029/review/`).
- **המקשן (`review/MAKSHAN.md`):** הניסוח נכון, הבנייה תואמת את המשפט, ושום מקור נגיש לא נותן ערך גדול יותר. **תיקו שנשאר:** מאמר אחד מ-2014 שלא היה נגיש.
- **אין תעודת Lean:** הגודל גדול מדי ל- `decide` . גם זה נרשם.
- **תוצאה שלילית:** כ-20 אלף ניסיונות על 8 תאים קטנים יותר, בלי שיפור. תועד.
- **ממתין לאריאל:** פנייה ל-Kim או ל-Monroe, הערה קצרה ב-arXiv, עדכון ויקיפדיה. כולם פעולות כלפי חוץ.

עדכון 2026-11-11: הלוח העברי של הרמב"ם ב-Lean, מאומת בקרנל

רמת טענה: L3 (אומת ב-kernel של Lean, בלי `native_decide` , בלי `sorry` ; האקסיומות: `propext` , `Classical.choice` , `Quot.sound` בלבד). בנייה נקייה שוחזרה בסשן הראשי (5.5 דקות). **עדיין בלי ביקורת מקשן.**

חבילה: `lean/Hebrew/` . ההגדרות עוקבות אחרי הלכות קידוש החודש (פרקים ו–ח) עם ציטוט ההלכה ליד כל הגדרה (`experiments/hebrew-calendar/SOURCES.md`).

משפט	מה הוא אומר	מצב בספרות
`lo_adu`	ראש השנה לעולם לא ביום א', ד', ו'	הוכח ב-Russinoff (ACL2, 2022; **ראשון ב-Lean**

משפט	מה הוא אומר	מצב בספרות
`yearLength_legal`	אורך שנה ∈ {353,354,355,383,384,385}	כנ"ל
`exactly_14_keviyot`	**לכל היותר** 14 קביעות, **וכל אחת מהן מתרחשת** (שנת עדות לכל אחת)	ה"מתרחשת" **חדש**: ב-ACL2 רק "לכל היותר"
`keviyah_period`	הקביעה חוזרת אחרי 689,472 שנה	הוכח ב-ACL2 לאורכי שנים
`greatPeriod_minimal`	**אין מחזור קצר יותר**	**לא נמצאה הוכחה פורמלית בשום מקום**
`rambam_eq_dr'`	הנוהל של הרמב"ם שקול לנוסחה של Dershowitz–Reingold	חדש

שיטה: בלי מעבר על 689,472 שנים. מצמצמים ליום המולד בשבוע ולשעה, 7 מקרים, ו- `omega` . בדיקה צולבת מול Python ב-10,001 שנים (L1).

תיקו: משפט לנדאו על המולד החודשי; מינימליות לאורכי שנים בלבד; AFP לא היה נגיש.

נספח א: מילון מונחים

הספר הזה נע בין לוגיקה, תורת המספרים, מדעי המחשב, למידת מכונה ופיזיקה תיאורטית, ולכל תחום יש שפה משלו. המילון מרכז כ-120 מונחים שמופיעים לאורך הפרקים ובמסמכי המעבדה (`docs/`), כדי שקורא שנתקל במונח לא מוכר יוכל למצוא אותו במקום אחד. כל ערך כולל את המונח בעברית, את המקבילה באנגלית ושורת הגדרה אחת. ההגדרות קצרות במכוון: הן נועדו להזכיר, לא להחליף את ההסבר שבפרקים. הערכים ממוינים לפי האלף-בית העברי; מונחים שאין להם שם עברי מקובל (כמו Lean או mathlib) מופיעים בתעתיק עברי.

א

- **אוטו-פורמליזציה** (autoformalization): תרגום אוטומטי, בדרך כלל בעזרת מודל שפה, של מתמטיקה משפה טבעית לשפה פורמלית כמו Lean.
- **אולימפיאדה בין-לאומית במתמטיקה** (International Mathematical Olympiad, IMO): תחרות לתלמידי תיכון; מדד לשיפור מערכות AI (כסף 2024, זהב 2025).
- **אי-כריעות** (undecidability): תכונה של בעיה שאין אלגוריתם שמכריע אותה בכל המקרים, כמו בעיית העצירה.
- **אי-שלמות, משפטי** (incompleteness theorems): משפטי גדל (1931): במערכת עקבית וחזקה דיה יש טענות אמיתיות שאינן ניתנות להוכחה, והיא אינה מוכיחה את עקביותה.
- **אינדוקציה** (induction): שיטת הוכחה: מוכיחים מקרה בסיס ומראים שמכל מקרה נובע הבא אחריו; ב-Lean היא נגזרת מטיפוסים אינדוקטיביים.
- **אינווריאנט** (invariant): גודל או תכונה שאינם משתנים תחת פעולה מסוימת; כלי מרכזי להוכחת אי-אפשרות.
- **אינטואיציוניזם** (intuitionism): גישה ליסודות המתמטיקה (ברואר) שדוחה את חוק השלישי הנמנע ודורשת הוכחות קונסטרוקטיביות.
- **אמפליטודת פיזור** (scattering amplitude): הגודל שקובע את ההסתברות לתוצאות של התנגשות חלקיקים; תחום שבו AI תרם נוסחאות ב-2024–2026.
- **אקסיומה** (axiom): הנחת יסוד שמתקבלת בלי הוכחה ומשמשת נקודת מוצא לגזירת משפטים.
- **אקסיומות סטנדרטיות של Lean** (standard axioms): `propext` , `Classical.choice` ו- `Quot.sound` ; ה-verifier דורש שהוכחה לא תשתמש בשום אקסיומה אחרת.
- **אקסיומת הבחירה** (axiom of choice): מכל אוסף של קבוצות לא ריקות אפשר לבחור איבר מכל אחת; ב-Lean מופיעה כ- `Classical.choice` .

- **אריזת כדורים** (sphere packing): בעיית הסידור הצפוף ביותר של כדורים זהים; נפתרה במימדים 3, 8 ו-24.

ב

- **בדיקת טיפוסים** (type checking): וידוא שכל ביטוי מקיים את הטיפוס שלו; לפי קרי–הוארד זו בדיקת הוכחה.
- **בודק הוכחות** (proof checker): תוכנה שמוודאת באופן מכני שכל צעד בהוכחה פורמלית תקף.
- **בונה עסוק** (Busy Beaver): $BB(n)$: מספר הצעדים המרבי של מכונת טיורינג עוצרת עם n מצבים; $BB(5) = 47,176,870$ נקבע ב-2024.
- **בנייה** (construction): אובייקט מפורש (צביעה, גרף, קונפיגורציה) שמוכיח חסם או קיום; תוצר עיקרי של construction-agent.
- **בנצ'מרק** (benchmark): מבחן סטנדרטי להשוואת מערכות, כמו miniF2F, PutnamBench ו-FrontierMath.
- **בנק המטרות** (goal bank): רשימת הבעיות של המעבדה לפי שכבות A/B/C/D/P, בקובץ `data/goals.csv`.
- **בעיות ארדש** (Erdős problems): מאות בעיות שהציע פול ארדש, מרוכזות באתר erdosproblems.com; זירת מבחן מרכזית ל-AI ב-2025–2026.
- **בעיות המילניום** (Millennium Prize Problems): שבע בעיות שמכון קליי הכריז עליהן ב-2000, עם פרס של מיליון דולר לכל אחת; רק פואנקרה נפתרה.
- **בעיית ההכרעה** (Entscheidungsproblem): שאלת הילברט אם יש שיטה מכנית שמכריעה כל טענה לוגית; נפתרה בשלילה בידי צ'רץ' וטיורינג (1936).
- **בעיית העצירה** (halting problem): השאלה אם תוכנית תעצור על קלט נתון; טיורינג הוכיח שאינה כריעה.

ג

- **גאומטריה לא-אוקלידית** (non-Euclidean geometry): גאומטריה שבה אקסיומת המקבילים אינה מתקיימת (לובצ'בסקי, בויאי).
- **גרעין** (kernel): החלק הקטן והאמין של עוזר הוכחות שבודק בפועל כל הוכחה.
- **גרעין אמון** (trust kernel): במעבדה: הגרעין של Lean ובודקי התעודות; הרכיב היחיד שרשאי להעלות טענה ל-L3.
- **גשרי חוכמה** (wisdom bridges): תחום במעבדה שמשתמש בגמרא, בקבלה ובחוכמות עתיקות כמשל ושיטת חשיבה; תוצריו תמיד ברמה L0.

ד

- **דוגמה נגדית** (counterexample): מקרה יחיד שמפריך טענה כללית.
- **דילוג (sorry)** (`sorry`): מילת מפתח ב-Lean שמדלגת על הוכחה; הוכחה עם `sorry` אינה נחשבת מאומתת.
- **דפוס הוכחה** (proof pattern): אסטרטגיה חוזרת בהוכחות יצירתיות, כמו ניסוח מחדש או חיפוש תוכניות; ראו `docs/creative-proofs.md`.

ה

- **הדפסת אקסיומות** (#print axioms): פקודת Lean שמציגה על אילו אקסיומות הוכחה נשענת; חלק מבדיקת ה-verifier.
- **הוכחה** (proof): שרשרת של צעדים תקפים מאקסיומות ומשפטים קודמים אל טענה.
- **הוכחה בדרך השלילה** (proof by contradiction): מניחים שהטענה שגויה ומגיעים לסתירה.
- **הוכחה חברתית** (social proof): הוכחה בשפה טבעית שמתקבלת דרך קריאה ושיפוט של הקהילה; מקבילה לרמות L2 ו-L4.
- **הוכחה ממוחשבת** (computer-assisted proof): הוכחה שחלק מהותי ממנה הוא חישוב, כמו משפט ארבעת הצבעים (1976).
- **הוכחה פורמלית** (formal proof): הוכחה בשפה מוגדרת במדויק, שכל צעד בה ניתן לבדיקה מכנית בידי תוכנה.
- **הוכחה קונסטרוקטיבית** (constructive proof): הוכחת קיום שבונה את האובייקט בפועל; מקבילה לאלגוריתם.
- **היוריסטיקה** (heuristic): כלל אצבע שמכוון חיפוש בלי להבטיח הצלחה.
- **השמטת סימטריות** (symmetry breaking): הוספת אילוצים לנוסחת SAT כדי לחסוך פתרונות שקולים; חייבת הצדקה בהוכחה.
- **השערה** (conjecture): טענה שמאמינים שהיא נכונה אך טרם הוכחה.
- **השערת abc** (abc conjecture): השערה על היחס בין c לרדיקל של abc כש-$a + b = c$; ההוכחה של מוצ'יזוקי (2012) שנויה במחלוקת.
- **השערת גולדבך** (Goldbach conjecture): כל מספר זוגי גדול מ-2 הוא סכום של שני ראשוניים; פתוחה (הגרסה החלשה הוכחה ב-2013).
- **השערת הרצף** (continuum hypothesis): אין עוצמה בין עוצמת הטבעיים לעוצמת הממשיים; בלתי תלויה ב-ZFC (גדל 1940, כהן 1963).
- **השערת קולץ** (Collatz conjecture): תהליך $n \mapsto n/2$ או $3n + 1$ מגיע תמיד ל-1; פתוחה.
- **השערת רימן** (Riemann hypothesis): כל האפסים הלא-טריוויאליים של פונקציית זטא נמצאים על הישר $\mathrm{Re}(s) = 1/2$; בעיית מילניום.

- **התאמת AdS/CFT** (AdS/CFT correspondence): שקילות משוערת בין כבידה במרחב אנטי-דה-סיטר לתורת שדות קונפורמית על השפה שלו (מלדסנה, 1997).
- **התפוצצות** (blow-up): היווצרות סינגולריות (ערך אינסופי) בפתרון של משוואה דיפרנציאלית בזמן סופי.

ז

- **זיהום** (contamination): דליפה של בעיות מבחן לנתוני האימון של מודל, שמנפחת את ציוניו.
- **זרימת ריצ'י** (Ricci flow): משוואה שמחליקה גאומטריה של מרחב לאורך זמן; הכלי של פרלמן בהוכחת השערת פואנקרה.

ח

- **חוק השלישי הנמנע** (law of excluded middle): $P \vee \neg P$ לכל טענה P; מתקבל בלוגיקה קלאסית ונדחה באינטואיציוניזם.
- **חיפוש ספרות** (literature search): חיפוש שיטתי האם בעיה כבר נפתרה; שלב חובה לפני כל עבודה, באחריות literature-agent.
- **חסם** (bound): ערך שגודל לא ידוע לא יכול לעבור (עליון) או לרדת מתחתיו (תחתון).
- **חשבון אינטרוולים** (interval arithmetic): חישוב עם קטעים במקום מספרים מקורבים, כך שהתוצאה מובטחת; הופך נומריקה לתעודה.

ט

- **טיפוס** (type): סיווג של ביטוי בתורת הטיפוסים; ב-Lean גם טענות הן טיפוסים.
- **טקטיקה** (tactic): הוראה ב-Lean שבונה חלק מהוכחה, כמו `simp`, `induction` או `exact`.

י

- **יאנג–מילס ופער המסה** (Yang–Mills mass gap): בעיית מילניום: להוכיח קיום של תורת יאנג–מילס קוונטית ושלחלקיק הקל בה יש מסה חיובית.
- **יומן** (ledger): הקובץ `data/ledger.jsonl` שבו נרשם כל ניסיון של המעבדה, ומאפשר רק הוספה (append-only).

כ

- **כפל מטריצות** (matrix multiplication): בעיה אלגוריתמית שבה AlphaEvolve מצא ב-2025 דרך לכפול מטריצות מרוכבות 4×4 ב-48 כפלים.

ל

- **לוגיקה מסדר ראשון** (first-order logic): לוגיקה שבה כמתים חלים על איברים בלבד, ולא על קבוצות או תכונות; הבסיס של ZFC ושל רוב ה-ATP.
- **לולאה אבולוציונית** (evolutionary loop): חיפוש שבו מודל שפה משכתב תוכניות, מעריך נותן ציון והטובות שורדות; FunSearch ו-AlphaEvolve.
- **לייק** (Lake): כלי הבנייה והניהול של פרויקטי Lean 4; `lake build` נקי הוא תנאי ל-L3.
- **לין** (Lean): עוזר הוכחות ושפת תכנות (דה מורה, 2013; Lean 4 מ-2021), הבסיס הטכני של המעבדה.
- **למה** (lemma): משפט עזר שמשמש בדרך להוכחת משפט גדול יותר.
- **למידת חיזוק** (reinforcement learning, RL): אימון מודל באמצעות תגמול; ב-AlphaProof התגמול הוא הוכחה שעוברת ב-Lean.

מ

- **מודל איזינג** (Ising model): מודל של ספינים על סריג; המעריכים הקריטיים שלו בתלת-ממד הם יעד של ה-bootstrap.
- **מודל שפה גדול** (large language model, LLM): רשת נוירונים שאומנה על טקסט רב ומייצרת טקסט וקוד; במעבדה, מחולל לא-אמין.
- **מוודא** (verifier): רכיב שבודק תוצר באופן אמין; במעבדה הוא היחיד שקובע רמה L3.
- **מוכיח אוטומטי** (automated theorem prover, ATP): תוכנה שמחפשת הוכחה בעצמה, בדרך כלל בלוגיקה מסדר ראשון; למשל Vampire ו-E.
- **מוכיח בשפה טבעית** (informal prover): מודל שכותב הוכחות בשפה טבעית (Gemini Deep Think); דורש בדיקה אנושית או פורמליזציה.
- **מוכיח משפטים נוירוני** (neural theorem prover): מערכת שמשלבת רשת נוירונים עם חיפוש כדי למצוא הוכחות פורמליות; למשל DeepSeek-Prover.
- **מחולל** (generator): רכיב שמציע רעיונות, בניות או הוכחות בלי ערובה לנכונות; כך המעבדה רואה מודל שפה.
- **מטרה** (goal): ב-Lean: הטענה שנותר להוכיח בנקודה מסוימת; במעבדה: שורה בבנק המטרות.
- **מכונת טיורינג** (Turing machine): מודל חישוב מופשט עם סרט אינסופי וראש קורא-כותב; הגדרה של "חישוב מכני".

- **מספר נשיקה** (kissing number): מספר הכדורים הזהים המרבי שיכולים לגעת בכדור מרכזי; AlphaEvolve שיפר את החסם במימד 11 ל-593.
- **מספרי ון דר ורדן** (van der Waerden numbers): $W(r,k)$: הגודל המינימלי שמבטיח סדרה חשבונית חד-צבעית באורך k בכל צביעה ב-r צבעים.
- **מספרי רמזי** (Ramsey numbers): $R(s,t)$: הגודל המינימלי שבו כל צביעה בשני צבעים של גרף שלם מכילה קליקה חד-צבעית בגודל s או t.
- **מספרי שור** (Schur numbers): $S(k)$: הגדול ביותר n שאפשר לצבוע את $1..n$ ב-k צבעים בלי $x + y = z$ חד-צבעי; $S(4) = 44$, $S(5) = 160$.
- **מעריך** (evaluator / scorer): פונקציה שנותנת ציון לבנייה או לתוכנית; חייב להיות מדויק וחסין לניצול באגים.
- **מעריך קריטי** (critical exponent): מספר שמתאר איך גדלים פיזיקליים מתנהגים ליד מעבר פאזה.
- **משוואות נאוויה–סטוקס** (Navier–Stokes equations): משוואות זרימת נוזל צמיג; בעיית מילניום שואלת אם פתרונות חלקים בתלת-ממד נשארים חלקים.
- **משפט** (theorem): טענה שהוכחה.
- **משפט ארבעת הצבעים** (four color theorem): כל מפה מישורית ניתנת לצביעה בארבעה צבעים; אפל והייקן 1976, Coq 2005.
- **משפט פרמה האחרון** (Fermat's Last Theorem, FLT): אין פתרון בשלמים חיוביים ל-$x^n + y^n = z^n$ עבור $n > 2$; ויילס 1994; פורמליזציה מלאה ב-Lean דווחה ב-2026.
- **מתזמר** (orchestrator): הרכיב במעבדה שבוחר את המטרה הבאה ומנהל את יומן הניסיונות; ראו `orchestrator/`.
- **מתליב** (mathlib): הספרייה הקהילתית של מתמטיקה פורמלית ב-Lean, עם מאות אלפי משפטים.

נ

- **נחיל סוכנים** (agent swarm): קבוצת סוכני AI עם תפקידים מוגדרים שעובדים יחד על מטרה; ראו `agents/`.
- **ניסוח** (statement): הטענה הפורמלית המדויקת שמוכיחים; ניסוח שגוי הוא הכשל השקט המסוכן ביותר.
- **נתונים סינתטיים** (synthetic data): נתוני אימון שנוצרו בידי מכונה, כמו טענות שעברו אוטו-פורמליזציה.

ס

- **סוכן** (agent): מודל שפה שפועל בלולאה עם כלים (קוד, חיפוש, Lean) כדי להשיג מטרה.
- **ספיקות** (satisfiability): האם יש השמה לנוסחה בוליאנית שהופכת אותה לאמת; בעיית SAT היא NP-שלמה.

ע

- **עדות נומרית** (numerical evidence): בדיקה חישובית של מקרים רבים; ראיה (L1) אך לא הוכחה.
- **עוזר הוכחות** (proof assistant): סביבה לכתיבת הוכחות פורמליות בעזרת אוטומציה, עם בודק אמין; Lean, Coq/Rocq, Isabelle.
- **עקביות** (consistency): מערכת שבה אי אפשר להוכיח סתירה.
- **עקרון דה-ברויין** (de Bruijn criterion): הבודק צריך להיות קטן ופשוט מספיק כדי שאדם יוכל לבדוק אותו.

פ

- **פורמליזציה** (formalization): תרגום של מתמטיקה לשפה פורמלית עד שעוזר הוכחות מקבל אותה.
- **פותר** (SAT solver) **SAT**: תוכנה שמכריעה ספיקות של נוסחאות בוליאניות; למשל CaDiCaL ו-Kissat.
- **פיגום** (blueprint): מסמך שמפרק הוכחה גדולה ללמות עם גרף תלויות, כבסיס לפורמליזציה מבוזרת (PFR, FLT).
- **פיזיקה תיאורטית** (theoretical physics): ענף הפיזיקה שמנסח ומנתח מודלים מתמטיים של הטבע; שכבה P בבנק המטרות.
- **פיזלין** (PhysLean): פרויקט לפורמליזציה של פיזיקה ב-Lean; יעד תרומה של physics-agent.
- **פער מסה** (mass gap): ההפרש בין האנרגיה של מצב היסוד למצב המעורר הנמוך ביותר בתורה קוונטית.
- **פרדוקס ראסל** (Russell’s paradox): קבוצת כל הקבוצות שאינן איברים של עצמן מובילה לסתירה (1901–1902).

צ

- **צביעה** (coloring): השמה של צבע לכל איבר; הבסיס לבעיות רמזי, שור, ון דר ורדן וארבעת הצבעים.
- **צרמלו–פרנקל (ZFC)** (Zermelo–Fraenkel with Choice): מערכת האקסיומות הסטנדרטית של תורת הקבוצות; היסוד המקובל של המתמטיקה.

ק

- **קבוצת כובע** (cap set): קבוצה ב-$\mathbb{F}_3^n$ בלי שלוש נקודות על ישר; FunSearch מצא ב-2023 קבוצה בגודל 512 במימד 8.
- **קומבינטוריקה** (combinatorics): ענף המתמטיקה שעוסק בספירה, סידור ומבנים בדידים.
- **קונפורמל בוטסטרפ** (conformal bootstrap): שיטה לחסום תורות שדה קונפורמיות מתוך עקביות בלבד; פתרונות נומריים הם L1 עד שהופכים לתעודה.
- **קפלר, השערת** (Kepler conjecture): סידור התפוזים בשוק הוא הצפוף ביותר בתלת-ממד; הייל 1998, Flyspeck 2014.
- **קרי–הוארד, התאמת** (Curry–Howard correspondence): טענות הן טיפוסים והוכחות הן תוכניות; הבסיס של Lean.

ר

- **ראשוניים תאומים** (twin primes): זוג ראשוניים שההפרש ביניהם 2; ההשערה שיש אינסוף כאלה פתוחה.
- **רגרסיה סימבולית** (symbolic regression): חיפוש נוסחה מתמטית שמתאימה לנתונים, למשל PySR.
- **רמות טענה** (claim levels): הסולם L0–L4 של המעבדה: רעיון, עדות נומרית, הוכחה לא-פורמלית, מאומת מכונה, פורסם.

ש

- **שכבה** (tier): קטגוריה בבנק המטרות: A מונשוט, B מפורסמות, C ברות-השגה, D פורמליזציה, P פיזיקה.
- **שלשות פיתגוריות בוליאניות** (Boolean Pythagorean triples): אי אפשר לצבוע את 1..7825 בשני צבעים בלי שלשה פיתגורית חד-צבעית (SAT, 2016).

ת

- **תורת הטיפוסים** (type theory): יסוד למתמטיקה שבו לכל אובייקט יש טיפוס; חלופה ל-ZFC שעליה בנוי Lean.
- **תורת הקבוצות** (set theory): יסוד למתמטיקה שבו הכול הוא קבוצה; הגרסה הסטנדרטית היא ZFC.
- **תורת שדות קוונטית** (quantum field theory, QFT): המסגרת של פיזיקת החלקיקים; לרוב חסרה לה הגדרה מתמטית מלאה.
- **תזת צ'רץ'–טיורינג** (Church–Turing thesis): כל חישוב מכני ניתן לביצוע במכונת טיורינג.
- **תחשיב הקונסטרוקציות** (Calculus of Constructions): תורת טיפוסים של קוקאן והואה (1988); הבסיס של Coq ובגרסה אינדוקטיבית גם של Lean.
- **תחשיב למדא** (lambda calculus): מודל חישוב של צ'רץ' המבוסס על פונקציות; שקול למכונת טיורינג.
- **תעודה** (certificate): ראיה שניתנת לבדיקה עצמאית לתוצאה חישובית, כמו הוכחת DRAT או צביעה מפורשת.
- **תעודת DRAT / LRAT** (DRAT / LRAT): פורמטים של תעודות UNSAT שפותרי SAT מפיקים ובודקים עצמאיים מאמתים.

בקצרה

- המילון מכסה כ-120 מונחים מחמישה תחומים: לוגיקה ויסודות, Lean ועוזרי הוכחות, AI וחיפוש, קומבינטוריקה ותורת המספרים, ופיזיקה תיאורטית.
- כל מונח מופיע בעברית ובאנגלית, כפי שנדרש במדריך הסגנון בפעם הראשונה שמונח מופיע בפרק.
- מונחי המעבדה עצמה (רמות טענה, גרעין אמון, בנק המטרות, יומן, מתזמר) מוגדרים כאן באופן שתואם את `docs/architecture.md` ו-`docs/goal-bank.md`.
- מונח חדש שנכנס לפרק צריך להיכנס גם למילון, במקום הנכון לפי האלף-בית.

רעיונות לתוכן

1. **"מילה ביום: המילון של ESH27"** זווית: סדרת פוסטים קצרים, מונח אחד בכל פעם (למשל "גרעין אמון" או "תעודה"), עם דוגמה מהמעבדה. קהל: עוקבי S27 ברשתות החברתיות.
2. **"10 מילים שכל מי שקורא על AI ומתמטיקה צריך להכיר"** זווית: כתבת מבוא שמסבירה את המונחים שחוזרים בכל כותרת (Lean, אוטו-פורמליזציה, verifier, sorry, בנצ'מרק). קהל: קוראי S27 News.
3. **"מילון עברי למתמטיקה פורמלית"** זווית: עמוד באתר עם חיפוש, שמציע תרגומים עקביים למונחים שאין להם עדיין שם עברי מקובל. קהל: סטודנטים, מורים וכותבי מדע בעברית.

נספח ב: ביבליוגרפיה

הנספח הזה מאחד את המקורות שמופיעים במסמכים שבתיקייה `docs/` (`ai-math-lessons.md`, `architecture.md`, `creative-proofs.md`, `goal-bank.md`, `theoretical-physics.md`, `tools.md`, `wisdom-bridges.md`) ובקובצי הנתונים שהם מפנים אליהם (`data/creative_proofs.csv`, `data/goals.csv`, `data/tools.csv`, `data/wisdom_projects.csv`). כל מקור מופיע פעם אחת, תחת הנושא המתאים ביותר, גם אם כמה מסמכים מפנים אליו.

מקורות שלא נקראו ישירות, או שפרטיהם לא אומתו, מסומנים [לא אומת], כמו במסמכי המקור. מקור בלי URL הוא ספר או מאמר שהמסמכים מצטטים בלי קישור; כשמופיע מזהה arXiv, הקישור נבנה ממנו. הספר מפנה לנספח מתוך הערות השוליים בפרקים.

1. פורמליזציה ועוזרי הוכחות

פרויקטים גדולים

- **Liquid Tensor Experiment** (Scholze; Commelin ושותפים, 2020–2022).
  https://leanprover-community.github.io/blog/posts/lte-final/ ;
  https://github.com/leanprover-community/lean-liquid
- **PFR** (Gowers, Green, Manners, Tao, 2023), ההוכחה:
  https://arxiv.org/abs/2311.05762 ; הפורמליזציה: https://teorth.github.io/pfr/ ;
  Tao, “A slightly longer Lean 4 proof tour”:
  https://terrytao.wordpress.com/2023/12/05/a-slightly-longer-lean-4-proof-tour/
- **Equational Theories Project** (Tao ושותפים, 2024–2025).
  https://teorth.github.io/equational_theories/ ;
  https://github.com/teorth/equational_theories
- **FLT** (Imperial College, Buzzard). פרויקט
  https://imperialcollegelondon.github.io/FLT/
- Anthropic, “Formalizing Fermat’s Last Theorem” (2026-09-04).
  https://www.anthropic.com/research/formalizing-fermats-last-theorem

- K. Buzzard, “FLT: Anthropic has beaten me to it” (2026-09-04). https://xenaproject.wordpress.com/2026/09/04/flt-anthropic-has-beaten-me-to-it/ [לא אומת: לא נקרא ישירות]
- **Math Inc. Gauss, Strong PNT** (2025). https://math-inc.github.io/strongpnt ; דיון ב-Zulip: https://leanprover-community.github.io/archive/stream/219941-Machine-Learning-for-Theorem-Proving/topic/Math.2C.20Inc.2C.20and.20Gauss.20discussion.html
- **mathlib stats**. https://leanprover-community.github.io/mathlib_stats.html

פורמליזציה של הוכחות ממוחשבות

- G. Gonthier, “Formal Proof—The Four-Color Theorem”, Notices AMS 55 (2008).
- T. Hales et al., “A formal proof of the Kepler conjecture” (Flyspeck), Forum of Math, Pi 5 (2017).
- B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro, M. Heule, אימות ב-Lean של משפט המשושה הריק, ITP 2024.
- **BB(5)**, bbchallenge (הודעה ביולי 2024; הוכחה ב-Coq). https://bbchallenge.org

2. מוכיחי משפטים נוירוניים ומערכות AI למתמטיקה

- GPT-f (Polu, Sutskever, 2020). https://arxiv.org/abs/2009.03393
- miniF2F (2021). https://arxiv.org/abs/2109.00110
- HTPS (Lample et al., 2022). https://arxiv.org/abs/2205.11491
- LeanDojo / ReProver (Yang et al., 2023). https://arxiv.org/abs/2306.15626
- PutnamBench (Tsoukalas et al., 2024). https://arxiv.org/abs/2407.11214
- DeepSeek-Prover V1 / V1.5 / V2. https://arxiv.org/abs/2405.14333 ; https://arxiv.org/abs/2408.08152 ; https://arxiv.org/abs/2504.21801
- Goedel-Prover (2025). https://arxiv.org/abs/2502.07640
- Kimina-Prover (2025). https://arxiv.org/abs/2504.11354
- Seed-Prover (ByteDance, 2025). https://arxiv.org/abs/2507.23726 ; https://seed.bytedance.com/en/blog/bytedance-seed-prover-achieves-silver-medal-score-in-imo-2025

- AlphaGeometry (Trinh et al.), Nature 625 (2024). https://www.nature.com/articles/s41586-023-06747-5
- AlphaProof, IMO 2024. https://deepmind.google/discover/blog/ai-solves-/imo-problems-at-silver-medal-level
- AlphaProof (Hubert et al.), Nature (2025-11-12). https://www.nature.com/articles/s41586-025-09833-y
- Harmonic Aristotle (2025). https://arxiv.org/abs/2510.01346
- Gemini Deep Think, IMO 2025. https://deepmind.google/discover/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-/mathematical-olympiad
- First Proof (2026). https://1stproof.org/ ; https://arxiv.org/abs/2602.05192 ; https://ethz.ch/staffnet/en/news-and-events/internal-news/archive/2026/07/eth-team-wins-first-proof-challenge-for-ai-assisted-mathematical-research.html [לא אומת: תוצאות סבב 3 מול הדוח הרשמי]
- W. McCune, פתרון השערת Robbins במוכיח EQP, J. Automated Reasoning 19 (1997).

3. חיפוש, גילוי ובנייה בעזרת מחשב

- A. Z. Wagner, “Constructions in combinatorics via neural networks” (2021). https://arxiv.org/abs/2104.14516
- G. Raayoni et al., Ramanujan Machine, Nature 590 (2021). https://www.nature.com/articles/s41586-021-03229-4
- A. Davies et al., ML מנחה קשרים ותורת הצגות, Nature 600 (2021). https://www.nature.com/articles/s41586-021-04086-x
- A. Fawzi et al., AlphaTensor, Nature 610 (2022).
- B. Romera-Paredes et al., FunSearch, Nature 625 (2024). https://www.nature.com/articles/s41586-023-06924-6
- F. Charton, J. Ellenberg, A. Z. Wagner, G. Williamson, PatternBoost (2024). https://arxiv.org/abs/2411.00566
- DeepMind, AlphaEvolve (מאי 2025). https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/ ; https://arxiv.org/abs/2506.13131

- B. Georgiev, J. Gómez-Serrano, T. Tao, A. Z. Wagner (2025), AlphaEvolve על 67 בעיות. https://arxiv.org/abs/2511.02864

SAT והוכחות ממוחשבות

- K. Appel, W. Haken, משפט ארבעת הצבעים, Illinois J. Math. 21 (1977).
- T. Hales, “A proof of the Kepler conjecture”, Annals 162 (2005).
- B. Konev, A. Lisitsa, אי-התאמה של ארדש ב-SAT, SAT 2014.
- M. Heule, O. Kullmann, V. Marek, שלשות פיתגוריות בוליאניות, SAT 2016.
- M. Heule, “Schur Number Five”, AAAI 2018.
- A. de Grey, צביעת המישור, Geombinatorics 28 (2018).
- J. Brakensiek, M. Heule, J. Mackey, D. Narváez, השערת Keller במימד 7, IJCAR 2020.
- G. Gardam, דוגמה נגדית להשערת היחידות של Kaplansky, Annals 194 (2021).
- B. Bašić, מספר Heesch 6, Discrete Comput. Geom. 65 (2021).
- M. Heule, M. Scheucher, משפט המשושה הריק, TACAS 2024.
- J. Steininger, S. Yurkevich, Noperthedron (2025). https://arxiv.org/abs/2508.18475

4. בעיות ארדש

- ויקי תרומות AI לבעיות ארדש (Tao ושותפים; עודכן עד 30 ביוני 2026). https://github.com/teorth/erdosproblems/wiki/AI-contributions-to-Erd%C5%91s-problems
- erdosproblems.com (Thomas Bloom). https://www.erdosproblems.com
- בעיה #728, תיאור הפתרון (ינואר 2026). https://arxiv.org/abs/2601.07421
- סיקור פרשת אוקטובר 2025. https://www.eweek.com/news/openai-math-hype/
- T. Tao, פתרון בעיית אי-ההתאמה של ארדש, Discrete Analysis 2016:1.

5. בנצ’מרקים

- FrontierMath (Epoch AI). https://epoch.ai/frontiermath
- מחלוקת המימון של FrontierMath (ינואר 2025). https://techcrunch.com/2025/01/19/ai-benchmarking-organization-criticized-for-waiting-to-disclose-funding-from-openai

- Humanity’s Last Exam. https://lastexam.ai/ [לא אומת: שיעור התשובות השגויות לפי ביקורות מאוחרות]

6. הוכחות יצירתיות קלאסיות ומודרניות

הרשימה מאחדת את המקורות של `data/creative_proofs.csv`, לפי סדר כרונולוגי. המזהה בסוגריים מפנה לשורה בקובץ.

- Euclid, *Elements*, ספר IX, טענה 20 (P001).
- Archimedes, *The Method*; J. L. Heiberg (1906); R. Netz, W. Noel, *The Archimedes Codex* (2007) (P002).
- Johann Bernoulli, Acta Eruditorum (1697) (P003).
- L. Euler, “Solutio problematis ad geometriam situs pertinentis” (1736) (P004).
- C. F. Gauss, *Disquisitiones Arithmeticae* (1801), פרק VII (P005).
- P. G. L. Dirichlet, Abh. Königl. Preuss. Akad. Wiss. (1837) (P006).
- É. Galois, כתבי יד, פורסמו על ידי Liouville (1846) (P007).
- W. W. Johnson, W. E. Story, Amer. J. Math. 2 (1879) (P009).
- G. Cantor, Jahresber. DMV 1 (1891) (P008).
- M. Dehn, Math. Ann. 55 (1901) (P010).
- E. Sperner, Abh. Math. Sem. Hamburg 6 (1928) (P011).
- K. Gödel, Monatsh. Math. Phys. 38 (1931) (P012).
- M. Black, *Critical Thinking* (1946); J. McCarthy (1964) (P013).
- P. Erdős, Bull. AMS 53 (1947) (P014).
- H. Furstenberg, Amer. Math. Monthly 62 (1955) (P015); J. Analyse Math. 31 (1977) (P018).
- D. Schattschneider, על המחומשים של Marjorie Rice, Math. Magazine 51 (1978) (P019).
- P. Candelas, X. de la Ossa, P. Green, L. Parkes, Nucl. Phys. B 359 (1991) (P020).
- R. Borcherds, Invent. Math. 109 (1992) (P021).
- A. Wiles, Annals 141 (1995); R. Taylor, A. Wiles, Annals 141 (1995) (P022).
- G. Perelman (2002–2003). https://arxiv.org/abs/math/0211159 ; https://arxiv.org/abs/math/0303109 ; https://arxiv.org/abs/math/0307245

(P026)

- Z. Dvir, J. AMS 22 (2009) (P027).
- D.H.J. Polymath, Annals 175 (2012) (P031); Res. Math. Sci. 1 (2014) (P030).
- Y. Zhang, Annals 179 (2014) (P028).
- J. Maynard, Annals 181 (2015) (P029).
- J. Ellenberg, D. Gijswijt, Annals 185 (2017); E. Croot, V. Lev, P. P. Pach, Annals 185 (2017) (P035).
- M. Viazovska, Annals 185 (2017); H. Cohn et al., Annals 185 (2017) (P036).
- H. Huang, Annals 190 (2019) (P038).
- D. Smith, J. S. Myers, C. S. Kaplan, C. Goodman-Strauss, “An aperiodic monotile”, Combinatorial Theory 4 (2024). https://arxiv.org/abs/2303.10798 (P048)
- H. Wang, J. Zahl, Kakeya ב-$\mathbb{R}^3$ (2025). https://arxiv.org/abs/2502.17655 (P055)
- R. B. Nelsen, *Proofs Without Words*, MAA (1993) (P060).

7. פיזיקה תיאורטית

רקע ובעיות פתוחות

- Clay Mathematics Institute, Millennium Problems. https://www.claymath.org/millennium-problems/
- A. Jaffe, F. Quinn, “‘Theoretical mathematics’: Toward a cultural synthesis of mathematics and theoretical physics”, Bull. AMS 29 (1993). (מצוטט בפרק 10 בלבד, לא במסמכי `docs/`.)
- Navier–Stokes existence and smoothness. https://en.wikipedia.org/wiki/Navier%E2%80%93Stokes_existence_and_smoothness
- J. Chen, T. Hou, התפוצצות של אוילר בעזרת מחשב (2022–2023). https://arxiv.org/abs/2210.07191
- Wang et al. (2025), גילוי סינגולריות לא-יציבות בעזרת PINNs [לא אומת: preprint, מזהה לא צוין במסמכים].

Conformal bootstrap

- S. El-Showk et al. (2014). https://arxiv.org/abs/1403.4545
- F. Kos, D. Poland, D. Simmons-Duffin, A. Vichi, “Precision islands in the Ising and O(N) models” (2016). https://arxiv.org/abs/1603.04436
- C.-H. Chang et al., “Bootstrapping the 3d Ising stress tensor” (2024) [לא אומת: ערכים מדויקים].
- D. Simmons-Duffin, SDPB. https://arxiv.org/abs/1502.02033 ; https://github.com/davidsd/sdpb

אמפליטודות, הולוגרפיה ואינטגרביליות

- N. Arkani-Hamed, J. Trnka, “The Amplituhedron” (2013). https://arxiv.org/abs/1312.2007
- Guevara, Lupsasca, Skinner, Strominger, Weil, “Single-minus gluon tree amplitudes are nonzero” (2026). סיקור: https://phys.org/news/2026-02-chatbot-author-ai-stalled-gluon.html ; ניתוח ביקורתי: https://huggingface.co/blog/dlouapre/gpt-single-minus-gluons
- F. Pastawski, B. Yoshida, D. Harlow, J. Preskill, HaPPY (2015). קוד: https://arxiv.org/abs/1503.06237
- M. de Leeuw, A. Pribytok, P. Ryan, “Classifying integrable spin-1/2 chains with nearest neighbour interactions” (2019).

למידת מכונה בפיזיקה

- S.-M. Udrescu, M. Tegmark, “AI Feynman”, Science Advances (2020). https://www.science.org/doi/10.1126/sciadv.aay2631
- M. Cranmer, PySR (2023). https://github.com/MilesCranmer/PySR
- Normalizing flows ל-lattice field theory (Albergo, Kanwar, Shanahan, 2019). https://arxiv.org/abs/1904.12072
- Y.-H. He, ML ב-string landscape (2017). https://arxiv.org/abs/1706.02714
- J. Carifio, J. Halverson, D. Krioukov, B. Nelson (2017). https://arxiv.org/abs/1707.00655

פיזיקה ב-Lean

- J. Tooby-Smith, “HepLean: Digitalising high energy physics”, Comput. Phys. Commun. 308 (2025) 109457. https://arxiv.org/abs/2405.08863 ; https://physlean.com/ ; https://github.com/HEPLean/PhysLean

מטרות P בבנק (`data/goals.csv`)

- G069, Smale 6: Albouy–Kaloshin, Annals 176 (2012). https://annals.math.princeton.edu/2012/176-1/p10 ; https://en.wikipedia.org/wiki/Smale%27s_problems
- G070, השערת Haldane. https://en.wikipedia.org/wiki/Haldane_conjecture ; Gosset–Mozgunov (2016).
- G071, חוק השטח: M. Hastings (2007). https://arxiv.org/abs/0705.2024
- G072, אי-שוויוני Bell: K. F. Pál, T. Vértesi, Phys. Rev. A 82, 022116 (2010).
- G073, Lieb–Thirring: Frank–Hundertmark–Jex–Nam (2021); Frank–Gontier–Lewin (2021).
- G074, הבעיה השישית של הילברט: Deng, Hani, Ma (2025). https://arxiv.org/abs/2503.01800 [בבדיקה בקהילה]
- G075, הליכה עצמית-נמנעת. https://arxiv.org/abs/1007.0575
- G076, יציבות Kerr: Klainerman, Szeftel, Giorgi (2022). https://arxiv.org/abs/2205.14808

8. בנק המטרות: מקורות למטרות הראשונות

- G050: https://en.wikipedia.org/wiki/Schur_number ; https://en.wikipedia.org/wiki/Van_der_Waerden_number ; L. D. Baumert (1965); V. Chvátal (1970); https://lean-lang.org/
- G023, G052: erdosproblems.com, ויקי תרומות ה-AI (סעיף 4), ו-Formal Conjectures (DeepMind). https://github.com/google-deepmind/formal-conjectures
- G027, G029: טבלאות Rabung–Lotts ו-Monroe ל-$W(r,k)$; S. Radziszowski, “Small Ramsey Numbers” (סקר דינמי, Electronic Journal of Combinatorics) [URL לא אומת: המסמכים לא נותנים].

9. גשרי חוכמה: מקורות יהודיים, היסטוריה של המתמטיקה ומתודולוגיה

מקורות יהודיים

- רמב”ם, הקדמה לשמונה פרקים (“שמע האמת ממי שאמרה”).

- רמב"ם, משנה תורה, ספר זמנים, הלכות קידוש החודש (פרקים ו–י: חשבון הלוח הקבוע; יא–יט: ראיית הלבנה; יז, כד: עיקרון הראיות). [יז, כד אומת דרך מקורות משניים, למשל https://outorah.org/p/47556 ; החלוקה בין הפרקים לא אומתה מול הטקסט]
- ספר יצירה, נוסחים שונים; A. Kaplan, *Sefer Yetzirah: The Book of Creation in Theory and Practice* (1990). "שתי אבנים בונות שני בתים": משנה 4:12 במספור Kaplan (נוסח הגר"א). [לא אומת מול טקסט עברי]
- תלמוד בבלי, תענית כג ע"א ("או חברותא או מיתותא").
- משנה, אבות ג, יח ("תקופות וגימטריאות פרפראות לחכמה").
- גימטריה/https://www.yeshiva.org.il/wiki/index.php
- ברייתא דרבי ישמעאל (שלוש-עשרה מידות), פתיחת הספרא.
- רבי לוי בן גרשום (רלב"ג), *מעשה חושב* (1321).
- רבי משה חיים לוצאטו (רמח"ל), *קל"ח פתחי חכמה*; *דרך ה'*; *דרך תבונות*.
- רבי שלום שרעבי (הרש"ש), *נהר שלום*; סידור הרש"ש (הכוונות).
- אברהם אבן עזרא, *ספר המספר*.

היסטוריה ופילוסופיה של המתמטיקה

- I. Lakatos, *Proofs and Refutations*, Cambridge University Press (1976).
- N. L. Rabinovitch, *Probability and Statistical Inference in Ancient and Medieval Jewish Literature*, University of Toronto Press (1973).
- D. F. Mansfield, N. J. Wildberger, "Plimpton 322 is Babylonian exact sexagesimal trigonometry", Historia Mathematica 44 (2017); והביקורת של E. Robson ואחרים.
- B. Cutler, ספירת סידורי ה-17,152 Stomachion (2003): 536 עד כדי סידורים, סימטריה.
- E. M. Reingold, N. Dershowitz, *Calendrical Calculations*, Cambridge University Press.
- Pingala, *Chandaḥśāstra* (המאה ה-10); Halayudha; Madhava ובית הספר של קרלה; Qin Jiushao (1247), *Sunzi Suanjing*; אל-ח'וארזמי, *al-jabr wa'l-muqābala* (כ-820).

מתודולוגיה סטטיסטית

- D. Witztum, E. Rips, Y. Rosenberg, Statistical Science 9 (1994).
- B. McKay, D. Bar-Natan, M. Bar-Hillel, G. Kalai, "Solving the Bible Code Puzzle", Statistical Science 14 (1999).

10. כלים

הקטלוג המלא (140 כלים, 19 קטגוריות) נמצא ב- `data/tools.csv` . כאן מופיעים הכלים שהספר מזכיר.

עוזרי הוכחות וספריות Lean

- Lean 4: https://lean-lang.org ; elan: https://github.com/leanprover/elan ; Lake: https://github.com/leanprover/lean4/tree/master/src/lake
- mathlib4: https://github.com/leanprover-community/mathlib4
- LeanSearch: https://leansearch.net ; Loogle: https://loogle.lean-lang.org
- Lean REPL: https://github.com/leanprover-community/repl ; lean-lsp-mcp: https://github.com/oOo0oOo/lean-lsp-mcp
- leanblueprint: https://github.com/PatrickMassot/leanblueprint
- LeanDojo: https://github.com/lean-dojo/LeanDojo
- lean-action (CI): https://github.com/leanprover/lean-action
- Isabelle/HOL: https://isabelle.in.tum.de ; Rocq (לשעבר Coq): https://rocq-prover.org ; HOL Light: https://github.com/jrh13/hol-light

SAT, SMT וחיפוש

- CaDiCaL: https://github.com/arminbiere/cadical ; Kissat: https://github.com/arminbiere/kissat
- DRAT-trim: https://github.com/marijnheule/drat-trim ; cake_lpr: https://github.com/tanyongkiam/cake_lpr
- PySAT: https://pysathq.github.io ; Z3: https://github.com/Z3Prover/z3
- OpenEvolve: https://github.com/codelion/openevolve

אלגברה ממוחשבת ונומריקה

- SageMath: https://www.sagemath.org ; PARI/GP: https://pari.math.u-bordeaux.fr ; SymPy: https://www.sympy.org
- FLINT: https://flintlib.org ; Arb: https://arblib.org ; mpmath: https://mpmath.org ; NetworkX: https://networkx.org
- SDPB, PySR, AI Feynman: ראו סעיף 7. AI Feynman (קוד): https://github.com/SJ001/AI-Feynman

מודלים ותשתית סוכנים

- Claude API: https://docs.anthropic.com ; Claude Agent SDK: https://docs.anthropic.com/en/docs/claude-code/sdk ; Claude Code: https://docs.anthropic.com/en/docs/claude-code/overview
- Model Context Protocol: https://modelcontextprotocol.io

ספרות ונתונים

- arXiv: https://arxiv.org ; arXiv API: https://info.arxiv.org/help/api/index.html
- zbMATH Open: https://zbmath.org ; Semantic Scholar API: https://www.semanticscholar.org/product/api ; Google Scholar: https://scholar.google.com
- OEIS: https://oeis.org ; LMFDB: https://www.lmfdb.org

טקסטים עבריים

- Sefaria API: https://developers.sefaria.org ; Sefaria-Export: https://github.com/Sefaria/Sefaria-Export
- Dicta: https://dicta.org.il ; HebrewBooks.org: https://hebrewbooks.org
- אוצר החכמה: https://www.otzar.org ; פרויקט השו"ת של בר-אילן: https://www.responsa.co.il

11. מקורות פנימיים של המעבדה

- מעקב בנק המטרות ב-Google Sheets: https://docs.google.com/spreadsheets/d/1V9coJlUiul7Z8uRsovujRcw9KB6DWcYMtOQhWudvhNo/edit
- `lab/README.md` , `docs/*.md` , `data/*.csv` , `news/README.md` , `site/README.md` .

בקצרה

- הנספח מאחד את המקורות של כל המסמכים ב- `docs/` ושל קובצי הנתונים, בלי כפילויות, לפי אחד-עשר נושאים.
- מקורות שלא נקראו ישירות או שפרטיהם לא נבדקו מסומנים [לא אומת], כמו במסמכי המקור.
- כשמסמך מציין רק מזהה arXiv, הקישור נבנה ממנו; ספרים ומאמרים בלי קישור מופיעים עם פרטי הפרסום.

- כל מקור חדש בפרק מתווסף גם לכאן, בסעיף המתאים.

רעיונות לתוכן

1. **"ספריית המעבדה": עמוד באתר.** הביבליוגרפיה כרשימה מסוננת לפי נושא, עם קישור לפרקים שמצטטים כל מקור. קהל יעד: סטודנטים וחוקרים שמצטרפים למעבדה.
2. **"עשרה מאמרים שכל מי שעוסק ב-AI למתמטיקה צריך לקרוא": פוסט.** בחירה מסעיפים 1–3 עם משפט הסבר לכל מאמר. קהל יעד: מהנדסי AI, סטודנטים לתארים מתקדמים.
3. **"מספר יצירה ועד FunSearch": שרשור.** קו זמן של מקורות מהנספח, מהעת העתיקה ועד 2026, בלי לטעון לקשר סיבתי ביניהם. קהל יעד: קהל רחב שמתעניין בהיסטוריה של המדע.

---

1. AlphaProof, IMO 2024: https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/↩

2. הודעות Google DeepMind ו-OpenAI, יולי 2025; ראו `docs/ai-math-lessons.md`, סעיף 2.↩

3. DeepMind, AlphaEvolve (מאי 2025); arXiv:2506.13131.↩

4. A. Wiles, Annals 141 (1995); R. Taylor, A. Wiles, Annals 141 (1995).↩

5. K. Gödel, "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I", *Monatshefte für Mathematik und Physik* 38 (1931).↩

6. A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem", *Proc. London Math. Soc.* (1936); A. Church, "An Unsolvable Problem of Elementary Number Theory", *Amer. J. Math.* (1936).↩

7. W. A. Howard, "The formulae-as-types notion of construction" (כתב יד 1969), פורסם ב-*To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism* (1980).↩

8. R. A. De Millo, R. J. Lipton, A. J. Perlis, "Social Processes and Proofs of Theorems and Programs", *Communications of the ACM* 22(5) (1979).↩

9. V. Voevodsky, “The Origins and Motivations of Univalent Foundations”, *The Institute Letter*, IAS (קיץ 2014). התרגום החופשי שלנו; הפרטים על השנים 1989/1998/2013 לקוחים מאותו מקור. ↩

10. ראו `docs/ai-math-lessons.md`, סעיף 1, וכן הבלוג של K. Buzzard, “Xena”. ↩

11. D. Hilbert, “Mathematische Probleme”, הרצאה בקונגרס הבין-לאומי של המתמטיקאים, פריז, 8 באוגוסט 1900; תרגום לאנגלית: *Bulletin of the AMS* 8 (1902). ↩

12. Clay Mathematics Institute, Millennium Problems: https://www.claymath.org/millennium-problems ↩

13. D. Platt, T. Trudgian, “The Riemann hypothesis is true up to $3 \cdot 10^{12}$”, *Bulletin of the LMS* (2021). ↩

14. J. Chen, T. Hou, arXiv:2210.07191. ↩

15. T. Oliveira e Silva, S. Herzog, S. Pardi, “Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$”, *Mathematics of Computation* (2014). ↩

16. D. Barina, “Convergence verification of the Collatz problem”, *The Journal of Supercomputing* (2021; גרסה מקוונת 2020). ייתכן שהחסם עודכן מאז [לא אומת]. T. Tao, “Almost all orbits of the Collatz map attain almost bounded values”, arXiv:1909.03562 (2019). ↩

17. P. Scholze, J. Stix, “Why abc is still a conjecture” (2018); S. Mochizuki, “Inter-universal Teichmüller Theory I–IV”, *Publ. RIMS* 57 (2021). ↩

18. A. Newell, H. A. Simon, “The Logic Theory Machine: A Complex Information Processing System”, *IRE Transactions on Information Theory* (1956). ↩

19. N. G. de Bruijn, “The mathematical language AUTOMATH, its usage, and some of its extensions” (1970); R. P. Nederpelt, J. H. Geuvers, R. C. de Vrijer (עורכים), *Selected Papers on Automath* (1994). ↩

20. K. Appel, W. Haken, “Every planar map is four colorable”, *Illinois Journal of Mathematics* 21 (1977). ↩

21. G. Gonthier, “Formal Proof—The Four-Color Theorem”, Notices AMS 55 (2008). ↩

22. T. Hales, “A proof of the Kepler conjecture”, Annals of Math. 162 (2005). ↩

23. T. Hales et al., “A formal proof of the Kepler conjecture”, Forum of Math, Pi 5 (2017). ↩

24. M. J. H. Heule, O. Kullmann, V. W. Marek, “Solving and Verifying the Boolean Pythagorean Triples problem via Cube-and-Conquer”, SAT 2016. ↩

25. J. Brakensiek, M. Heule, J. Mackey, D. Narváez, “The Resolution of Keller’s Conjecture”, IJCAR 2020. ↩

26. https://bbchallenge.org (הודעה ביולי 2024). ↩

27. Liquid Tensor Experiment: https://leanprover-community.github.io/blog/posts/lte-final ↩

28. W. T. Gowers, B. Green, F. Manners, T. Tao, arXiv:2311.05762; https://teorth.github.io/pfr ↩

29. T. Tymoczko, “The Four-Color Problem and Its Philosophical Significance”, *The Journal of Philosophy* 76(2) (1979). ↩

30. Liquid Tensor Experiment: https://leanprover-community.github.io/blog/posts/lte-final ↩

31. mathlib stats: https://leanprover-community.github.io/mathlib_stats.html ↩

32. W. T. Gowers, B. Green, F. Manners, T. Tao, arXiv:2311.05762; https://teorth.github.io/pfr ↩

33. Polu & Sutskever, GPT-f: https://arxiv.org/abs/2009.03393 ↩

34. miniF2F: https://arxiv.org/abs/2109.00110 ↩

35. HTPS: https://arxiv.org/abs/2205.11491 ↩

36. LeanDojo: https://arxiv.org/abs/2306.15626 ↩

37. DeepSeek-Prover V1/V1.5/V2: https://arxiv.org/abs/2405.14333 ; https://arxiv.org/abs/2408.08152 ; https://arxiv.org/abs/2504.21801 ↩

38. PutnamBench: https://arxiv.org/abs/2407.11214 ↩

39. Goedel-Prover: https://arxiv.org/abs/2502.07640 ↩

40. Kimina-Prover: https://arxiv.org/abs/2504.11354 ↩

41. Seed-Prover: https://arxiv.org/abs/2507.23726 ; https://seed.bytedance.com/en/blog/bytedance-seed-prover-achieves-silver-medal-score-in-imo-2025 ↩

42. AlphaGeometry, Nature 2024: https://www.nature.com/articles/s41586-023-06747-5 ↩

43. AlphaProof, IMO 2024: https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/ ↩

44. AlphaProof, Nature (2025-11-12): https://www.nature.com/articles/s41586-025-09833-y ↩

45. Aristotle (Harmonic): https://arxiv.org/abs/2510.01346 ↩

46. Gemini Deep Think, IMO 2025: https://deepmind.google/discover/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/ ↩

47. Wagner 2021: https://arxiv.org/abs/2104.14516 ↩

48. Ramanujan Machine: https://www.nature.com/articles/s41586-021-03229-4 ↩

49. A. Davies et al., Nature 600 (2021): https://www.nature.com/articles/s41586-021-04086-x ↩

50. B. Romera-Paredes et al., Nature 625 (2024): https://www.nature.com/articles/s41586-023-06924-6 ↩

51. PatternBoost: https://arxiv.org/abs/2411.00566 ↩

52. DeepMind, AlphaEvolve (מאי 2025); arXiv:2506.13131. ↩

53. Georgiev, Gómez-Serrano, Tao, Wagner (2025): https://arxiv.org/abs/2511.02864 ↩

54. https://teorth.github.io/equational_theories/ ↩

55. C. F. Gauss, *Disquisitiones Arithmeticae* (1801), פרק VII. ↩

56. A. Wiles, Annals 141 (1995); R. Taylor, A. Wiles, Annals 141 (1995). ↩

57. Anthropic, “Formalizing Fermat’s Last Theorem” (2026-09-04): https://www.anthropic.com/research/formalizing-fermats-last-theorem ↩

58. K. Buzzard, “FLT: Anthropic has beaten me to it” (2026-09-04): https://xenaproject.wordpress.com/2026/09/04/flt-anthropic-has-beaten-me-to-it/ (לא נקרא ישירות) ↩

59. סיקור פרשת אוקטובר 2025: https://www.eweek.com/news/openai-math-hype/ ↩

60. arXiv:2601.07421; ויקי תרומות ה-AI לבעיות ארדש: https://github.com/teorth/erdosproblems/wiki/AI-contributions-to-Erd%C5%91s-problems ↩

61. AI contributions to Erdős problems: https://github.com/teorth/erdosproblems/wiki/AI-contributions-to-Erd%C5%91s-problems ↩

62. FrontierMath: https://epoch.ai/frontiermath ↩

63. מחלוקת המימון: https://techcrunch.com/2025/01/19/ai-benchmarking-organization-criticized-for-waiting-to-disclose-funding-from-openai ↩

64. Humanity’s Last Exam: https://lastexam.ai/ ↩

65. First Proof: https://1stproof.org/ ; https://arxiv.org/abs/2602.05192 ; https://ethz.ch/staffnet/en/news-and-events/internal-news/archive/2026/07/eth-team-wins-first-proof-challenge-for-ai-assisted-mathematical-research.html ↩

66. S.-M. Udrescu, M. Tegmark, “AI Feynman”, Science Advances (2020): https://www.science.org/doi/10.1126/sciadv.aay2631 ↩

67. סיקור: https://phys.org/news/2026-02-chatbot-author-ai-stalled-gluon.html ; ניתוח ביקורתי: https://huggingface.co/blog/dlouapre/gpt-single-minus-gluons ↩

68. Y.-H. He, arXiv:1706.02714; J. Carifio et al., arXiv:1707.00655. ↩

69. Normalizing flows for lattice field theory: https://arxiv.org/abs/1904.12072 ↩

70. `docs/ai-math-lessons.md` , סעיף 4; סיקור: https://www.eweek.com/news/openai-math-hype/ ↩

71. arXiv:2601.07421; ויקי תרומות ה-AI לבעיות ארדש: https://github.com/teorth/erdosproblems/wiki/AI-contributions-to-Erd%C5%91s-problems ↩

72. L. Euler, “Solutio problematis ad geometriam situs pertinentis” (1736). ↩

73. C. F. Gauss, *Disquisitiones Arithmeticae* (1801), פרק VII. ↩

74. K. Appel, W. Haken, Illinois J. Math. 21 (1977). ↩

75. G. Gonthier, “Formal Proof—The Four-Color Theorem”, Notices AMS 55 (2008). ↩

76. T. Hales, “A proof of the Kepler conjecture”, Annals of Math. 162 (2005). ↩

77. T. Hales et al., “A formal proof of the Kepler conjecture”, Forum of Math, Pi 5 (2017). ↩

78. M. Heule, O. Kullmann, V. Marek, SAT 2016. ↩

79. M. Heule, “Schur Number Five”, AAAI 2018. ↩

80. A. Wiles, Annals 141 (1995); R. Taylor, A. Wiles, Annals 141 (1995). ↩

81. M. Viazovska, Annals 185 (2017); H. Cohn et al., Annals 185 (2017). ↩

82. W. W. Johnson, W. E. Story, Amer. J. Math. 2 (1879). ↩

83. M. Black, *Critical Thinking* (1946); J. McCarthy (1964) על הבעיה כאתגר למוכיחים אוטומטיים.↩

84. D. Smith, J. S. Myers, C. S. Kaplan, C. Goodman-Strauss, “An aperiodic monotile”, Combinatorial Theory 4 (2024); arXiv:2303.10798.↩

85. A. Davies et al., Nature 600 (2021): https://www.nature.com/articles/s41586-021-04086-x↩

86. B. Romera-Paredes et al., Nature 625 (2024): https://www.nature.com/articles/s41586-023-06924-6↩

87. DeepMind, AlphaEvolve (מאי 2025); arXiv:2506.13131.↩

88. B. Georgiev, J. Gómez-Serrano, T. Tao, A. Z. Wagner (2025), arXiv:2511.02864.↩

89. G. Gardam, “A counterexample to the unit conjecture for group rings”, Annals 194 (2021).↩

90. M. Heule, M. Scheucher, TACAS 2024; B. Subercaseaux et al., ITP 2024.↩

91. P. Erdős, “Some remarks on the theory of graphs”, Bull. AMS 53 (1947).↩

92. H. Huang, Annals 190 (2019).↩

93. Archimedes, *The Method*; J. L. Heiberg (1906); R. Netz, W. Noel, *The Archimedes Codex* (2007).↩

94. P. Candelas, X. de la Ossa, P. Green, L. Parkes, Nucl. Phys. B 359 (1991).↩

95. W. T. Gowers, B. Green, F. Manners, T. Tao, arXiv:2311.05762; https://teorth.github.io/pfr/↩

96. https://teorth.github.io/equational_theories/↩

97. https://bbchallenge.org (הודעה ביולי 2024).↩

98. A. Jaffe, F. Quinn, “‘Theoretical mathematics’: Toward a cultural synthesis of mathematics and theoretical physics”, Bull. AMS 29 (1993).↩

99. Clay Mathematics Institute, Millennium Problems: https://www.claymath.org/millennium-problems ↩

100. J. Chen, T. Hou, arXiv:2210.07191 ↩.

101. F. Kos, D. Poland, D. Simmons-Duffin, A. Vichi, “Precision islands in the Ising and O(N) models”, arXiv:1603.04436 ↩.

102. D. Simmons-Duffin, “A semidefinite program solver for the conformal bootstrap”, arXiv:1502.02033; https://github.com/davidsd/sdpb ↩

103. N. Arkani-Hamed, J. Trnka, “The Amplituhedron”, arXiv:1312.2007 ↩.

104. סיקור: https://phys.org/news/2026-02-chatbot-author-ai-stalled-gluon.html ; ניתוח ביקורתי: https://huggingface.co/blog/dlouapre/gpt-single-minus-gluons ↩

105. M. de Leeuw, A. Pribytok, P. Ryan, “Classifying integrable spin-1/2 chains with nearest neighbour interactions” (2019) ↩.

106. F. Pastawski, B. Yoshida, D. Harlow, J. Preskill, arXiv:1503.06237 ↩.

107. Y.-H. He, arXiv:1706.02714; J. Carifio et al., arXiv:1707.00655 ↩.

108. M. Cranmer, PySR (2023) ↩.

109. S.-M. Udrescu, M. Tegmark, “AI Feynman”, Science Advances (2020): https://www.science.org/doi/10.1126/sciadv.aay2631 ↩

110. J. Tooby-Smith, “HepLean: Digitalising high energy physics”, arXiv:2405.08863; Comput. Phys. Commun. 308 (2025) 109457; https://physlean.com ↩/

111. S. El-Showk et al., arXiv:1403.4545 ↩.

112. I. Lakatos, *Proofs and Refutations: The Logic of Mathematical Discovery*, Cambridge University Press (1976) ↩.

113. רבי לוי בן גרשום, *מעשה חושב* (1321) ↩.

114. N. L. Rabinovitch, *Probability and Statistical Inference in Ancient and Medieval Jewish Literature*, University of Toronto Press (1973). ↩

115. על הנוסח 221 מול 231 ראו `docs/wisdom-bridges.md`, סעיף ב.1. [לא אומת: לא נבדק מול כתבי יד או מהדורה מדעית.] ↩

116. A. Kaplan, *Sefer Yetzirah: The Book of Creation in Theory and Practice* (1990; מהדורה מתוקנת 1997), משנה 4:12. המספור אומת דרך תיאורים משניים בלבד; בנוסחים אחרים (למשל נוסח רס"ג) הקטע מופיע במקום אחר. ↩

117. רמב"ם, משנה תורה, ספר זמנים, הלכות קידוש החודש. ↩

118. שם, יז, כד. המובן אומת בחיפוש מול מקורות משניים (למשל https://outorah.org/p/47556); הנוסח לא נבדק מול מהדורה מודפסת. ↩

119. E. M. Reingold, N. Dershowitz, *Calendrical Calculations*, Cambridge University Press. ↩

120. D. F. Mansfield, N. J. Wildberger, "Plimpton 322 is Babylonian exact sexagesimal trigonometry", Historia Mathematica 44 (2017). ↩

121. D. Witztum, E. Rips, Y. Rosenberg, Statistical Science 9 (1994); B. McKay, D. Bar-Natan, M. Bar-Hillel, G. Kalai, "Solving the Bible Code Puzzle", Statistical Science 14 (1999). ↩

122. משנה, אבות ג, יח (רבי אליעזר בן חסמא). על הפירושים השונים ל"גימטריאות" ראו: https://www.yeshiva.org.il/wiki/index.php/גימטריה ↩

123. https://github.com/teorth/erdosproblems/wiki/AI-contributions-to-Erd%C5%91s-problems (עודכן עד 30 ביוני 2026, לפי `docs/ai-math-lessons.md`). ↩

124. `data/goals.csv`, שורה G050; https://en.wikipedia.org/wiki/Schur_number ; https://en.wikipedia.org/wiki/Van_der_Waerden_number ; B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro, M. Heule, ITP 2024 (Lean-ב empty hexagon). ↩